Discrete Mathematics and Its Applications

Introductory Lecture

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

What is Discrete Mathematics?

- Discrete mathematics is the part of mathematics devoted to the study of discrete (as opposed to continuous) objects.
- Calculus deals with continuous objects and is not part of discrete mathematics.
- Examples of discrete objects: integers, steps taken by a computer program, distinct paths to travel from point A to point B on a map along a road network, ways to pick a winning set of numbers in a lottery.
- A course in discrete mathematics provides the mathematical background needed for all subsequent courses in computer science and for all subsequent courses in the many branches of discrete mathematics.

Kinds of Problems Solved Using Discrete Mathematics

- How many ways can a password be chosen following specific rules?
- How many valid Internet addresses are there?
- What is the probability of winning a particular lottery?
- Is there a link between two computers in a network?
- How can I identify spam email messages?
- How can I encrypt a message so that no unintended recipient can read it?
- How can we build a circuit that adds two integers?

Kinds of Problems Solved Using Discrete Mathematics

- What is the shortest path between two cities using a transportation system?
- Find the shortest tour that visits each of a group of cities only once and then ends in the starting city.
- How can we represent English sentences so that a computer can reason with them?
- How can we prove that there are infinitely many prime numbers?
- How can a list of integers be sorted so that the integers are in increasing order?
- How many steps are required to do such a sorting?
- How can it be proved that a sorting algorithm always correctly sorts a list?

Goals of a Course in Discrete Mathematics

- **Mathematical Reasoning**: Ability to read, understand, and construct mathematical arguments and proofs.
- **Combinatorial Analysis**: Techniques for counting objects of different kinds.
- **Discrete Structures**: Abstract mathematical structures that represent objects and the relationships between them. Examples are sets, permutations, relations, graphs, trees, and finite state machines.

Goals of a Course in Discrete Mathematics

- **Algorithmic Thinking**: One way to solve many problems is to specify an algorithm. An algorithm is a sequence of steps that can be followed to solve any instance of a particular problem. Algorithmic thinking involves specifying algorithms, analyzing the memory and time required by an execution of the algorithm, and verifying that the algorithm will produce the correct answer.
- **Applications and Modeling**: It is important to appreciate and understand the wide range of applications of the topics in discrete mathematics and develop the ability to develop new models in various domains. Concepts from discrete mathematics have not only been used to address problems in computing, but have been applied to solve problems in many areas such as chemistry, biology, linguistics, geography, business, etc.

Discrete Mathematics is a Gateway Course

- Topics in discrete mathematics will be important in many courses that you will take in the future:
  - **Computer Science**: Computer Architecture, Data Structures, Algorithms, Programming Languages, Compilers, Computer Security, Databases, Artificial Intelligence, Networking, Graphics, Game Design, Theory of Computation, ……
  - **Mathematics**: Logic, Set Theory, Probability, Number Theory, Abstract Algebra, Combinatorics, Graph Theory, Game Theory, Network Optimization, …
    - The concepts learned will also be helpful in continuous areas of mathematics.
  - **Other Disciplines**: You may find concepts learned here useful in courses in philosophy, economics, linguistics, and other departments.

The Foundations: Logic and Proofs

Chapter 1, Part I: Propositional Logic

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Propositional Logic
  - The Language of Propositions
  - Applications
  - Logical Equivalences
- Predicate Logic
  - The Language of Quantifiers
  - Logical Equivalences
  - Nested Quantifiers
- Proofs
  - Rules of Inference
  - Proof Methods
  - Proof Strategy

Propositional Logic Summary

- The Language of Propositions
  - Connectives
  - Truth Values
  - Truth Tables
- Applications
  - Translating English Sentences
  - System Specifications
  - Logic Puzzles
  - Logic Circuits
- Logical Equivalences
  - Important Equivalences
  - Showing Equivalence
  - Satisfiability

Propositional Logic

Section 1.1

Section Summary

- Propositions
- Connectives
  - Negation
  - Conjunction
  - Disjunction
  - Implication; contrapositive, inverse, converse
  - Biconditional
- Truth Tables

Propositions

- A *proposition* is a declarative sentence that is either true or false.
- Examples of propositions:
  a) The Moon is made of green cheese.
  b) Trenton is the capital of New Jersey.
  c) Toronto is the capital of Canada.
  d) $1 + 0 = 1$
  e) $0 + 0 = 2$
- Examples that are not propositions.
  a) Sit down!
  b) What time is it?
  c) $x + 1 = 2$
  d) $x + y = z$

Propositional Logic

- Constructing Propositions
  - Propositional Variables: *p*, *q*, *r*, *s*, …
  - The proposition that is always true is denoted by **T** and the proposition that is always false is denoted by **F**.
  - Compound Propositions; constructed from logical connectives and other propositions
    - Negation ¬
    - Conjunction ∧
    - Disjunction ∨
    - Implication →
    - Biconditional ↔

Compound Propositions: Negation

- The *negation* of a proposition p is denoted by $\neg p$ and has this truth table:

p	$\neg p$
T	F
F	T

- **Example**: If p denotes "The earth is round.", then $\neg p$ denotes "It is not the case that the earth is round," or more simply "The earth is not round."

Conjunction

- The *conjunction* of propositions p and q is denoted by $p \wedge q$ and has this truth table:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

- **Example**: If p denotes "I am at home." and q denotes "It is raining." then $p \wedge q$ denotes "I am at home and it is raining."

Disjunction

- The *disjunction* of propositions p and q is denoted by $p \vee q$ and has this truth table:

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

- **Example**: If p denotes "I am at home." and q denotes "It is raining." then $p \vee q$ denotes "I am at home or it is raining."

The Connective Or in English

- In English "or" has two distinct meanings.
  - "Inclusive Or" - In the sentence "Students who have taken CS202 or Math120 may take this class," we assume that students need to have taken one of the prerequisites, but may have taken both. This is the meaning of disjunction. For $p \vee q$ to be true, either one or both of p and q must be true.
  - "Exclusive Or" - When reading the sentence "Soup or salad comes with this entrée," we do not expect to be able to get both soup and salad. This is the meaning of Exclusive Or (Xor). In $p \oplus q$, one of p and q must be true, but not both. The truth table for $\oplus$ is:

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

Implication

- If p and q are propositions, then $p \rightarrow q$ is a *conditional statement* or *implication* which is read as "if p, then q" and has this truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

- **Example**: If p denotes "I am at home." and q denotes "It is raining." then $p \rightarrow q$ denotes "If I am at home then it is raining."
- In $p \rightarrow q$, p is the *hypothesis* (*antecedent* or *premise*) and q is the *conclusion* (or *consequence*).

Understanding Implication

- In $p \rightarrow q$ there does not need to be any connection between the antecedent or the consequent. The "meaning" of $p \rightarrow q$ depends only on the truth values of p and q.
- These implications are perfectly fine, but would not be used in ordinary English.
  - "If the moon is made of green cheese, then I have more money than Bill Gates. "
  - "If the moon is made of green cheese then I'm on welfare."
  - "If 1 + 1 = 3, then your grandma wears combat boots."

Understanding Implication (cont)

- One way to view the logical conditional is to think of an obligation or contract.
  - "If I am elected, then I will lower taxes."
  - "If you get 100% on the final, then you will get an A."
- If the politician is elected and does not lower taxes, then the voters can say that he or she has broken the campaign pledge. Something similar holds for the professor. This corresponds to the case where p is true and q is false.

Different Ways of Expressing $p \rightarrow q$

- **if** p, **then** q
- **if** p, q
- q **unless** $\neg p$
- q **if** p
- q **whenever** p
- q **follows from** p
- p **implies** q
- p **only if** q
- q **when** p
- p **is sufficient for** q
- q **is necessary for** p
- **a necessary condition for** p **is** q
- **a sufficient condition for** q **is** p

Converse, Contrapositive, and Inverse

- From $p \rightarrow q$ we can form new conditional statements .
  - $q \rightarrow p$ is the **converse** of $p \rightarrow q$
  - $\neg q \rightarrow \neg p$ is the **contrapositive** of $p \rightarrow q$
  - $\neg p \rightarrow \neg q$ is the **inverse** of $p \rightarrow q$

Example: Find the converse, inverse, and contrapositive of "It raining is a sufficient condition for my not going to town."

Solution:

converse: If I do not go to town, then it is raining.
inverse: If it is not raining, then I will go to town.
contrapositive: If I go to town, then it is not raining.

Biconditional

- If p and q are propositions, then we can form the *biconditional* proposition $p \leftrightarrow q$, read as "p if and only if q." The biconditional $p \leftrightarrow q$ denotes the proposition with this truth table:

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

- If p denotes "I am at home." and q denotes "It is raining." then $p \leftrightarrow q$ denotes "I am at home if and only if it is raining."

Expressing the Biconditional

- Some alternative ways "*p* if and only if *q*" is expressed in English:
  - *p* **is necessary and sufficient for** *q*
  - **if** *p* **then** *q* **, and conversely**
  - *p* **iff** *q*

Truth Tables For Compound Propositions

- Construction of a truth table:
- Rows
  - Need a row for every possible combination of values for the atomic propositions.
- Columns
  - Need a column for the compound proposition (usually at far right)
  - Need a column for the truth value of each expression that occurs in the compound proposition as it is built up.
    - This includes the atomic propositions

Example Truth Table

- Construct a truth table for $p \vee q \rightarrow \neg r$

p	q	r	¬r	p ∨ q	p ∨ q → ¬r
T	T	T	F	T	F
T	T	F	T	T	T
T	F	T	F	T	F
T	F	F	T	T	T
F	T	T	F	T	F
F	T	F	T	T	T
F	F	T	F	F	T
F	F	F	T	F	T

Equivalent Propositions

- Two propositions are **e***quivalent* if they always have the same truth value.
- **Example**: Show using a truth table that the conditional is equivalent to the contrapositive.

  Solution:

p	*q*	¬ *p*	¬ *q*	*p*→*q*	¬*q*→ ¬ *p*
T	T	F	F	T	T
T	F	F	T	F	F
F	T	T	F	T	T
F	F	T	T	T	T

Using a Truth Table to Show Non-Equivalence

Example: Show using truth tables that neither the converse nor inverse of an implication are not equivalent to the implication.

Solution:

p	*q*	¬ *p*	¬ *q*	*p*→*q*	¬ *p*→¬ *q*	*q*→ *p*
T	T	F	F	T	T	T
T	F	F	T	F	T	T
F	T	T	F	T	F	F
F	F	T	T	T	T	T

Problem

- How many rows are there in a truth table with *n* propositional variables?

  Solution: 2^n We will see how to do this in Chapter 6.

- Note that this means that with n propositional variables, we can construct 2^n distinct (i.e., not equivalent) propositions.

Precedence of Logical Operators

Operator	Precedence
¬	1
∧ ∨	2 3
→ ↔	4 5

$p \vee q \rightarrow \neg r$ is equivalent to $(p \vee q) \rightarrow \neg r$
If the intended meaning is $p \vee (q \rightarrow \neg r)$
then parentheses must be used.

Applications of Propositional Logic

Section 1.2

Applications of Propositional Logic: Summary

- Translating English to Propositional Logic
- System Specifications
- Boolean Searching
- Logic Puzzles
- Logic Circuits
- AI Diagnosis Method (Optional)

Translating English Sentences

- Steps to convert an English sentence to a statement in propositional logic
  - Identify atomic propositions and represent using propositional variables.
  - Determine appropriate logical connectives
- "If I go to Harry's or to the country, I will not go shopping."
  - *p*: I go to Harry's
  - q: I go to the country.
  - *r*: I will go shopping.

If *p* or *q* then not *r*.

$$(p \vee q) \rightarrow \neg r$$

Example

Problem: Translate the following sentence into propositional logic:

"You can access the Internet from campus only if you are a computer science major or you are not a freshman."

One Solution: Let *a*, *c*, and *f* represent respectively "You can access the internet from campus," "You are a computer science major," and "You are a freshman."

$$a \rightarrow (c \vee \neg f)$$

System Specifications

- System and Software engineers take requirements in English and express them in a precise specification language based on logic.

Example: Express in propositional logic:

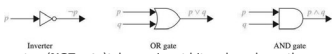
"The automated reply cannot be sent when the file system is full"

Solution: One possible solution: Let *p* denote "The automated reply can be sent" and *q* denote "The file system is full."

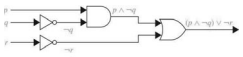
$$q \rightarrow \neg p$$

Logic Circuits (Studied in depth in Chapter 12)

- Electronic circuits; each input/output signal can be viewed as a 0 or 1.
  - 0 represents **False**
  - 1 represents **True**
- Complicated circuits are constructed from three basic circuits called gates.



  - The inverter (**NOT gate**)takes an input bit and produces the negation of that bit.
  - The **OR gate** takes two input bits and produces the value equivalent to the disjunction of the two bits.
  - The **AND gate** takes two input bits and produces the value equivalent to the conjunction of the two bits.
- More complicated digital circuits can be constructed by combining these basic circuits to produce the desired output given the input signals by building a circuit for each piece of the output expression and then combining them. For example:



Propositional Equivalences

Section 1.3

Section Summary

- Tautologies, Contradictions, and Contingencies.
- Logical Equivalence
  - Important Logical Equivalences
  - Showing Logical Equivalence
- Normal Forms (*optional, covered in exercises in text*)
  - Disjunctive Normal Form
  - Conjunctive Normal Form
- Propositional Satisfiability
  - Sudoku Example

Tautologies, Contradictions, and Contingencies

- A tautology is a proposition which is always true.
  - Example: $p \vee \neg p$
- A *contradiction* is a proposition which is always false.
  - Example: $p \wedge \neg p$
- A *contingency* is a proposition which is neither a tautology nor a contradiction, such as *p*

p	$\neg p$	$p \vee \neg p$	$p \wedge \neg p$
T	F	T	F
F	T	T	F

Logically Equivalent

- Two compound propositions p and q are logically equivalent if $p \leftrightarrow q$ is a tautology.
- We write this as $p \Leftrightarrow q$ or as $p \equiv q$ where p and q are compound propositions.
- Two compound propositions p and q are equivalent if and only if the columns in a truth table giving their truth values agree.
- This truth table shows that $\neg p \vee q$ is equivalent to $p \rightarrow q$.

p	q	$\neg p$	$\neg p \vee q$	$p \rightarrow q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

De Morgan's Laws

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$
$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$



Augustus De Morgan

1806-1871

This truth table shows that De Morgan's Second Law holds.

p	q	$\neg p$	$\neg q$	$(p \vee q)$	$\neg(p \vee q)$	$\neg p \wedge \neg q$
T	T	F	F	T	F	F
T	F	F	T	T	F	F
F	T	T	F	T	F	F
F	F	T	T	F	T	T

Key Logical Equivalences

- Identity Laws: $p \wedge T \equiv p$, $p \vee F \equiv p$
- Domination Laws: $p \vee T \equiv T$, $p \wedge F \equiv F$
- Idempotent laws: $p \vee p \equiv p$, $p \wedge p \equiv p$
- Double Negation Law: $\neg(\neg p) \equiv p$
- Negation Laws: $p \vee \neg p \equiv T$, $p \wedge \neg p \equiv F$

Key Logical Equivalences (*cont*)

- Commutative Laws: $p \vee q \equiv q \vee p$, $p \wedge q \equiv q \wedge p$
- Associative Laws: $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
  $(p \vee q) \vee r \equiv p \vee (q \vee r)$
- Distributive Laws: $(p \vee (q \wedge r)) \equiv (p \vee q) \wedge (p \vee r)$
  $(p \wedge (q \vee r)) \equiv (p \wedge q) \vee (p \wedge r)$
- Absorption Laws: $p \vee (p \wedge q) \equiv p$ $p \wedge (p \vee q) \equiv p$

More Logical Equivalences

TABLE 7 Logical Equivalences Involving Conditional Statements.

$p \rightarrow q \equiv \neg p \vee q$
$p \rightarrow q \equiv \neg q \rightarrow \neg p$
$p \vee q \equiv \neg p \rightarrow q$
$p \wedge q \equiv \neg(p \rightarrow \neg q)$
$\neg(p \rightarrow q) \equiv p \wedge \neg q$
$(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$
$(p \rightarrow r) \wedge (q \rightarrow r) \equiv (p \vee q) \rightarrow r$
$(p \rightarrow q) \vee (p \rightarrow r) \equiv p \rightarrow (q \vee r)$
$(p \rightarrow r) \vee (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$

TABLE 8 Logical Equivalences Involving Biconditional Statements.

$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
$p \leftrightarrow q \equiv \neg p \leftrightarrow \neg q$
$p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$
$\neg(p \leftrightarrow q) \equiv p \leftrightarrow \neg q$

Constructing New Logical Equivalences

- We can show that two expressions are logically equivalent by developing a series of logically equivalent statements.
- To prove that $A \equiv B$ we produce a series of equivalences beginning with A and ending with B.

$$A \equiv A_1$$
$$\vdots$$
$$A_n \equiv B$$

- Keep in mind that whenever a proposition (represented by a propositional variable) occurs in the equivalences listed earlier, it may be replaced by an arbitrarily complex compound proposition.

Equivalence Proofs

Example: Show that $\neg(p \vee (\neg p \wedge q))$ is logically equivalent to $\neg p \wedge \neg q$

Solution:

$\neg(p \vee (\neg p \wedge q))$	$\equiv$	$\neg p \wedge \neg(\neg p \wedge q)$	by the second De Morgan law
	$\equiv$	$\neg p \wedge [\neg(\neg p) \vee \neg q]$	by the first De Morgan law
	$\equiv$	$\neg p \wedge (p \vee \neg q)$	by the double negation law
	$\equiv$	$(\neg p \wedge p) \vee (\neg p \wedge \neg q)$	by the second distributive law
	$\equiv$	$F \vee (\neg p \wedge \neg q)$	because $\neg p \wedge p \equiv F$
	$\equiv$	$(\neg p \wedge \neg q) \vee F$	by the commutative law for disjunction
	$\equiv$	$(\neg p \wedge \neg q)$	by the identity law for $\mathbf{F}$

Equivalence Proofs

Example: Show that $(p \wedge q) \rightarrow (p \vee q)$ is a tautology.

Solution:

$(p \wedge q) \rightarrow (p \vee q)$	$\equiv$	$\neg(p \wedge q) \vee (p \vee q)$	by truth table for $\rightarrow$
	$\equiv$	$(\neg p \vee \neg q) \vee (p \vee q)$	by the first De Morgan law
	$\equiv$	$(\neg p \vee p) \vee (\neg q \vee q)$	by associative and commutative laws for disjunction
	$\equiv$	$T \vee T$	by truth tables
	$\equiv$	T	by the domination law

7

Propositional Satisfiability

- A compound proposition is *satisfiable* if there is an assignment of truth values to its variables that make it true. When no such assignments exist, the compound proposition is *unsatisfiable*.
- A compound proposition is unsatisfiable if and only if its negation is a tautology.

Questions on Propositional Satisfiability

Example: Determine the satisfiability of the following compound propositions:

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p)$$

Solution: Satisfiable. Assign **T** to p, q, and r.

$$(p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$$

Solution: Satisfiable. Assign **T** to p and $\boldsymbol{F}$ to q.

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p) \wedge (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$$

Solution: Not satisfiable. Check each possible assignment of truth values to the propositional variables and none will make the proposition true.

Solving Satisfiability Problems

- To solve a Sudoku puzzle, we need to find an assignment of truth values to the 729 variables of the form *p(i,j,n)* that makes the conjunction of the assertions true. Those variables that are assigned T yield a solution to the puzzle.
- A truth table can always be used to determine the satisfiability of a compound proposition. But this is too complex even for modern computers for large problems.
- There has been much work on developing efficient methods for solving satisfiability problems as many practical problems can be translated into satisfiability problems.

The Foundations: Logic and Proofs

Chapter 1, Part II: Predicate Logic

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Summary

- Predicate Logic (First-Order Logic (FOL), Predicate Calculus)
  - The Language of Quantifiers
  - Logical Equivalences
  - Nested Quantifiers
  - Translation from Predicate Logic to English
  - Translation from English to Predicate Logic

Predicates and Quantifiers

Section 1.4

Section Summary

- Predicates
- Variables
- Quantifiers
  - Universal Quantifier
  - Existential Quantifier
- Negating Quantifiers
  - De Morgan's Laws for Quantifiers
- Translating English to Logic
- Logic Programming (*optional*)

Propositional Logic Not Enough

- If we have:
  "All men are mortal."
  "Socrates is a man."
- Does it follow that "Socrates is mortal?"
- Can't be represented in propositional logic. Need a language that talks about objects, their properties, and their relations.
- Later we'll see how to draw inferences.

Introducing Predicate Logic

- Predicate logic uses the following new features:
  - Variables: *x*, *y*, *z*
  - Predicates: *P*(*x*), *M*(*x*)
  - Quantifiers (*to be covered in a few slides*):
- *Propositional functions* are a generalization of propositions.
  - They contain variables and a predicate, e.g., *P*(*x*)
  - Variables can be replaced by elements from their *domain*.

Propositional Functions

- Propositional functions become propositions (and have truth values) when their variables are each replaced by a value from the *domain* (or *bound* by a quantifier, as we will see later).
- The statement *P(x)* is said to be the value of the propositional function *P* at *x*.
- For example, let *P(x)* denote "$x > 0$" and the domain be the integers. Then:
  P(-3) is false.
  P(0) is false.
  P(3) is true.
- Often the domain is denoted by *U*. So in this example *U* is the integers.

Examples of Propositional Functions

- Let "*x* + *y* = *z*" be denoted by *R*(*x, y, z*) and *U* (for all three variables) be the integers. Find these truth values:
  R(2,-1,5)
  Solution: F
  R(3,4,7)
  Solution: T
  R(*x*, 3, *z*)
  Solution: Not a Proposition
- Now let "*x* - *y* = *z*" be denoted by *Q*(*x*, *y*, *z*), with U as the integers. Find these truth values:
  Q(2,-1,3)
  Solution: T
  Q(3,4,7)
  Solution: F
  Q(*x*, 3, *z*)
  Solution: Not a Proposition

Compound Expressions

- Connectives from propositional logic carry over to predicate logic.
- If *P(x)* denotes "$x > 0$," find these truth values:
  P(3) ∨ P(-1) **Solution**: T
  P(3) ∧ P(-1) **Solution**: F
  P(3) → P(-1) **Solution**: F
  P(3) → ¬P(-1) **Solution**: T
- Expressions with variables are not propositions and therefore do not have truth values. For example,
  P(3) ∧ P(*y*)
  P(*x*) → P(*y*)
- When used with quantifiers (to be introduced next), these expressions (propositional functions) become propositions.

Quantifiers



Charles Peirce (1839-1914)

- We need *quantifiers* to express the meaning of English words including *all* and *some*:
  - "All men are Mortal."
  - "Some cats do not have fur."
- The two most important quantifiers are:
  - *Universal Quantifier,* "For all," symbol: ∀
  - *Existential Quantifier*, "There exists," symbol: ∃
- We write as in ∀*x P*(*x*) and ∃*x P*(*x*).
- ∀*x P*(*x*) asserts *P*(*x*) is true for <u>every</u> *x* in the *domain*.
- ∃*x P*(*x*) asserts *P*(*x*) is true for <u>some</u> *x* in the *domain*.
- The quantifiers are said to bind the variable *x* in these expressions.

Universal Quantifier

- ∀*x P*(*x*) is read as "For all *x*, P(*x*)" or "For every *x*, P(*x*)"

  Examples:

  1) If *P(x)* denotes "$x > 0$" and U is the integers, then ∀*x P*(*x*) is false.
  2) If *P(x)* denotes "$x > 0$" and U is the positive integers, then ∀*x P*(*x*) is true.
  3) If *P(x)* denotes "*x* is even" and U is the integers, then ∀ *x P*(*x*) is false.

Existential Quantifier

- ∃*x P*(*x*) is read as "For some *x*, P(*x*)", or as "There is an *x* such that P(*x*)," or "For at least one *x*, P(*x*)."

  Examples:

  1. If *P(x)* denotes "$x > 0$" and U is the integers, then ∃*x P*(*x*) is true. It is also true if U is the positive integers.
  2. If *P(x)* denotes "$x < 0$" and U is the positive integers, then ∃*x P*(*x*) is false.
  3. If *P(x)* denotes "*x* is even" and U is the integers, then ∃*x P*(*x*) is true.

Thinking about Quantifiers

- When the domain of discourse is finite, we can think of quantification as looping through the elements of the domain.
- To evaluate ∀*x P*(*x*) loop through all *x* in the domain.
  - If at every step P(*x*) is true, then ∀*x P*(*x*) is true.
  - If at a step P(*x*) is false, then ∀*x P*(*x*) is false and the loop terminates.
- To evaluate ∃*x P*(*x*) loop through all *x* in the domain.
  - If at some step, P(*x*) is true, then ∃*x P*(*x*) is true and the loop terminates.
  - If the loop ends without finding an *x* for which P(*x*) is true, then ∃*x P*(*x*) is false.
- Even if the domains are infinite, we can still think of the quantifiers this fashion, but the loops will not terminate in some cases.

Properties of Quantifiers

- The truth value of $\exists x\, P(x)$ and $\forall x\, P(x)$ depend on both the propositional function $P(x)$ and on the domain U.
- **Examples**:
  1. If U is the positive integers and $P(x)$ is the statement "$x < 2$", then $\exists x\, P(x)$ is true, but $\forall x\, P(x)$ is false.
  2. If U is the negative integers and $P(x)$ is the statement "$x < 2$", then both $\exists x\, P(x)$ and $\forall x\, P(x)$ are true.
  3. If U consists of 3, 4, and 5, and $P(x)$ is the statement "$x > 2$", then both $\exists x\, P(x)$ and $\forall x\, P(x)$ are true. But if $P(x)$ is the statement "$x < 2$", then both $\exists x\, P(x)$ and $\forall x\, P(x)$ are false.

Precedence of Quantifiers

- The quantifiers $\forall$ and $\exists$ have higher precedence than all the logical operators.
- For example, $\forall x\, P(x) \vee Q(x)$ means $(\forall x\, P(x)) \vee Q(x)$
- $\forall x\, (P(x) \vee Q(x))$ means something different.
- Unfortunately, often people write $\forall x\, P(x) \vee Q(x)$ when they mean $\forall x\, (P(x) \vee Q(x))$.

Returning to the Socrates Example

- Introduce the propositional functions *Man(x)* denoting "*x* is a man" and *Mortal(x)* denoting "*x* is mortal." Specify the domain as all people.
- The two premises are: $\forall x(Man(x) \rightarrow Mortal(x))$
  $Man(Socrates)$
- The conclusion is: $Mortal(Socrates)$
- Later we will show how to prove that the conclusion follows from the premises.

Equivalences in Predicate Logic

- Statements involving predicates and quantifiers are *logically equivalent* if and only if they have the same truth value
  - for every predicate substituted into these statements and
  - for every domain of discourse used for the variables in the expressions.
- The notation $S \equiv T$ indicates that S and T are logically equivalent.
- **Example**: $\forall x \neg\neg S(x) \equiv \forall x\, S(x)$

Thinking about Quantifiers as Conjunctions and Disjunctions

- If the domain is finite, a universally quantified proposition is equivalent to a conjunction of propositions without quantifiers and an existentially quantified proposition is equivalent to a disjunction of propositions without quantifiers.
- If U consists of the integers 1,2, and 3:

$$\forall x P(x) \equiv P(1) \wedge P(2) \wedge P(3)$$

$$\exists x P(x) \equiv P(1) \vee P(2) \vee P(3)$$

- Even if the domains are infinite, you can still think of the quantifiers in this fashion, but the equivalent expressions without quantifiers will be infinitely long.

Negating Quantified Expressions

- Consider $\forall x\, J(x)$
  "Every student in your class has taken a course in Java."
  Here $J(x)$ is "x has taken a course in Java" and the domain is students in your class.
- Negating the original statement gives "It is not the case that every student in your class has taken Java." This implies that "There is a student in your class who has not taken Java."

  Symbolically $\neg \forall x\, J(x)$ and $\exists x\, \neg J(x)$ are equivalent

Negating Quantified Expressions (*continued*)

- Now Consider $\exists x\, J(x)$
  "There is a student in this class who has taken a course in Java."
  Where $J(x)$ is "x has taken a course in Java."
- Negating the original statement gives "It is not the case that there is a student in this class who has taken Java." This implies that "Every student in this class has not taken Java"

  Symbolically $\neg \exists x\, J(x)$ and $\forall x\, \neg J(x)$ are equivalent

De Morgan's Laws for Quantifiers

- The rules for negating quantifiers are:

TABLE 2 **De Morgan's Laws for Quantifiers.**

Negation	*Equivalent Statement*	*When Is Negation True?*	*When False?*
$\neg\exists x P(x)$	$\forall x \neg P(x)$	For every x, $P(x)$ is false.	There is an x for which $P(x)$ is true.
$\neg\forall x P(x)$	$\exists x \neg P(x)$	There is an x for which $P(x)$ is false.	$P(x)$ is true for every x.

- The reasoning in the table shows that:

$$\neg\forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg\exists x P(x) \equiv \forall x \neg P(x)$$

- These are important. You will use these.

Translation from English to Logic

Examples:

1. "Some student in this class has visited Mexico."
   Solution: Let $M(x)$ denote "x has visited Mexico" and $S(x)$ denote "x is a student in this class," and U be all people.
   $$\exists x\ (S(x) \wedge M(x))$$
2. "Every student in this class has visited Canada or Mexico."
   Solution: Add $C(x)$ denoting "x has visited Canada."
   $$\forall x\ (S(x) \rightarrow (M(x) \vee C(x)))$$

Lewis Carroll Example



Charles Lutwidge Dodgson
(AKA Lewis Caroll)
(1832-1898)

- The first two are called *premises* and the third is called the *conclusion*.
  1. "All lions are fierce."
  2. "Some lions do not drink coffee."
  3. "Some fierce creatures do not drink coffee."
- Here is one way to translate these statements to predicate logic. Let P(x), Q(x), and R(x) be the propositional functions "x is a lion," "x is fierce," and "x drinks coffee," respectively.
  1. $\forall x\ (P(x) \rightarrow Q(x))$
  2. $\exists x\ (P(x) \wedge \neg R(x))$
  3. $\exists x\ (Q(x) \wedge \neg R(x))$
- Later we will see how to prove that the conclusion follows from the premises.

System Specification Example

- Predicate logic is used for specifying properties that systems must satisfy.
- For example, translate into predicate logic:
  - "Every mail message larger than one megabyte will be compressed."
  - "If a user is active, at least one network link will be available."
- Decide on predicates and domains (left implicit here) for the variables:
  - Let $L(m, y)$ be "Mail message m is larger than y megabytes."
  - Let $C(m)$ denote "Mail message m will be compressed."
  - Let $A(u)$ represent "User u is active."
  - Let $S(n, x)$ represent "Network link n is state x.
- Now we have:
  $$\forall m(L(m, 1) \rightarrow C(m))$$
  $$\exists u\, A(u) \rightarrow \exists n\, S(n, available)$$

Nested Quantifiers

Section 1.4

Section Summary

- Nested Quantifiers
- Order of Quantifiers
- Translating from Nested Quantifiers into English
- Translating Mathematical Statements into Statements involving Nested Quantifiers.
- Translated English Sentences into Logical Expressions.
- Negating Nested Quantifiers.

Nested Quantifiers

- Nested quantifiers are often necessary to express the meaning of sentences in English as well as important concepts in computer science and mathematics.
  Example: "Every real number has an inverse" is
  $$\forall x \exists y(x + y = 0)$$
  where the domains of x and y are the real numbers.
- We can also think of nested propositional functions:
  $\forall x \exists y(x + y = 0)$ can be viewed as $\forall x\, Q(x)$ where $Q(x)$ is $\exists y\, P(x, y)$ where $P(x, y)$ is $(x + y = 0)$

Thinking of Nested Quantification

- Nested Loops
  - To see if $\forall x \forall y P(x,y)$ is true, loop through the values of x:
    - At each step, loop through the values for y.
    - If for some pair of x and y, $P(x,y)$ is false, then $\forall x\ \forall y P(x,y)$ is false and both the outer and inner loop terminate.

    $\forall x\ \forall y\ P(x,y)$ is true if the outer loop ends after stepping through each x.
  - To see if $\forall x \exists y P(x,y)$ is true, loop through the values of x:
    - At each step, loop through the values for y.
    - The inner loop ends when a pair x and y is found such that $P(x, y)$ is true.
    - If no y is found such that $P(x, y)$ is true the outer loop terminates as $\forall x\ \exists y P(x,y)$ has been shown to be false.

    $\forall x \exists y\ P(x,y)$ is true if the outer loop ends after stepping through each x.
- If the domains of the variables are infinite, then this process can not actually be carried out.

Order of Quantifiers

Examples:

1. Let $P(x,y)$ be the statement "$x + y = y + x$." Assume that U is the real numbers. Then $\forall x\ \forall y P(x,y)$ and $\forall y\ \forall x P(x,y)$ have the same truth value.
2. Let $Q(x,y)$ be the statement "$x + y = 0$." Assume that U is the real numbers. Then $\forall x \exists y Q(x,y)$ is true, but $\exists y\ \forall x Q(x,y)$ is false.

Questions on Order of Quantifiers

Example 1: Let *U* be the real numbers,

Define *P(x,y) : x · y = 0*

What is the truth value of the following:

1. $\forall x \forall y P(x,y)$
   Answer: False
2. $\forall x \exists y P(x,y)$
   Answer: True
3. $\exists x \forall y\, P(x,y)$
   Answer: True
4. $\exists x\, \exists y\, P(x,y)$
   Answer: True

Questions on Order of Quantifiers

Example 2: Let *U* be the real numbers,

Define *P(x,y) : x / y = 1*

What is the truth value of the following:

1. $\forall x \forall y P(x,y)$
   Answer: False
2. $\forall x \exists y P(x,y)$
   Answer: False
3. $\exists x \forall y\, P(x,y)$
   Answer: False
4. $\exists x\, \exists y\, P(x,y)$
   Answer: True

Quantifications of Two Variables

Statement	When True?	When False
$\forall x \forall y P(x,y)$ $\forall y \forall x P(x,y)$	*P*(*x,y*) is true for every pair *x,y*.	There is a pair *x, y* for which *P*(*x,y*) is false.
$\forall x \exists y P(x,y)$	For every *x* there is a *y* for which *P*(*x,y*) is true.	There is an x such that *P*(*x,y*) is false for every *y*.
$\exists x \forall y P(x,y)$	There is an *x* for which *P*(*x,y*) is true for every *y*.	For every *x* there is a y for which *P*(x,y) is false.
$\exists x \exists y P(x,y)$ $\exists y \exists x P(x,y)$	There is a pair *x, y* for which *P*(*x,y*) is true.	*P*(x,y) is false for every pair *x,y*

Translating Nested Quantifiers into English

Example 1: Translate the statement

$\forall x\ (C(x) \vee \exists y\ (C(y) \wedge F(x, y)))$

where C(x) is "*x* has a computer," and *F*(*x*,*y*) is "*x* and *y* are friends," and the domain for both *x* and *y* consists of all students in your school.

Solution: Every student in your school has a computer or has a friend who has a computer.

Example 2: Translate the statement

$\exists x\ \forall y\ \forall z\ ((F(x, y) \wedge F(x,z) \wedge (y \neq z)) \rightarrow \neg F(y,z))$

Solution: There is a student none of whose friends are also friends with each other.

Translating Mathematical Statements into Predicate Logic

Example : Translate "The sum of two positive integers is always positive" into a logical expression.

Solution:

1. Rewrite the statement to make the implied quantifiers and domains explicit:
   "For every two integers, if these integers are both positive, then the sum of these integers is positive."
2. Introduce the variables *x* and *y*, and specify the domain, to obtain:
   "For all positive integers *x* and *y*, *x* + *y* is positive."
3. The result is:
   $\forall x\ \forall\ y((x > 0) \wedge (y > 0) \rightarrow (x + y > 0))$
   where the domain of both variables consists of all integers

Translating English into Logical Expressions Example

Example: Use quantifiers to express the statement "There is a woman who has taken a flight on every airline in the world."

Solution:

1. Let *P(w,f)* be "*w* has taken *f* " and *Q*(*f,a*) be "*f* is a flight on *a* ."
2. The domain of *w* is all women, the domain of *f* is all flights, and the domain of *a* is all airlines.
3. Then the statement can be expressed as:
   $\exists w\ \forall a\ \exists f\ (P(w,f) \wedge Q(f,a))$

Calculus in Logic (*optional*)

Example: Use quantifiers to express the definition of the limit of *a* real-valued function *f(x)* of a real variable *x* at a point *a* in its domain.

Solution: Recall the definition of the statement

$\lim_{x \to a} f(x) = L$

is "For every real number $\varepsilon > 0$, there exists a real number $\delta > 0$ such that $|f(x) - L| < \varepsilon$ whenever $0 < |x - a| < \delta$."

Using quantifiers:

$\forall \epsilon \exists \delta \forall x (0 < |\ x - a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)$

Where the domain for the variables ε and δ consists of all positive real numbers and the domain for *x* consists of all real numbers.

Questions on Translation from English

Choose the obvious predicates and express in predicate logic.

Example 1: "Brothers are siblings."
Solution: $\forall x\ \forall y\ (B(x,y) \rightarrow S(x,y))$

Example 2: "Siblinghood is symmetric."
Solution: $\forall x\ \forall y\ (S(x,y) \rightarrow S(y,x))$

Example 3: "Everybody loves somebody."
Solution: $\forall x\ \exists y\ L(x,y)$

Example 4: "There is someone who is loved by everyone."
Solution: $\exists y\ \forall x\ L(x,y)$

Example 5: "There is someone who loves someone."
Solution: $\exists x\ \exists y\ L(x,y)$

Example 6: "Everyone loves himself"
Solution: $\forall x\ L(x,x)$

Negating Nested Quantifiers

Example 1: Recall the logical expression developed three slides back:

$\exists w \forall a \exists f\ (P(w,f) \wedge Q(f,a))$

Part 1: Use quantifiers to express the statement that "There does not exist a woman who has taken a flight on every airline in the world."

Solution: $\neg\exists w \forall a \exists f\ (P(w,f) \wedge Q(f,a))$

Part 2: Now use De Morgan's Laws to move the negation as far inwards as possible.

Solution:

1. $\neg\exists w \forall a \exists f\ (P(w,f) \wedge Q(f,a))$
2. $\forall w \neg \forall a \exists f\ (P(w,f) \wedge Q(f,a))$ by De Morgan's for $\exists$
3. $\forall w \exists a \neg \exists f\ (P(w,f) \wedge Q(f,a))$ by De Morgan's for $\forall$
4. $\forall w \exists a \forall f \neg\ (P(w,f) \wedge Q(f,a))$ by De Morgan's for $\exists$
5. $\forall w \exists a \forall f (\neg\ P(w,f) \vee \neg\ Q(f,a))$ by De Morgan's for $\wedge$.

Part 3: Can you translate the result back into English?

Solution:

"For every woman there is an airline such that for all flights, this woman has not taken that flight or that flight is not on this airline"

Return to Calculus and Logic (*Opt*)

Example : Recall the logical expression developed in the calculus example three slides back.

Use quantifiers and predicates to express that $\lim_{x\to a} f(x)$ does not exist.

1. We need to say that for all real numbers L, $\lim_{x\to a} f(x) \neq L$
2. The result from the previous example can be negated to yield:

$$\neg\forall\epsilon\exists\delta\forall x(0 < |\ x-a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)$$

3. Now we can repeatedly apply the rules for negating quantified expressions:

$$\begin{aligned}
&\neg\forall\epsilon\exists\delta\forall x(0 < |\ x-a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)\\
&\equiv \exists\epsilon\neg\exists\delta\forall x(0 < |\ x-a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)\\
&\equiv \exists\epsilon\forall\delta\neg\forall x(0 < |\ \ x-a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)\\
&\equiv \exists\epsilon\forall\delta\exists x\neg(0 < |\ \ x-a\ | < \delta \rightarrow |\ f(x) - L\ | < \epsilon)\\
&\equiv \exists\epsilon\forall\delta\exists x\neg(0 < |\ x-a\ | < \ \delta \wedge\ |\ f(x) - L\ | \geq \epsilon)
\end{aligned}$$

The last step uses the equivalence $\neg(p\rightarrow q) \equiv p\wedge\neg q$

Calculus in Predicate Logic (*optional*)

4. Therefore, to say that $\lim_{x\to a} f(x)$ does not exist means that for all real numbers L, $\lim_{x\to a} f(x) \neq L$ can be expressed as:

   $\forall L\exists\epsilon\forall\delta\exists x\neg(0 < |\ x-a\ | < \delta \wedge\ |\ f(x) - L\ | \geq \epsilon)$

   Remember that ε and δ range over all positive real numbers and x over all real numbers.
5. Translating back into English we have, for every real number L, there is a real number $\varepsilon > 0$, such that for every real number $\delta > 0$, there exists a real number x such that $0 < |\ x - a\ | < \delta$ and $|f(x) - L\ | \geq \varepsilon$.

Some Questions about Quantifiers (*optional*)

- Can you switch the order of quantifiers?
  - Is this a valid equivalence? $\forall x\forall y P(x,y) \equiv \forall y\forall x P(x,y)$
    Solution: Yes! The left and the right side will always have the same truth value. The order in which x and y are picked does not matter.
  - Is this a valid equivalence? $\forall x\exists y P(x,y) \equiv \exists y\forall x P(x,y)$
    Solution: No! The left and the right side may have different truth values for some propositional functions for *P*. Try "x + y = 0" for *P(x,y)* with *U* being the integers. The order in which the values of x and y are picked does matter.
- Can you distribute quantifiers over logical connectives?
  - Is this a valid equivalence? $\forall x(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$
    Solution: Yes! The left and the right side will always have the same truth value no matter what propositional functions are denoted by *P(x)* and *Q(x)*.
  - Is this a valid equivalence? $\forall x(P(x) \vee Q(x)) \equiv \forall x P(x) \vee \forall x Q(x)$
    Solution: No! The left and the right side may have different truth values. Pick "x is a fish" for P(x) and "x has scales" for Q(x) with the domain of discourse being all animals. Then the left side is false, because there are some fish that do not have scales. But the right side is true since not all animals are fish.

The Foundations: Logic and Proofs

Chapter 1, Part III: Proofs

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Summary

- Valid Arguments and Rules of Inference
- Proof Methods
- Proof Strategies

Rules of Inference

Section 1.6

Section Summary

- Valid Arguments
- Inference Rules for Propositional Logic
- Using Rules of Inference to Build Arguments
- Rules of Inference for Quantified Statements
- Building Arguments for Quantified Statements

Revisiting the Socrates Example

- We have the two premises:
  - "All men are mortal."
  - "Socrates is a man."
- And the conclusion:
  - "Socrates is mortal."
- How do we get the conclusion from the premises?

The Argument

- We can express the premises (above the line) and the conclusion (below the line) in predicate logic as an argument:

$$\frac{\begin{array}{c}\forall x(Man(x) \rightarrow Mortal(x)) \\ Man(Socrates)\end{array}}{\therefore \quad Mortal(Socrates)}$$

- We will see shortly that this is a valid argument.

Valid Arguments

- We will show how to construct valid arguments in two stages; first for propositional logic and then for predicate logic. The rules of inference are the essential building block in the construction of valid arguments.
  1. Propositional Logic
     Inference Rules
  2. Predicate Logic
     Inference rules for propositional logic plus additional inference rules to handle variables and quantifiers.

Arguments in Propositional Logic

- A *argument* in propositional logic is a sequence of propositions. All but the final proposition are called *premises*. The last statement is the *conclusion*.
- The argument is valid if the premises imply the conclusion. An *argument form* is an argument that is valid no matter what propositions are substituted into its propositional variables.
- If the premises are $p_1, p_2, \ldots, p_n$ and the conclusion is q then
  $(p_1 \wedge p_2 \wedge \ldots \wedge p_n) \rightarrow q$ is a tautology.
- Inference rules are all argument simple argument forms that will be used to construct more complex argument forms.

Rules of Inference for Propositional Logic: Modus Ponens

$$\frac{\begin{array}{l}p \rightarrow q \\ p\end{array}}{\therefore q}$$

Corresponding Tautology:
$(p \wedge (p \rightarrow q)) \rightarrow q$

Example:
Let *p* be "It is snowing."
Let *q* be "I will study discrete math."

"If it is snowing, then I will study discrete math."
"It is snowing."

"Therefore , I will study discrete math."

Modus Tollens

$$\frac{\begin{array}{l}p \rightarrow q \\ \neg q\end{array}}{\therefore \neg p}$$

Corresponding Tautology:
$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$

Example:
Let *p* be "it is snowing."
Let *q* be "I will study discrete math."

"If it is snowing, then I will study discrete math."
"I will not study discrete math."

"Therefore , it is not snowing."

Hypothetical Syllogism

$$\frac{\begin{array}{l}p \rightarrow q \\ q \rightarrow r\end{array}}{\therefore p \rightarrow r}$$

Corresponding Tautology:
$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$

Example:
Let *p* be "it snows."
Let *q* be "I will study discrete math."
Let *r* be "I will get an A."

"If it snows, then I will study discrete math."
"If I study discrete math, I will get an A."

"Therefore , If it snows, I will get an A."

Disjunctive Syllogism

$$\frac{\begin{array}{l}p \vee q \\ \neg p\end{array}}{\therefore q}$$

Corresponding Tautology:
$(\neg p \wedge (p \vee q)) \rightarrow q$

Example:
Let *p* be "I will study discrete math."
Let *q* be "I will study English literature."

"I will study discrete math or I will study English literature."
"I will not study discrete math."

"Therefore , I will study English literature."

Addition

$$\frac{p}{\therefore p \vee q}$$

Corresponding Tautology:
$p \rightarrow (p \vee q)$

Example:
Let *p* be "I will study discrete math."
Let *q* be "I will visit Las Vegas."

"I will study discrete math."

"Therefore, I will study discrete math or I will visit Las Vegas."

Simplification

$$\frac{p \wedge q}{\therefore q}$$

Corresponding Tautology:
$(p \wedge q) \rightarrow p$

Example:
Let *p* be "I will study discrete math."
Let *q* be "I will study English literature."

"I will study discrete math and English literature"

"Therefore, I will study discrete math."

Conjunction

$$\frac{\begin{array}{l} p \\ q \end{array}}{\therefore p \wedge q}$$

Corresponding Tautology:
$((p) \wedge (q)) \rightarrow (p \wedge q)$

Example:
Let *p* be "I will study discrete math."
Let *q* be "I will study English literature."

"I will study discrete math."
"I will study English literature."

"Therefore, I will study discrete math and I will study English literature."

Resolution

Resolution plays an important role in AI and is used in Prolog.

$$\frac{\begin{array}{l} \neg p \vee r \\ p \vee q \end{array}}{\therefore q \vee r}$$

Corresponding Tautology:
$((\neg p \vee r) \wedge (p \vee q)) \rightarrow (q \vee r)$

Example:
Let *p* be "I will study discrete math."
Let *r* be "I will study English literature."
Let q be "I will study databases."

"I will not study discrete math or I will study English literature."
"I will study discrete math or I will study databases."

"Therefore, I will study databases or I will study English literature."

Using the Rules of Inference to Build Valid Arguments

- A *valid argument* is a sequence of statements. Each statement is either a premise or follows from previous statements by rules of inference. The last statement is called conclusion.
- A valid argument takes the following form:

S_1
S_2
.
.
.
S_n
$\therefore C$

Valid Arguments

Example 1: From the single proposition

$$p \wedge (p \rightarrow q)$$

Show that *q* is a conclusion.

Solution:

Step	**Reason**
1. $p \wedge (p \rightarrow q)$	Premise
2. p	Simplification using (1)
3. $p \rightarrow q$	Simplification using (1)
4. q	Modus Ponens using (2) and (3)

Valid Arguments

Example 2:

- With these hypotheses:
  "It is not sunny this afternoon and it is colder than yesterday."
  "We will go swimming only if it is sunny."
  "If we do not go swimming, then we will take a canoe trip."
  "If we take a canoe trip, then we will be home by sunset."
- Using the inference rules, construct a valid argument for the conclusion:
  "We will be home by sunset."

Solution:

1. Choose propositional variables:
   p : "It is sunny this afternoon." *r* : "We will go swimming." *t* : "We will be home by sunset."
   q : "It is colder than yesterday." *s* : "We will take a canoe trip."
2. Translation into propositional logic:
   Hypotheses: $\neg p \wedge q,\ r \rightarrow p,\ \neg r \rightarrow s,\ s \rightarrow t$
   Conclusion: t

Continued on next slide →

Valid Arguments

3. Construct the Valid Argument

Step	**Reason**
1. $\neg p \wedge q$	Premise
2. $\neg p$	Simplification using (1)
3. $r \rightarrow p$	Premise
4. $\neg r$	Modus tollens using (2) and (3)
5. $\neg r \rightarrow s$	Premise
6. s	Modus ponens using (4) and (5)
7. $s \rightarrow t$	Premise
8. t	Modus ponens using (6) and (7)

Handling Quantified Statements

- Valid arguments for quantified statements are a sequence of statements. Each statement is either a premise or follows from previous statements by rules of inference which include:
  - Rules of Inference for Propositional Logic
  - Rules of Inference for Quantified Statements
- The rules of inference for quantified statements are introduced in the next several slides.

Universal Instantiation (UI)

$$\frac{\forall x P(x)}{\therefore P(c)}$$

Example:

Our domain consists of all dogs and Fido is a dog.

"All dogs are cuddly."

"Therefore, Fido is cuddly."

Universal Generalization (UG)

$$\frac{P(c) \text{ for an arbitrary } c}{\therefore \forall x P(x)}$$

Used often implicitly in Mathematical Proofs.

Existential Instantiation (EI)

$$\frac{\exists x P(x)}{\therefore P(c) \text{ for some element } c}$$

Example:

"There is someone who got an A in the course."
"Let's call her *a* and say that *a* got an A"

Existential Generalization (EG)

$$\frac{P(c) \text{ for some element } c}{\therefore \exists x P(x)}$$

Example:

"Michelle got an A in the class."
"Therefore, someone got an A in the class."

Using Rules of Inference

Example 1: Using the rules of inference, construct a valid argument to show that
"John Smith has two legs"
is a consequence of the premises:
"Every man has two legs." "John Smith is a man."

Solution: Let $M(x)$ denote "x is a man" and $L(x)$ "x has two legs" and let John Smith be a member of the domain.

Valid Argument:

Step	Reason
1. $\forall x(M(x) \rightarrow L(x))$	Premise
2. $M(J) \rightarrow L(J)$	UI from (1)
3. $M(J)$	Premise
4. $L(J)$	Modus Ponens using (2) and (3)

Using Rules of Inference

Example 2: Use the rules of inference to construct a valid argument showing that the conclusion
"Someone who passed the first exam has not read the book."
follows from the premises
"A student in this class has not read the book."
"Everyone in this class passed the first exam."

Solution: Let $C(x)$ denote "x is in this class," $B(x)$ denote "x has read the book," and $P(x)$ denote "x passed the first exam."

First we translate the premises and conclusion into symbolic form.

$$\frac{\exists x(C(x) \wedge \neg B(x)) \quad \forall x(C(x) \rightarrow P(x))}{\therefore \exists x(P(x) \wedge \neg B(x))}$$

Continued on next slide →

Using Rules of Inference

Valid Argument:

Step	Reason
1. $\exists x(C(x) \wedge \neg B(x))$	Premise
2. $C(a) \wedge \neg B(a)$	EI from (1)
3. $C(a)$	Simplification from (2)
4. $\forall x(C(x) \rightarrow P(x))$	Premise
5. $C(a) \rightarrow P(a)$	UI from (4)
6. $P(a)$	MP from (3) and (5)
7. $\neg B(a)$	Simplification from (2)
8. $P(a) \wedge \neg B(a)$	Conj from (6) and (7)
9. $\exists x(P(x) \wedge \neg B(x))$	EG from (8)

Returning to the Socrates Example

$$\forall x(Man(x) \rightarrow Mortal(x))$$

$$Man(Socrates)$$

$$\therefore \quad Mortal(Socrates)$$

Solution for Socrates Example

Valid Argument

Step	**Reason**
1. $\forall x(Man(x) \rightarrow Mortal(x))$	Premise
2. $Man(Socrates) \rightarrow Mortal(Socrates)$	UI from (1)
3. $Man(Socrates)$	Premise
4. $Mortal(Socrates)$	MP from (2) and (3)

Universal Modus Ponens

Universal Modus Ponens combines universal instantiation and modus ponens into one rule.

$$\forall x(P(x) \rightarrow Q(x))$$
$$P(a), \text{where } a \text{ is a particular element in the domain}$$
$$\therefore Q(a)$$

This rule could be used in the Socrates example.

Introduction to Proofs

Section 1.7

Section Summary

- Mathematical Proofs
- Forms of Theorems
- Direct Proofs
- Indirect Proofs
  - Proof of the Contrapositive
  - Proof by Contradiction

Proofs of Mathematical Statements

- A *proof* is a valid argument that establishes the truth of a statement.
- In math, CS, and other disciplines, informal proofs which are generally shorter, are generally used.
  - More than one rule of inference are often used in a step.
  - Steps may be skipped.
  - The rules of inference used are not explicitly stated.
  - Easier for to understand and to explain to people.
  - But it is also easier to introduce errors.
- Proofs have many practical applications:
  - verification that computer programs are correct
  - establishing that operating systems are secure
  - enabling programs to make inferences in artificial intelligence
  - showing that system specifications are consistent

Definitions

- A *theorem* is a statement that can be shown to be true using:
  - definitions
  - other theorems
  - *axioms* (statements which are given as true)
  - rules of inference
- A *lemma* is a 'helping theorem' or a result which is needed to prove a theorem.
- A *corollary* is a result which follows directly from a theorem.
- Less important theorems are sometimes called *propositions*.
- A *conjecture* is a statement that is being proposed to be true. Once a proof of a conjecture is found, it becomes a theorem. It may turn out to be false.

Forms of Theorems

- Many theorems assert that a property holds for all elements in a domain, such as the integers, the real numbers, or some of the discrete structures that we will study in this class.
- Often the universal quantifier (needed for a precise statement of a theorem) is omitted by standard mathematical convention.

For example, the statement:

"If $x > y$, where x and y are positive real numbers, then $x^2 > y^2$ "

really means

"For all positive real numbers x and y, if $x > y$, then $x^2 > y^2$."

Proving Theorems

- Many theorems have the form:

$$\forall x(P(x) \rightarrow Q(x))$$

- To prove them, we show that where *c* is an arbitrary element of the domain,

$$P(c) \rightarrow Q(c)$$

- By universal generalization the truth of the original formula follows.
- So, we must prove something of the form $p \rightarrow q$

Even and Odd Integers

Definition: The integer *n* is even if there exists an integer *k* such that *n* = 2*k*, and *n* is odd if there exists an integer *k*, such that *n* = 2*k* + 1. Note that every integer is either even or odd and no integer is both even and odd.

We will need this basic fact about the integers in some of the example proofs to follow. We will learn more about the integers in Chapter 4.

Proving Conditional Statements: $p \rightarrow q$

- *Direct Proof*: Assume that p is true. Use rules of inference, axioms, and logical equivalences to show that q must also be true.

Example: Give a direct proof of the theorem "If *n* is an odd integer, then n^2 is odd."

Solution: Assume that *n* is odd. Then *n* = 2*k* + 1 for an integer *k*. Squaring both sides of the equation, we get:

$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1 = 2r + 1$,

where $r = 2k^2 + 2k$, an integer.

We have proved that if n is an odd integer, then n^2 is an odd integer. ◄

(◄ marks the end of the proof. Sometimes **QED** is used instead.)

Proving Conditional Statements: $p \rightarrow q$

Definition: The real number *r* is *rational* if there exist integers *p* and *q* where $q \neq 0$ such that *r* = *p*/*q*

Example: Prove that the sum of two rational numbers is rational.

Solution*:* Assume *r* and *s* are two rational numbers. Then there must be integers *p, q* and also *t, u* such that

$r = p/q, \ \ s = t/u, \ \ u \neq 0, \ q \neq 0$

$r + s = \frac{p}{q} + \frac{t}{u} = \frac{pu+qt}{qu} = \frac{v}{w}$ where $v = pu + qt$, $w = qu \neq 0$

Thus the sum is rational. ◄

Proving Conditional Statements: $p \rightarrow q$

- *Proof by Contraposition*: Assume $\neg q$ and show $\neg p$ is true also. This is sometimes called an *indirect proof* method. If we give a direct proof of $\neg q \rightarrow \neg p$ then we have a proof of $p \rightarrow q$.

Why does this work?

Example: Prove that if *n* is an integer and 3*n* + 2 is odd, then *n* is odd.

Solution*:* Assume *n* is even. So, *n* = 2*k* for some integer *k*. Thus

$3n + 2 = 3(2k) + 2 = 6k + 2 = 2(3k + 1) = 2j$ for $j = 3k + 1$

Therefore 3*n* + 2 is even. Since we have shown $\neg q \rightarrow \neg p$, $p \rightarrow q$ must hold as well. If *n* is an integer and 3*n* + 2 is odd (not even) , then *n* is odd (not even). ◄

Proving Conditional Statements: $p \rightarrow q$

Example: Prove that for an integer *n,* if n^2 is odd, then *n* is odd.

Solution: Use proof by contraposition. Assume *n* is even (i.e., not odd). Therefore, there exists an integer *k* such that *n* = 2*k*. Hence,

$n^2 = 4k^2 = 2(2k^2)$

and n^2 is even(i.e., not odd).

We have shown that if *n* is an even integer, then n^2 is even. Therefore by contraposition, for an integer *n,* if n^2 is odd, then *n* is odd. ◄

Proving Conditional Statements: $p \rightarrow q$

- *Proof by Contradiction*: (AKA *reductio ad absurdum***)**.

To prove p, assume $\neg p$ and derive a contradiction such as $p \wedge \neg p$. (an indirect form of proof). Since we have shown that $\neg p \rightarrow \mathbf{F}$ is true , it follows that the contrapositive $\mathbf{T} \rightarrow p$ also holds.

Example: Prove that if you pick 22 days from the calendar, at least 4 must fall on the same day of the week.

Solution: Assume that no more than 3 of the 22 days fall on the same day of the week. Because there are 7 days of the week, we could only have picked 21 days. This contradicts the assumption that we have picked 22 days.

◄

Proof by Contradiction

- A preview of Chapter 4.

Example: Use a proof by contradiction to give a proof that $\sqrt{2}$ is irrational.

Solution: Suppose $\sqrt{2}$ is rational. Then there exists integers *a* and *b* with $\sqrt{2} = a/b$, where $b \neq 0$ and *a* and *b* have no common factors (see Chapter 4). Then

$2 = \frac{a^2}{b^2}$ $\quad 2b^2 = a^2$

Therefore a^2 must be even. If a^2 is even then *a* must be even (an exercise). Since *a* is even, $a = 2c$ for some integer *c*. Thus,

$2b^2 = 4c^2$ $\quad b^2 = 2c^2$

Therefore b^2 is even. Again then *b* must be even as well.

But then 2 must divide both *a* and *b*. This contradicts our assumption that *a* and *b* have no common factors. We have proved by contradiction that our initial assumption must be false and therefore $\sqrt{2}$ is irrational .

◄

Proof by Contradiction

- A preview of Chapter 4.

Example: Prove that there is no largest prime number.

Solution: Assume that there is a largest prime number. Call it p_n. Hence, we can list all the primes 2,3,.., p_n. Form

$$r = p_1 \times p_2 \times \ldots \times p_n + 1$$

None of the prime numbers on the list divides *r*. Therefore, by a theorem in Chapter 4, either *r* is prime or there is a smaller prime that divides *r*. This contradicts the assumption that there is a largest prime. Therefore, there is no largest prime.

◄

Theorems that are Biconditional Statements

- To prove a theorem that is a biconditional statement, that is, a statement of the form $p \leftrightarrow q$, we show that $p \rightarrow q$ and $q \rightarrow p$ are both true.

Example: Prove the theorem: "If n is an integer, then n is odd if and only if n^2 is odd."

Solution: We have already shown (previous slides) that both $p \rightarrow q$ and $q \rightarrow p$. Therefore we can conclude $p \leftrightarrow q$.

Sometimes *iff* is used as an abbreviation for "if an only if," as in "If n is an integer, then n is odd iif n^2 is odd."

What is wrong with this?

"Proof" that *1 = 2*

Step	Reason
1. $a = b$	Premise
2. $a^2 = a \times b$	Multiply both sides of (1) by a
3. $a^2 - b^2 = a \times b - b^2$	Subtract b^2 from both sides of (2)
4. $(a - b)(a + b) = b(a - b)$	Algebra on (3)
5. $a + b = b$	Divide both sides by $a - b$
6. $2b = b$	Replace a by b in (5) because $a = b$
7. $2 = 1$	Divide both sides of (6) by b

Solution: Step 5. a - b = 0 by the premise and division by 0 is undefined.

Looking Ahead

- If direct methods of proof do not work:
  - We may need a clever use of a proof by contraposition.
  - Or a proof by contradiction.
  - In the next section, we will see strategies that can be used when straightforward approaches do not work.
  - In Chapter 5, we will see mathematical induction and related techniques.
  - In Chapter 6, we will see combinatorial proofs

Proof Methods and Strategy

Section 1.8

Section Summary

- Proof by Cases
- Existence Proofs
  - Constructive
  - Nonconstructive
- Disproof by Counterexample
- Nonexistence Proofs
- Uniqueness Proofs
- Proof Strategies
- Proving Universally Quantified Assertions
- Open Problems

Proof by Cases

- To prove a conditional statement of the form:

$$(p_1 \vee p_2 \vee \ldots \vee p_n) \rightarrow q$$

- Use the tautology

$$[(p_1 \vee p_2 \vee \ldots \vee p_n) \rightarrow q] \leftrightarrow [(p_1 \rightarrow q) \wedge (p_2 \rightarrow q) \wedge \ldots \wedge (p_n \rightarrow q)]$$

- Each of the implications $p_i \rightarrow q$ is a *case*.

Proof by Cases

Example: Let a @ b = max{a, b} = a if $a \geq b$, otherwise a @ b = max{a, b} = b.

Show that for all real numbers a, b, c

(a @b) @ c = a @ (b @ c)

(This means the operation @ is associative.)

Proof: Let a, b, and c be arbitrary real numbers.

Then one of the following 6 cases must hold.

1. $a \geq b \geq c$
2. $a \geq c \geq b$
3. $b \geq a \geq c$
4. $b \geq c \geq a$
5. $c \geq a \geq b$
6. $c \geq b \geq a$

Continued on next slide →

Proof by Cases

Case 1: a ≥ b ≥ c

(a @ b) = a, a @ c = a, b @ c = b

Hence (a @ b) @ c = a = a @ (b @ c)

Therefore the equality holds for the first case.

A complete proof requires that the equality be shown to hold for all 6 cases. But the proofs of the remaining cases are similar. Try them.

◄

Without Loss of Generality

Example: Show that if *x* and *y* are integers and both *x·y and x+y* are even, then both *x* and *y* are even.

Proof: Use a proof by contraposition. Suppose *x* and *y* are not both even. Then, one or both are odd. Without loss of generality, assume that *x* is odd. Then $x = 2m + 1$ for some integer *m*.

Case 1: *y* is even. Then $y = 2n$ for some integer *n*, so $x + y = (2m + 1) + 2n = 2(m + n) + 1$ is odd.

Case 2: *y* is odd. Then $y = 2n + 1$ for some integer *n*, so $x \cdot y = (2m + 1)(2n + 1) = 2(2m \cdot n + m + n) + 1$ is odd.

◄

We only cover the case where *x* is odd because the case where *y* is odd is similar. The use phrase *without loss of generality* (WLOG) indicates this.

Existence Proofs



Srinivasa Ramanujan
(1887-1920)

- Proof of theorems of the form $\exists x P(x)$.
- **Constructive** existence proof:
  - Find an explicit value of *c*, for which *P(c)* is true.
  - Then $\exists x P(x)$ is true by Existential Generalization (EG).

Example: Show that there is a positive integer that can be written as the sum of cubes of positive integers in two different ways:

Proof: 1729 is such a number since

$$1729 = 10^3 + 9^3 = 12^3 + 1^3$$

◄



Godfrey Harold Hardy
(1877-1947)

Nonconstructive Existence Proofs

- In a *nonconstructive* existence proof, we assume no *c* exists which makes *P(c)* true and derive a contradiction.

Example: Show that there exist irrational numbers *x* and *y* such that x^y is rational.

Proof: We know that $\sqrt{2}$ is irrational. Consider the number $\sqrt{2}^{\sqrt{2}}$. If it is rational, we have two irrational numbers x and y with x^y rational, namely $x = \sqrt{2}$ and $y = \sqrt{2}$. But if $\sqrt{2}^{\sqrt{2}}$ is irrational, then we can let $x = \sqrt{2}^{\sqrt{2}}$ and $y = \sqrt{2}$ so that $x^y = (\sqrt{2}^{\sqrt{2}})^{\sqrt{2}} = \sqrt{2}^{(\sqrt{2}\sqrt{2})} = \sqrt{2}^2 = 2$. ◄

Counterexamples

- Recall $\exists x \neg P(x) \equiv \neg \forall x P(x)$.
- To establish that $\neg \forall x P(x)$ is true (or $\forall x P(x)$ is false) find a *c* such that $\neg P(c)$ is true or $P(c)$ is false.
- In this case *c* is called a *counterexample* to the assertion $\forall x P(x)$.

Example: "Every positive integer is the sum of the squares of 3 integers." The integer 7 is a counterexample. So the claim is false.

Uniqueness Proofs

- Some theorems asset the existence of a unique element with a particular property, $\exists! x\, P(x)$. The two parts of a *uniqueness proof* are
  - *Existence*: We show that an element *x* with the property exists.
  - *Uniqueness*: We show that if $y \neq x$, then *y* does not have the property.

Example: Show that if *a* and *b* are real numbers and $a \neq 0$, then there is a unique real number r such that $ar + b = 0$.

Solution:

- Existence: The real number $r = -b/a$ is a solution of $ar + b = 0$ because $a(-b/a) + b = -b + b = 0$.
- Uniqueness: Suppose that *s* is a real number such that $as + b = 0$. Then $ar + b = as + b$, where $r = -b/a$. Subtracting *b* from both sides and dividing by *a* shows that $r = s$.

◄

Proof Strategies for proving $p \rightarrow q$

- Choose a method.
  1. First try a direct method of proof.
  2. If this does not work, try an indirect method (e.g., try to prove the contrapositive).
- For whichever method you are trying, choose a strategy.
  1. First try *forward reasoning*. Start with the axioms and known theorems and construct a sequence of steps that end in the conclusion. Start with *p* and prove *q*, or start with $\neg q$ and prove $\neg p$.
  2. If this doesn't work, try *backward reasoning*. When trying to prove *q*, find a statement p that we can prove with the property $p \rightarrow q$.

Backward Reasoning

Example: Suppose that two people play a game taking turns removing, 1, 2, or 3 stones at a time from a pile that begins with 15 stones. The person who removes the last stone wins the game. Show that the first player can win the game no matter what the second player does.

Proof: Let *n* be the last step of the game.

Step n: Player_1 can win if the pile contains 1,2, or 3 stones.

Step n-1: Player_2 will have to leave such a pile if the pile that he/she is faced with has 4 stones.

Step n-2: Player_1 can leave 4 stones when there are 5,6, or 7 stones left at the beginning of his/her turn.

Step n-3: Player_2 must leave such a pile, if there are 8 stones .

Step n-4: Player_1 has to have a pile with 9,10, or 11 stones to ensure that there are 8 left.

Step n-5: Player_2 needs to be faced with 12 stones to be forced to leave 9,10, or 11.

Step n-6: Player_1 can leave 12 stones by removing 3 stones.

Now reasoning forward, the first player can ensure a win by removing 3 stones and leaving 12.

Universally Quantified Assertions

- To prove theorems of the form $\forall x P(x)$ assume x is an arbitrary member of the domain and show that $P(x)$ must be true. Using UG it follows that $\forall x P(x)$

Example: An integer x is even if and only if x^2 is even.

Solution: The quantified assertion is

$\forall x$ [x is even $\leftrightarrow$ x^2 is even]

We assume x is arbitrary.

Recall that $p \leftrightarrow q$ is equivalent to $(p \rightarrow q) \wedge (q \rightarrow p)$

So, we have two cases to consider. These are considered in turn.

Continued on next slide →

Universally Quantified Assertions

Case 1. We show that if x is even then x^2 *is* even using a direct proof (the *only if* part or *necessity*).

If x is even then $x = 2k$ for some integer k.

Hence $x^2 = 4k^2 = 2(2k^2)$ which is even since it is an integer divisible by 2.

This completes the proof of case 1.

Case 2 on next slide →

Universally Quantified Assertions

Case 2. We show that if x^2 is even then x must be even (the *if* part or *sufficiency*). We use a proof by contraposition.

Assume x is not even and then show that x^2 is not even.

If x is not even then it must be odd. So, $x = 2k + 1$ for some k. Then $x^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$

which is odd and hence not even. This completes the proof of case 2.

Since x was arbitrary, the result follows by UG.

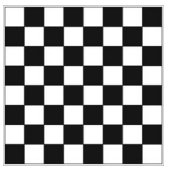
Therefore we have shown that x is even if and only if x^2 is even.

◄

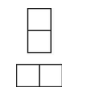
Proof and Disproof: Tilings

Example 1: Can we tile the standard checkerboard using dominos?

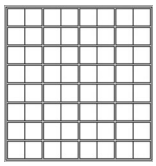
Solution: Yes! One example provides a constructive existence proof.



The Standard Checkerboard



Two Dominoes



One Possible Solution

Tilings

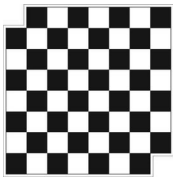
Example 2: Can we tile a checkerboard obtained by removing one of the four corner squares of a standard checkerboard?

Solution:

- Our checkerboard has $64 - 1 = 63$ squares.
- Since each domino has two squares, a board with a tiling must have an even number of squares.
- The number 63 is not even.
- We have a contradiction. ◄

Tilings

Example 3: Can we tile a board obtained by removing both the upper left and the lower right squares of a standard checkerboard?



Nonstandard Checkerboard



Dominoes

Continued on next slide →

Tilings

Solution:

- There are 62 squares in this board.
- To tile it we need 31 dominos.
- *Key fact*: Each domino covers one black and one white square.
- Therefore the tiling covers 31 black squares and 31 white squares.
- Our board has either 30 black squares and 32 white squares or 32 black squares and 30 white squares.
- Contradiction! ◄

The Role of Open Problems

- Unsolved problems have motivated much work in mathematics. Fermat's Last Theorem was conjectured more than 300 years ago. It has only recently been finally solved.

Fermat's Last Theorem: The equation $x^n + y^n = z^n$ has no solutions in integers x, y, and z, with $xyz \neq 0$ whenever n is an integer with $n > 2$.

A proof was found by Andrew Wiles in the 1990s.

An Open Problem

- **The $3x + 1$ Conjecture**: Let T be the transformation that sends an even integer x to $x/2$ and an odd integer x to $3x + 1$. For all positive integers x, when we repeatedly apply the transformation T, we will eventually reach the integer 1.

  For example, starting with $x = 13$:

  $T(13) = 3 \cdot 13 + 1 = 40, T(40) = 40/2 = 20, T(20) = 20/2 = 10,$
  $T(10) = 10/2 = 5, T(5) = 3 \cdot 5 + 1 = 16, T(16) = 16/2 = 8,$
  $T(8) = 8/2 = 4, T(4) = 4/2 = 2, T(2) = 2/2 = 1$

  The conjecture has been verified using computers up to $5.6 \cdot 10^{13}$.

Additional Proof Methods

- Later we will see many other proof methods:
  - Mathematical induction, which is a useful method for proving statements of the form $\forall n\, P(n)$, where the domain consists of all positive integers.
  - Structural induction, which can be used to prove such results about recursively defined sets.
  - Cantor diagonalization is used to prove results about the size of infinite sets.
  - Combinatorial proofs use counting arguments.

Basic Structures: Sets, Functions, Sequences, Sums, and Matrices

Chapter 2

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Sets
  - The Language of Sets
  - Set Operations
  - Set Identities
- Functions
  - Types of Functions
  - Operations on Functions
  - Computability
- Sequences and Summations
  - Types of Sequences
  - Summation Formulae
- Set Cardinality
  - Countable Sets
- Matrices
  - Matrix Arithmetic

Sets

Section 2.1

Section Summary

- Definition of sets
- Describing Sets
  - Roster Method
  - Set-Builder Notation
- Some Important Sets in Mathematics
- Empty Set and Universal Set
- Subsets and Set Equality
- Cardinality of Sets
- Tuples
- Cartesian Product

Introduction

- Sets are one of the basic building blocks for the types of objects considered in discrete mathematics.
  - Important for counting.
  - Programming languages have set operations.
- Set theory is an important branch of mathematics.
  - Many different systems of axioms have been used to develop set theory.
  - Here we are not concerned with a formal set of axioms for set theory. Instead, we will use what is called naïve set theory.

Sets

- A *set* is an unordered collection of objects.
  - the students in this class
  - the chairs in this room
- The objects in a set are called the *elements*, or *members* of the set. A set is said to *contain* its elements.
- The notation $a \in A$ denotes that a is an element of the set A.
- If a is not a member of A, write $a \notin A$

Describing a Set: Roster Method

- $S = \{a,b,c,d\}$
- Order not important
  $S = \{a,b,c,d\} = \{b,c,a,d\}$
- Each distinct object is either a member or not; listing more than once does not change the set.
  $S = \{a,b,c,d\} = \{a,b,c,b,c,d\}$
- Elipses (…) may be used to describe a set without listing all of the members when the pattern is clear.
  $S = \{a,b,c,d, \ldots\ldots,z\}$

Roster Method

- Set of all vowels in the English alphabet:
  $V = \{a,e,i,o,u\}$
- Set of all odd positive integers less than 10:
  $O = \{1,3,5,7,9\}$
- Set of all positive integers less than 100:
  $S = \{1,2,3,\ldots\ldots\ldots,99\}$
- Set of all integers less than 0:
  $S = \{\ldots., -3,-2,-1\}$

Some Important Sets

$\mathbf{N}$ = *natural numbers* = $\{0,1,2,3\ldots.\}$

$\mathbf{Z}$ = *integers* = $\{\ldots,-3,-2,-1,0,1,2,3,\ldots\}$

$\mathbf{Z^+}$ = *positive integers* = $\{1,2,3,\ldots..\}$

$\mathbf{R}$ = set of *real numbers*

$\mathbf{R^+}$ = set of *positive real numbers*

$\mathbf{C}$ = set of *complex numbers*.

$\mathbf{Q}$ = set of rational numbers

Set-Builder Notation

- Specify the property or properties that all members must satisfy:
  $S = \{x \mid x \text{ is a positive integer less than } 100\}$
  $O = \{x \mid x \text{ is an odd positive integer less than } 10\}$
  $O = \{x \in \mathbf{Z^+} \mid x \text{ is odd and } x < 10\}$
- A predicate may be used:
  $S = \{x \mid P(x)\}$
- Example: $S = \{x \mid \text{Prime}(x)\}$
- Positive rational numbers*:*
  $\mathbf{Q^+} = \{x \in \mathbf{R} \mid x = p/q, \text{ for some positive integers } p,q\}$

Interval Notation

$[a,b] = \{x \mid a \le x \le b\}$

$[a,b) = \{x \mid a \le x < b\}$

$(a,b] = \{x \mid a < x \le b\}$

$(a,b) = \{x \mid a < x < b\}$

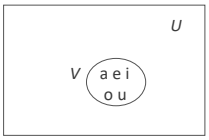
closed interval [a,b]

open interval (a,b)

Universal Set and Empty Set

- The *universal set U* is the set containing everything currently under consideration.
  - Sometimes implicit
  - Sometimes explicitly stated.
  - Contents depend on the context.
- The empty set is the set with no elements. Symbolized ∅, but {} also used.

Venn Diagram





John Venn (1834-1923)
Cambridge, UK

Russell's Paradox

- Let *S* be the set of all sets which are not members of themselves. A paradox results from trying to answer the question "Is *S* a member of itself?"
- Related Paradox:
  - Henry is a barber who shaves all people who do not shave themselves. A paradox results from trying to answer the question "Does Henry shave himself?"



Bertrand Russell (1872-1970)
Cambridge, UK
Nobel Prize Winner

Some things to remember

- Sets can be elements of sets.
  $\{\{1,2,3\},a,\{b,c\}\}$
  $\{\mathbf{N},\mathbf{Z},\mathbf{Q},\mathbf{R}\}$
- The empty set is different from a set containing the empty set.
  $\emptyset \neq \{\emptyset\}$

Set Equality

Definition: Two sets are *equal* if and only if they have the same elements.

- Therefore if A and B are sets, then A and B are equal if and only if $\forall x(x \in A \leftrightarrow x \in B)$
- We write $A = B$ if A and B are equal sets.

$$\{1,3,5\} = \{3,5,1\}$$

$$\{1,5,5,5,3,3,1\} = \{1,3,5\}$$

Subsets

Definition: The set A is a *subset* of B, if and only if every element of A is also an element of B.

- The notation $A \subseteq B$ is used to indicate that A is a subset of the set B.
- $A \subseteq B$ holds if and only if $\forall x(x \in A \rightarrow x \in B)$ is true.
  1. Because $a \in \emptyset$ is always false, $\emptyset \subseteq S$, for every set S.
  2. Because $a \in S \rightarrow a \in S$, $S \subseteq S$, for every set S.

Showing a Set is or is not a Subset of Another Set

- **Showing that A is a Subset of B**: To show that $A \subseteq B$, show that if x belongs to A, then x also belongs to B.
- **Showing that A is not a Subset of B**: To show that A is not a subset of B, $A \not\subseteq B$, find an element $x \in A$ with $x \notin B$. (Such an x is a counterexample to the claim that $x \in A$ implies $x \in B$.)

Examples:

1. The set of all computer science majors at your school is a subset of all students at your school.
2. The set of integers with squares less than 100 is not a subset of the set of nonnegative integers.

Another look at Equality of Sets

- Recall that two sets A and B are *equal*, denoted by $A = B$, iff

$$\forall x(x \in A \leftrightarrow x \in B)$$

- Using logical equivalences we have that $A = B$ iff

$$\forall x[(x \in A \rightarrow x \in B) \wedge (x \in B \rightarrow x \in A)]$$

- This is equivalent to

$A \subseteq B$ and $B \subseteq A$

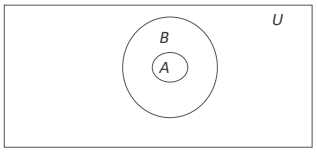
Proper Subsets

Definition: If $A \subseteq B$, but $A \neq B$, then we say A is a *proper subset* of B, denoted by $A \subset B$. If $A \subset B$, then

$$\forall x(x \in A \rightarrow x \in B) \wedge \exists x(x \in B \wedge x \notin A)$$

is true.

Venn Diagram



Set Cardinality

Definition: If there are exactly n distinct elements in S where n is a nonnegative integer, we say that S is *finite*. Otherwise it is *infinite*.

Definition: The *cardinality* of a finite set A, denoted by $|A|$, is the number of (distinct) elements of A.

Examples:

1. $|\emptyset| = 0$
2. Let S be the letters of the English alphabet. Then $|S| = 26$
3. $|\{1,2,3\}| = 3$
4. $|\{\emptyset\}| = 1$
5. The set of integers is infinite.

Power Sets

Definition: The set of all subsets of a set A, denoted $\mathcal{P}(A)$, is called the *power set* of A.

Example: If $A = \{a,b\}$ then

$\mathcal{P}(A) = \{\emptyset, \{a\},\{b\},\{a,b\}\}$

- If a set has n elements, then the cardinality of the power set is 2^n. (In Chapters 5 and 6, we will discuss different ways to show this.)

Tuples

- The *ordered n-tuple* $(a_1,a_2,\ldots,a_n)$ is the ordered collection that has a_1 as its first element and a_2 as its second element and so on until a_n as its last element.
- Two n-tuples are equal if and only if their corresponding elements are equal.
- 2-tuples are called *ordered pairs*.
- The ordered pairs (a,b) and (c,d) are equal if and only if $a = c$ and $b = d$.

Cartesian Product



René Descartes
(1596-1650)

Definition: The *Cartesian Product* of two sets *A* and *B*, denoted by $A \times B$ is the set of ordered pairs (a,b) where $a \in A$ and $b \in B$.

$$A \times B = \{(a, b)|a \in A \wedge b \in B\}$$

Example:

$A = \{a,b\}$ $B = \{1,2,3\}$

$A \times B = \{(a,1),(a,2),(a,3), (b,1),(b,2),(b,3)\}$

- **Definition**: A subset *R* of the Cartesian product $A \times B$ is called a *relation* from the set A to the set B. (Relations will be covered in depth in Chapter 9.)

Cartesian Product

Definition: The cartesian products of the sets $A_1,A_2,\ldots\ldots,A_n$, denoted by $A_1 \times A_2 \times \ldots\ldots \times A_n$, is the set of ordered n-tuples $(a_1,a_2,\ldots\ldots,a_n)$ where a_i belongs to A_i for $i = 1, \ldots n$.

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, a_2, \ldots, a_n)|a_i \in A_i \text{ for } i = 1, 2, \ldots n\}$$

Example: What is $A \times B \times C$ where $A = \{0,1\}$, $B = \{1,2\}$ and $C = \{0,1,2\}$

Solution: $A \times B \times C$ = {(0,1,0), (0,1,1), (0,1,2),(0,2,0), (0,2,1), (0,2,2),(1,1,0), (1,1,1), (1,1,2), (1,2,0), (1,2,1), (1,2,2)}

Truth Sets of Quantifiers

- Given a predicate *P* and a domain *D*, we define the *truth set* of *P* to be the set of elements in *D* for which *P*(*x*) is true. The truth set of *P*(x) is denoted by

$$\{x \in D|P(x)\}$$

- **Example**: The truth set of *P*(*x*) where the domain is the integers and *P*(*x*) is "$|x| = 1$" is the set {-1,1}

Set Operations

Section 2.2

Section Summary

- Set Operations
  - Union
  - Intersection
  - Complementation
  - Difference
- More on Set Cardinality
- Set Identities
- Proving Identities
- Membership Tables

Boolean Algebra

- Propositional calculus and set theory are both instances of an algebraic system called a *Boolean Algebra*. This is discussed in Chapter 12.
- The operators in set theory are analogous to the corresponding operator in propositional calculus.
- As always there must be a universal set *U*. All sets are assumed to be subsets of *U*.

Union

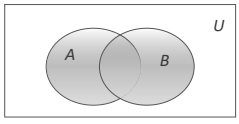
- **Definition**: Let *A* and *B* be sets. The *union* of the sets *A* and *B*, denoted by $A \cup B$, is the set:

$$\{x|x \in A \vee x \in B\}$$

- **Example**: What is $\{1,2,3\} \cup \{3, 4, 5\}$?

Solution: {1,2,3,4,5}

Venn Diagram for $A \cup B$



Intersection

- **Definition**: The *intersection* of sets *A* and *B*, denoted by $A \cap B$, is

$$\{x|x \in A \wedge x \in B\}$$

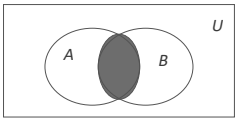
- Note if the intersection is empty, then *A* and *B* are said to be *disjoint*.
- **Example**: What is? $\{1,2,3\} \cap \{3,4,5\}$?

  Solution: $\{3\}$
- **Example:**What is?

  $\{1,2,3\} \cap \{4,5,6\}$?

  Solution: $\emptyset$

Venn Diagram for $A \cap B$



Complement

Definition: If *A* is a set, then the complement of the *A* (with respect to *U*), denoted by $\bar{A}$ is the set *U - A*

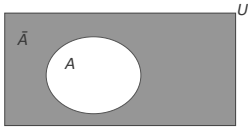
$$\bar{A} = \{x \in U \mid x \notin A\}$$

(The complement of A is sometimes denoted by A^c .)

Example: If *U* is the positive integers less than 100, what is the complement of $\{x \mid x > 70\}$

Solution: $\{x \mid x \leq 70\}$

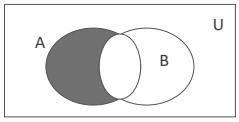
Venn Diagram for Complement



Difference

- **Definition**: Let *A* and *B* be sets. The *difference* of *A* and *B*, denoted by *A* – *B*, is the set containing the elements of *A* that are not in *B*. The difference of *A* and *B* is also called the complement of *B* with respect to *A*.

$$A - B = \{x \mid x \in A \wedge x \notin B\} = A \cap \overline{B}$$

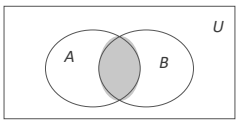


Venn Diagram for $A - B$

The Cardinality of the Union of Two Sets

- Inclusion-Exclusion

$|A \cup B| = |A| + |B| - |A \cap B|$



Venn Diagram for *A*, *B*, $A \cap B$, $A \cup B$

- **Example**: Let *A* be the math majors in your class and *B* be the CS majors. To count the number of students who are either math majors or CS majors, add the number of math majors and the number of CS majors, and subtract the number of joint CS/math majors.
- We will return to this principle in Chapter 6 and Chapter 8 where we will derive a formula for the cardinality of the union of *n* sets, where *n* is a positive integer.

Review Questions

Example: $U = \{0,1,2,3,4,5,6,7,8,9,10\}$ $A = \{1,2,3,4,5\}$, $B = \{4,5,6,7,8\}$

1. $A \cup B$
   Solution: $\{1,2,3,4,5,6,7,8\}$
2. $A \cap B$
   Solution: $\{4,5\}$
3. $\bar{A}$
   Solution: $\{0,6,7,8,9,10\}$
4. $\bar{B}$ **Solution:** $\{0,1,2,3,9,10\}$
5. $A - B$
   Solution: $\{1,2,3\}$
6. $B - A$
   Solution: $\{6,7,8\}$

Symmetric Difference (*optional*)

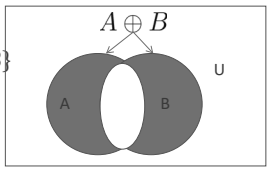
Definition: The *symmetric difference* of **A** and **B**, denoted by $A \oplus B$ is the set

$$(A - B) \cup (B - A)$$

Example:

$U = \{0,1,2,3,4,5,6,7,8,9,10\}$
$A = \{1,2,3,4,5\}$ $B = \{4,5,6,7,8\}$
What is $A \oplus B$:

- **Solution**: $\{1,2,3,6,7,8\}$



Venn Diagram

Set Identities

- Identity laws

$$A \cup \emptyset = A \qquad A \cap U = A$$

- Domination laws

$$A \cup U = U \qquad A \cap \emptyset = \emptyset$$

- Idempotent laws

$$A \cup A = A \qquad A \cap A = A$$

- Complementation law

$$\overline{(\overline{A})} = A$$

Continued on next slide →

Set Identities

- Commutative laws

$$A \cup B = B \cup A \qquad A \cap B = B \cap A$$

- Associative laws

$$A \cup (B \cup C) = (A \cup B) \cup C$$
$$A \cap (B \cap C) = (A \cap B) \cap C$$

- Distributive laws

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$
$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

Continued on next slide →

Set Identities

- De Morgan's laws

$$\overline{A \cup B} = \overline{A} \cap \overline{B} \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}$$

- Absorption laws

$$A \cup (A \cap B) = A \qquad A \cap (A \cup B) = A$$

- Complement laws

$$A \cup \overline{A} = U \qquad A \cap \overline{A} = \emptyset$$

Proving Set Identities

- Different ways to prove set identities:
  1. Prove that each set (side of the identity) is a subset of the other.
  2. Use set builder notation and propositional logic.
  3. Membership Tables: Verify that elements in the same combination of sets always either belong or do not belong to the same side of the identity. Use 1 to indicate it is in the set and a 0 to indicate that it is not.

Proof of Second De Morgan Law

Example: Prove that $\overline{A \cap B} = \overline{A} \cup \overline{B}$

Solution: We prove this identity by showing that:

1) $\overline{A \cap B} \subseteq \overline{A} \cup \overline{B}$ and

2) $\overline{A} \cup \overline{B} \subseteq \overline{A \cap B}$

Continued on next slide →

Proof of Second De Morgan Law

These steps show that: $\overline{A \cap B} \subseteq \overline{A} \cup \overline{B}$

$x \in \overline{A \cap B}$	by assumption
$x \notin A \cap B$	defn. of complement
$\neg((x \in A) \wedge (x \in B))$	defn. of intersection
$\neg(x \in A) \vee \neg(x \in B)$	1st De Morgan Law for Prop Logic
$x \notin A \vee x \notin B$	defn. of negation
$x \in \overline{A} \vee x \in \overline{B}$	defn. of complement
$x \in \overline{A} \cup \overline{B}$	defn. of union

Continued on next slide →

Proof of Second De Morgan Law

These steps show that: $\overline{A} \cup \overline{B} \subseteq \overline{A \cap B}$

$x \in \overline{A} \cup \overline{B}$	by assumption
$(x \in \overline{A}) \vee (x \in \overline{B})$	defn. of union
$(x \notin A) \vee (x \notin B)$	defn. of complement
$\neg(x \in A) \vee \neg(x \in B)$	defn. of negation
$\neg((x \in A) \wedge (x \in B))$	by 1st De Morgan Law for Prop Logic
$\neg(x \in A \cap B)$	defn. of intersection
$x \in \overline{A \cap B}$	defn. of complement

◄

Set-Builder Notation: Second De Morgan Law

$\overline{A \cap B}$	=	$\{x \mid x \notin A \cap B\}$	by defn. of complement
	=	$\{x \mid \neg(x \in (A \cap B))\}$	by defn. of does not belong symbol
	=	$\{x \mid \neg(x \in A \wedge x \in B\}$	by defn. of intersection
	=	$\{x \mid \neg(x \in A) \vee \neg(x \in B)\}$	by 1st De Morgan law for Prop Logic
	=	$\{x \mid x \notin A \vee x \notin B\}$	by defn. of not belong symbol
	=	$\{x \mid x \in \overline{A} \vee x \in \overline{B}\}$	by defn. of complement
	=	$\{x \mid x \in \overline{A} \cup \overline{B}\}$	by defn. of union
	=	$\overline{A} \cup \overline{B}$	by meaning of notation

◄

Membership Table

Example: Construct a membership table to show that the distributive law holds.

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

Solution:

A	B	C	$B \cap C$	$A \cup (B \cap C)$	$A \cup B$	$A \cup C$	$(A \cup B) \cap (A \cup C)$
1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	0	0	0	1	1	1	1
0	1	1	1	1	1	1	1
0	1	0	0	0	1	0	0
0	0	1	0	0	0	1	0
0	0	0	0	0	0	0	0

Generalized Unions and Intersections

- Let $A_1, A_2, \ldots, A_n$ be an indexed collection of sets.
  We define:

$$\bigcup_{i=1}^{n} A_i = A_1 \cup A_2 \cup \ldots \cup A_n$$

$$\bigcap_{i=1}^{n} A_i = A_1 \cap A_2 \cap \ldots \cap A_n$$

  These are well defined, since union and intersection are associative.
- For $i = 1,2,\ldots$, let $A_i = \{i, i+1, i+2, \ldots.\}$. Then,

$$\bigcup_{i=1}^{n} A_i = \bigcup_{i=1}^{n} \{i, i+1, i+2, \ldots\} = \{1, 2, 3, \ldots\}$$

$$\bigcap_{i=1}^{n} A_i = \bigcap_{i=1}^{n} \{i, i+1, i+2, \ldots\} = \{n, n+1, n+2, \ldots..\} = A_n$$

Functions

Section 2.3

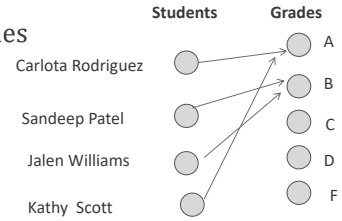
Section Summary

- Definition of a Function.
  - Domain, Codomain
  - Image, Preimage
- Injection, Surjection, Bijection
- Inverse Function
- Function Composition
- Graphing Functions
- Floor, Ceiling, Factorial
- Partial Functions (optional)

Functions

Definition: Let *A* and *B* be nonempty sets. A *function* f from *A* to *B*, denoted $f: A \to B$ is an assignment of each element of *A* to exactly one element of *B*. We write $f(a) = b$ if *b* is the unique element of *B* assigned by the function f to the element *a* of *A*.

- Functions are sometimes called *mappings* or *transformations*.



Functions

- A function $f: A \to B$ can also be defined as a subset of $A \times B$ (a relation). This subset is restricted to be a relation where no two elements of the relation have the same first element.
- Specifically, a function f from *A* to *B* contains one, and only one ordered pair (*a, b*) for every element *a*∈ *A*.

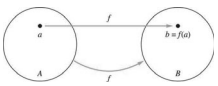
$$\forall x[x \in A \to \exists y[y \in B \wedge (x, y) \in f]]$$

and

$$\forall x, y_1, y_2[[(x, y_1) \in f \wedge (x, y_2) \in f] \to y_1 = y_2]$$

Functions

Given a function $f: A \to B$:



- We say *f maps A* to *B or* *f* is a *mapping* from *A* to *B*.
- *A* is called the *domain* of *f*.
- *B* is called the *codomain* of *f*.
- If $f(a) = b$,
  - then *b* is called the *image* of *a* under *f*.
  - *a* is called the *preimage* of *b.*
- The range of *f* is the set of all images of points in **A** under *f*. We denote it by $f(\boldsymbol{A})$.
- Two functions are *equal* when they have the same domain, the same codomain and map each element of the domain to the same element of the codomain.

Representing Functions

- Functions may be specified in different ways:
  - An explicit statement of the assignment.
    Students and grades example.
  - A formula.
    $f(x) = x + 1$
  - A computer program.
    - A Java program that when given an integer *n*, produces the *n*th Fibonacci Number (covered in the next section and also inChapter 5).

Questions

f(a) = ? z

The image of d is ? z

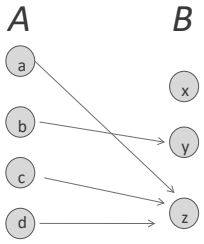
The domain of f is ? *A*

The codomain of f is ? *B*

The preimage of y is ? b

$f(A)$ = ? {y,z}

The preimage(s) of z is (are) ? {a,c,d}



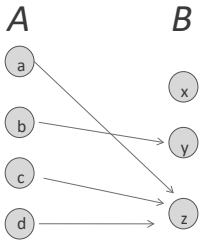
Question on Functions and Sets

- If $f : A \to B$ and S is a subset of A, then

$$f(S) = \{f(s) | s \in S\}$$

f {a,b,c,} is ? {y,z}

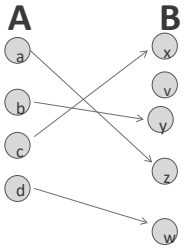
f {c,d} is ? {z}



Injections

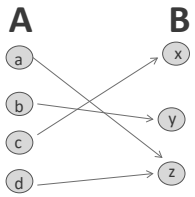
Definition: A function f is said to be *one-to-one* , or *injective*, if and only if $f(a) = f(b)$ implies that $a = b$ for all *a* and *b* in the domain of *f*. A function is said to be an *injection* if it is one-to-one.





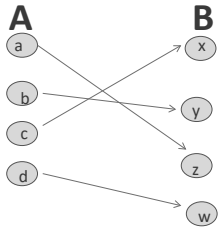
Surjections

Definition: A function *f* from *A* to *B* is called *onto* or *surjective*, if and only if for every element $b \in B$ there is an element $a \in A$ with $f(a) = b$. A function *f* is called a *surjection* if it is *onto*.



Bijections

Definition: A function f is a *one-to-one correspondence*, or a *bijection*, if it is both one-to-one and onto (surjective and injective).



Showing that *f* is one-to-one or onto

Suppose that $f : A \to B$.

To show that f is injective Show that if $f(x) = f(y)$ for arbitrary $x, y \in A$ with $x \neq y$, then $x = y$.

To show that f is not injective Find particular elements $x, y \in A$ such that $x \neq y$ and $f(x) = f(y)$.

To show that f is surjective Consider an arbitrary element $y \in B$ and find an element $x \in A$ such that $f(x) = y$.

To show that f is not surjective Find a particular $y \in B$ such that $f(x) \neq y$ for all $x \in A$.

Showing that *f* is one-to-one or onto

Example 1: Let *f* be the function from {*a,b,c,d*} to {1,2,3} defined by *f*(*a*) = 3, *f*(*b*) = 2, *f*(*c*) = 1, and *f*(*d*) = 3. Is *f* an onto function?

Solution: Yes, *f* is onto since all three elements of the codomain are images of elements in the domain. If the codomain were changed to {1,2,3,4}, *f* would not be onto.

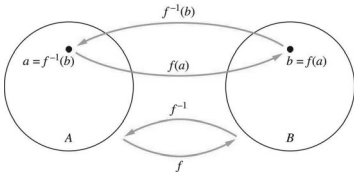
Example 2: Is the function $f(x) = x^2$ from the set of integers to the set of integers onto?

Solution: No, *f* is not onto because there is no integer *x* with $x^2 = -1$, for example.

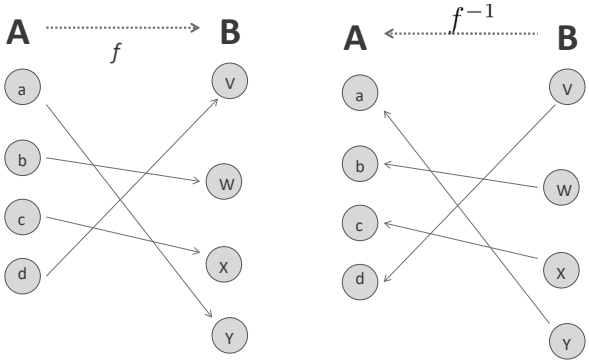
Inverse Functions

Definition: Let *f* be a bijection from *A* to *B*. Then the *inverse* of *f*, denoted f^{-1}, is the function from *B* to *A* defined as $f^{-1}(y) = x \text{ iff } f(x) = y$.

No inverse exists unless *f* is a bijection. Why?



Inverse Functions



Questions

Example 1: Let *f* be the function from {*a,b,c*} to {1,2,3} such that *f*(*a*) = 2, *f*(*b*) = 3, and *f*(*c*) = 1. Is *f* invertible and if so what is its inverse?

Solution: The function *f* is invertible because it is a one-to-one correspondence. The inverse function f^{-1} reverses the correspondence given by *f*, so $f^{-1}(1) = c$, $f^{-1}(2) = a$, and $f^{-1}(3) = b$.

Questions

Example 2: Let *f*: **Z** → **Z** be such that *f*(*x*) = *x* + 1. Is *f* invertible, and if so, what is its inverse?

Solution: The function *f* is invertible because it is a one-to-one correspondence. The inverse function f^{-1} reverses the correspondence so $f^{-1}(y) = y - 1$.

Questions

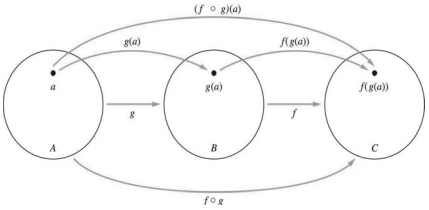
Example 3: Let *f:* **R** → **R** be such that $f(x) = x^2$. Is *f* invertible, and if so, what is its inverse?

Solution: The function *f* is not invertible because it is not one-to-one .

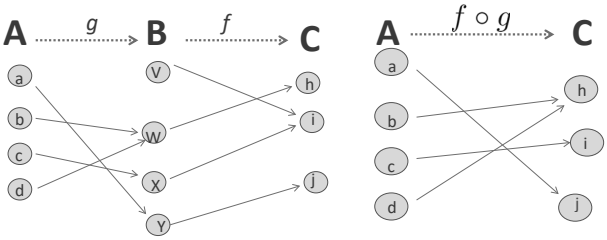
Composition

- **Definition**: Let *f*: *B* → *C*, *g*: *A* → *B*. The *composition of f with g*, denoted $f \circ g$ is the function from *A* to *C* defined by

$$f \circ g(x) = f(g(x))$$



Composition



Composition

Example 1: If $f(x) = x^2$ and $g(x) = 2x + 1$, then

$$f(g(x)) = (2x+1)^2$$

and

$$g(f(x)) = 2x^2 + 1$$

Composition Questions

Example 2: Let *g* be the function from the set {*a,b,c*} to itself such that *g*(*a*) = *b*, *g*(*b*) = *c*, and *g*(*c*) = *a*. Let *f* be the function from the set {*a,b,c*} to the set {1,2,3} such that *f*(*a*) = 3, *f*(*b*) = 2, and *f*(*c*) = 1.

What is the composition of *f* and *g*, and what is the composition of *g* and *f*.

Solution: The composition $f \circ g$ is defined by

$f \circ g\ (a) = f(g(a)) = f(b) = 2.$
$f \circ g\ (b) = f(g(b)) = f(c) = 1.$
$f \circ g\ (c) = f(g(c)) = f(a) = 3.$

Note that $g \circ f$ is not defined, because the range of *f* is not a subset of the domain of *g*.

Composition Questions

Example 2: Let f and g be functions from the set of integers to the set of integers defined by *f*(*x*) = 2*x* + 3 and *g*(*x*) = 3*x* + 2.

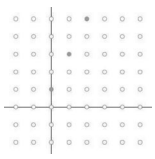
What is the composition of *f* and *g*, and also the composition of *g* and *f* ?

Solution:

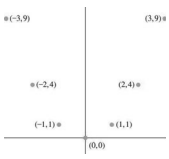
$f \circ g\ (x) = f(g(x)) = f(3x+2) = 2(3x+2) + 3 = 6x + 7$
$g \circ f\ (x) = g(f(x)) = g(2x+3) = 3(2x+3) + 2 = 6x + 11$

Graphs of Functions

- Let *f* be a function from the set *A* to the set *B*. The *graph* of the function *f* is the set of ordered pairs $\{(a,b) \mid a \in A \text{ and } f(a) = b\}$.



Graph of *f*(*n*) = 2*n* + 1 from Z to Z



Graph of *f*(*x*) = *x*^{2} from Z to Z

Some Important Functions

- The *floor* function, denoted

$$f(x) = \lfloor x \rfloor$$

is the largest integer less than or equal to *x*.

- The *ceiling* function, denoted

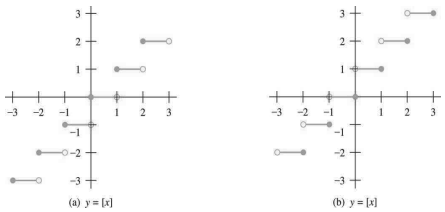
$$f(x) = \lceil x \rceil$$

is the smallest integer greater than or equal to *x*

Example: $\lceil 3.5 \rceil = 4 \qquad \lfloor 3.5 \rfloor = 3$

$\lceil -1.5 \rceil = -1 \quad \lfloor -1.5 \rfloor = -2$

Floor and Ceiling Functions



Graph of (a) Floor and (b) Ceiling Functions

Floor and Ceiling Functions

TABLE 1 Useful Properties of the Floor and Ceiling Functions.
(n is an integer, x is a real number)

(1a)	$\lfloor x \rfloor = n$ if and only if $n \le x < n+1$
(1b)	$\lceil x \rceil = n$ if and only if $n-1 < x \le n$
(1c)	$\lfloor x \rfloor = n$ if and only if $x-1 < n \le x$
(1d)	$\lceil x \rceil = n$ if and only if $x \le n < x+1$
(2)	$x-1 < \lfloor x \rfloor \le x \le \lceil x \rceil < x+1$
(3a)	$\lfloor -x \rfloor = -\lceil x \rceil$
(3b)	$\lceil -x \rceil = -\lfloor x \rfloor$
(4a)	$\lfloor x+n \rfloor = \lfloor x \rfloor + n$
(4b)	$\lceil x+n \rceil = \lceil x \rceil + n$

Proving Properties of Functions

Example: Prove that x is a real number, then

$$\lfloor 2x \rfloor = \lfloor x \rfloor + \lfloor x + 1/2 \rfloor$$

Solution: Let $x = n + \varepsilon$, where n is an integer and $0 \le \varepsilon < 1$.

Case 1: $\varepsilon < ½$

- $2x = 2n + 2\varepsilon$ and $\lfloor 2x \rfloor = 2n$, since $0 \le 2\varepsilon < 1$.
- $\lfloor x + 1/2 \rfloor = n$, since $x + ½ = n + (1/2 + \varepsilon)$ and $0 \le ½ + \varepsilon < 1$.
- Hence, $\lfloor 2x \rfloor = 2n$ and $\lfloor x \rfloor + \lfloor x + 1/2 \rfloor = n + n = 2n$.

Case 2: $\varepsilon \ge ½$

- $2x = 2n + 2\varepsilon = (2n+1) + (2\varepsilon - 1)$ and $\lfloor 2x \rfloor = 2n + 1$, since $0 \le 2\varepsilon - 1 < 1$.
- $\lfloor x + 1/2 \rfloor = \lfloor n + (1/2 + \varepsilon) \rfloor = \lfloor n + 1 + (\varepsilon - 1/2) \rfloor = n + 1$ since $0 \le \varepsilon - 1/2 < 1$.
- Hence, $\lfloor 2x \rfloor = 2n + 1$ and $\lfloor x \rfloor + \lfloor x + 1/2 \rfloor = n + (n+1) = 2n + 1$.

◀

Factorial Function

Definition: f: **N** → **Z⁺** , denoted by $f(n) = n!$ is the product of the first n positive integers when n is a nonnegative integer.

$f(n) = 1 \cdot 2 \cdots (n-1) \cdot n, \qquad f(0) = 0! = 1$

Examples:

$f(1) = 1! = 1$

$f(2) = 2! = 1 \cdot 2 = 2$

$f(6) = 6! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 = 720$

$f(20) = 2{,}432{,}902{,}008{,}176{,}640{,}000.$

Stirling's Formula:

$n! \sim \sqrt{2\pi n}(n/e)^n$

$f(n) \sim g(n) \doteq lim_{n\to\infty} f(n)/g(n) = 1$

Partial Functions (*optional*)

Definition: A *partial function* f from a set A to a set B is an assignment to each element a in a subset of A, called the *domain of definition* of f, of a unique element b in B.

- The sets A and B are called the *domain* and *codomain* of f, respectively.
- We day that f is *undefined* for elements in A that are not in the domain of definition of f.
- When the domain of definition of f equals A, we say that f is a *total function*.

Example: f: **N** → **R** where $f(n) = \sqrt{n}$ is a partial function from **Z** to **R** where the domain of definition is the set of nonnegative integers. Note that f is undefined for negative integers.

Sequences and Summations

Section 2.4

Section Summary

- Sequences.
  - Examples: Geometric Progression, Arithmetic Progression
- Recurrence Relations
  - Example: Fibonacci Sequence
- Summations
- Special Integer Sequences (*optional*)

Introduction

- Sequences are ordered lists of elements.
  - 1, 2, 3, 5, 8
  - 1, 3, 9, 27, 81, …….
- Sequences arise throughout mathematics, computer science, and in many other disciplines, ranging from botany to music.
- We will introduce the terminology to represent sequences and sums of the terms in the sequences.

Sequences

Definition: A *sequence* is a function from a subset of the integers (usually either the set {0, 1, 2, 3, 4, …..} or {1, 2, 3, 4, ….}) to a set *S*.

- The notation a_n is used to denote the image of the integer *n*. We can think of a_n as the equivalent of *f(n)* where *f* is a function from {0,1,2,…..} to *S*. We call a_n a *term* of the sequence.

Sequences

Example: Consider the sequence $\{a_n\}$ where

$$a_n = \frac{1}{n} \qquad \{a_n\} = \{a_1, a_2, a_3, \ldots\}$$

$$1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4} \ldots$$

Geometric Progression

Definition: A *geometric progression* is a sequence of the form: $a, ar, ar^2, \ldots, ar^n, \ldots$

where the *initial term a* and the *common ratio r* are real numbers.

Examples:

1. Let $a = 1$ and $r = -1$. Then:
   $\{b_n\} = \{b_0, b_1, b_2, b_3, b_4, \ldots\} = \{1, -1, 1, -1, 1, \ldots\}$
2. Let $a = 2$ and $r = 5$. Then:
   $\{c_n\} = \{c_0, c_1, c_2, c_3, c_4, \ldots\} = \{2, 10, 50, 250, 1250, \ldots\}$
3. Let $a = 6$ and $r = 1/3$. Then:
   $\{d_n\} = \{d_0, d_1, d_2, d_3, d_4, \ldots\} = \{6, 2, \frac{2}{3}, \frac{2}{9}, \frac{2}{27}, \ldots\}$

Arithmetic Progression

Definition: A *arithmetic progression* is a sequence of the form: $a, a + d, a + 2d, \ldots, a + nd, \ldots$

where the *initial term a* and the *common difference d* are real numbers.

Examples:

1. Let $a = -1$ and $d = 4$:
   $\{s_n\} = \{s_0, s_1, s_2, s_3, s_4, \ldots\} = \{-1, 3, 7, 11, 15, \ldots\}$
2. Let $a = 7$ and $d = -3$:
   $\{t_n\} = \{t_0, t_1, t_2, t_3, t_4, \ldots\} = \{7, 4, 1, -2, -5, \ldots\}$
3. Let $a = 1$ and d = 2:
   $\{u_n\} = \{u_0, u_1, u_2, u_3, u_4, \ldots\} = \{1, 3, 5, 7, 9, \ldots\}$

Fibonacci Sequence

Definition: Define the *Fibonacci sequence*, $f_0, f_1, f_2, \ldots$, by:

- Initial Conditions: $f_0 = 0$, $f_1 = 1$
- Recurrence Relation: $f_n = f_{n-1} + f_{n-2}$

Example: Find f_2, f_3, f_4, f_5 and f_6 .

Answer:

$f_2 = f_1 + f_0 = 1 + 0 = 1,$
$f_3 = f_2 + f_1 = 1 + 1 = 2,$
$f_4 = f_3 + f_2 = 2 + 1 = 3,$
$f_5 = f_4 + f_3 = 3 + 2 = 5,$
$f_6 = f_5 + f_4 = 5 + 3 = 8.$

Useful Sequences

TABLE 1 Some Useful Sequences.

nth Term	*First 10 Terms*
n^2	1, 4, 9, 16, 25, 36, 49, 64, 81, 100, …
n^3	1, 8, 27, 64, 125, 216, 343, 512, 729, 1000, …
n^4	1, 16, 81, 256, 625, 1296, 2401, 4096, 6561, 10000, …
2^n	2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, …
3^n	3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049, …
$n!$	1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800, …
f_n	1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

Summations

- Sum of the terms $a_m, a_{m+1}, \ldots, a_n$ from the sequence $\{a_n\}$
- The notation:

$$\sum_{j=m}^{n} a_j \qquad \sum_{j=m}^{n} a_j \qquad \sum_{m \le j \le n} a_j$$

represents $a_m + a_{m+1} + \cdots + a_n$

- The variable *j* is called the *index of summation*. It runs through all the integers starting with its *lower limit m* and ending with its *upper limit n*.

Summations

- More generally for a set *S*:

$$\sum_{j \in S} a_j$$

- **Examples**:

$$r^0 + r^1 + r^2 + r^3 + \cdots + r^n = \sum_{0}^{n} r^j$$

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots = \sum_{1}^{\infty} \frac{1}{i}$$

$$\text{If } S = \{2, 5, 7, 10\} \text{ then } \sum_{j \in S} a_j = a_2 + a_5 + a_7 + a_{10}$$

Product Notation (*optional*)

- Product of the terms $a_m, a_{m+1}, \ldots, a_n$ from the sequence $\{a_n\}$
- The notation:

$$\prod_{j=m}^{n} a_j \qquad \prod_{j=m}^{n} a_j \qquad \prod_{m \le j \le n} a_j$$

represents $a_m \times a_{m+1} \times \cdots \times a_n$

Geometric Series

Sums of terms of geometric progressions

$$\sum_{j=0}^{n} ar^j = \begin{cases} \frac{ar^{n+1}-a}{r-1} & r \neq 1 \\ (n+1)a & r = 1 \end{cases}$$

Proof: Let $S_n = \sum_{j=0}^{n} ar^j$ To compute S_n, first multiply both sides of the equality by r and then manipulate the resulting sum as follows:

$$rS_n = r\sum_{j=0}^{n} ar^j$$

$$= \sum_{j=0}^{n} ar^{j+1}$$

Continued on next slide →

Geometric Series

$= \sum_{j=0}^{n} ar^{j+1}$ From previous slide.

$= \sum_{k=1}^{n+1} ar^k$ Shifting the index of summation with $k = j + 1$.

$= \left(\sum_{k=0}^{n} ar^k\right) + (ar^{n+1} - a)$ Removing $k = n + 1$ term and adding $k = 0$ term.

$= S_n + (ar^{n+1} - a)$ Substituting S for summation formula

∴ $rS_n = S_n + (ar^{n+1} - a)$

$S_n = \frac{ar^{n+1} - a}{r - 1}$ if r ≠1

$S_n = \sum_{j=0}^{n} ar^j = \sum_{j=0}^{n} a = (n+1)a$ if r = 1

Some Useful Summation Formulae

TABLE 2 Some Useful Summation Formulae.

Sum	*Closed Form*
$\sum_{k=0}^{n} ar^k \ (r \neq 0)$	$\frac{ar^{n+1}-a}{r-1}, r \neq 1$
$\sum_{k=1}^{n} k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^{n} k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^{n} k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{\infty} x^k, \lvert x\rvert < 1$	$\frac{1}{1-x}$
$\sum_{k=1}^{\infty} kx^{k-1}, \lvert x\rvert < 1$	$\frac{1}{(1-x)^2}$

Geometric Series: We just proved this.

Later we will prove some of these by induction.

Proof in text (requires calculus)

Section Summary

- Cardinality
- Countable Sets
- Computability

Cardinality of Sets

Section 2.5

Cardinality

Definition: The *cardinality* of a set *A* is equal to the cardinality of a set *B*, denoted

$|A| = |B|$,

if and only if there is a one-to-one correspondence (*i.e.*, a bijection) from *A* to *B*.

- If there is a one-to-one function (*i.e.*, an injection) from *A* to *B*, the cardinality of *A* is less than or the same as the cardinality of *B* and we write $|A| \le |B|$.
- When $|A| \le |B|$ and *A* and *B* have different cardinality, we say that the cardinality of A is less than the cardinality of *B* and write $|A| < |B|$.

Cardinality

- **Definition**: A set that is either finite or has the same cardinality as the set of positive integers ($\mathbf{Z}^+$) is called *countable*. A set that is not countable is *uncountable*.
- The set of real numbers **R** is an uncountable set.
- When an infinite set is countable (*countably infinite*) its cardinality is $\aleph_0$ (where $\aleph$ is aleph, the 1st letter of the Hebrew alphabet). We write $|S| = \aleph_0$ and say that *S* has cardinality "aleph null."

Showing that a Set is Countable

- An infinite set is countable if and only if it is possible to list the elements of the set in a sequence (indexed by the positive integers).
- The reason for this is that a one-to-one correspondence f from the set of positive integers to a set S can be expressed in terms of a sequence $a_1, a_2, \ldots, a_n, \ldots$ where $a_1 = f(1)$, $a_2 = f(2), \ldots, a_n = f(n), \ldots$

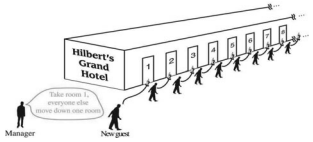
Hilbert's Grand Hotel



David Hilbert

The Grand Hotel (example due to David Hilbert) has countably infinite number of rooms, each occupied by a guest. We can always accommodate a new guest at this hotel. How is this possible?

Explanation: Because the rooms of Grand Hotel are countable, we can list them as Room 1, Room 2, Room 3, and so on. When a new guest arrives, we move the guest in Room 1 to Room 2, the guest in Room 2 to Room 3, and in general the guest in Room n to Room $n + 1$, for all positive integers n. This frees up Room 1, which we assign to the new guest, and all the current guests still have rooms.



The hotel can also accommodate a countable number of new guests, all the guests on a countable number of buses where each bus contains a countable number of guests (see exercises).

Showing that a Set is Countable

Example 1: Show that the set of positive even integers E is countable set.

Solution: Let $f(x) = 2x$.

$$\begin{array}{cccccccc} 1 & 2 & 3 & 4 & 5 & 6 & \ldots\ldots \\ \updownarrow & \updownarrow & \updownarrow & \updownarrow & \updownarrow & \updownarrow & \\ 2 & 4 & 6 & 8 & 10 & 12 & \ldots\ldots \end{array}$$

Then f is a bijection from **N** to E since f is both one-to-one and onto. To show that it is one-to-one, suppose that $f(n) = f(m)$. Then $2n = 2m$, and so $n = m$. To see that it is onto, suppose that t is an even positive integer. Then $t = 2k$ for some positive integer k and $f(k) = t$. ◄

Showing that a Set is Countable

Example 2: Show that the set of integers **Z** is countable.

Solution: Can list in a sequence:

$0, 1, -1, 2, -2, 3, -3, \ldots\ldots\ldots$

Or can define a bijection from **N** to **Z**:

- When n is even: $f(n) = n/2$
- When n is odd: $f(n) = -(n-1)/2$

◄

The Positive Rational Numbers are Countable

- **Definition**: A *rational number* can be expressed as the ratio of two integers p and q such that $q \neq 0$.
  - ¾ is a rational number
  - $\sqrt{2}$ is not a rational number.

Example 3: Show that the positive rational numbers are countable.

Solution:The positive rational numbers are countable since they can be arranged in a sequence:

$r_1, r_2, r_3, \ldots$

The next slide shows how this is done. →

The Positive Rational Numbers are Countable

First row $q = 1$.
Second row $q = 2$.
etc.

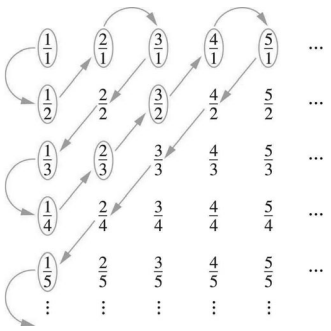
Constructing the List

First list p/q with $p + q = 2$.
Next list p/q with $p + q = 3$

And so on.

Terms not circled are not listed because they repeat previously listed terms

1, ½, 2, 3, 1/3,1/4, 2/3, ...



◄

Strings

Example 4: Show that the set of finite strings S over a finite alphabet A is countably infinite.

Assume an alphabetical ordering of symbols in A

Solution: Show that the strings can be listed in a sequence. First list

1. All the strings of length 0 in alphabetical order.
2. Then all the strings of length 1 in lexicographic (as in a dictionary) order.
3. Then all the strings of length 2 in lexicographic order.
4. And so on.

This implies a bijection from **N** to S and hence it is a countably infinite set.

◄

The set of all Java programs is countable.

Example 5: Show that the set of all Java programs is countable.

Solution: Let S be the set of strings constructed from the characters which can appear in a Java program. Use the ordering from the previous example. Take each string in turn:

- Feed the string into a Java compiler. (A Java compiler will determine if the input program is a syntactically correct Java program.)
- If the compiler says YES, this is a syntactically correct Java program, we add the program to the list.
- We move on to the next string.

In this way we construct an implied bijection from **N** to the set of Java programs. Hence, the set of Java programs is countable. ◄

The Real Numbers are Uncountable



Georg Cantor (1845-1918)

Example: Show that the set of real numbers is uncountable.

Solution: The method is called the Cantor diagnalization argument, and is a proof by contradiction.

1. Suppose **R** is countable. Then the real numbers between 0 and 1 are also countable (any subset of a countable set is countable - an exercise in the text).
2. The real numbers between 0 and 1 can be listed in order $r_1, r_2, r_3, \ldots$.
3. Let the decimal representation of this listing be

$$r_1 = 0.d_{11}d_{12}d_{13}d_{14}d_{15}d_{16}\ldots$$
$$r_2 = 0.d_{21}d_{22}d_{23}d_{24}d_{25}d_{26}\ldots$$
$$r_3 = 0.d_{31}d_{32}d_{33}d_{34}d_{35}d_{36}\ldots$$
$$\vdots$$

4. Form a new real number with the decimal expansion $r = .r_1r_2r_3r_4\ldots$ where $r_i = 3$ if $d_{ii} \neq 3$ and $r_i = 4$ if $d_{ii} = 3$
5. r is not equal to any of the $r_1, r_2, r_3, \ldots$ Because it differs from r_i in its ith position after the decimal point. Therefore there is a real number between 0 and 1 that is not on the list since every real number has a unique decimal expansion. Hence, all the real numbers between 0 and 1 cannot be listed, so the set of real numbers between 0 and 1 is uncountable.
6. Since a set with an uncountable subset is uncountable (an exercise), the set of real numbers is uncountable.

◄

Computability (Optional)

- **Definition**: We say that a function is **computable** if there is a computer program in some programming language that finds the values of this function. If a function is not computable we say it is **uncomputable**.
- There are uncomputable functions. We have shown that the set of Java programs is countable. Exercise 38 in the text shows that there are uncountably many different functions from a particular countably infinite set (i.e., the positive integers) to itself. Therefore (Exercise 39) there must be uncomputable functions.

Matrices

Section 2.6

Section Summary

- Definition of a Matrix
- Matrix Arithmetic
- Transposes and Powers of Arithmetic
- Zero-One matrices

Matrices

- Matrices are useful discrete structures that can be used in many ways. For example, they are used to:
  - describe certain types of functions known as linear transformations.
  - Express which vertices of a graph are connected by edges (see Chapter 10).
- In later chapters, we will see matrices used to build models of:
  - Transportation systems.
  - Communication networks.
- Algorithms based on matrix models will be presented in later chapters.
- Here we cover the aspect of matrix arithmetic that will be needed later.

Matrix

Definition: A *matrix* is a rectangular array of numbers. A matrix with m rows and n columns is called an $m \times n$ matrix.

- The plural of matrix is *matrices*.
- A matrix with the same number of rows as columns is called *square*.
- Two matrices are *equal* if they have the same number of rows and the same number of columns and the corresponding entries in every position are equal.

$$3 \times 2 \text{ matrix} \qquad \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 1 & 3 \end{bmatrix}$$

Notation

- Let m and n be positive integers and let

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ . & . & & . \\ . & . & & . \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix}$$

- The ith row of **A** is the $1 \times n$ matrix $[a_{i1}, a_{i2}, \ldots, a_{in}]$. The jth column of **A** is the $m \times 1$ *matrix:*

$$\begin{bmatrix} a_{1j} \\ a_{2j} \\ . \\ . \\ a_{mj} \end{bmatrix}$$

- The (i,j)th *element* or *entry* of **A** is the element a_{ij}. We can use $\mathbf{A} = [a_{ij}]$ to denote the matrix with its (i,j)th element equal to a_{ij}.

Matrix Arithmetic: Addition

Defintion: Let $\mathbf{A} = [a_{ij}]$ and $\mathbf{B} = [b_{ij}]$ be $m \times n$ matrices. The sum of **A** and **B**, denoted by **A** + **B**, is the $m \times n$ matrix that has $a_{ij} + b_{ij}$ as its (i,j)th element. In other words, $\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}]$.

Example:

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 2 & -3 \\ 3 & 4 & 0 \end{bmatrix} + \begin{bmatrix} 3 & 4 & -1 \\ 1 & -3 & 0 \\ -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 & -2 \\ 3 & -1 & -3 \\ 2 & 5 & 2 \end{bmatrix}$$

Note that matrices of different sizes can not be added.

Matrix Multiplication

Definition: Let **A** be an $m \times k$ matrix and **B** be a $k \times n$ matrix. The *product* of **A** and **B**, denoted by **AB**, is the $m \times n$ matrix that has its (i,j)th element equal to the sum of the products of the corresponding elements from the ith row of **A** and the jth column of **B**. In other words, if **AB** = $[c_{ij}]$ then $c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \ldots + a_{kj}b_{2j}$.

Example:

$$\begin{bmatrix} 1 & 0 & 4 \\ 2 & 1 & 1 \\ 3 & 1 & 0 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 1 & 1 \\ 3 & 0 \end{bmatrix} = \begin{bmatrix} 14 & 4 \\ 8 & 9 \\ 7 & 13 \\ 8 & 2 \end{bmatrix}$$

The product of two matrices is undefined when the number of columns in the first matrix is not the same as the number of rows in the second.

Illustration of Matrix Multiplication

- The Product of **A** = $[a_{ij}]$ and **B** = $[b_{ij}]$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1k} \\ a_{21} & a_{22} & \ldots & a_{2k} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ a_{i1} & a_{i2} & \ldots & a_{ik} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & & \cdot \\ a_{m1} & a_{m2} & \ldots & a_{mk} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \ldots & b_{1j} & \ldots & b_{1n} \\ b_{21} & b_{22} & \ldots & b_{2j} & \ldots & b_{2n} \\ \cdot & \cdot & & \cdot & & \\ \cdot & \cdot & & \cdot & & \\ b_{k1} & b_{k2} & \ldots & b_{kj} & \ldots & b_{kn} \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} c_{11} & c_{12} & \ldots & c_{1n} \\ c_{21} & c_{22} & \ldots & c_{2n} \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & c_{ij} & \cdot \\ \cdot & \cdot & & \cdot \\ c_{m1} & c_{m2} & \ldots & c_{mn} \end{bmatrix}$$

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{ik}b_{kj}$$

Matrix Multiplication is not Commutative

Example: Let

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

Does **AB** = **BA**?

Solution:

$$\mathbf{AB} = \begin{bmatrix} 2 & 2 \\ 5 & 3 \end{bmatrix} \qquad \mathbf{BA} = \begin{bmatrix} 4 & 3 \\ 3 & 2 \end{bmatrix}$$

AB $\neq$ **BA**

Identity Matrix and Powers of Matrices

Definition: The *identity matrix of order n* is the $n \times n$ matrix $\mathbf{I}_n = [\delta_{ij}]$, where $\delta_{ij} = 1$ if $i = j$ and $\delta_{ij} = 0$ if $i \neq j$.

$$\mathbf{I}_n = \begin{bmatrix} 1 & 0 & \ldots & 0 \\ 0 & 1 & \ldots & 0 \\ \cdot & \cdot & & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & & \cdot \\ 0 & 0 & \ldots & 1 \end{bmatrix}$$

$\mathbf{AI}_n = \mathbf{I}_m\mathbf{A} = \mathbf{A}$ when **A** is an $m \times n$ matrix

Powers of square matrices can be defined. When A is an $n \times n$ matrix, we have:

$\mathbf{A}^0 = \mathbf{I}_n$ $\quad \mathbf{A}^r = \underbrace{\mathbf{AAA}\cdots\mathbf{A}}_{\text{r times}}$

Transposes of Matrices

Definition: Let **A** = $[a_{ij}]$ be an $m \times n$ matrix. The *transpose* of **A**, denoted by $\mathbf{A}^t$, is the $n \times m$ matrix obtained by interchanging the rows and columns of **A**.

If $\mathbf{A}^t = [b_{ij}]$, then $b_{ij} = a_{ji}$ for $i = 1,2,\ldots,n$ and $j = 1,2,\ldots,m$.

The transpose of the matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ is the matrix $\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$.

Transposes of Matrices

Definition: A square matrix **A** is called symmetric if $\mathbf{A} = \mathbf{A}^t$. Thus $\mathbf{A} = [a_{ij}]$ is symmetric if $a_{ij} = a_{ji}$ for i and j with $1 \leq i \leq n$ and $1 \leq j \leq n$.

The matrix $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ is square.

Square matrices do not change when their rows and columns are interchanged.

Zero-One Matrices

Definition: A matrix all of whose entries are either 0 or 1 is called a *zero-one matrix*. (These will be used in Chapters 9 and 10.)

Algorithms operating on discrete structures represented by zero-one matrices are based on Boolean arithmetic defined by the following Boolean operations:

$$b_1 \wedge b_2 = \begin{cases} 1 & \text{if } b_1 = b_2 = 1 \\ 0 & \text{otherwise} \end{cases} \qquad b_1 \vee b_2 = \begin{cases} 1 & \text{if } b_1 = 1 \text{ or } b_2 = 1 \\ 0 & \text{otherwise} \end{cases}$$

Zero-One Matrices

Definition: Let **A** = $[a_{ij}]$ and **B** = $[b_{ij}]$ be an $m \times n$ zero-one matrices.

- The *join* of **A** and **B** is the zero-one matrix with (i,j)th entry $a_{ij} \vee b_{ij}$. The *join* of **A** and **B** is denoted by $\mathbf{A} \vee \mathbf{B}$.
- The meet of of **A** and **B** is the zero-one matrix with (i,j)th entry $a_{ij} \wedge b_{ij}$. The *meet* of **A** and **B** is denoted by $\mathbf{A} \wedge \mathbf{B}$.

Joins and Meets of Zero-One Matrices

Example: Find the join and meet of the zero-one matrices

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

Solution: The join of **A** and **B** is

$$\mathbf{A} \vee \mathbf{B} = \begin{bmatrix} 1 \vee 0 & 0 \vee 1 & 1 \vee 0 \\ 0 \vee 1 & 1 \vee 1 & 0 \vee 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

The meet of **A** and **B** is

$$\mathbf{A} \wedge \mathbf{B} = \begin{bmatrix} 1 \wedge 0 & 0 \wedge 1 & 1 \wedge 0 \\ 0 \wedge 1 & 1 \wedge 1 & 0 \wedge 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Boolean Product of Zero-One Matrices

Definition: Let $\mathbf{A} = [a_{ij}]$ be an $m \times k$ zero-one matrix and $\mathbf{B} = [b_{ij}]$ be a $k \times n$ zero-one matrix. The *Boolean product* of **A** and **B**, denoted by $\mathbf{A} \odot \mathbf{B}$, is the $m \times n$ zero-one matrix with (i,j)th entry

$$c_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \ldots \vee (a_{ik} \wedge b_{kj}).$$

Example: Find the Boolean product of **A** and **B**, where

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathbf{B} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

Continued on next slide →

Boolean Product of Zero-One Matrices

Solution: The Boolean product $\mathbf{A} \odot \mathbf{B}$ is given by

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \\ (0 \wedge 1) \vee (1 \wedge 0) & (0 \wedge 1) \vee (1 \wedge 1) & (0 \wedge 0) \vee (1 \wedge 1) \\ (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \end{bmatrix}$$

$$= \begin{bmatrix} 1 \vee 0 & 1 \vee 0 & 0 \vee 0 \\ 0 \vee 0 & 0 \vee 1 & 0 \vee 1 \\ 1 \vee 0 & 1 \vee 0 & 0 \vee 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

Boolean Powers of Zero-One Matrices

Definition: Let **A** be a square zero-one matrix and let r be a positive integer. The rth Boolean power of **A** is the Boolean product of r factors of **A**, denoted by $\mathbf{A}^{[r]}$. Hence,

$$\mathbf{A}^{[r]} = \underbrace{\mathbf{A} \odot \mathbf{A} \odot \ldots \odot \mathbf{A}}_{r \text{ times}}.$$

We define $\mathbf{A}^{[r]}$ to be $\mathbf{I}_n$.

(The Boolean product is well defined because the Boolean product of matrices is associative.)

Boolean Powers of Zero-One Matrices

Example: Let $\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}$.

Find $\mathbf{A}^n$ for all positive integers n.

Solution:

$$\mathbf{A}^{[2]} = \mathbf{A} \odot \mathbf{A} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \qquad \mathbf{A}^{[3]} = \mathbf{A}^{[2]} \odot \mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{A}^{[4]} = \mathbf{A}^{[3]} \odot \mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{A}^{[5]} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \mathbf{A}^{[n]} = \mathbf{A}^5 \text{ for all positive integers } n \text{ with } n \geq 5.$$

Algorithms

Chapter 3

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Algorithms
  - Example Algorithms
  - Algorithmic Paradigms
- Growth of Functions
  - Big-*O* and other Notation
- Complexity of Algorithms

Algorithms

Section 3.1

Section Summary

- Properties of Algorithms
- Algorithms for Searching and Sorting
- Greedy Algorithms
- Halting Problem

Problems and Algorithms

- In many domains there are key general problems that ask for output with specific properties when given valid input.
- The first step is to precisely state the problem, using the appropriate structures to specify the input and the desired output.
- We then solve the general problem by specifying the steps of a procedure that takes a valid input and produces the desired output. This procedure is called an *algorithm*.

Algorithms



Abu Ja'far Mohammed Ibin Musa Al-Khowarizmi (780-850)

Definition: An *algorithm* is a finite set of precise instructions for performing a computation or for solving a problem.

Example: Describe an algorithm for finding the maximum value in a finite sequence of integers.

Solution: Perform the following steps:

1. Set the temporary maximum equal to the first integer in the sequence.
2. Compare the next integer in the sequence to the temporary maximum.
   - If it is larger than the temporary maximum, set the temporary maximum equal to this integer.
3. Repeat the previous step if there are more integers. If not, stop.
4. When the algorithm terminates, the temporary maximum is the largest integer in the sequence.

Specifying Algorithms

- Algorithms can be specified in different ways. Their steps can be described in English or in *pseudocode.*
- Pseudocode is an intermediate step between an English language description of the steps and a coding of these steps using a programming language.
- The form of pseudocode we use is specified in Appendix 3. It uses some of the structures found in popular languages such as C++ and Java.
- Programmers can use the description of an algorithm in pseudocode to construct a program in a particular language.
- Pseudocode helps us analyze the time required to solve a problem using an algorithm, independent of the actual programming language used to implement algorithm.

Properties of Algorithms

- *Input*: An algorithm has input values from a specified set.
- *Output*: From the input values, the algorithm produces the output values from a specified set. The output values are the solution.
- *Correctness*: An algorithm should produce the correct output values for each set of input values.
- *Finiteness*: An algorithm should produce the output after a finite number of steps for any input.
- *Effectiveness*: It must be possible to perform each step of the algorithm correctly and in a finite amount of time.
- *Generality*: The algorithm should work for all problems of the desired form.

Finding the Maximum Element in a Finite Sequence

- The algorithm in pseudocode:

```
procedure max(a1, a2, …., an: integers)
   max := a1
   for i := 2 to n
      if max < ai then max := ai
   return max{max is the largest element}
```

- Does this algorithm have all the properties listed on the previous slide?

Some Example Algorithm Problems

- Three classes of problems will be studied in this section.
  1. *Searching Problems*: finding the position of a particular element in a list.
  2. *Sorting problems*: putting the elements of a list into increasing order.
  3. *Optimization Problems*: determining the optimal value (maximum or minimum) of a particular quantity over all possible inputs.

Searching Problems

Definition: The general *searching problem* is to locate an element x in the list of distinct elements $a_1, a_2, \ldots, a_n$, or determine that it is not in the list.

- The solution to a searching problem is the location of the term in the list that equals x (that is, i is the solution if $x = a_i$) or 0 if x is not in the list.
- For example, a library might want to check to see if a patron is on a list of those with overdue books before allowing him/her to checkout another book.
- We will study two different searching algorithms; linear search and binary search.

Linear Search Algorithm

- The linear search algorithm locates an item in a list by examining elements in the sequence one at a time, starting at the beginning.
  - First compare x with a_1. If they are equal, return the position 1.
  - If not, try a_2. If $x = a_2$, return the position 2.
  - Keep going, and if no match is found when the entire list is scanned, return 0.

```
procedure linear search(x:integer,
          a1, a2, …,an: distinct integers)
i := 1
while (i ≤ n and x ≠ ai)
    i := i + 1
if i ≤ n then location := i
else location := 0
return location{location is the subscript of the term that equals
   x, or is 0 if x is not found}
```

Binary Search

- Assume the input is a list of items in increasing order.
- The algorithm begins by comparing the element to be found with the middle element.
  - If the middle element is lower, the search proceeds with the upper half of the list.
  - If it is not lower, the search proceeds with the lower half of the list (through the middle position).
- Repeat this process until we have a list of size 1.
  - If the element we are looking for is equal to the element in the list, the position is returned.
  - Otherwise, 0 is returned to indicate that the element was not found.
- In Section 3.3, we show that the binary search algorithm is much more efficient than linear search.

Binary Search

- Here is a description of the binary search algorithm in pseudocode.

```
procedure binary search(x: integer, a1,a2,…, an: increasing integers)
 i := 1 {i is the left endpoint of interval}
 j := n {j is right endpoint of interval}
 while i < j
     m := ⌊(i + j)/2⌋
     if x > am then i := m + 1
     else j := m
 if x = ai then location := i
 else location := 0
 return location{location is the subscript i of the term ai equal to x,
                 or 0 if x is not found}
```

Binary Search

Example: The steps taken by a binary search for 19 in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

1. The list has 16 elements, so the midpoint is 8. The value in the 8th position is 10. Since 19 > 10, further search is restricted to positions 9 through 16.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

2. The midpoint of the list (positions 9 through 16) is now the 12th position with a value of 16. Since 19 > 16, further search is restricted to the 13th position and above.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

3. The midpoint of the current list is now the 14th position with a value of 19. Since 19 ≥ 19, further search is restricted to the portion from the 13th through the 14th positions.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

4. The midpoint of the current list is now the 13th position with a value of 18. Since 19 > 18, search is restricted to the portion from the 14th position through the 14th.

   1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

5. Now the list has a single element and the loop ends. Since 19=19, the location 14 is returned.

Sorting

- To *sort* the elements of a list is to put them in increasing order (numerical order, alphabetic, and so on).
- Sorting is an important problem because:
  - A nontrivial percentage of all computing resources are devoted to sorting different kinds of lists, especially applications involving large databases of information that need to be presented in a particular order (e.g., by customer, part number etc.).
  - An amazing number of fundamentally different algorithms have been invented for sorting. Their relative advantages and disadvantages have been studied extensively.
  - Sorting algorithms are useful to illustrate the basic notions of computer science.
- A variety of sorting algorithms are studied in this book; binary, insertion, bubble, selection, merge, quick, and tournament.
- In Section 3.3, we'll study the amount of time required to sort a list using the sorting algorithms covered in this section.

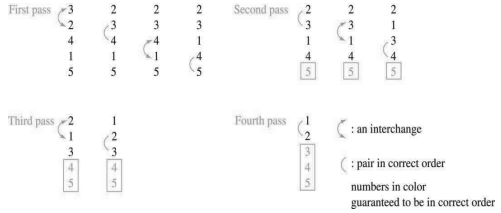
Bubble Sort

- *Bubble sort* makes multiple passes through a list. Every pair of elements that are found to be out of order are interchanged.

```
procedure bubblesort(a1,…,an: real numbers
                     with n ≥ 2)
  for i := 1 to n − 1
    for j := 1 to n − i
      if aj >aj+1 then interchange aj and aj+1
{a1,…, an is now in increasing order}
```

Bubble Sort

Example: Show the steps of bubble sort with 3 2 4 1 5



- At the first pass the largest element has been put into the correct position
- At the end of the second pass, the 2nd largest element has been put into the correct position.
- In each subsequent pass, an additional element is put in the correct position.

Insertion Sort

- *Insertion sort* begins with the 2nd element. It compares the 2nd element with the 1st and puts it before the first if it is not larger.

•Next the 3rd element is put into the correct position among the first 3 elements.
•In each subsequent pass, the n+1st element is put into its correct position among the first n+1 elements.
•Linear search is used to find the correct position.

```
procedure insertion sort
 (a1,…,an:
    real numbers with n ≥ 2)
   for j := 2 to n
     i := 1
     while aj > ai
        i := i + 1
     m := aj
     for k := 0 to j − i − 1
        aj-k := aj-k-1
     ai := m
{Now a1,…,an is in increasing order}
```

Insertion Sort

Example: Show all the steps of insertion sort with the input: 3 2 4 1 5

i. 2 3 4 1 5 (*first two positions are interchanged*)

ii. 2 3 4 1 5 (*third element remains in its position*)

iii. 1 2 3 4 5 (*fourth is placed at beginning*)

iv. 1 2 3 4 5 (*fifth element remains in its position*)

Greedy Algorithms



- *Optimization problems* minimize or maximize some parameter over all possible inputs.
- Among the many optimization problems we will study are:
  - Finding a route between two cities with the smallest total mileage.
  - Determining how to encode messages using the fewest possible bits.
  - Finding the fiber links between network nodes using the least amount of fiber.
- Optimization problems can often be solved using a *greedy algorithm*, which makes the "best" choice at each step. Making the "best choice" at each step does not necessarily produce an optimal solution to the overall problem, but in many instances, it does.
- After specifying what the "best choice" at each step is, we try to prove that this approach always produces an optimal solution, or find a counterexample to show that it does not.
- The greedy approach to solving problems is an example of an algorithmic paradigm, which is a general approach for designing an algorithm. We return to algorithmic paradigms in Section 3.3.

Greedy Algorithms: Making Change



Example: Design a greedy algorithm for making change (in U.S. money) of n cents with the following coins: quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cent) , using the least total number of coins.

Idea: At each step choose the coin with the largest possible value that does not exceed the amount of change left.

1. If $n = 67$ cents, first choose a quarter leaving $67-25 = 42$ cents. Then choose another quarter leaving $42 - 25 = 17$ cents
2. Then choose 1 dime, leaving $17 - 10 = 7$ cents.
3. Choose 1 nickel, leaving $7 - 5 = 2$ cents.
4. Choose a penny, leaving one cent. Choose another penny leaving 0 cents.

Greedy Change-Making Algorithm

Solution: Greedy change-making algorithm for n cents. The algorithm works with any coin denominations $c_1, c_2, \ldots, c_r$.

procedure *change*($c_1, c_2, \ldots, c_r$: values of coins, where $c_1 > c_2 > \ldots > c_r$; n: a positive integer)
for $i := 1$ to r
  $d_i := 0$ [d_i counts the coins of denomination c_i]
  while $n \geq c_i$
    $d_i := d_i + 1$ [add a coin of denomination c_i]
    $n = n - c_i$
[d_i counts the coins c_i]

- For the example of U.S. currency, we may have quarters, dimes, nickels and pennies, with $c_1 = 25$, $c_2 = 10$, $c_3 = 5$, and $c_4 = 1$.

Proving Optimality for U.S. Coins

- Show that the change making algorithm for *U.S.* coins is optimal.

Lemma 1: If n is a positive integer, then n cents in change using quarters, dimes, nickels, and pennies, using the fewest coins possible has at most 2 dimes, 1 nickel, 4 pennies, and cannot have 2 dimes and a nickel. The total amount of change in dimes, nickels, and pennies must not exceed 24 cents.

Proof: By contradiction

- If we had 3 dimes, we could replace them with a quarter and a nickel.
- If we had 2 nickels, we could replace them with 1 dime.
- If we had 5 pennies, we could replace them with a nickel.
- If we had 2 dimes and 1 nickel, we could replace them with a quarter.
- The allowable combinations, have a maximum value of 24 cents; 2 dimes and 4 pennies.

Proving Optimality for U.S. Coins

Theorem: The greedy change-making algorithm for U.S. coins produces change using the fewest coins possible.

Proof: By contradiction.

1. Assume there is a positive integer n such that change can be made for n cents using quarters, dimes, nickels, and pennies, with a fewer total number of coins than given by the algorithm.
2. Then, $q' \leq q$ where q' is the number of quarters used in this optimal way and q is the number of quarters in the greedy algorithm's solution. But this is not possible by Lemma 1, since the value of the coins other than quarters can not be greater than 24 cents.
3. Similarly, by Lemma 1, the two algorithms must have the same number of dimes, nickels, and quarters.

Greedy Change-Making Algorithm

- Optimality depends on the denominations available.
- For U.S. coins, optimality still holds if we add half dollar coins (50 cents) and dollar coins (100 cents).
- But if we allow only quarters (25 cents), dimes (10 cents), and pennies (1 cent), the algorithm no longer produces the minimum number of coins.
  - Consider the example of 31 cents. The optimal number of coins is 4, i.e., 3 dimes and 1 penny. What does the algorithm output?

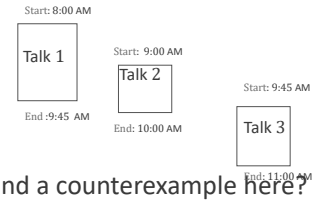
Greedy Scheduling

Example: We have a group of proposed talks with start and end times. Construct a greedy algorithm to schedule as many as possible in a lecture hall, under the following assumptions:

- When a talk starts, it continues till the end.
- No two talks can occur at the same time.
- A talk can begin at the same time that another ends.
- Once we have selected some of the talks, we cannot add a talk which is incompatible with those already selected because it overlaps at least one of these previously selected talks.
- How should we make the "best choice" at each step of the algorithm? That is, which talk do we pick ?
  - The talk that starts earliest among those compatible with already chosen talks?
  - The talk that is shortest among those already compatible?
  - The talk that ends earliest among those compatible with already chosen talks?

Greedy Scheduling

- Picking the shortest talk doesn't work.



- Can you find a counterexample here?
- But picking the one that ends soonest does work. The algorithm is specified on the next page.

Greedy Scheduling algorithm

Solution: At each step, choose the talks with the earliest ending time among the talks compatible with those selected.

procedure *schedule*($s_1 \le s_2 \le \ldots \le s_n$: start times, $e_1 \le e_2 \le \ldots \le e_n$: end times)
sort talks by finish time and reorder so that $e_1 \le e_2 \le \ldots \le e_n$
$S := \emptyset$
for $j := 1$ to n
  if talk j is compatible with S then
    S := S $\emptyset$U{talk j}
return S [S is the set of talks scheduled]

- Will be proven correct by induction in Chapter 5.

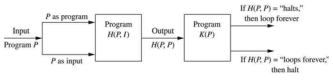
Halting Problem

Example: Can we develop a procedure that takes as input a computer program along with its input and determines whether the program will eventually halt with that input.

- **Solution**: Proof by contradiction.
- Assume that there is such a procedure and call it $H(P,I)$. The procedure $H(P,I)$ takes as input a program P and the input I to P.
  - H outputs "halt" if it is the case that P will stop when run with input I.
  - Otherwise, H outputs "loops forever."

Halting Problem

- Since a program is a string of characters, we can call $H(P,P)$. Construct a procedure $K(P)$, which works as follows.
  - If $H(P,P)$ outputs "loops forever" then $K(P)$ halts.
  - If $H(P,P)$ outputs "halt" then $K(P)$ goes into an infinite loop printing "ha" on each iteration.



Halting Problem

- Now we call K with K as input, i.e. $K(K)$.
  - If the output of $H(K,K)$ is "loops forever" then $K(K)$ halts. A Contradiction.
  - If the output of $H(K,K)$ is "halts" then $K(K)$ loops forever. A Contradiction.
- Therefore, there can not be a procedure that can decide whether or not an arbitrary program halts. The halting problem is unsolvable.

The Growth of Functions

Section 3.2

Section Summary



Donald E. Knuth
(Born 1938)

- Big-O Notation
- Big-O Estimates for Important Functions
- Big-Omega and Big-Theta Notation



Edmund Landau
(1877-1938)



Paul Gustav Heinrich Bachmann
(1837-1920)

The Growth of Functions

- In both computer science and in mathematics, there are many times when we care about how fast a function grows.
- In computer science, we want to understand how quickly an algorithm can solve a problem as the size of the input grows.
  - We can compare the efficiency of two different algorithms for solving the same problem.
  - We can also determine whether it is practical to use a particular algorithm as the input grows.
  - We'll study these questions in Section 3.3.
- Two of the areas of mathematics where questions about the growth of functions are studied are:
  - number theory (covered in Chapter 4)
  - combinatorics (covered in Chapters 6 and 8)

Big-*O* Notation

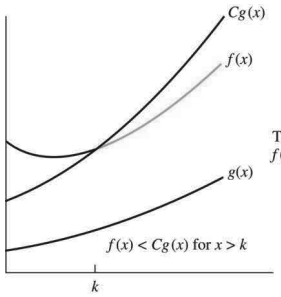
Definition: Let *f* and *g* be functions from the set of integers or the set of real numbers to the set of real numbers. We say that *f*(*x*) is *O*(*g*(*x*)) if there are constants *C* and *k* such that

$$|f(x)| \leq C|g(x)|$$

whenever $x > k$. (illustration on next slide)

- This is read as "*f*(*x*) is big-*O* of *g*(*x*)" or "*g* asymptotically dominates *f*."
- The constants C and k are called *witnesses* to the relationship *f*(*x*) is *O*(*g*(*x*)). Only one pair of witnesses is needed.

Illustration of Big-*O* Notation



f(*x*) is *O*(*g*(*x*))

The part of the graph of $f(x)$ that satisfies $f(x) < Cg(x)$ is shown in color.

Some Important Points about Big-*O* Notation

- If one pair of witnesses is found, then there are infinitely many pairs. We can always make the *k* or the *C* larger and still maintain the inequality $|f(x)| \leq C|g(x)|$.
  - Any pair C' and k' where $C < C'$ and $k < k'$ is also a pair of witnesses since $|f(x)| \leq C|g(x)| \leq C'|g(x)|$ whenever $x > k' > k$.

You may see "*f*(*x*) = *O*(*g*(*x*))" instead of "*f*(*x*) is *O*(*g*(*x*))."

- But this is an abuse of the equals sign since the meaning is that there is an inequality relating the values of *f* and *g*, for sufficiently large values of x.
- It is ok to write *f*(*x*) ∈ *O*(*g*(*x*)), because *O*(*g*(*x*)) represents the set of functions that are *O*(*g*(*x*)).

- Usually, we will drop the absolute value sign since we will always deal with functions that take on positive values.

Using the Definition of Big-*O* Notation

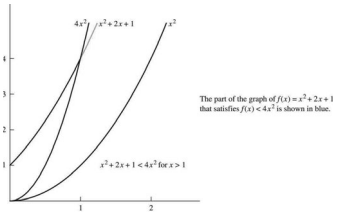
Example: Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.
Solution: Since when $x > 1$, $x < x^2$ and $1 < x^2$

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

- Can take *C* = 4 and *k* = 1 as witnesses to show that $f(x)$ is $O(x^2)$. (see graph on next slide)
- Alternatively, when $x > 2$, we have $2x \leq x^2$ and $1 < x^2$. Hence, when $x > 2$, $0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$
  - Can take *C* = 3 and *k* = 2 as witnesses instead.

Illustration of Big-*O* Notation

$f(x) = x^2 + 2x + 1$ is $O(x^2)$



The part of the graph of $f(x) = x^2 + 2x + 1$ that satisfies $f(x) < 4x^2$ is shown in blue.

Big-*O* Notation

- Both $f(x) = x^2 + 2x + 1$ and $g(x) = x^2$ are such that $f(x)$ is $O(g(x))$ and $g(x)$ is $O(f(x))$.
  We say that the two functions are of the *same order*. (More on this later)
- If $f(x)$ is $O(g(x))$ and *h(x)* is larger than *g(x)* for all positive real numbers, then $f(x)$ is $O(h(x))$.
- Note that if $|f(x)| \leq C|g(x)|$ for $x > k$ and if $|h(x)| > |g(x)|$ for all *x*, then $|f(x)| \leq C|h(x)|$ if $x > k$. Hence, $f(x)$ is $O(h(x))$
- For many applications, the goal is to select the function *g*(*x*) in *O(g(x))* as small as possible (up to multiplication by a constant, of course).

Using the Definition of Big-*O* Notation

Example: Show that $7x^2$ is $O(x^3)$.

Solution: When $x > 7$, $7x^2 < x^3$. Take $C = 1$ and $k = 7$ as witnesses to establish that $7x^2$ is $O(x^3)$.

(Would $C = 7$ and $k = 1$ work?)

Example: Show that n^2 is not $O(n)$.

Solution: Suppose there are constants *C* and *k* for which $n^2 \leq Cn$, whenever $n > k$. Then (by dividing both sides of $n^2 \leq Cn$) by *n*, then $n \leq C$ must hold for all $n > k$. A contradiction!

Big-*O* Estimates for Polynomials

Example: Let $f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_o$ where $a_0, a_1, \ldots, a_n$ are real numbers with $a_n \neq 0$. Then *f*(*x*) is $O(x^n)$.

Proof: $|f(x)| = |a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0|$ (Uses triangle inequality, an exercise in Section 1.8.)

$\leq |a_n|x^n + |a_{n-1}| x^{n-1} + \cdots + |a_1|x^1 + |a_0|$

Assuming $x > 1$ $= x^n (|a_n| + |a_{n-1}| /x + \cdots + |a_1|/x^{n-1} + |a_0|/ x^n)$

$\leq x^n (|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|)$

- Take $C = |a_n| + |a_{n-1}| + \cdots + |a_0|$ and $k = 1$. Then *f*(*x*) is $O(x^n)$.
- The leading term $a_n x^n$ of a polynomial dominates its growth.

Big-*O* Estimates for some Important Functions

Example: Use big-*O* notation to estimate the sum of the first *n* positive integers.

Solution: $1 + 2 + \cdots + n \leq n + n + \cdots n = n^2$

$1 + 2 + \ldots + n$ is $O(n^2)$ taking $C = 1$ and $k = 1$.

Example: Use big-*O* notation to estimate the factorial function $f(n) = n! = 1 \times 2 \times \cdots \times n$.

Solution:

$$n! = 1 \times 2 \times \cdots \times n \leq n \times n \times \cdots \times n = n^n$$

$n!$ is $O(n^n)$ taking $C = 1$ and $k = 1$.

Continued →

Big-*O* Estimates for some Important Functions

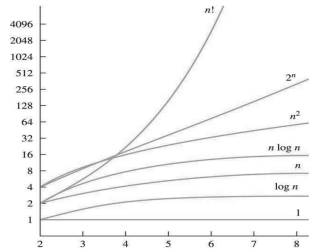
Example: Use big-*O* notation to estimate log *n*!

Solution: Given that $n! \leq n^n$ (previous slide)

then $\log(n!) \leq n \cdot \log(n)$.

Hence, log(*n*!) is $O(n \cdot \log(n))$ taking $C = 1$ and $k = 1$.

Display of Growth of Functions



Note the difference in behavior of functions as *n* gets larger

Useful Big-*O* Estimates Involving Logarithms, Powers, and Exponents

- If $d > c > 1$, then
  n^c is $O(n^d)$, but n^d is not $O(n^c)$.
- If $b > 1$ and c and d are positive, then
  $(\log_b n)^c$ is $O(n^d)$, but n^d is not $O((\log_b n)^c)$.
- If $b > 1$ and d is positive, then
  n^d is $O(b^n)$, but b^n is not $O(n^d)$.
- If $c > b > 1$, then
  b^n is $O(c^n)$, but c^n is not $O(b^n)$.

Combinations of Functions

- If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then
  $(f_1 + f_2)(x)$ is $O(\max(|g_1(x)|, |g_2(x)|))$.
  - See next slide for proof
- If $f_1(x)$ and $f_2(x)$ are both $O(g(x))$ then
  $(f_1 + f_2)(x)$ is $O(g(x))$.
  - See text for argument
- If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then
  $(f_1 f_2)(x)$ is $O(g_1(x)g_2(x))$.
  - See text for argument

Combinations of Functions

- If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then
  $(f_1 + f_2)(x)$ is $O(\max(|g_1(x)|, |g_2(x)|))$.
  - By the definition of big-*O* notation, there are constants C_1, C_2, k_1, k_2 such that $|f_1(x)| \leq C_1|g_1(x)|$ when $x > k_1$ and $|f_2(x)| \leq C_2|g_2(x)|$ when $x > k_2$.
  - $|(f_1 + f_2)(x)| = |f_1(x) + f_2(x)|$
    $\leq |f_1(x)| + |f_2(x)|$ by the triangle inequality $|a + b| \leq |a| + |b|$
  - $|f_1(x)| + |f_2(x)| \leq C_1|g_1(x)| + C_2|g_2(x)|$
    $\leq C_1|g(x)| + C_2|g(x)|$ where $g(x) = \max(|g_1(x)|, |g_2(x)|)$
    $= (C_1 + C_2)|g(x)|$
    $= C|g(x)|$ where $C = C_1 + C_2$
  - Therefore $|(f_1 + f_2)(x)| \leq C|g(x)|$ whenever $x > k$, where $k = \max(k_1, k_2)$.

Ordering Functions by Order of Growth

- Put the functions below in order so that each function is big-O of the next function on the list.
- $f_1(n) = (1.5)^n$
- $f_2(n) = 8n^3 + 17n^2 + 111$
- $f_3(n) = (\log n)^2$
- $f_4(n) = 2^n$
- $f_5(n) = \log(\log n)$
- $f_6(n) = n^2 (\log n)^3$
- $f_7(n) = 2^n (n^2 + 1)$
- $f_8(n) = n^3 + n(\log n)^2$
- $f_9(n) = 10000$
- $f_{10}(n) = n!$

We solve this exercise by successively finding the function that grows slowest among all those left on the list.

- $f_9(n) = 10000$ (constant, does not increase with n)
- $f_5(n) = \log(\log n)$ (grows slowest of all the others)
- $f_3(n) = (\log n)^2$ (grows next slowest)
- $f_6(n) = n^2 (\log n)^3$ (next largest, $(\log n)^3$ factor smaller than any power of n)
- $f_2(n) = 8n^3 + 17n^2 + 111$ (tied with the one below)
- $f_8(n) = n^3 + n(\log n)^2$ (tied with the one above)
- $f_1(n) = (1.5)^n$ (next largest, an exponential function)
- $f_4(n) = 2^n$ (grows faster than one above since $2 > 1.5$)
- $f_7(n) = 2^n (n^2 + 1)$ (grows faster than above because of the $n^2 + 1$ factor)
- $f_{10}(n) = n!$ ($n!$ grows faster than c^n for every c)

Big-Omega Notation

Definition: Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that
$f(x)$ is $\Omega(g(x))$
if there are constants C and k such that
$|f(x)| \geq C|g(x)|$ when $x > k$.

- We say that "$f(x)$ is big-Omega of $g(x)$."

Ω is the upper case version of the lower case Greek letter ω.

- Big-*O* gives an upper bound on the growth of a function, while Big-Omega gives a lower bound. Big-Omega tells us that a function grows at least as fast as another.
- $f(x)$ is $\Omega(g(x))$ if and only if $g(x)$ is $O(f(x))$. This follows from the definitions. See the text for details.

Big-Omega Notation

Example: Show that $f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(g(x))$ where $g(x) = x^3$.

Solution: $f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3$ for all positive real numbers *x*.

- Is it also the case that $g(x) = x^3$ is $O(8x^3 + 5x^2 + 7)$?

Big-Theta Notation

Θ is the upper case version of the lower case Greek letter θ.

- **Definition**: Let *f* and *g* be functions from the set of integers or the set of real numbers to the set of real numbers. The function $f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ and $f(x)$ is $\Omega(g(x))$.
- We say that "f is big-Theta of *g*(*x*)" and also that "*f*(*x*) is of *order g*(*x*)" and also that "*f*(*x*) and *g*(*x*) are of the *same order*."
- $f(x)$ is $\Theta(g(x))$ if and only if there exists constants C_1, C_2 and *k* such that $C_1 g(x) < f(x) < C_2 g(x)$ if $x > k$. This follows from the definitions of big-*O* and big-Omega.

Big Theta Notation

Example: Show that the sum of the first *n* positive integers is $\Theta(n^2)$.

Solution: Let $f(n) = 1 + 2 + \cdots + n$.

- We have already shown that $f(n)$ is $O(n^2)$.
- To show that $f(n)$ is $\Omega(n^2)$, we need a positive constant *C* such that $f(n) > Cn^2$ for sufficiently large *n*. Summing only the terms greater than $n/2$ we obtain the inequality

$$\begin{aligned} 1 + 2 + \cdots + n &\geq \lceil n/2 \rceil + (\lceil n/2 \rceil + 1) + \cdots + n \\ &\geq \lceil n/2 \rceil + \lceil n/2 \rceil + \cdots + \lceil n/2 \rceil \\ &= (n - \lceil n/2 \rceil + 1)\lceil n/2 \rceil \\ &\geq (n/2)(n/2) = n^2/4 \end{aligned}$$

- Taking $C = 1/4$, $f(n) > Cn^2$ for all positive integers *n*. Hence, $f(n)$ is $\Omega(n^2)$, and we can conclude that $f(n)$ is $\Theta(n^2)$.

Big-Theta Notation

Example: Sh0w that $f(x) = 3x^2 + 8x \log x$ is $\Theta(x^2)$.

Solution:

- $3x^2 + 8x \log x \leq 11x^2$ for $x > 1$, since $0 \leq 8x \log x \leq 8x^2$.
  - Hence, $3x^2 + 8x \log x$ is $O(x^2)$.
- x^2 is clearly $O(3x^2 + 8x \log x)$
- Hence, $3x^2 + 8x \log x$ is $\Theta(x^2)$.

Big-Theta Notation

- When $f(x)$ is $\Theta(g(x))$ it must also be the case that $g(x)$ is $\Theta(f(x))$.
- Note that $f(x)$ is $\Theta(g(x))$ if and only if it is the case that $f(x)$ is $O(g(x))$ and $g(x)$ is $O(f(x))$.
- Sometimes writers are careless and write as if big-*O* notation has the same meaning as big-Theta.

Big-Theta Estimates for Polynomials

Theorem: Let $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_o$ where $a_0, a_1, \ldots, a_n$ are real numbers with $a_n \neq 0$. Then $f(x)$ is of order x^n (or $\Theta(x^n)$).

(The proof is an exercise.)

Example:

The polynomial $f(x) = 8x^5 + 5x^2 + 10$ is order of x^5 (or $\Theta(x^5)$).

The polynomial $f(x) = 8x^{199} + 7x^{100} + x^{99} + 5x^2 + 25$ is order of x^{199} (or $\Theta(x^{199})$).

Complexity of Algorithms

Section 3.3

Section Summary

- Time Complexity
- Worst-Case Complexity
- Algorithmic Paradigms
- Understanding the Complexity of Algorithms

The Complexity of Algorithms

- Given an algorithm, how efficient is this algorithm for solving a problem given input of a particular size? To answer this question, we ask:
  - How much time does this algorithm use to solve a problem?
  - How much computer memory does this algorithm use to solve a problem?
- When we analyze the time the algorithm uses to solve the problem given input of a particular size, we are studying the *time complexity* of the algorithm.
- When we analyze the computer memory the algorithm uses to solve the problem given input of a particular size, we are studying the *space complexity* of the algorithm.

The Complexity of Algorithms

- In this course, we focus on time complexity. The space complexity of algorithms is studied in later courses.
- We will measure time complexity in terms of the number of operations an algorithm uses and we will use big-*O* and big-Theta notation to estimate the time complexity.
- We can use this analysis to see whether it is practical to use this algorithm to solve problems with input of a particular size. We can also compare the efficiency of different algorithms for solving the same problem.
- We ignore implementation details (including the data structures used and both the hardware and software platforms) because it is extremely complicated to consider them.

Time Complexity

- To analyze the time complexity of algorithms, we determine the number of operations, such as comparisons and arithmetic operations (addition, multiplication, etc.). We can estimate the time a computer may actually use to solve a problem using the amount of time required to do basic operations.
- We ignore minor details, such as the "house keeping" aspects of the algorithm.
- We will focus on the *worst-case time* complexity of an algorithm. This provides an upper bound on the number of operations an algorithm uses to solve a problem with input of a particular size.
- It is usually much more difficult to determine the *average case time complexity* of an algorithm. This is the average number of operations an algorithm uses to solve a problem over all inputs of a particular size.

Complexity Analysis of Algorithms

Example: Describe the time complexity of the algorithm for finding the maximum element in a finite sequence.

```
procedure max(a1, a2, ...., an: integers)
  max := a1
  for i := 2 to n
    if max < ai then max := ai
  return max{max is the largest element}
```

Solution: Count the number of comparisons.

- The $max < a_i$ comparison is made $n - 1$ times.
- Each time i is incremented, a test is made to see if $i \leq n$.
- One last comparison determines that $i > n$.
- Exactly $2(n-1) + 1 = 2n - 1$ comparisons are made.

Hence, the time complexity of the algorithm is $\Theta(n)$.

Worst-Case Complexity of Linear Search

Example: Determine the time complexity of the linear search algorithm.

```
procedure linear search(x:integer,
          a1, a2, ...,an: distinct integers)
i := 1
while (i ≤ n and x ≠ ai)
    i := i + 1
if i ≤ n then location := i
else location := 0
return location{location is the subscript of the term that equals x, or is 0 if x is
      not found}
```

Solution: Count the number of comparisons.

- At each step two comparisons are made; $i \leq n$ and $x \neq a_i$.
- To end the loop, one comparison $i \leq n$ is made.
- After the loop, one more $i \leq n$ comparison is made.

If $x = a_i$, $2i + 1$ comparisons are used. If x is not on the list, $2n + 1$ comparisons are made and then an additional comparison is used to exit the loop. So, in the worst case $2n + 2$ comparisons are made. Hence, the complexity is $\Theta(n)$.

Average-Case Complexity of Linear Search

Example: Describe the average case performance of the linear search algorithm. (Although usually it is very difficult to determine average-case complexity, it is easy for linear search.)

Solution: Assume the element is in the list and that the possible positions are equally likely. By the argument on the previous slide, if $x = a_i$, the number of comparisons is $2i + 1$.

$$\frac{3+5+7+\ldots+(2n+1)}{n} = \frac{2(1+2+3+\ldots+n)+n}{n} = \frac{2[\frac{n(n+1)}{2}]}{n} + 1 = n + 2$$

Hence, the average-case complexity of linear search is $\Theta(n)$.

Worst-Case Complexity of Binary Search

Example: Describe the time complexity of binary search in terms of the number of comparisons used.

```
procedure binary search(x: integer, a1,a2,..., an: increasing integers)
 i := 1 {i is the left endpoint of interval}
 j := n {j is right endpoint of interval}
 while i < j
     m := ⌊(i + j)/2⌋
     if x > am then i := m + 1
     else j := m
 if x = ai then location := i
 else location := 0
 return location{location is the subscript i of the term ai equal to x, or 0 if x is not found}
```

Solution: Assume (for simplicity) $n = 2^k$ elements. Note that $k = \log n$.

- Two comparisons are made at each stage; $i < j$, and $x > a_m$.
- At the first iteration the size of the list is 2^k and after the first iteration it is 2^{k-1}. Then 2^{k-2} and so on until the size of the list is $2^1 = 2$.
- At the last step, a comparison tells us that the size of the list is the size is $2^0 = 1$ and the element is compared with the single remaining element.
- Hence, at most $2k + 2 = 2 \log n + 2$ comparisons are made.
- Therefore, the time complexity is $\Theta(\log n)$, better than linear search.

Worst-Case Complexity of Bubble Sort

Example: What is the worst-case complexity of bubble sort in terms of the number of comparisons made?

```
procedure bubblesort(a1,...,an: real numbers
                     with n ≥ 2)
  for i := 1 to n − 1
    for j := 1 to n − i
      if aj > aj+1 then interchange aj and aj+1
{a1,..., an is now in increasing order}
```

Solution: A sequence of $n-1$ passes is made through the list. On each pass $n - i$ comparisons are made.

$$(n-1) + (n-2) + \ldots + 2 + 1 = \frac{n(n-1)}{2}$$

The worst-case complexity of bubble sort is $\Theta(n^2)$ since $\frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$

Worst-Case Complexity of Insertion Sort

Example: What is the worst-case complexity of insertion sort in terms of the number of comparisons made?

Solution: The total number of comparisons are:

$$2 + 3 + \cdots + n = \frac{n(n-1)}{2} - 1$$

Therefore the complexity is $\Theta(n^2)$.

```
procedure insertion sort(a_1,...,a_n:
          real numbers with n ≥ 2)
  for j := 2 to n
    i := 1
    while a_j > a_i
      i := i + 1
    m := a_j
    for k := 0 to j − i − 1
      a_{j-k} := a_{j-k-1}
    a_i := m
```

Matrix Multiplication Algorithm

- The definition for matrix multiplication can be expressed as an algorithm; **C** = **A B** where **C** is an $m \times n$ matrix that is the product of the $m \times k$ matrix **A** and the $k \times n$ matrix **B**.
- This algorithm carries out matrix multiplication based on its definition.

```
procedure matrix multiplication(A,B: matrices)
  for i := 1 to m
    for j := 1 to n
      c_ij := 0
      for q := 1 to k
        c_ij := c_ij + a_iq b_qj
return C{C = [c_ij] is the product of A and B}
```

$\mathbf{A} = [a_{ij}]$ is a $m \times k$ matrix
$\mathbf{B} = [b_{ij}]$ is a $k \times n$ matrix

Complexity of Matrix Multiplication

Example: How many additions of integers and multiplications of integers are used by the matrix multiplication algorithm to multiply two $n \times n$ matrices.

Solution: There are n^2 entries in the product. Finding each entry requires n multiplications and $n - 1$ additions. Hence, n^3 multiplications and $n^2(n - 1)$ additions are used.

Hence, the complexity of matrix multiplication is $O(n^3)$.

Boolean Product Algorithm

- The definition of Boolean product of zero-one matrices can also be converted to an algorithm.

```
procedure Boolean product(A,B: zero-one matrices)
  for i := 1 to m
    for j := 1 to n
      c_ij := 0
      for q := 1 to k
        c_ij := c_ij ∨ (a_iq ∧ b_qj)
return C{C = [c_ij] is the Boolean product of A and B}
```

Complexity of Boolean Product Algorithm

Example: How many bit operations are used to find $\mathbf{A} \odot \mathbf{B}$, where A and B are $n \times n$ zero-one matrices?

Solution: There are n^2 entries in the $\mathbf{A} \odot \mathbf{B}$. A total of n 0rs and n ANDs are used to find each entry. Hence, each entry takes $2n$ bit operations. A total of $2n^3$ operations are used.

Therefore the complexity is $O(n^3)$

Matrix-Chain Multiplication

- How should the *matrix-chain* $\mathbf{A}_1\mathbf{A}_2 \cdots \mathbf{A}_n$ be computed using the fewest multiplications of integers, where $\mathbf{A}_1, \mathbf{A}_2, \cdots, \mathbf{A}_n$ are $m_1 \times m_2, m_2 \times m_3, \cdots m_n \times m_{n+1}$ integer matrices. Matrix multiplication is associative (exercise in Section 2.6).

Example: In which order should the integer matrices $\mathbf{A}_1\mathbf{A}_2\mathbf{A}_3$ - where $\mathbf{A}_1$ is 30×20 $\mathbf{A}_2$ 20×40, $\mathbf{A}_3$ 40×10 - be multiplied to use the least number of multiplications.

Solution: There are two possible ways to compute $\mathbf{A}_1\mathbf{A}_2\mathbf{A}_3$.

- $\mathbf{A}_1(\mathbf{A}_2\mathbf{A}_3)$: $\mathbf{A}_2\mathbf{A}_3$ takes $20 \cdot 40 \cdot 10 = 8000$ multiplications. Then multiplying $\mathbf{A}_1$ by the 20×10 matrix $\mathbf{A}_2\mathbf{A}_3$ takes $30 \cdot 20 \cdot 10 = 6000$ multiplications. So the total number is $8000 + 6000 = 14{,}000$.
- $(\mathbf{A}_1\mathbf{A}_2)\mathbf{A}_3$: $\mathbf{A}_1\mathbf{A}_2$ takes $30 \cdot 20 \cdot 40 = 24{,}000$ multiplications. Then multiplying the 30×40 matrix $\mathbf{A}_1\mathbf{A}_2$ by $\mathbf{A}_3$ takes $30 \cdot 40 \cdot 10 = 12{,}000$ multiplications. So the total number is $24{,}000 + 12{,}000 = 36{,}000$.

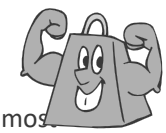
So the first method is best.

An efficient algorithm for finding the best order for matrix-chain multiplication can be based on the algorithmic paradigm known as *dynamic programming*. (see Ex. 57 in Section 8.1)

Algorithmic Paradigms

- An *algorithmic paradigm* is a a general approach based on a particular concept for constructing algorithms to solve a variety of problems.
  - Greedy algorithms were introduced in Section 3.1.
  - We discuss brute-force algorithms in this section.
  - We will see divide-and-conquer algorithms (Chapter 8), dynamic programming (Chapter 8), backtracking (Chapter 11), and probabilistic algorithms (Chapter 7). There are many other paradigms that you may see in later courses.

Brute-Force Algorithms



- A *brute-force* algorithm is solved in the most straightforward manner, without taking advantage of any ideas that can make the algorithm more efficient.
- Brute-force algorithms we have previously seen are sequential search, bubble sort, and insertion sort.

Computing the Closest Pair of Points by Brute-Force

Example: Construct a brute-force algorithm for finding the closest pair of points in a set of n points in the plane and provide a worst-case estimate of the number of arithmetic operations.

Solution: Recall that the distance between (x_i, y_i) and (x_j, y_j) is $\sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$. A brute-force algorithm simply computes the distance between all pairs of points and picks the pair with the smallest distance.

Note: There is no need to compute the square root, since the square of the distance between two points is smallest when the distance is smallest.

Continued →

Computing the Closest Pair of Points by Brute-Force

- Algorithm for finding the closest pair in a set of n points.

```
procedure closest pair((x1, y1), (x2, y2), ... ,(xn, yn): xi, yi real numbers)
min = ∞
  for i := 1 to n
    for j := 1 to i
      if (xj − xi)^2 + (yj − yi)^2 < min
        then min := (xj − xi)^2 + (yj − yi)^2
          closest pair := (xi, yi), (xj, yj)
return closest pair
```

- The algorithm loops through $n(n-1)/2$ pairs of points, computes the value $(x_j - x_i)^2 + (y_j - y_i)^2$ and compares it with the minimum, etc. So, the algorithm uses $\Theta(n^2)$ arithmetic and comparison operations.
- We will develop an algorithm with $O(\log n)$ worst-case complexity in Section 8.3.

Understanding the Complexity of Algorithms

TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

Complexity	*Terminology*
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Understanding the Complexity of Algorithms

TABLE 2 The Computer Time Used by Algorithms.

Problem Size	*Bit Operations Used*					
n	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

Times of more than 10^{100} years are indicated with an *.

Complexity of Problems

- *Tractable Problem*: There exists a polynomial time algorithm to solve this problem. These problems are said to belong to the *Class P*.
- *Intractable Problem*: There does not exist a polynomial time algorithm to solve this problem
- *Unsolvable Problem* : No algorithm exists to solve this problem, e.g., halting problem.
- *Class NP*: Solution can be checked in polynomial time. But no polynomial time algorithm has been found for finding a solution to problems in this class.
- *NP Complete Class*: If you find a polynomial time algorithm for one member of the class, it can be used to solve all the problems in the class.

P Versus NP Problem



Stephen Cook
(Born 1939)

- The *P versus NP problem* asks whether the class P = NP? Are there problems whose solutions can be checked in polynomial time, but can not be solved in polynomial time?
  - Note that just because no one has found a polynomial time algorithm is different from showing that the problem can not be solved by a polynomial time algorithm.
- If a polynomial time algorithm for any of the problems in the NP complete class were found, then that algorithm could be used to obtain a polynomial time algorithm for every problem in the NP complete class.
  - Satisfiability (in Section 1.3) is an NP complete problem.
- It is generally believed that P≠NP since no one has been able to find a polynomial time algorithm for any of the problems in the NP complete class.
- The problem of P versus NP remains one of the most famous unsolved problems in mathematics (including theoretical computer science). The Clay Mathematics Institute has offered a prize of $1,000,000 for a solution.

Number Theory and Cryptography

Chapter 4

With Question/Answer Animations

Chapter Motivation

- *Number theory* is the part of mathematics devoted to the study of the integers and their properties.
- Key ideas in number theory include divisibility and the primality of integers.
- Representations of integers, including binary and hexadecimal representations, are part of number theory.
- Number theory has long been studied because of the beauty of its ideas, its accessibility, and its wealth of open questions.
- We'll use many ideas developed in Chapter 1 about proof methods and proof strategy in our exploration of number theory.
- Mathematicians have long considered number theory to be pure mathematics, but it has important applications to computer science and cryptography studied in Sections 4.5 and 4.6.

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Divisibility and Modular Arithmetic
- Integer Representations and Algorithms
- Primes and Greatest Common Divisors
- Solving Congruences
- Applications of Congruences
- Cryptography

Divisibility and Modular Arithmetic

Section 4.1

Section Summary

- Division
- Division Algorithm
- Modular Arithmetic

Division

Definition: If a and b are integers with $a \neq 0$, then *a divides b* if there exists an integer c such that $b = ac$.

- When a divides b we say that a is a *factor* or *divisor* of b and that b is a multiple of a.
- The notation $a \mid b$ denotes that a divides b.
- If $a \mid b$, then b/a is an integer.
- If a does not divide b, we write $a \nmid b$.

Example: Determine whether $3 \mid 7$ and whether $3 \mid 12$.

Properties of Divisibility

Theorem 1: Let a, b, and c be integers, where $a \neq 0$.

i. If $a \mid b$ and $a \mid c$, then $a \mid (b + c)$;
ii. If $a \mid b$, then $a \mid bc$ for all integers c;
iii. If $a \mid b$ and $b \mid c$, then $a \mid c$.

Proof: (i) Suppose $a \mid b$ and $a \mid c$, then it follows that there are integers s and t with $b = as$ and $c = at$. Hence,
$b + c = as + at = a(s + t)$. Hence, $a \mid (b + c)$

(Exercises 3 and 4 ask for proofs of parts (ii) and (iii).)

Corollary: If a, b, and c be integers, where $a \neq 0$, such that $a \mid b$ and $a \mid c$, then $a \mid mb + nc$ whenever m and n are integers. ◀

Can you show how it follows easily from from (ii) and (i) of Theorem 1?

Division Algorithm

- When an integer is divided by a positive integer, there is a quotient and a remainder. This is traditionally called the "Division Algorithm," but is really a theorem.

Division Algorithm: If a is an integer and d a positive integer, then there are unique integers q and r, with $0 \leq r < d$, such that $a = dq + r$ (*proved in Section* 5.2).

- d is called the *divisor*.
- a is called the *dividend*.
- q is called the *quotient*.
- r is called the *remainder*.

Definitions of Functions **div** and **mod**

$q = a$ **div** d
$r = a$ **mod** d

Examples:

- What are the quotient and remainder when 101 is divided by 11?
  Solution: The quotient when 101 is divided by 11 is $9 = 101$ **div** 11, and the remainder is $2 = 101$ **mod** 11.
- What are the quotient and remainder when -11 is divided by 3?
  Solution: The quotient when -11 is divided by 3 is $-4 = -11$ **div** 3, and the remainder is $1 = -11$ **mod** 3.

Congruence Relation

Definition: If a and b are integers and m is a positive integer, then a is *congruent* to b *modulo* m if m divides $a - b$.

- The notation $a \equiv b \pmod{m}$ says that a is congruent to b modulo m.
- We say that $a \equiv b \pmod{m}$ is a *congruence* and that m is its *modulus.*
- Two integers are congruent mod m if and only if they have the same remainder when divided by m.
- If a is not congruent to b modulo m, we write $a \not\equiv b \pmod{m}$

Example: Determine whether 17 is congruent to 5 modulo 6 and whether 24 and 14 are congruent modulo 6.

Solution:

- $17 \equiv 5 \pmod{6}$ because 6 divides $17 - 5 = 12$.
- $24 \not\equiv 14 \pmod{6}$ since $24 - 14 = 10$ is not divisible by 6.

More on Congruences

Theorem 4: Let m be a positive integer. The integers a and b are congruent modulo m if and only if there is an integer k such that $a = b + km$.

Proof:

- If $a \equiv b \pmod{m}$, then (by the definition of congruence) $m \mid a - b$. Hence, there is an integer k such that $a - b = km$ and equivalently $a = b + km$.
- Conversely, if there is an integer k such that $a = b + km$, then $km = a - b$. Hence, $m \mid a - b$ and $a \equiv b \pmod{m}$.

◀

The Relationship between (mod m) and **mod** m Notations

- The use of "mod" in $a \equiv b \pmod m$ and a **mod** $m = b$ are different.
  - $a \equiv b \pmod m$ is a relation on the set of integers.
  - In a **mod** $m = b$, the notation **mod** denotes a function.
- The relationship between these notations is made clear in this theorem.
- **Theorem 3**: Let a and b be integers, and let m be a positive integer. Then $a \equiv b \pmod m$ if and only if a **mod** $m = b$ **mod** m. (*Proof in the exercises*)

Congruences of Sums and Products

Theorem 5: Let m be a positive integer. If $a \equiv b \pmod m$ and $c \equiv d \pmod m$, then

$a + c \equiv b + d \pmod m$ and $ac \equiv bd \pmod m$

Proof:

- Because $a \equiv b \pmod m$ and $c \equiv d \pmod m$, by Theorem 4 there are integers s and t with $b = a + sm$ and $d = c + tm$.
- Therefore,
  - $b + d = (a + sm) + (c + tm) = (a + c) + m(s + t)$ and
  - $b\,d = (a + sm)(c + tm) = ac + m(at + cs + stm)$.
- Hence, $a + c \equiv b + d \pmod m$ and $ac \equiv bd \pmod m$.

Example: Because $7 \equiv 2 \pmod 5$ and $11 \equiv 1 \pmod 5$, it follows from Theorem 5 that

$18 = 7 + 11 \equiv 2 + 1 = 3 \pmod 5$

$77 = 7 \cdot 11 \equiv 2 \cdot 1 = 2 \pmod 5$

Algebraic Manipulation of Congruences

- Multiplying both sides of a valid congruence by an integer preserves validity.
  If $a \equiv b \pmod m$ holds then $c \cdot a \equiv c \cdot b \pmod m$, where c is any integer, holds by Theorem 5 with $d = c$.
- Adding an integer to both sides of a valid congruence preserves validity.
  If $a \equiv b \pmod m$ holds then $c + a \equiv c + b \pmod m$, where c is any integer, holds by Theorem 5 with $d = c$.
- Dividing a congruence by an integer does not always produce a valid congruence.

  Example: The congruence $14 \equiv 8 \pmod 6$ holds. But dividing both sides by 2 does not produce a valid congruence since $14/2 = 7$ and $8/2 = 4$, but $7 \not\equiv 4 \pmod 6$.

  See Section 4.3 for conditions when division is ok.

Computing the **mod** m Function of Products and Sums

- We use the following corollary to Theorem 5 to compute the remainder of the product or sum of two integers when divided by m from the remainders when each is divided by m.

  Corollary: Let m be a positive integer and let a and b be integers. Then

  $(a + b)$ **mod** m = $((a \textbf{ mod } m) + (b \textbf{ mod } m))$ **mod** m

  and

  ab **mod** $m = ((a \textbf{ mod } m)(b \textbf{ mod } m))$ **mod** m.

  (*proof in text*)

Arithmetic Modulo m

Definitions: Let $\mathbf{Z}_m$ be the set of nonnegative integers less than m: $\{0, 1, \ldots, m-1\}$

- The operation $+_m$ is defined as $a +_m b = (a + b)$ **mod** m. This is *addition modulo m*.
- The operation $\cdot_m$ is defined as $a \cdot_m b = (a \cdot b)$ **mod** m. This is *multiplication modulo m*.
- Using these operations is said to be doing *arithmetic modulo m*.

Example: Find $7 +_{11} 9$ and $7 \cdot_{11} 9$.

Solution: Using the definitions above:

- $7 +_{11} 9 = (7 + 9) \textbf{ mod } 11 = 16 \textbf{ mod } 11 = 5$
- $7 \cdot_{11} 9 = (7 \cdot 9) \textbf{ mod } 11 = 63 \textbf{ mod } 11 = 8$

Arithmetic Modulo m

- The operations $+_m$ and $\cdot_m$ satisfy many of the same properties as ordinary addition and multiplication.
  - *Closure*: If a and b belong to $\mathbf{Z}_m$, then $a +_m b$ and $a \cdot_m b$ belong to $\mathbf{Z}_m$.
  - *Associativity*: If a, b, and c belong to $\mathbf{Z}_m$, then $(a +_m b) +_m c = a +_m (b +_m c)$ and $(a \cdot_m b) \cdot_m c = a \cdot_m (b \cdot_m c)$.
  - *Commutativity*: If a and b belong to $\mathbf{Z}_m$, then $a +_m b = b +_m a$ and $a \cdot_m b = b \cdot_m a$.
  - *Identity elements*: The elements 0 and 1 are identity elements for addition and multiplication modulo m, respectively.
    - If a belongs to $\mathbf{Z}_m$, then $a +_m 0 = a$ and $a \cdot_m 1 = a$.

continued →

Arithmetic Modulo m

- *Additive inverses*: If $a \neq 0$ belongs to $\mathbf{Z}_m$, then $m - a$ is the additive inverse of a modulo m and 0 is its own additive inverse.
  - $a +_m (m - a) = 0$ and $0 +_m 0 = 0$
- *Distributivity*: If a, b, and c belong to $\mathbf{Z}_m$, then
  - $a \cdot_m (b +_m c) = (a \cdot_m b) +_m (a \cdot_m c)$ and $(a +_m b) \cdot_m c = (a \cdot_m c) +_m (b \cdot_m c)$.
- Exercises 42-44 ask for proofs of these properties.
- Multiplicatative inverses have not been included since they do not always exist. For example, there is no multiplicative inverse of 2 modulo 6.
- (*optional*) Using the terminology of abstract algebra, $\mathbf{Z}_m$ with $+_m$ is a commutative group and $\mathbf{Z}_m$ with $+_m$ and $\cdot_m$ is a commutative ring.

Integer Representations and Algorithms

Section 4.2

Section Summary

- Integer Representations
  - Base *b* Expansions
  - Binary Expansions
  - Octal Expansions
  - Hexadecimal Expansions
- Base Conversion Algorithm
- Algorithms for Integer Operations

Representations of Integers

- In the modern world, we use *decimal,* or *base* 10, *notation* to represent integers. For example when we write 965, we mean $9 \cdot 10^2 + 6 \cdot 10^1 + 5 \cdot 10^0$.
- We can represent numbers using any base *b*, where *b* is a positive integer greater than 1.
- The bases *b* = 2 (*binary*), *b* = 8 (*octal*) , and *b*= 16 (*hexadecimal*) are important for computing and communications
- The ancient Mayans used base 20 and the ancient Babylonians used base 60.

Base *b* Representations

- We can use positive integer *b* greater than 1 as a base, because of this theorem:

  Theorem 1: Let *b* be a positive integer greater than 1. Then if *n* is a positive integer, it can be expressed uniquely in the form:

  $n = a_k b^k + a_{k-1} b^{k-1} + \ldots + a_1 b + a_0$

  where *k* is a nonnegative integer, $a_0, a_1, \ldots a_k$ are nonnegative integers less than *b*, and $a_k \neq 0$. The a_j, $j = 0,\ldots,k$ are called the base-*b* digits of the representation.

  (We will prove this using mathematical induction in Section 5.1.)
- The representation of n given in Theorem 1 is called the *base b expansion of n* and is denoted by $(a_k a_{k-1} \ldots a_1 a_0)_b$.
- We usually omit the subscript 10 for base 10 expansions.

Binary Expansions

Most computers represent integers and do arithmetic with binary (base 2) expansions of integers. In these expansions, the only digits used are 0 and 1.

Example: What is the decimal expansion of the integer that has $(1\ 0101\ 1111)_2$ as its binary expansion?

Solution:

$(1\ 0101\ 1111)_2 = 1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 351.$

Example: What is the decimal expansion of the integer that has $(11011)_2$ as its binary expansion?

Solution: $(11011)_2 = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 27.$

Octal Expansions

The octal expansion (base 8) uses the digits $\{0,1,2,3,4,5,6,7\}$.

Example: What is the decimal expansion of the number with octal expansion $(7016)_8$?

Solution: $7 \cdot 8^3 + 0 \cdot 8^2 + 1 \cdot 8^1 + 6 \cdot 8^0 = 3598$

Example: What is the decimal expansion of the number with octal expansion $(111)_8$?

Solution: $1 \cdot 8^2 + 1 \cdot 8^1 + 1 \cdot 8^0 = 64 + 8 + 1 = 73$

Hexadecimal Expansions

The hexadecimal expansion needs 16 digits, but our decimal system provides only 10. So letters are used for the additional symbols. The hexadecimal system uses the digits $\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$. The letters A through F represent the decimal numbers 10 through 15.

Example: What is the decimal expansion of the number with hexadecimal expansion $(2AE0B)_{16}$?

Solution:

$2 \cdot 16^4 + 10 \cdot 16^3 + 14 \cdot 16^2 + 0 \cdot 16^1 + 11 \cdot 16^0 = 175627$

Example: What is the decimal expansion of the number with hexadecimal expansion $(E5)_{16}$?

Solution: $14 \cdot 16^1 + 5 \cdot 16^0 = 224 + 5 = 229$

Base Conversion

To construct the base *b* expansion of an integer *n*:

- Divide *n* by *b* to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$
- The remainder, a_0 , is the rightmost digit in the base *b* expansion of *n*. Next, divide q_0 by *b*.

  $q_0 = bq_1 + a_1 \quad 0 \leq a_1 < b$
- The remainder, a_1, is the second digit from the right in the base *b* expansion of *n*.
- Continue by successively dividing the quotients by *b*, obtaining the additional base *b* digits as the remainder. The process terminates when the quotient is 0.

continued →

Algorithm: Constructing Base *b* Expansions

procedure *base b expansion*(*n, b*: positive integers with $b > 1$)
$q := n$
$k := 0$
while $(q \neq 0)$
  $a_k := q$ **mod** b
  $q := q$ **div** b
  $k := k + 1$
return$(a_{k-1}, \ldots, a_1, a_0)$ {$(a_{k-1} \ldots a_1 a_0)_b$ is base *b* expansion of *n*}

- *q* represents the quotient obtained by successive divisions by *b*, starting with $q = n$.
- The digits in the base *b* expansion are the remainders of the division given by *q* **mod** *b*.
- The algorithm terminates when $q = 0$ is reached.

Base Conversion

Example: Find the octal expansion of $(12345)_{10}$

Solution: Successively dividing by 8 gives:

- $12345 = 8 \cdot 1543 + 1$
- $1543 = 8 \cdot 192 + 7$
- $192 = 8 \cdot 24 + 0$
- $24 = 8 \cdot 3 + 0$
- $3 = 8 \cdot 0 + 3$

The remainders are the digits from right to left yielding $(30071)_8$.

Comparison of Hexadecimal, Octal, and Binary Representations

TABLE 1 Hexadecimal, Octal, and Binary Representation of the Integers 0 through 15.

Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Octal	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
Binary	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

Initial 0s are not shown

Each octal digit corresponds to a block of 3 binary digits.
Each hexadecimal digit corresponds to a block of 4 binary digits.
So, conversion between binary, octal, and hexadecimal is easy.

Conversion Between Binary, Octal, and Hexadecimal Expansions

Example: Find the octal and hexadecimal expansions of $(11\ 1110\ 1011\ 1100)_2$.

Solution:

- To convert to octal, we group the digits into blocks of three $(011\ 111\ 010\ 111\ 100)_2$, adding initial 0s as needed. The blocks from left to right correspond to the digits 3,7,2,7, and 4. Hence, the solution is $(37274)_8$.
- To convert to hexadecimal, we group the digits into blocks of four $(0011\ 1110\ 1011\ 1100)_2$, adding initial 0s as needed. The blocks from left to right correspond to the digits 3,E,B, and C. Hence, the solution is $(3EBC)_{16}$.

Binary Addition of Integers

- Algorithms for performing operations with integers using their binary expansions are important as computer chips work with binary numbers. Each digit is called a *bit*.

procedure *add*(*a, b*: positive integers)
{the binary expansions of *a* and *b* are $(a_{n-1},a_{n-2},\ldots,a_0)_2$ and $(b_{n-1},b_{n-2},\ldots,b_0)_2$, respectively}
$c := 0$
for $j := 0$ to $n-1$
  $d := \lfloor (a_j + b_j + c)/2 \rfloor$
  $s_j := a_j + b_j + c - 2d$
  $c := d$
$s_n := c$
return$(s_0,s_1,\ldots,s_n)$ {the binary expansion of the sum is $(s_n,s_{n-1},\ldots,s_0)_2$}

- The number of additions of bits used by the algorithm to add two *n*-bit integers is $O(n)$.

Binary Multiplication of Integers

- Algorithm for computing the product of two *n* bit integers.

procedure *multiply*(*a, b*: positive integers)
{the binary expansions of a and b are $(a_{n-1},a_{n-2},\ldots,a_0)_2$ and $(b_{n-1},b_{n-2},\ldots,b_0)_2$, respectively}
for $j := 0$ to $n-1$
  if $b_j = 1$ **then** $c_j = a$ shifted j places
  else $c_j := 0$
{$c_0, c_1, \ldots, c_{n-1}$ are the partial products}
$p := 0$
for $j := 0$ to $n-1$
  $p := p + c_j$
return p {p is the value of ab}

- The number of additions of bits used by the algorithm to multiply two *n*-bit integers is $O(n^2)$.

Binary Modular Exponentiation

- In cryptography, it is important to be able to find b^n **mod** m efficiently, where b, n, and m are large integers.
- Use the binary expansion of n, $n = (a_{k-1},\ldots,a_1,a_0)_2$, to compute b^n. Note that:

$$b^n = b^{a_{k-1}\cdot 2^{k-1}+\cdots+a_1\cdot 2+a_0} = b^{a_{k-1}\cdot 2^{k-1}} \cdots b^{a_1\cdot 2} \cdot b^{a_0}.$$

- Therefore, to compute b^n, we need only compute the values of b, b^2, $(b^2)^2 = b^4$, $(b^4)^2 = b^8$, ..., b^{2^k} and the multiply the terms in this list, where $a_j = 1$. b^{2^j}

Example: Compute 3^{11} using this method.

Solution: Note that $11 = (1011)_2$ so that $3^{11} = 3^8\,3^2\,3^1 = ((3^2)^2)^2\,3^2\,3^1 = (9^2)^2 \cdot 9 \cdot 3 = (81)^2 \cdot 9 \cdot 3 = 6561 \cdot 9 \cdot 3 = 117{,}147$.

continued →

Binary Modular Exponentiation Algorithm

- The algorithm successively finds b **mod** m, b^2 **mod** m, b^4 **mod** m, ..., $b^{2^{k-1}}$ **mod** m, and multiplies together the terms b^{2^j} where $a_j = 1$.

procedure *modular exponentiation*(b: integer, $n = (a_{k-1}a_{k-2}\ldots a_1a_0)_2$, m: positive integers)
$x := 1$
$power := b$ **mod** m
for $i := 0$ to $k-1$
  if $a_i = 1$ **then** $x := (x \cdot power)$ **mod** m
  $power := (power \cdot power)$ **mod** m
return x {x equals b^n **mod** m }

- ~~$O((\log m)^2 \log n)$ bit operations are used to find b^n **mod** m.~~

Primes and Greatest Common Divisors

Section 4.3

Section Summary

- Prime Numbers and their Properties
- Conjectures and Open Problems About Primes
- Greatest Common Divisors and Least Common Multiples
- The Euclidian Algorithm
- gcds as Linear Combinations

Primes

Definition: A positive integer p greater than 1 is called *prime* if the only positive factors of p are 1 and p. A positive integer that is greater than 1 and is not prime is called *composite*.

Example: The integer 7 is prime because its only positive factors are 1 and 7, but 9 is composite because it is divisible by 3.

The Fundamental Theorem of Arithmetic

Theorem: Every positive integer greater than 1 can be written uniquely as a prime or as the product of two or more primes where the prime factors are written in order of nondecreasing size.

Examples:

- $100 = 2 \cdot 2 \cdot 5 \cdot 5 = 2^2 \cdot 5^2$
- $641 = 641$
- $999 = 3 \cdot 3 \cdot 3 \cdot 37 = 3^3 \cdot 37$
- $1024 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^{10}$

The Sieve of Erastosthenes



Erastothenes
(276-194 B.C.)

- The *Sieve of Erastosthenes* can be used to find all primes not exceeding a specified positive integer. For example, begin with the list of integers between 1 and 100.
  a. Delete all the integers, other than 2, divisible by 2.
  b. Delete all the integers, other than 3, divisible by 3.
  c. Next, delete all the integers, other than 5, divisible by 5.
  d. Next, delete all the integers, other than 7, divisible by 7.
  e. Since all the remaining integers are not divisible by any of the previous integers, other than 1, the primes are:

{2,3,5,7,11,15,1719,23,29,31,37,41,43,47,53,
59,61,67,71,73,79,83,89, 97}

continued →

The Sieve of Erastosthenes

TABLE 1 **The Sieve of Eratosthenes.**

Integers divisible by 2 other than 2 receive an underline.									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Integers divisible by 3 other than 3 receive an underline.									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Integers divisible by 5 other than 5 receive an underline.									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Integers divisible by 7 other than 7 receive an underline; integers in color are prime.									
1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

If an integer n is a composite integer, then it has a prime divisor less than or equal to $\sqrt{n}$.

To see this, note that if $n = ab$, then $a \leq \sqrt{n}$ or $b \leq \sqrt{n}$.

Trial division, a very inefficient method of determining if a number n is prime, is to try every integer $i \leq \sqrt{n}$ and see if n is divisible by i.

Infinitude of Primes



Euclid
(325 B.C.E. – 265 B.C.E.)

Theorem: There are infinitely many primes. (Euclid)

Proof: Assume finitely many primes: $p_1, p_2, \ldots, p_n$

- Let $q = p_1 p_2 \cdots p_n + 1$
- Either q is prime or by the fundamental theorem of arithmetic it is a product of primes.
  - But none of the primes p_j divides q since if $p_j \mid q$, then p_j divides $q - p_1 p_2 \cdots p_n = 1$.
  - Hence, there is a prime not on the list $p_1, p_2, \ldots, p_n$. It is either q, or if q is composite, it is a prime factor of q. This contradicts the assumption that $p_1, p_2, \ldots, p_n$ are all the primes.
- Consequently, there are infinitely many primes.

This proof was given by Euclid *The Elements*. The proof is considered to be one of the most beautiful in all mathematics. It is the first proof in *The Book*, inspired by the famous mathematician Paul Erdős' imagined collection of perfect proofs maintained by God.



Paul Erdős
(1913-1996)

Mersene Primes



Marin Mersenne
(1588-1648)

Definition: Prime numbers of the form $2^p - 1$, where p is prime, are called *Mersene primes*.

- $2^2 - 1 = 3$, $2^3 - 1 = 7$, $2^5 - 1 = 37$, and $2^7 - 1 = 127$ are Mersene primes.
- $2^{11} - 1 = 2047$ is not a Mersene prime since $2047 = 23 \cdot 89$.
- There is an efficient test for determining if $2^p - 1$ is prime.
- The largest known prime numbers are Mersene primes.
- As of mid 2011, 47 Mersene primes were known, the largest is $2^{43,112,609} - 1$, which has nearly 13 million decimal digits.
- The *Great Internet Mersene Prime Search* (*GIMPS*) is a distributed computing project to search for new Mersene Primes.
  http://www.mersenne.org/

Distribution of Primes

- Mathematicians have been interested in the distribution of prime numbers among the positive integers. In the nineteenth century, the *prime number theorem* was proved which gives an asymptotic estimate for the number of primes not exceeding x.

  Prime Number Theorem: The ratio of the number of primes not exceeding x and $x/\ln x$ approaches 1 as x grows without bound. ($\ln x$ is the natural logarithm of x)
  - The theorem tells us that the number of primes not exceeding x, can be approximated by $x/\ln x$.
  - The odds that a randomly selected positive integer less than n is prime are approximately $(n/\ln n)/n = 1/\ln n$.

Primes and Arithmetic Progressions (*optional*)

- Euclid's proof that there are infinitely many primes can be easily adapted to show that there are infinitely many primes in the following $4k + 3$, $k = 1,2,\ldots$ (See Exercise 55)
- In the 19th century G. Lejuenne Dirchlet showed that every arithmetic progression $ka + b$, $k = 1,2, \ldots$, where a and b have no common factor greater than 1 contains infinitely many primes. (The proof is beyond the scope of the text.)
- Are there long arithmetic progressions made up entirely of primes?
  - 5,11, 17, 23, 29 is an arithmetic progression of five primes.
  - 199, 409, 619, 829, 1039,1249,1459,1669,1879,2089 is an arithmetic progression of ten primes.
- In the 1930s, Paul Erdős conjectured that for every positive integer n greater than 1, there is an arithmetic progression of length n made up entirely of primes. This was proven in 2006, by Ben Green and Terrence Tau.



Terence Tao
(Born 1975)

Generating Primes

- The problem of generating large primes is of both theoretical and practical interest.
- We will see (in Section 4.6) that finding large primes with hundreds of digits is important in cryptography.
- So far, no useful closed formula that always produces primes has been found. There is no simple function $f(n)$ such that $f(n)$ is prime for all positive integers n.
- But $f(n) = n^2 - n + 41$ is prime for all integers $1,2,\ldots, 40$. Because of this, we might conjecture that $f(n)$ is prime for all positive integers n. But $f(41) = 41^2$ is not prime.
- More generally, there is no polynomial with integer coefficients such that $f(n)$ is prime for all positive integers n. (See supplementary Exercise 23.)
- Fortunately, we can generate large integers which are almost certainly primes. See Chapter 7.

Conjectures about Primes

- Even though primes have been studied extensively for centuries, many conjectures about them are unresolved, including:
- *Goldbach's Conjecture*: Every even integer n, $n > 2$, is the sum of two primes. It has been verified by computer for all positive even integers up to $1.6 \cdot 10^{18}$. The conjecture is believed to be true by most mathematicians.
- There are infinitely many primes of the form $n^2 + 1$, where n is a positive integer. But it has been shown that there are infinitely many primes of the form $n^2 + 1$, where n is a positive integer or the product of at most two primes.
- *The Twin Prime Conjecture*: The twin prime conjecture is that there are infinitely many pairs of twin primes. Twin primes are pairs of primes that differ by 2. Examples are 3 and 5, 5 and 7, 11 and 13, etc. The current world's record for twin primes (as of mid 2011) consists of numbers $65{,}516{,}468{,}355 \cdot 2^{33{,}333} \pm 1$, which have 100,355 decimal digits.

Greatest Common Divisor

Definition: Let a and b be integers, not both zero. The largest integer d such that $d \mid a$ and also $d \mid b$ is called the greatest common divisor of a and b. The greatest common divisor of a and b is denoted by $\gcd(a,b)$.

One can find greatest common divisors of small numbers by inspection.

Example:What is the greatest common divisor of 24 and 36?

Solution: $\gcd(24, 36) = 12$

Example:What is the greatest common divisor of 17 and 22?

Solution: $\gcd(17,22) = 1$

Greatest Common Divisor

Definition: The integers a and b are *relatively prime* if their greatest common divisor is 1.

Example: 17 and 22

Definition: The integers $a_1, a_2, \ldots, a_n$ are *pairwise relatively prime* if $\gcd(a_i, a_j) = 1$ whenever $1 \leq i < j \leq n$.

Example: Determine whether the integers 10, 17 and 21 are pairwise relatively prime.

Solution: Because $\gcd(10,17) = 1$, $\gcd(10,21) = 1$, and $\gcd(17,21) = 1$, 10, 17, and 21 are pairwise relatively prime.

Example: Determine whether the integers 10, 19, and 24 are pairwise relatively prime.

Solution: Because $\gcd(10,24) = 2$, 10, 19, and 24 are not pairwise relatively prime.

Greatest Common Divisor

Definition: The integers a and b are *relatively prime* if their greatest common divisor is 1.

Example: 17 and 22

Definition: The integers $a_1, a_2, \ldots, a_n$ are *pairwise relatively prime* if $\gcd(a_i, a_j) = 1$ whenever $1 \leq i < j \leq n$.

Example: Determine whether the integers 10, 17 and 21 are pairwise relatively prime.

Solution: Because $\gcd(10,17) = 1$, $\gcd(10,21) = 1$, and $\gcd(17,21) = 1$, 10, 17, and 21 are pairwise relatively prime.

Example: Determine whether the integers 10, 19, and 24 are pairwise relatively prime.

Solution: Because $\gcd(10,24) = 2$, 10, 19, and 24 are not pairwise relatively prime.

Finding the Greatest Common Divisor Using Prime Factorizations

- Suppose the prime factorizations of a and b are:

$$a = p_1^{a_1} p_2^{a_2} \cdots p_n^{a_n}, \qquad b = p_1^{b_1} p_2^{b_2} \cdots p_n^{b_n},$$

where each exponent is a nonnegative integer, and where all primes occurring in either prime factorization are included in both. Then:

$$\gcd(a,b) = p_1^{\min(a_1,b_1)} p_2^{\min(a_2,b_2)} \cdots p_n^{\min(a_n,b_n)}.$$

- This formula is valid since the integer on the right (of the equals sign) divides both a and b. No larger integer can divide both a and b.

  Example: $120 = 2^3 \cdot 3 \cdot 5$ $\quad 500 = 2^2 \cdot 5^3$

  $\gcd(120,500) = 2^{\min(3,2)} \cdot 3^{\min(1,0)} \cdot 5^{\min(1,3)} = 2^2 \cdot 3^0 \cdot 5^1 = 20$
- Finding the gcd of two positive integers using their prime factorizations is not efficient because there is no efficient algorithm for finding the prime factorization of a positive integer.

Least Common Multiple

Definition: The least common multiple of the positive integers a and b is the smallest positive integer that is divisible by both a and b. It is denoted by $\text{lcm}(a,b)$.

- The least common multiple can also be computed from the prime factorizations.

$$\text{lcm}(a,b) = p_1^{\max(a_1,b_1)} p_2^{\max(a_2,b_2)} \cdots p_n^{\max(a_n,b_n)}$$

This number is divided by both a and b and no smaller number is divided by a and b.

Example: $\text{lcm}(2^3 3^5 7^2, 2^4 3^3) = 2^{\max(3,4)} 3^{\max(5,3)} 7^{\max(2,0)} = 2^4 3^5 7^2$

- The greatest common divisor and the least common multiple of two integers are related by:

  Theorem 5: Let a and b be positive integers. Then

  $ab = \gcd(a,b) \cdot \text{lcm}(a,b)$

  (*proof is Exercise* 31)

Euclidean Algorithm



Euclid
(325 B.C.E. – 265 B.C.E.)

- The Euclidian algorithm is an efficient method for computing the greatest common divisor of two integers. It is based on the idea that gcd(*a*,*b*) is equal to gcd(*a*,*c*) when *a* > *b* and *c* is the remainder when a is divided by *b*.

Example: Find gcd(91, 287):

- $287 = 91 \cdot 3 + 14$
- $91 = 14 \cdot 6 + 7$
- $14 = 7 \cdot 2 + 0$

Divide 287 by 91
Divide 91 by 14
Divide 14 by 7
Stopping condition

$\gcd(287, 91) = \gcd(91, 14) = \gcd(14, 7) = 7$

continued →

Euclidean Algorithm

- The Euclidean algorithm expressed in pseudocode is:

procedure *gcd*(*a, b*: positive integers)
x := *a*
y := *b*
while $y \neq 0$
  r := *x* **mod** *y*
  x := *y*
  y := *r*
return *x* {gcd(*a*,*b*) is *x*}

- In Section 5.3, we'll see that the time complexity of the algorithm is $O(\log b)$, where $a > b$.

Correctness of Euclidean Algorithm

Lemma 1: Let $a = bq + r$, where a, b, q, and r are integers. Then $\gcd(a,b) = \gcd(b,r)$.

Proof:

- Suppose that d divides both a and b. Then d also divides $a - bq = r$ (by Theorem 1 of Section 4.1). Hence, any common divisor of a and b must also be any common divisor of b and r.
- Suppose that d divides both b and r. Then d also divides $bq + r = a$. Hence, any common divisor of a and b must also be a common divisor of b and r.
- Therefore, $\gcd(a,b) = \gcd(b,r)$. ◄

Correctness of Euclidean Algorithm

- Suppose that a and b are positive integers with $a \geq b$.
  Let $r_0 = a$ and $r_1 = b$.
  Successive applications of the division algorithm yields:

$$\begin{aligned} r_0 &= r_1 q_1 + r_2 & 0 \leq r_2 < r_1, \\ r_1 &= r_2 q_2 + r_3 & 0 \leq r_3 < r_2, \\ &\vdots \\ r_{n-2} &= r_{n-1} q_{n-1} + r_n & 0 \leq r_n < r_{n-1}, \\ r_{n-1} &= r_n q_n . \end{aligned}$$

- Eventually, a remainder of zero occurs in the sequence of terms: $a = r_0 > r_1 > r_2 > \cdots \geq 0$. The sequence can't contain more than a terms.
- By Lemma 1
  $\gcd(a,b) = \gcd(r_0,r_1) = \cdots = \gcd(r_{n-1},r_n) = \gcd(r_n, 0) = r_n$.
- Hence the greatest common divisor is the last nonzero remainder in the sequence of divisions. ◄

gcds as Linear Combinations

Étienne Bézout
(1730-1783)



Bézout's Theorem: If a and b are positive integers, then there exist integers s and t such that $\gcd(a,b) = sa + tb$.
(*proof in exercises of Section* 5.2)

Definition: If a and b are positive integers, then integers s and t such that $\gcd(a,b) = sa + tb$ are called *Bézout coefficients* of a and b. The equation $\gcd(a,b) = sa + tb$ is called *Bézout's identity.*

- By Bézout's Theorem, the gcd of integers a and b can be expressed in the form $sa + tb$ where s and t are integers. This is a *linear combination* with integer coefficients of a and b.
  - $\gcd(6,14) = (-2)\cdot 6 + 1\cdot 14$

Finding gcds as Linear Combinations

Example: Express $\gcd(252,198) = 18$ as a linear combination of 252 and 198.

Solution: First use the Euclidean algorithm to show $\gcd(252,198) = 18$

i. $252 = 1\cdot 198 + 54$
ii. $198 = 3\cdot 54 + 36$
iii. $54 = 1\cdot 36 + 18$
iv. $36 = 2\cdot 18$

- Now working backwards, from iii and i above
  - $18 = 54 - 1\cdot 36$
  - $36 = 198 - 3\cdot 54$
- Substituting the 2nd equation into the 1st yields:
  - $18 = 54 - 1\cdot(198 - 3\cdot 54) = 4\cdot 54 - 1\cdot 198$
- Substituting $54 = 252 - 1\cdot 198$ (from i)) yields:
  - $18 = 4\cdot(252 - 1\cdot 198) - 1\cdot 198 = 4\cdot 252 - 5\cdot 198$

- This method illustrated above is a two pass method. It first uses the Euclidian algorithm to find the gcd and then works backwards to express the gcd as a linear combination of the original two integers. A one pass method, called the *extended Euclidean algorithm*, is developed in the exercises.

Consequences of Bézout's Theorem

Lemma 2: If a, b, and c are positive integers such that $\gcd(a, b) = 1$ and $a \mid bc$, then $a \mid c$.

Proof: Assume $\gcd(a, b) = 1$ and $a \mid bc$

- Since $\gcd(a, b) = 1$, by Bézout's Theorem there are integers s and t such that $sa + tb = 1$.
- Multiplying both sides of the equation by c, yields $sac + tbc = c$.
- From Theorem 1 of Section 4.1:
  $a \mid tbc$ (part ii) and a divides $sac + tbc$ since $a \mid sac$ and $a \mid tbc$ (part i)
- We conclude $a \mid c$, since $sac + tbc = c$.

Lemma 3: If p is prime and $p \mid a_1 a_2 \cdots a_n$, then $p \mid a_i$ for some i. ◄
(*proof uses mathematical induction; see Exercise* 64 *of Section* 5.1)

- Lemma 3 is crucial in the proof of the uniqueness of prime factorizations.

Uniqueness of Prime Factorization

- We will prove that a prime factorization of a positive integer where the primes are in nondecreasing order is unique. (This part of the fundamental theorem of arithmetic. The other part, which asserts that every positive integer has a prime factorization into primes, will be proved in Section 5.2.)

Proof: (*by contradiction*) Suppose that the positive integer n can be written as a product of primes in two distinct ways:

$n = p_1 p_2 \cdots p_s$ and $n = q_1 q_2 \cdots p_t$.

- Remove all common primes from the factorizations to get
  $p_{i_1} p_{i_2} \cdots p_{i_u} = q_{j_1} q_{j_2} \cdots q_{j_v}$.
- By Lemma 3, it follows that p_{i_1} divides q_{j_k} for some k, contradicting the assumption that p_{i_1} and q_{j_k} are distinct primes.
- Hence, there can be at most one factorization of n into primes in nondecreasing order. ◄

Dividing Congruences by an Integer

- Dividing both sides of a valid congruence by an integer does not always produce a valid congruence (see Section 4.1).
- But dividing by an integer relatively prime to the modulus does produce a valid congruence:

Theorem 7: Let m be a positive integer and let a, b, and c be integers. If $ac \equiv bc \pmod{m}$ and $\gcd(c,m) = 1$, then $a \equiv b \pmod{m}$.

Proof: Since $ac \equiv bc \pmod{m}$, $m \mid ac - bc = c(a - b)$ by Lemma 2 and the fact that $\gcd(c,m) = 1$, it follows that $m \mid a - b$. Hence, $a \equiv b \pmod{m}$. ◄

Solving Congruences

Section 4.4

Section Summary

- Linear Congruences
- The Chinese Remainder Theorem
- Computer Arithmetic with Large Integers (*not currently included in slides, see text*)
- Fermat's Little Theorem
- Pseudoprimes
- Primitive Roots and Discrete Logarithms

Linear Congruences

Definition: A congruence of the form

$$ax \equiv b \pmod{m},$$

where m is a positive integer, a and b are integers, and x is a variable, is called a *linear congruence*.

- The solutions to a linear congruence $ax \equiv b \pmod{m}$ are all integers x that satisfy the congruence.

Definition: An integer $\bar{a}$ such that $\bar{a}a \equiv 1 \pmod{m}$ is said to be an *inverse* of a modulo m.

Example: 5 is an inverse of 3 modulo 7 since $5 \cdot 3 = 15 \equiv 1 \pmod{7}$

- One method of solving linear congruences makes use of an inverse $\bar{a}$, if it exists. Although we can not divide both sides of the congruence by a, we can multiply by $\bar{a}$ to solve for x.

Inverse of a modulo m

- The following theorem guarantees that an inverse of a modulo m exists whenever a and m are relatively prime. Two integers a and b are relatively prime when $\gcd(a,b) = 1$.

Theorem 1: If a and m are relatively prime integers and $m > 1$, then an inverse of a modulo m exists. Furthermore, this inverse is unique modulo m. (This means that there is a unique positive integer $\bar{a}$ less than m that is an inverse of a modulo m and every other inverse of a modulo m is congruent to $\bar{a}$ modulo m.)

Proof: Since $\gcd(a,m) = 1$, by Theorem 6 of Section 4.3, there are integers s and t such that $sa + tm = 1$.

- Hence, $sa + tm \equiv 1 \pmod{m}$.
- Since $tm \equiv 0 \pmod{m}$, it follows that $sa \equiv 1 \pmod{m}$
- Consequently, s is an inverse of a modulo m.
- The uniqueness of the inverse is Exercise 7.

◄

Finding Inverses

- The Euclidean algorithm and Bézout coefficients gives us a systematic approaches to finding inverses.

Example: Find an inverse of 3 modulo 7.

Solution: Because $\gcd(3,7) = 1$, by Theorem 1, an inverse of 3 modulo 7 exists.

- Using the Euclidian algorithm: $7 = 2 \cdot 3 + 1$.
- From this equation, we get $-2 \cdot 3 + 1 \cdot 7 = 1$, and see that -2 and 1 are Bézout coefficients of 3 and 7.
- Hence, -2 is an inverse of 3 modulo 7.
- Also every integer congruent to -2 modulo 7 is an inverse of 3 modulo 7, i.e., 5, -9, 12, etc.

Finding Inverses

Example: Find an inverse of 101 modulo 4620.

Solution: First use the Euclidian algorithm to show that $\gcd(101,4620) = 1$.

$$42620 = 45 \cdot 101 + 75$$
$$101 = 1 \cdot 75 + 26$$
$$75 = 2 \cdot 26 + 23$$
$$26 = 1 \cdot 23 + 3$$
$$23 = 7 \cdot 3 + 2$$
$$3 = 1 \cdot 2 + 1$$
$$2 = 2 \cdot 1$$

Since the last nonzero remainder is 1, $\gcd(101,4260) = 1$

Working Backwards:

$$1 = 3 - 1 \cdot 2$$
$$1 = 3 - 1 \cdot (23 - 7 \cdot 3) = -1 \cdot 23 + 8 \cdot 3$$
$$1 = -1 \cdot 23 + 8 \cdot (26 - 1 \cdot 23) = 8 \cdot 26 - 9 \cdot 23$$
$$1 = 8 \cdot 26 - 9 \cdot (75 - 2 \cdot 26) = 26 \cdot 26 - 9 \cdot 75$$
$$1 = 26 \cdot (101 - 1 \cdot 75) - 9 \cdot 75$$
$$= 26 \cdot 101 - 35 \cdot 75$$
$$1 = 26 \cdot 101 - 35 \cdot (42620 - 45 \cdot 101)$$
$$= -35 \cdot 42620 + 1601 \cdot 101$$

Bézout coefficients : -35 and 1601

1601 is an inverse of 101 modulo 42620

Using Inverses to Solve Congruences

- We can solve the congruence $ax \equiv b \pmod{m}$ by multiplying both sides by $\bar{a}$.

Example: What are the solutions of the congruence $3x \equiv 4 \pmod{7}$.

Solution: We found that -2 is an inverse of 3 modulo 7 (two slides back). We multiply both sides of the congruence by -2 giving

$$-2 \cdot 3x \equiv -2 \cdot 4 \pmod{7}.$$

Because $-6 \equiv 1 \pmod{7}$ and $-8 \equiv 6 \pmod{7}$, it follows that if x is a solution, then $x \equiv -8 \equiv 6 \pmod{7}$

We need to determine if every x with $x \equiv 6 \pmod{7}$ is a solution. Assume that $x \equiv 6 \pmod{7}$. By Theorem 5 of Section 4.1, it follows that $3x \equiv 3 \cdot 6 = 18 \equiv 4 \pmod{7}$ which shows that all such x satisfy the congruence.

The solutions are the integers x such that $x \equiv 6 \pmod{7}$, namely, 6,13,20 … and $-1, -8, -15$,…

The Chinese Remainder Theorem

- In the first century, the Chinese mathematician Sun-Tsu asked:

  There are certain things whose number is unknown. When divided by 3, the remainder is 2; when divided by 5, the remainder is 3; when divided by 7, the remainder is 2. What will be the number of things?
- This puzzle can be translated into the solution of the system of congruences:

  $x \equiv 2 \pmod 3$,
  $x \equiv 3 \pmod 5$,
  $x \equiv 2 \pmod 7$?
- We'll see how the theorem that is known as the *Chinese Remainder Theorem* can be used to solve Sun-Tsu's problem.

The Chinese Remainder Theorem

Theorem 2: (*The Chinese Remainder Theorem*) Let $m_1, m_2, \ldots, m_n$ be pairwise relatively prime positive integers greater than one and $a_1, a_2, \ldots, a_n$ arbitrary integers. Then the system

$x \equiv a_1 \pmod{m_1}$
$x \equiv a_2 \pmod{m_2}$
.
.
.
$x \equiv a_n \pmod{m_n}$

has a unique solution modulo $m = m_1 m_2 \cdots m_n$.

(That is, there is a solution x with $0 \leq x < m$ and all other solutions are congruent modulo m to this solution.)

- **Proof**: We'll show that a solution exists by describing a way to construct the solution. Showing that the solution is unique modulo m is Exercise 30.

continued →

The Chinese Remainder Theorem

To construct a solution first let $M_k = m/m_k$ for $k = 1,2,\ldots,n$ and $m = m_1 m_2 \cdots m_n$.

Since $\gcd(m_k, M_k) = 1$, by Theorem 1, there is an integer y_k, an inverse of M_k modulo m_k, such that

$M_k y_k \equiv 1 \pmod{m_k}$.

Form the sum

$x = a_1 M_1 y_1 + a_2 M_2 y_2 + \cdots + a_n M_n y_n$.

Note that because $M_j \equiv 0 \pmod{m_k}$ whenever $j \neq k$, all terms except the kth term in this sum are congruent to 0 modulo m_k.

Because $M_k y_k \equiv 1 \pmod{m_k}$, we see that $x \equiv a_k M_k y_k \equiv a_k \pmod{m_k}$, for $k = 1,2,\ldots,n$.

Hence, x is a simultaneous solution to the n congruences.

$x \equiv a_1 \pmod{m_1}$
$x \equiv a_2 \pmod{m_2}$
.
.
.
$x \equiv a_n \pmod{m_n}$

◄

The Chinese Remainder Theorem

Example: Consider the 3 congruences from Sun-Tsu's problem:

$x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$.

- Let $m = 3 \cdot 5 \cdot 7 = 105$, $M_1 = m/3 = 35$, $M_3 = m/5 = 21$, $M_3 = m/7 = 15$.
- We see that
  - 2 is an inverse of $M_1 = 35$ modulo 3 since $35 \cdot 2 \equiv 2 \cdot 2 \equiv 1 \pmod 3$
  - 1 is an inverse of $M_2 = 21$ modulo 5 since $21 \equiv 1 \pmod 5$
  - 1 is an inverse of $M_3 = 15$ modulo 7 since $15 \equiv 1 \pmod 7$
- Hence,

$$x = a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3$$
$$= 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 = 233 \equiv 23 \pmod{105}$$

- We have shown that 23 is the smallest positive integer that is a simultaneous solution. Check it!

Back Substitution

- We can also solve systems of linear congruences with pairwise relatively prime moduli by rewriting a congruences as an equality using Theorem 4 in Section 4.1, substituting the value for the variable into another congruence, and continuing the process until we have worked through all the congruences. This method is known as *back substitution*.

  Example: Use the method of back substitution to find all integers x such that $x \equiv 1 \pmod 5$, $x \equiv 2 \pmod 6$, and $x \equiv 3 \pmod 7$.

  Solution: By Theorem 4 in Section 4.1, the first congruence can be rewritten as $x = 5t + 1$, where t is an integer.
  - Substituting into the second congruence yields $5t + 1 \equiv 2 \pmod 6$.
  - Solving this tells us that $t \equiv 5 \pmod 6$.
  - Using Theorem 4 again gives $t = 6u + 5$ where u is an integer.
  - Substituting this back into $x = 5t + 1$, gives $x = 5(6u + 5) + 1 = 30u + 26$.
  - Inserting this into the third equation gives $30u + 26 \equiv 3 \pmod 7$.
  - Solving this congruence tells us that $u \equiv 6 \pmod 7$.
  - By Theorem 4, $u = 7v + 6$, where v is an integer.
  - Substituting this expression for u into $x = 30u + 26$, tells us that $x = 30(7v + 6) + 26 = 210u + 206$.

  Translating this back into a congruence we find the solution $x \equiv 206 \pmod{210}$.

Fermat's Little Theorem



Pierre de Fermat
(1601-1665)

Theorem 3: (*Fermat's Little Theo*rem) If p is prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod p$

Furthermore, for every integer a we have $a^p \equiv a \pmod p$

(*proof outlined in Exercise 19*)

Fermat's little theorem is useful in computing the remainders modulo p of large powers of integers.

Example: Find 7^{222} **mod** 11.

By Fermat's little theorem, we know that $7^{10} \equiv 1 \pmod{11}$, and so $(7^{10})^k \equiv 1 \pmod{11}$, for every positive integer k. Therefore,

$$7^{222} = 7^{22 \cdot 10 + 2} = (7^{10})^{22} 7^2 \equiv (1)^{22} \cdot 49 \equiv 5 \pmod{11}.$$

Hence, 7^{222} **mod** $11 = 5$.

Pseudoprimes

- By Fermat's little theorem $n > 2$ is prime, where

  $2^{n-1} \equiv 1 \pmod n$.
- But if this congruence holds, n may not be prime. Composite integers n such that $2^{n-1} \equiv 1 \pmod n$ are called *pseudoprimes* to the base 2.

  Example: The integer 341 is a pseudoprime to the base 2.

  $341 = 11 \cdot 31$
  $2^{340} \equiv 1 \pmod{341}$ (*see in Exercise* 37)
- We can replace 2 by any integer $b \geq 2$.

  Definition: Let b be a positive integer. If n is a composite integer, and $b^{n-1} \equiv 1 \pmod n$, then n is called a *pseudoprime to the base b*.

Pseudoprimes

- Given a positive integer n, such that $2^{n-1} \equiv 1 \pmod n$:
  - If n does not satisfy the congruence, it is composite.
  - If n does satisfy the congruence, it is either prime or a pseudoprime to the base 2.
- Doing similar tests with additional bases b, provides more evidence as to whether n is prime.
- Among the positive integers not exceeding a positive real number x, compared to primes, there are relatively few pseudoprimes to the base b.
  - For example, among the positive integers less than 10^{10} there are 455,052,512 primes, but only 14,884 pseudoprimes to the base 2.

Carmichael Numbers (*optional*)



Robert Carmichael (1879-1967)

- There are composite integers n that pass all tests with bases b such that $\gcd(b,n) = 1$.

  Definition: A composite integer n that satisfies the congruence $b^{n-1} \equiv 1 \pmod{n}$ for all positive integers b with $\gcd(b,n) = 1$ is called a *Carmichael* number.

  Example: The integer 561 is a Carmichael number. To see this:
  - 561 is composite, since $561 = 3 \cdot 11 \cdot 13$.
  - If $\gcd(b, 561) = 1$, then $\gcd(b, 3) = 1$, then $\gcd(b, 11) = \gcd(b, 17) = 1$.
  - Using Fermat's Little Theorem: $b^2 \equiv 1 \pmod 3$, $b^{10} \equiv 1 \pmod{11}$, $b^{16} \equiv 1 \pmod{17}$.
  - Then
    $b^{560} = (b^2)^{280} \equiv 1 \pmod 3$,
    $b^{560} = (b^{10})^{56} \equiv 1 \pmod{11}$,
    $b^{560} = (b^{16})^{35} \equiv 1 \pmod{17}$.
  - It follows (*see Exercise* 29) that $b^{560} \equiv 1 \pmod{561}$ for all positive integers b with $\gcd(b,561) = 1$. Hence, 561 is a Carmichael number.
- Even though there are infinitely many Carmichael numbers, there are other tests (described in the exercises) that form the basis for efficient probabilistic primality testing. (*see Chapter* 7)

Primitive Roots

Definition: A primitive root modulo a prime p is an integer r in $\mathbf{Z}_p$ such that every nonzero element of $\mathbf{Z}_p$ is a power of r.

Example: Since every element of $\mathbf{Z}_{11}$ is a power of 2, 2 is a primitive root of 11.

Powers of 2 modulo 11: $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 5$, $2^5 = 10$, $2^6 = 9$, $2^7 = 7$, $2^8 = 3$, $2^{10} = 2$.

Example: Since not all elements of $\mathbf{Z}_{11}$ are powers of 3, 3 is not a primitive root of 11.

Powers of 3 modulo 11: $3^1 = 3$, $3^2 = 9$, $3^3 = 5$, $3^4 = 4$, $3^5 = 1$, and the pattern repeats for higher powers.

Important Fact: There is a primitive root modulo p for every prime number p.

Discrete Logarithms

Suppose p is prime and r is a primitive root modulo p. If a is an integer between 1 and $p-1$, that is an element of $\mathbf{Z}_p$, there is a unique exponent e such that $r^e = a$ in $\mathbf{Z}_p$, that is, $r^e \bmod p = a$.

Definition: Suppose that p is prime, r is a primitive root modulo p, and a is an integer between 1 and $p-1$, inclusive. If $r^e \bmod p = a$ and $1 \le e \le p-1$, we say that e is the *discrete logarithm* of a modulo p to the base r and we write $\log_r a = e$ (where the prime p is understood).

Example 1: We write $\log_2 3 = 8$ since the discrete logarithm of 3 modulo 11 to the base 2 is 8 as $2^8 = 3$ modulo 11.

Example 2: We write $\log_2 5 = 4$ since the discrete logarithm of 5 modulo 11 to the base 2 is 4 as $2^4 = 5$ modulo 11.

There is no known polynomial time algorithm for computing the discrete logarithm of a modulo p to the base r (when given the prime p, a root r modulo p, and a positive integer $a \in \mathbf{Z}_p$). The problem plays a role in cryptography as will be discussed in Section 4.6.

Applications of Congruences

Section 4.5

Section Summary

- Hashing Functions
- Pseudorandom Numbers
- Check Digits

Hashing Functions

Definition: A *hashing function* h assigns memory location $h(k)$ to the record that has k as its key.
- A common hashing function is $h(k) = k \bmod m$, where m is the number of memory locations.
- Because this hashing function is onto, all memory locations are possible.

Example: Let $h(k) = k \bmod 111$. This hashing function assigns the records of customers with social security numbers as keys to memory locations in the following manner:

$h(064212848) = 064212848 \bmod 111 = 14$
$h(037149212) = 037149212 \bmod 111 = 65$
$h(107405723) = 107405723 \bmod 111 = 14$, but since location 14 is already occupied, the record is assigned to the next available position, which is 15.

- The hashing function is not one-to-one as there are many more possible keys than memory locations. When more than one record is assigned to the same location, we say a *collision* occurs. Here a collision has been resolved by assigning the record to the first free location.
- For collision resolution, we can use a *linear probing function*:

  $h(k,i) = (h(k) + i) \bmod m$, where i runs from 0 to $m-1$.
- There are many other methods of handling with collisions. You may cover these in a later CS course.

Pseudorandom Numbers

- Randomly chosen numbers are needed for many purposes, including computer simulations.
- *Pseudorandom numbers* are not truly random since they are generated by systematic methods.
- The *linear congruential method* is one commonly used procedure for generating pseudorandom numbers.
- Four integers are needed: the *modulus* m, the *multiplier* a, the *increment* c, and *seed* x_0, with $2 \le a < m$, $0 \le c < m$, $0 \le x_0 < m$.
- We generate a sequence of pseudorandom numbers $\{x_n\}$, with $0 \le x_n < m$ for all n, by successively using the recursively defined function

  $x_{n+1} = (ax_n + c) \bmod m$.

  (*an example of a recursive definition, discussed in Section* 5.3)
- If psudorandom numbers between 0 and 1 are needed, then the generated numbers are divided by the modulus, x_n/m.

Pseudorandom Numbers

- **Example**: Find the sequence of pseudorandom numbers generated by the linear congruential method with modulus $m = 9$, multiplier $a = 7$, increment $c = 4$, and seed $x_0 = 3$.
- **Solution**: Compute the terms of the sequence by successively using the congruence $x_{n+1} = (7x_n + 4) \bmod 9$, with $x_0 = 3$.

  $x_1 = 7x_0 + 4 \bmod 9 = 7 \cdot 3 + 4 \bmod 9 = 25 \bmod 9 = 7$,
  $x_2 = 7x_1 + 4 \bmod 9 = 7 \cdot 7 + 4 \bmod 9 = 53 \bmod 9 = 8$,
  $x_3 = 7x_2 + 4 \bmod 9 = 7 \cdot 8 + 4 \bmod 9 = 60 \bmod 9 = 6$,
  $x_4 = 7x_3 + 4 \bmod 9 = 7 \cdot 6 + 4 \bmod 9 = 46 \bmod 9 = 1$,
  $x_5 = 7x_4 + 4 \bmod 9 = 7 \cdot 1 + 4 \bmod 9 = 11 \bmod 9 = 2$,
  $x_6 = 7x_5 + 4 \bmod 9 = 7 \cdot 2 + 4 \bmod 9 = 18 \bmod 9 = 0$,
  $x_7 = 7x_6 + 4 \bmod 9 = 7 \cdot 0 + 4 \bmod 9 = 4 \bmod 9 = 4$,
  $x_8 = 7x_7 + 4 \bmod 9 = 7 \cdot 4 + 4 \bmod 9 = 32 \bmod 9 = 5$,
  $x_9 = 7x_8 + 4 \bmod 9 = 7 \cdot 5 + 4 \bmod 9 = 39 \bmod 9 = 3$.

  The sequence generated is 3,7,8,6,1,2,0,4,5,3,7,8,6,1,2,0,4,5,3,...
  It repeats after generating 9 terms.
- Commonly, computers use a linear congruential generator with increment $c = 0$. This is called a *pure multiplicative generator*. Such a generator with modulus $2^{31} - 1$ and multiplier $7^5 = 16{,}807$ generates $2^{31} - 2$ numbers before repeating.

Check Digits: UPCs

- A common method of detecting errors in strings of digits is to add an extra digit at the end, which is evaluated using a function. If the final digit is not correct, then the string is assumed not to be correct.

Example: Retail products are identified by their *Universal Product Codes* (*UPC*s). Usually these have 12 decimal digits, the last one being the check digit. The check digit is determined by the congruence:

$3x_1 + x_2 + 3x_3 + x_4 + 3x_5 + x_6 + 3x_7 + x_8 + 3x_9 + x_{10} + 3x_{11} + x_{12} \equiv 0 \pmod{10}$.

a. Suppose that the first 11 digits of the UPC are 79357343104. What is the check digit?

b. Is 041331021641 a valid UPC?

Solution:

a. $3\cdot7 + 9 + 3\cdot3 + 5 + 3\cdot7 + 3 + 3\cdot4 + 3 + 3\cdot1 + 0 + 3\cdot4 + x_{12} \equiv 0 \pmod{10}$
$21 + 9 + 9 + 5 + 21 + 3 + 12 + 3 + 3 + 0 + 12 + x_{12} \equiv 0 \pmod{10}$
$98 + x_{12} \equiv 0 \pmod{10}$
$x_{12} \equiv 2 \pmod{10}$ So, the check digit is 2.

b. $3\cdot0 + 4 + 3\cdot1 + 3 + 3\cdot3 + 1 + 3\cdot0 + 2 + 3\cdot1 + 6 + 3\cdot4 + 1 \equiv 0 \pmod{10}$
$0 + 4 + 3 + 3 + 9 + 1 + 0 + 2 + 3 + 6 + 12 + 1 = 44 \equiv 4 \not\equiv 0 \pmod{10}$
Hence, 041331021641 is not a valid UPC.

Check Digits:ISBNs

Books are identified by an *International Standard Book Number* (ISBN-10), a 10 digit code. The first 9 digits identify the language, the publisher, and the book. The tenth digit is a check digit, which is determined by the following congruence

$$x_{10} \equiv \sum_{i=1}^{9} i x_i \pmod{11}.$$

The validity of an ISBN-10 number can be evaluated with the equivalent

$$\sum_{i=1}^{10} i x_i \equiv 0 \pmod{11}.$$

a. Suppose that the first 9 digits of the ISBN-10 are 007288008. What is the check digit?

b. Is 084930149X a valid ISBN10?

Solution:

a. $x_{10} \equiv 1\cdot0 + 2\cdot0 + 3\cdot7 + 4\cdot2 + 5\cdot8 + 6\cdot8 + 7\cdot0 + 8\cdot0 + 9\cdot8 \pmod{11}$.
$x_{10} \equiv 0 + 0 + 21 + 8 + 40 + 48 + 0 + 0 + 72 \pmod{11}$.
$x_{10} \equiv 189 \equiv 2 \pmod{11}$. Hence, $x_{10} = 2$.

b. $1\cdot0 + 2\cdot8 + 3\cdot4 + 4\cdot9 + 5\cdot3 + 6\cdot0 + 7\cdot1 + 8\cdot4 + 9\cdot9 + 10\cdot10 =$
$0 + 16 + 12 + 36 + 15 + 0 + 7 + 32 + 81 + 100 = 299 \equiv 2 \not\equiv 0 \pmod{11}$
Hence, 084930149X is not a valid ISBN-10.

X is used for the digit 10.

- A *single error* is an error in one digit of an identification number and a *transposition error* is the accidental interchanging of two digits. Both of these kinds of errors can be detected by the check digit for ISBN-10. (*see text for more details*)

Cryptography

Section 4.6

Section Summary

- Classical Cryptography
- Cryptosystems
- Public Key Cryptography
- RSA Cryptosystem
- Crytographic Protocols
- Primitive Roots and Discrete Logarithms

Caesar Cipher



Julius Caesar created secret messages by shifting each letter three letters forward in the alphabet (sending the last three letters to the first three letters.) For example, the letter B is replaced by E and the letter X is replaced by A. This process of making a message secret is an example of *encryption*.

Here is how the encryption process works:

- Replace each letter by an integer from $\mathbf{Z}_{26}$, that is an integer from 0 to 25 representing one less than its position in the alphabet.
- The encryption function is $f(p) = (p + 3) \bmod 26$. It replaces each integer p in the set $\{0,1,2,\ldots,25\}$ by $f(p)$ in the set $\{0,1,2,\ldots,25\}$.
- Replace each integer p by the letter with the position $p + 1$ in the alphabet.

Example: Encrypt the message "MEET YOU IN THE PARK" using the Caesar cipher.

Solution: 12 4 4 19 24 14 20 8 13 19 7 4 15 0 17 10.

Now replace each of these numbers p by $f(p) = (p + 3) \bmod 26$.

15 7 7 22 1 17 23 11 16 22 10 7 18 3 20 13.

Translating the numbers back to letters produces the encrypted message

"PHHW BRX LQ WKH SDUN."

Caesar Cipher

- To recover the original message, use $f^{-1}(p) = (p-3) \bmod 26$. So, each letter in the coded message is shifted back three letters in the alphabet, with the first three letters sent to the last three letters. This process of recovering the original message from the encrypted message is called *decryption*.
- The Caesar cipher is one of a family of ciphers called *shift ciphers.* Letters can be shifted by an integer k, with 3 being just one possibility. The encryption function is
$f(p) = (p + k) \bmod 26$
and the decryption function is
$f^{-1}(p) = (p-k) \bmod 26$
The integer k is called a *key*.

Shift Cipher

Example 1: Encrypt the message "STOP GLOBAL WARMING" using the shift cipher with $k = 11$.

Solution: Replace each letter with the corresponding element of $\mathbf{Z}_{26}$.

18 19 14 15 6 11 14 1 0 11 22 0 17 12 8 13 6.

Apply the shift $f(p) = (p + 11) \bmod 26$, yielding

3 4 25 0 17 22 25 12 11 22 7 11 2 23 19 24 17.

Translating the numbers back to letters produces the ciphertext

"DEZA RWZMLW HLCXTYR."

Shift Cipher

Example 2: Decrypt the message "LEWLYPLUJL PZ H NYLHA ALHJOLY" that was encrypted using the shift cipher with $k = 7$.

Solution: Replace each letter with the corresponding element of $\mathbf{Z}_{26}$.

11 4 22 11 24 15 11 20 9 11 15 25 7 13 24 11 7 0 0 11 7 9 14 11 24.

Shift each of the numbers by $-k = -7$ modulo 26, yielding

4 23 15 4 17 8 4 13 2 4 8 18 0 6 17 4 0 19 19 4 0 2 7 4 17.

Translating the numbers back to letters produces the decrypted message

"EXPERIENCE IS A GREAT TEACHER."

Affine Ciphers

- Shift ciphers are a special case of *affine ciphers* which use functions of the form

  $f(p) = (ap + b) \bmod 26$,

  where a and b are integers, chosen so that f is a bijection.

  The function is a bijection if and only if $\gcd(a,26) = 1$.
- **Example**: What letter replaces the letter K when the function $f(p) = (7p + 3) \bmod 26$ is used for encryption.

  Solution: Since 10 represents K, $f(10) = (7 \cdot 10 + 3) \bmod 26 = 21$, which is then replaced by V.
- To decrypt a message encrypted by a shift cipher, the congruence $c \equiv ap + b \pmod{26}$ needs to be solved for p.
  - Subtract b from both sides to obtain $c - b \equiv ap \pmod{26}$.
  - Multiply both sides by the inverse of a modulo 26, which exists since $\gcd(a,26) = 1$.
  - $\bar{a}(c - b) \equiv \bar{a}ap \pmod{26}$, which simplifies to $\bar{a}(c - b) \equiv p \pmod{26}$.
  - $p \equiv \bar{a}(c - b) \pmod{26}$ is used to determine p in $\mathbf{Z}_{26}$.

Cryptanalysis of Affine Ciphers

- The process of recovering plaintext from ciphertext without knowledge both of the encryption method and the key is known as *cryptanalysis* or *breaking codes*.
- An important tool for cryptanalyzing ciphertext produced with a affine ciphers is the relative frequencies of letters. The nine most common letters in the English texts are E 13%, T 9%, A 8%, O 8%, I 7%, N 7%, S 7%, H 6%, and R 6%.
- To analyze ciphertext:
  - Find the frequency of the letters in the ciphertext.
  - Hypothesize that the most frequent letter is produced by encrypting E.
  - If the value of the shift from E to the most frequent letter is k, shift the ciphertext by $-k$ and see if it makes sense.
  - If not, try T as a hypothesis and continue.
- **Example**: We intercepted the message "ZNK KGXRE HOXJ MKZY ZNK CUXS" that we know was produced by a shift cipher. Let's try to cryptanalyze.
- **Solution**: The most common letter in the ciphertext is K. So perhaps the letters were shifted by 6 since this would then map E to K. Shifting the entire message by -6 gives us "THE EARLY BIRD GETS THE WORM."

Block Ciphers

- Ciphers that replace each letter of the alphabet by another letter are called *character* or *monoalphabetic* ciphers.
- They are vulnerable to cryptanalysis based on letter frequency. *Block ciphers* avoid this problem, by replacing blocks of letters with other blocks of letters.
- A simple type of block cipher is called the *transposition cipher*. The key is a permutation σ of the set $\{1,2,\ldots,m\}$, where m is an integer, that is a one-to-one function from $\{1,2,\ldots,m\}$ to itself.
- To encrypt a message, split the letters into blocks of size m, adding additional letters to fill out the final block. We encrypt $p_1,p_2,\ldots,p_m$ as $c_1,c_2,\ldots,c_m = p_{\sigma(1)},p_{\sigma(2)},\ldots,p_{\sigma(m)}$.
- To decrypt the $c_1,c_2,\ldots,c_m$ transpose the letters using the inverse permutation σ^{-1}.

Block Ciphers

Example: Using the transposition cipher based on the permutation σ of the set $\{1,2,3,4\}$ with $\sigma(1) = 3, \sigma(2) = 1, \sigma(3) = 4, \sigma(4) = 2$,

a. Encrypt the plaintext PIRATE ATTACK
b. Decrypt the ciphertext message SWUE TRAEOEHS, which was encryted using the same cipher.

Solution:

a. Split into four blocks PIRA TEAT TACK.
   Apply the permutation σ giving IAPR ETTA AKTC.
b. $\sigma^{-1}: \sigma^{-1}(1) = 2, \sigma^{-1}(2) = 4, \sigma^{-1}(3) = 1, \sigma^{-1}(4) = 3.$
   Apply the permutation σ^{-1} giving USEW ATER HOSE.
   Split into words to obtain USE WATER HOSE.

Cryptosystems

Definition: A *cryptosystem* is a five-tuple $(\mathcal{P},\mathcal{C},\mathcal{K},\mathcal{E},\mathcal{D})$, where

- $\mathcal{P}$ is the set of plainntext strings,
- $\mathcal{C}$ is the set of ciphertext strings,
- $\mathcal{K}$ is the *keyspace* (set of all possible keys),
- $\mathcal{E}$ is the set of encription functions, and
- $\mathcal{D}$ is the set of decryption functions.

- The encryption function in $\mathcal{E}$ corresponding to the key k is denoted by E_k and the decription function in $\mathcal{D}$ that decrypts cipher text enrypted using E_k is denoted by D_k. Therefore:

  $D_k(E_k(p)) = p$, for all plaintext strings p.

Cryptosystems

Example: Describe the family of shift ciphers as a cryptosystem.

Solution: Assume the messages are strings consisting of elements in $\mathbf{Z}_{26}$.

- $\mathcal{P}$ is the set of strings of elements in $\mathbf{Z}_{26}$,
- $\mathcal{C}$ is the set of strings of elements in $\mathbf{Z}_{26}$,
- $\mathcal{K} = \mathbf{Z}_{26}$,
- $\mathcal{E}$ consists of functions of the form $E_k(p) = (p + k) \bmod 26$, and
- $\mathcal{D}$ is the same as $\mathcal{E}$ where $D_k(p) = (p - k) \bmod 26$.

Public Key Cryptography

- All classical ciphers, including shift and affine ciphers, are *private key cryptosystems*. Knowing the encryption key allows one to quickly determine the decryption key.
- All parties who wish to communicate using a private key cryptosystem must share the key and keep it a secret.
- In public key cryptosystems, first invented in the 1970s, knowing how to encrypt a message does not help one to decrypt the message. Therefore, everyone can have a publicly known encryption key. The only key that needs to be kept secret is the decryption key.

The RSA Cryptosys



Clifford Cocks
(Born 1950)

- A public key cryptosystem, now known as the RSA system was introduced in 1976 by three researchers at MIT.

Ronald Rivest (Born 1948)



Adi Shamir (Born 1952)



Leonard Adelman (Born 1945)



- It is now known that the method was discovered earlier by Clifford Cocks, working secretly for the UK government.
- The public encryption key is (n,e), where $n = pq$ (the modulus) is the product of two large (200 digits) primes p and q, and an exponent e that is relatively prime to $(p-1)(q-1)$. The two large primes can be quickly found using probabilistic primality tests, discussed earlier. But $n = pq$, with approximately 400 digits, cannot be factored in a reasonable length of time.

RSA Encryption

- To encrypt a message using RSA using a key (n,e) :
  i. Translate the plaintext message M into sequences of two digit integers representing the letters. Use 00 for A, 01 for B, etc.
  ii. Concatenate the two digit integers into strings of digits.
  iii. Divide this string into equally sized blocks of $2N$ digits where $2N$ is the largest even number $2525\ldots25$ with $2N$ digits that does not exceed n.
  iv. The plaintext message M is now a sequence of integers $m_1, m_2, \ldots, m_k$.
  v. Each block (an integer) is encrypted using the function $C = M^e \bmod n$.

Example: Encrypt the message STOP using the RSA cryptosystem with key(2537,13).
- $2537 = 43 \cdot 59$,
- $p = 43$ and $q = 59$ are primes and $\gcd(e,(p-1)(q-1)) = \gcd(13, 42 \cdot 58) = 1$.

Solution: Translate the letters in STOP to their numerical equivalents 18 19 14 15.
- Divide into blocks of four digits (because $2525 < 2537 < 252525$) to obtain 1819 1415.
- Encrypt each block using the mapping $C = M^{13} \bmod 2537$.
- Since $1819^{13} \bmod 2537 = 2081$ and $1415^{13} \bmod 2537 = 2182$, the encrypted message is 2081 2182.

RSA Decryption

- To decrypt a RSA ciphertext message, the decryption key d, an inverse of e modulo $(p-1)(q-1)$ is needed. The inverse exists since $\gcd(e,(p-1)(q-1)) = \gcd(13, 42 \cdot 58) = 1$.
- With the decryption key d, we can decrypt each block with the computation $M = C^d \bmod p \cdot q$. (*see text for full derivation*)
- RSA works as a public key system since the only known method of finding d is based on a factorization of n into primes. There is currently no known feasible method for factoring large numbers into primes.

Example: The message 0981 0461 is received. What is the decrypted message if it was encrypted using the RSA cipher from the previous example.

Solution: The message was encrypted with $n = 43 \cdot 59$ and exponent 13. An inverse of 13 modulo $42 \cdot 58 = 2436$ (*exercise 2 in Section 4.4*) is $d = 937$.
- To decrypt a block C, $M = C^{937} \bmod 2537$.
- Since $0981^{937} \bmod 2537 = 0704$ and $0461^{937} \bmod 2537 = 1115$, the decrypted message is 0704 1115. Translating back to English letters, the message is HELP.

Cryptographic Protocols: Key Exchange

- *Cryptographic protocols* are exchanges of messages carried out by two or more parties to achieve a particular security goal.
- *Key exchange* is a protocol by which two parties can exchange a secret key over an insecure channel without having any past shared secret information. Here the *Diffe-Hellman key agreement protocol* is described by example.
  i. Suppose that Alice and Bob want to share a common key.
  ii. Alice and Bob agree to use a prime p and a primitive root a of p.
  iii. Alice chooses a secret integer k_1 and sends $a^{k1} \bmod p$ to Bob.
  iv. Bob chooses a secret integer k_2 and sends $a^{k2} \bmod p$ to Alice.
  v. Alice computes $(a^{k2})^{k1} \bmod p$.
  vi. Bob computes $(a^{k1})^{k2} \bmod p$.

At the end of the protocol, Alice and Bob have their shared key

$$(a^{k2})^{k1} \bmod p = (a^{k1})^{k2} \bmod p.$$

- To find the secret information from the public information would require the adversary to find k_1 and k_2 from $a^{k1} \bmod p$ and $a^{k2} \bmod p$ respectively. This is an instance of the discrete logarithm problem, considered to be computationally infeasible when p and a are sufficiently large.

Cryptographic Protocols: Digital Signatures

Adding a *digital signature* to a message is a way of ensuring the recipient that the message came from the purported sender.

- Suppose that Alice's RSA public key is (n,e) and her private key is d. Alice encrypts a plain text message x using $E_{(n,e)}(x) = x^d \bmod n$. She decrypts a ciphertext message y using $D_{(n,e)}(y) = y^d \bmod n$.
- Alice wants to send a message M so that everyone who receives the message knows that it came from her.
  1. She translates the message to numerical equivalents and splits into blocks, just as in RSA encryption.
  2. She then applies her decryption function $D_{(n,e)}$ to the blocks and sends the results to all intended recipients.
  3. The recipients apply Alice's encryption function and the result is the original plain text since $E_{(n,e)}(D_{(n,e)}(x)) = x$.

Everyone who receives the message can then be certain that it came from Alice.

Cryptographic Protocols: Digital Signatures

Example: Suppose Alice's RSA cryptosystem is the same as in the earlier example with key(2537,13), $2537 = 43 \cdot 59$, $p = 43$ and $q = 59$ are primes and $\gcd(e,(p-1)(q-1)) = \gcd(13, 42 \cdot 58) = 1$.

Her decryption key is d = 937.

She wants to send the message "MEET AT NOON" to her friends so that they can be certain that the message is from her.

Solution: Alice translates the message into blocks of digits 1204 0419 0019 1314 1413.
1. She then applies her decryption transformation $D_{(2537,13)}(x) = x^{937} \bmod 2537$ to each block.
2. She finds (using her laptop, programming skills, and knowledge of discrete mathematics) that $1204^{937} \bmod 2537 = 817$, $419^{937} \bmod 2537 = 555$, $19^{937} \bmod 2537 = 1310$, $1314^{937} \bmod 2537 = 2173$, and $1413^{937} \bmod 2537 = 1026$.
3. She sends 0817 0555 1310 2173 1026.

When one of her friends receive the message, they apply Alice's encryption transformation $E_{(2537,13)}$ to each block. They then obtain the original message which they translate back to English letters.

Induction and recursion

Chapter 5

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Mathematical Induction
- Strong Induction
- Well-Ordering
- Recursive Definitions
- Structural Induction
- Recursive Algorithms
- Program Correctness (*not yet included in overheads*)

Mathematical Induction

Section 5.1

Section Summary

- Mathematical Induction
- Examples of Proof by Mathematical Induction
- Mistaken Proofs by Mathematical Induction
- Guidelines for Proofs by Mathematical Induction

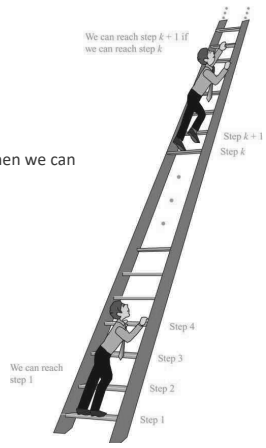
Climbing an Infinite Ladder

Suppose we have an infinite ladder:

1. We can reach the first rung of the ladder.
2. If we can reach a particular rung of the ladder, then we can reach the next rung.

From (1), we can reach the first rung. Then by applying (2), we can reach the second rung. Applying (2) again, the third rung. And so on. We can apply (2) any number of times to reach any particular rung, no matter how high up.

This example motivates proof by mathematical induction.



Principle of Mathematical Induction

Principle of Mathematical Induction: To prove that $P(n)$ is true for all positive integers n, we complete these steps:

- *Basis Step*: Show that $P(1)$ is true.
- *Inductive Step*: Show that $P(k) \rightarrow P(k+1)$ is true for all positive integers k.

To complete the inductive step, assuming the *inductive hypothesis* that $P(k)$ holds for an arbitrary integer k, show that must $P(k+1)$ be true.

Climbing an Infinite Ladder Example:

- BASIS STEP: By (1), we can reach rung 1.
- INDUCTIVE STEP: Assume the inductive hypothesis that we can reach rung k. Then by (2), we can reach rung $k+1$.

Hence, $P(k) \rightarrow P(k+1)$ is true for all positive integers k. We can reach every rung on the ladder. ◄

Important Points About Using Mathematical Induction

- Mathematical induction can be expressed as the rule of inference

$$(P(1) \wedge \forall k\,(P(k) \rightarrow P(k+1))) \rightarrow \forall n\,P(n),$$

where the domain is the set of positive integers.

- In a proof by mathematical induction, we don't assume that $P(k)$ is true for all positive integers! We show that if we assume that $P(k)$ is true, then $P(k+1)$ must also be true.
- Proofs by mathematical induction do not always start at the integer 1. In such a case, the basis step begins at a starting point b where b is an integer. We will see examples of this soon.

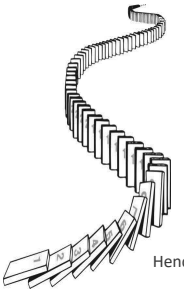
Validity of Mathematical Induction

- Mathematical induction is valid because of the well ordering property, which states that every nonempty subset of the set of positive integers has a least element (*see Section* 5.2 *and Appendix* 1). Here is the proof:
  - Suppose that $P(1)$ holds and $P(k) \rightarrow P(k+1)$ is true for all positive integers k.
  - Assume there is at least one positive integer n for which $P(n)$ is false. Then the set S of positive integers for which $P(n)$ is false is nonempty.
  - By the well-ordering property, S has a least element, say m.
  - We know that m can not be 1 since $P(1)$ holds.
  - Since m is positive and greater than 1, $m-1$ must be a positive integer. Since $m-1 < m$, it is not in S, so $P(m-1)$ must be true.
  - But then, since the conditional $P(k) \rightarrow P(k+1)$ for every positive integer k holds, $P(m)$ must also be true. This contradicts $P(m)$ being false.
  - Hence, $P(n)$ must be true for every positive integer n.

Remembering How Mathematical Induction Works

Consider an infinite sequence of dominoes, labeled 1,2,3, …, where each domino is standing.

Let $P(n)$ be the proposition that the nth domino is knocked over.



We know that the first domino is knocked down, i.e., $P(1)$ is true .

We also know that if whenever the kth domino is knocked over, it knocks over the $(k+1)$st domino, i.e, $P(k) \rightarrow P(k+1)$ is true for all positive integers k.

Hence, all dominos are knocked over.

$P(n)$ is true for all positive integers n.

Proving a Summation Formula by Mathematical Induction

Example: Show that: $\sum_{i=1}^{n} = \frac{n(n+1)}{2}$

Note: Once we have this conjecture, mathematical induction can be used to prove it correct.

Solution:

- BASIS STEP: $P(1)$ is true since $1(1+1)/2 = 1$.
- INDUCTIVE STEP: Assume true for $P(k)$.

The inductive hypothesis is $\sum_{i=1}^{k} = \frac{k(k+1)}{2}$

Under this assumption,

$$\begin{aligned} 1+2+\ldots+k+(k+1) &= \frac{k(k+1)}{2} + (k+1) \\ &= \frac{k(k+1)+2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

◄

Conjecturing and Proving Correct a Summation Formula

Example: Conjecture and prove correct a formula for the sum of the first n positive odd integers. Then prove your conjecture.

Solution: We have: $1 = 1, 1+3 = 4, 1+3+5 = 9, 1+3+5+7 = 16, 1+3+5+7+9 = 25$.

- We can conjecture that the sum of the first n positive odd integers is n^2, $1+3+5+\cdots+(2n-1) = n^2$.
- We prove the conjecture is correct with mathematical induction.
- BASIS STEP: $P(1)$ is true since $1^2 = 1$.
- INDUCTIVE STEP: $P(k) \rightarrow P(k+1)$ for every positive integer k.

Assume the inductive hypothesis holds and then show that $P(k+1)$ holds has well.

- So, assuming $P(k)$, it follows that

Inductive Hypothesis: $1+3+5+\cdots+(2k-1) = k^2$

$$\begin{aligned} 1+3+5+\cdots+(2k-1)+(2k+1) &= [1+3+5+\cdots+(2k-1)]+(2k+1) \\ &= k^2+(2k+1) \quad (\textit{by the inductive hypothesis}) \\ &= k^2+2k+1 \\ &= (k+1)^2 \end{aligned}$$

- Hence, we have shown that $P(k+1)$ follows from $P(k)$. Therefore the sum of the first n positive odd integers is n^2.

◄

Proving Inequalities

Example: Use mathematical induction to prove that $n < 2^n$ for all positive integers n.

Solution: Let $P(n)$ be the proposition that $n < 2^n$.

- BASIS STEP: $P(1)$ is true since $1 < 2^1 = 2$.
- INDUCTIVE STEP: Assume $P(k)$ holds, i.e., $k < 2^k$, for an arbitrary positive integer k.
- Must show that $P(k + 1)$ holds. Since by the inductive hypothesis, $k < 2^k$, it follows that:

$$k + 1 < 2^k + 1 \leq 2^k + 2^k = 2 \cdot 2^k = 2^{k+1}$$

Therefore $n < 2^n$ holds for all positive integers n. ◀

Proving Inequalities

Example: Use mathematical induction to prove that $2^n < n!$, for every integer $n \geq 4$.

Solution: Let $P(n)$ be the proposition that $2^n < n!$.

- BASIS STEP: $P(4)$ is true since $2^4 = 16 < 4! = 24$.
- INDUCTIVE STEP: Assume $P(k)$ holds, i.e., $2^k < k!$ for an arbitrary integer $k \geq 4$. To show that $P(k + 1)$ holds:

$$\begin{aligned} 2^{k+1} &= 2 \cdot 2^k \\ &< 2 \cdot k! \qquad \textit{(by the inductive hypothesis)} \\ &< (k + 1)k! \\ &= (k + 1)! \end{aligned}$$

Therefore, $2^n < n!$ holds, for every integer $n \geq 4$. ◀

Note that here the basis step is $P(4)$, since $P(0)$, $P(1)$, $P(2)$, and $P(3)$ are all false.

Proving Divisibility Results

Example: Use mathematical induction to prove that $n^3 - n$ is divisible by 3, for every positive integer n.

Solution: Let $P(n)$ be the proposition that $n^3 - n$ is divisible by 3.

- BASIS STEP: $P(1)$ is true since $1^3 - 1 = 0$, which is divisible by 3.
- INDUCTIVE STEP: Assume $P(k)$ holds, i.e., $k^3 - k$ is divisible by 3, for an arbitrary positive integer k. To show that $P(k + 1)$ follows:

$$\begin{aligned} (k + 1)^3 - (k + 1) &= (k^3 + 3k^2 + 3k + 1) - (k + 1) \\ &= (k^3 - k) + 3(k^2 + k) \end{aligned}$$

By the inductive hypothesis, the first term $(k^3 - k)$ is divisible by 3 and the second term is divisible by 3 since it is an integer multiplied by 3. So by part (i) of Theorem 1 in Section 4.1 , $(k + 1)^3 - (k + 1)$ is divisible by 3.

Therefore, $n^3 - n$ is divisible by 3, for every integer positive integer n. ◀

Number of Subsets of a Finite Set

Example: Use mathematical induction to show that if S is a finite set with n elements, where n is a nonnegative integer, then S has 2^n subsets.

(*Chapter 6 uses combinatorial methods to prove this result.*)

Solution: Let $P(n)$ be the proposition that a set with n elements has 2^n subsets.

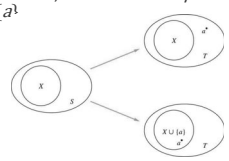
- Basis Step: $P(0)$ is true, because the empty set has only itself as a subset and $2^0 = 1$.
- Inductive Step: Assume $P(k)$ is true for an arbitrary nonnegative integer k.

continued →

Number of Subsets of a Finite Set

Inductive Hypothesis: For an arbitrary nonnegative integer k, every set with k elements has 2^k subsets.

- Let T be a set with $k + 1$ elements. Then $T = S \cup \{a\}$, where $a \in T$ and $S = T - \{a\}$. Hence $|S| = k$.
- For each subset X of S, there are exactly two subsets of T, i.e., X and $X \cup \{a\}$.



- By the inductive hypothesis S has 2^k subsets. Since there are two subsets of T for each subset of S, the number of subsets of T is $2 \cdot 2^k = 2^{k+1}$. ◀

Tiling Checkerboards

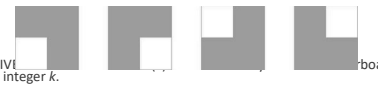
Example: Show that every $2^n \times 2^n$ checkerboard with one square removed can be tiled using right trominoes.

A right triomino is an L-shaped tile which covers three squares at a time.



Solution: Let $P(n)$ be the proposition that every $2^n \times 2^n$ checkerboard with one square removed can be tiled using right trominoes. Use mathematical induction to prove that $P(n)$ is true for all positive integers n.

- BASIS STEP: P(1) is true, because each of the four 2×2 checkerboards with one square removed can be tiled using one right triomino.



- INDUCTIVE … rboard, for some positive integer k.

continued →

Tiling Checkerboards

Inductive Hypothesis: Every $2^k \times 2^k$ checkerboard, for some positive integer k, with one square removed can be tiled using right trominoes.

- Consider a $2^{k+1} \times 2^{k+1}$ checkerboard with one square removed. Split this checkerboard into four checkerboards of size $2^k \times 2^k$, by dividing it in half in both directions.



- Remove a square from one of the four $2^k \times 2^k$ checkerboards. By the inductive hypothesis, this board can be tiled. Also by the inductive hypothesis, the other three boards can be tiled with the square from the corner of the center of the original board removed. We can then cover the three adjacent squares with a triominoe.
- Hence, the entire $2^{k+1} \times 2^{k+1}$ checkerboard with one square removed can be tiled using right trominoes. ◀

An Incorrect "Proof" by Mathematical Induction

Example: Let $P(n)$ be the statement that every set of n lines in the plane, no two of which are parallel, meet in a common point. Here is a "proof" that $P(n)$ is true for all positive integers $n \geq 2$.

- BASIS STEP: The statement $P(2)$ is true because any two lines in the plane that are not parallel meet in a common point.
- INDUCTIVE STEP: The inductive hypothesis is the statement that $P(k)$ is true for the positive integer $k \geq 2$, i.e., every set of k lines in the plane, no two of which are parallel, meet in a common point.
- We must show that if $P(k)$ holds, then $P(k + 1)$ holds, i.e., if every set of k lines in the plane, no two of which are parallel, $k \geq 2$, meet in a common point, then every set of k + 1 lines in the plane, no two of which are parallel, meet in a common point.

continued →

An Incorrect "Proof" by Mathematical Induction

Inductive Hypothesis: Every set of k lines in the plane, where $k \geq 2$, no two of which are parallel, meet in a common point.

- Consider a set of $k + 1$ distinct lines in the plane, no two parallel. By the inductive hypothesis, the first k of these lines must meet in a common point p_1. By the inductive hypothesis, the last k of these lines meet in a common point p_2.
- If p_1 and p_2 are different points, all lines containing both of them must be the same line since two points determine a line. This contradicts the assumption that the lines are distinct. Hence, $p_1 = p_2$ lies on all $k + 1$ distinct lines, and therefore $P(k + 1)$ holds. Assuming that $k \geq 2$, distinct lines meet in a common point, then every $k + 1$ lines meet in a common point.
- There must be an error in this proof since the conclusion is absurd. But where is the error?
  - **Answer**: $P(k) \rightarrow P(k + 1)$ only holds for $k \geq 3$. It is not the case that $P(2)$ implies $P(3)$. The first two lines must meet in a common point p_1 and the second two must meet in a common point p_2. They do not have to be the same point since only the second line is common to both sets of lines.

Guidelines: Mathematical Induction Proofs

Template for Proofs by Mathematical Induction

1. Express the statement that is to be proved in the form "for all $n \geq b$, $P(n)$" for a fixed integer b.
2. Write out the words "Basis Step." Then show that $P(b)$ is true, taking care that the correct value of b is used. This completes the first part of the proof.
3. Write out the words "Inductive Step."
4. State, and clearly identify, the inductive hypothesis, in the form "assume that $P(k)$ is true for an arbitrary fixed integer $k \geq b$."
5. State what needs to be proved under the assumption that the inductive hypothesis is true. That is, write out what $P(k + 1)$ says.
6. Prove the statement $P(k + 1)$ making use the assumption $P(k)$. Be sure that your proof is valid for all integers k with $k \geq b$, taking care that the proof works for small values of k, including $k = b$.
7. Clearly identify the conclusion of the inductive step, such as by saying "this completes the inductive step."
8. After completing the basis step and the inductive step, state the conclusion, namely that by mathematical induction, $P(n)$ is true for all integers n with $n \geq b$.

Strong Induction and Well-Ordering

Section 5.2

Section Summary

- Strong Induction
- Example Proofs using Strong Induction
- Using Strong Induction in Computational Geometry (*not yet included in overheads*)
- Well-Ordering Property

Strong Induction

- *Strong Induction*: To prove that $P(n)$ is true for all positive integers n, where $P(n)$ is a propositional function, complete two steps:
  - *Basis Step*: Verify that the proposition $P(1)$ is true.
  - *Inductive Step*: Show the conditional statement $[P(1) \wedge P(2) \wedge \cdots \wedge P(k)] \rightarrow P(k + 1)$ holds for all positive integers k.

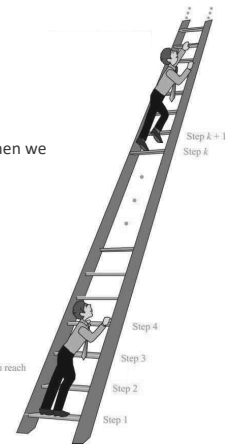
Strong Induction is sometimes called the *second principle of mathematical induction* or *complete induction*.

Strong Induction and the Infinite Ladder

Strong induction tells us that we can reach all rungs if:
1. We can reach the first rung of the ladder.
2. For every integer k, if we can reach the first k rungs, then we can reach the $(k + 1)$st rung.

To conclude that we can reach every rung by strong induction:
- BASIS STEP: $P(1)$ holds
- INDUCTIVE STEP: Assume $P(1) \wedge P(2) \wedge \cdots \wedge P(k)$ holds for an arbitrary integer k, and show that $P(k + 1)$ must also hold.

We will have then shown by strong induction that for every positive integer n, $P(n)$ holds, i.e., we can reach the nth rung of the ladder.



Proof using Strong Induction

Example: Suppose we can reach the first and second rungs of an infinite ladder, and we know that if we can reach a rung, then we can reach two rungs higher. Prove that we can reach every rung.

(Try this with mathematical induction.)

Solution: Prove the result using strong induction.
- BASIS STEP: We can reach the first step.
- INDUCTIVE STEP: The inductive hypothesis is that we can reach the first k rungs, for any $k \geq 2$. We can reach the $(k + 1)$st rung since we can reach the $(k - 1)$st rung by the inductive hypothesis.
- Hence, we can reach all rungs of the ladder.

◄

Which Form of Induction Should Be Used?

- We can always use strong induction instead of mathematical induction. But there is no reason to use it if it is simpler to use mathematical induction. (*See page* 335 *of text*.)
- In fact, the principles of mathematical induction, strong induction, and the well-ordering property are all equivalent. (*Exercises* 41-43)
- Sometimes it is clear how to proceed using one of the three methods, but not the other two.

Completion of the proof of the Fundamental Theorem of Arithmetic

Example: Show that if n is an integer greater than 1, then n can be written as the product of primes.

Solution: Let $P(n)$ be the proposition that n can be written as a product of primes.

- BASIS STEP: $P(2)$ is true since 2 itself is prime.
- INDUCTIVE STEP: The inductive hypothesis is $P(j)$ is true for all integers j with $2 \leq j \leq k$. To show that $P(k+1)$ must be true under this assumption, two cases need to be considered:
  - If $k+1$ is prime, then $P(k+1)$ is true.
  - Otherwise, $k+1$ is composite and can be written as the product of two positive integers a and b with $2 \leq a \leq b < k+1$. By the inductive hypothesis a and b can be written as the product of primes and therefore $k+1$ can also be written as the product of those primes.

Hence, it has been shown that every integer greater than 1 can be written as the product of primes. ◀

(*uniqueness proved in Section* 4.3)

Proof using Strong Induction

Example: Prove that every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.

Solution: Let $P(n)$ be the proposition that postage of n cents can be formed using 4-cent and 5-cent stamps.

- BASIS STEP: $P(12)$, $P(13)$, $P(14)$, and $P(15)$ hold.
  - $P(12)$ uses three 4-cent stamps.
  - $P(13)$ uses two 4-cent stamps and one 5-cent stamp.
  - $P(14)$ uses one 4-cent stamp and two 5-cent stamps.
  - $P(15)$ uses three 5-cent stamps.
- INDUCTIVE STEP: The inductive hypothesis states that $P(j)$ holds for $12 \leq j \leq k$, where $k \geq 15$. Assuming the inductive hypothesis, it can be shown that $P(k+1)$ holds.
- Using the inductive hypothesis, $P(k-3)$ holds since $k-3 \geq 12$. To form postage of $k+1$ cents, add a 4-cent stamp to the postage for $k-3$ cents.

Hence, $P(n)$ holds for all $n \geq 12$. ◀

Proof of Same Example using Mathematical Induction

Example: Prove that every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.

Solution: Let $P(n)$ be the proposition that postage of n cents can be formed using 4-cent and 5-cent stamps.

- BASIS STEP: Postage of 12 cents can be formed using three 4-cent stamps.
- INDUCTIVE STEP: The inductive hypothesis $P(k)$ for any positive integer k is that postage of k cents can be formed using 4-cent and 5-cent stamps. To show P($k+1$) where $k \geq 12$, we consider two cases:
  - If at least one 4-cent stamp has been used, then a 4-cent stamp can be replaced with a 5-cent stamp to yield a total of k + 1 cents.
  - Otherwise, no 4-cent stamp have been used and at least three 5-cent stamps were used. Three 5-cent stamps can be replaced by four 4-cent stamps to yield a total of k + 1 cents.

Hence, $P(n)$ holds for all $n \geq 12$.

◀

Well-Ordering Property

- *Well-ordering property*: Every nonempty set of nonnegative integers has a least element.
- The well-ordering property is one of the axioms of the positive integers listed in Appendix 1.
- The well-ordering property can be used directly in proofs, as the next example illustrates.
- The well-ordering property can be generalized.
  - **Definition:** A set is *well ordered if every subset has a least element.*
    - **N** is well ordered under ≤.
    - The set of finite strings over an alphabet using lexicographic ordering is well ordered.
  - We will see a generalization of induction to sets other than the integers in the next section.

Well-Ordering Property

Example: Use the well-ordering property to prove the division algorithm, which states that if a is an integer and d is a positive integer, then there are unique integers q and r with $0 \leq r < d$, such that $a = dq + r$.

Solution: Let S be the set of nonnegative integers of the form $a - dq$, where q is an integer. The set is nonempty since $-dq$ can be made as large as needed.

- By the well-ordering property, S has a least element $r = a - dq_0$. The integer r is nonnegative. It also must be the case that $r < d$. If it were not, then there would be a smaller nonnegative element in S, namely, $a - d(q_0 + 1) = a - dq_0 - d = r - d > 0$.
- Therefore, there are integers q and r with $0 \leq r < d$.

(*uniqueness of q and r is Exercise* 37) ◀

Recursive Definitions and Structural Induction

Section 5.3

Section Summary

- Recursively Defined Functions
- Recursively Defined Sets and Structures
- Structural Induction
- Generalized Induction

Recursively Defined Functions

Definition: A *recursive* or *inductive definition* of a function consists of two steps.

- BASIS STEP: Specify the value of the function at zero.
- RECURSIVE STEP: Give a rule for finding its value at an integer from its values at smaller integers.

- A function $f(n)$ is the same as a sequence $a_0, a_1, \ldots$, where a_i, where $f(i) = a_i$. This was done using recurrence relations in Section 2.4.

Recursively Defined Functions

Example: Suppose f is defined by:

$f(0) = 3$,

$f(n + 1) = 2f(n) + 3$

Find $f(1)$, $f(2)$, $f(3)$, $f(4)$

Solution:

- $f(1) = 2f(0) + 3 = 2 \cdot 3 + 3 = 9$
- $f(2) = 2f(1) + 3 = 2 \cdot 9 + 3 = 21$
- $f(3) = 2f(2) + 3 = 2 \cdot 21 + 3 = 45$
- $f(4) = 2f(3) + 3 = 2 \cdot 45 + 3 = 93$

Example: Give a recursive definition of the factorial function $n!$:

Solution:

$f(0) = 1$

$f(n + 1) = (n + 1) \cdot f(n)$

Recursively Defined Functions

Example: Give a recursive definition of:

$$\sum_{k=0}^{n} a_k.$$

Solution: The first part of the definition is

$$\sum_{k=0}^{0} a_k = a_0.$$

The second part is

$$\sum_{k=0}^{n+1} a_k = \left(\sum_{k=0}^{n} a_k\right) + a_{n+1}.$$

Fibonacci Numbers

Fibonacci (1170- 1250)



Example : The Fibonacci numbers are defined as follows:

$f_0 = 0$

$f_1 = 1$

$f_n = f_{n-1} + f_{n-2}$

Find f_2, f_3, f_4, f_5.

- $f_2 = f_1 + f_0 = 1 + 0 = 1$
- $f_3 = f_2 + f_1 = 1 + 1 = 2$
- $f_4 = f_3 + f_2 = 2 + 1 = 3$
- $f_5 = f_4 + f_3 = 3 + 2 = 5$

In Chapter 8, we will use the Fibonacci numbers to model population growth of rabbits. This was an application described by Fibonacci himself.

Next, we use strong induction to prove a result about the Fibonacci numbers.

Fibonacci Numbers

Example 4: Show that whenever $n \geq 3$, $f_n > \alpha^{n-2}$, where $\alpha = (1 + \sqrt{5})/2$.

Solution: Let $P(n)$ be the statement $f_n > \alpha^{n-2}$. Use strong induction to show that $P(n)$ is true whenever $n \geq 3$.

- BASIS STEP: $P(3)$ holds since $\alpha < 2 = f_3$

  $P(4)$ holds since $\alpha^2 = (3 + \sqrt{5})/2 < 3 = f_4$.
- INDUCTIVE STEP: Assume that $P(j)$ holds, i.e., $f_j > \alpha^{j-2}$ for all integers j with $3 \leq j \leq k$, where $k \geq 4$. Show that $P(k + 1)$ holds, i.e., $f_{k+1} > \alpha^{k-1}$.
  - Since $\alpha^2 = \alpha + 1$ (because α is a solution of $x^2 - x - 1 = 0$),

    $\alpha^{k-1} = \alpha^2 \cdot \alpha^{k-3} = (\alpha + 1) \cdot \alpha^{k-3} = \alpha \cdot \alpha^{k-3} + 1 \cdot \alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}$
  - By the inductive hypothesis, because $k \geq 4$ we have

    $f_{k-1} > \alpha^{k-3}$, $\quad f_k > \alpha^{k-2}$.
  - Therefore, it follows that

    $f_{k+1} = f_k + f_{k-1} > \alpha^{k-2} + \alpha^{k-3} = \alpha^{k-1}$.
  - Hence, $P(k + 1)$ is true.

Why does this equality hold?

Gabriel Lamé (1795-1870)



Lamé's Theorem

Lamé's Theorem: Let a and b be positive integers with $a \geq b$. Then the number of divisions used by the Euclidian algorithm to find gcd(a,b) is less than or equal to five times the number of decimal digits in b.

Proof: When we use the Euclidian algorithm to find gcd(a,b) with $a \geq b$,

- n divisions are used to obtain (with $a = r_0, b = r_1$):

$$\begin{aligned} r_0 &= r_1 q_1 + r_2 && 0 \leq r_2 < r_1, \\ r_1 &= r_2 q_2 + r_3 && 0 \leq r_3 < r_2, \\ &\vdots \\ r_{n-2} &= r_{n-1} q_{n-1} + r_n && 0 \leq r_n < r_{n-1}, \\ r_{n-1} &= r_n q_n. \end{aligned}$$

- Since each quotient $q_1, q_2, \ldots, q_{n-1}$ is at least 1 and $q_n \geq 2$:

$$\begin{aligned} r_n &\geq 1 = f_2, \\ r_{n-1} &\geq 2\, r_n \geq 2\, f_2 = f_3, \\ r_{n-2} &\geq r_{n-1} + r_n \geq f_3 + f_2 = f_4, \\ &\vdots \\ r_2 &\geq r_3 + r_4 \geq f_{n-1} + f_{n-2} = f_n, \\ b = r_1 &\geq r_2 + r_3 \geq f_n + f_{n-1} = f_{n+1}. \end{aligned}$$

continued →

Lamé's Theorem

- It follows that if n divisions are used by the Euclidian algorithm to find gcd(a,b) with $a \geq b$, then $b \geq f_{n+1}$. By Example 4, $f_{n+1} > \alpha^{n-1}$, for $n > 2$, where $\alpha = (1 + \sqrt{5})/2$. Therefore, $b > \alpha^{n-1}$.
- Because $\log_{10} \alpha \approx 0.208 > 1/5$, $\log_{10} b > (n-1) \log_{10} \alpha > (n-1)/5$. Hence,

  $n - 1 < 5 \cdot \log_{10} b$.
- Suppose that b has k decimal digits. Then $b < 10^k$ and $\log_{10} b < k$. It follows that $n - 1 < 5k$ and since k is an integer, $n \leq 5k$.
- As a consequence of Lamé's Theorem, $O(\log b)$ divisions are used by the Euclidian algorithm to find gcd(a,b) whenever $a > b$.
  - By Lamé's Theorem, the number of divisions needed to find gcd(a,b) with $a > b$ is less than or equal to 5 ($\log_{10} b + 1$) since the number of decimal digits in b (which equals $[\log_{10} b] + 1$) is less than or equal to $\log_{10} b + 1$.

Lamé's Theorem was the first result in computational complexity

Recursively Defined Sets and Structures

Recursive definitions of sets have two parts:

- The *basis step* specifies an initial collection of elements.
- The *recursive step* gives the rules for forming new elements in the set from those already known to be in the set.

- Sometimes the recursive definition has an *exclusion rule*, which specifies that the set contains nothing other than those elements specified in the basis step and generated by applications of the rules in the recursive step.
- We will always assume that the exclusion rule holds, even if it is not explicitly mentioned.
- We will later develop a form of induction, called *structural induction*, to prove results about recursively defined sets.

Recursively Defined Sets and Structures

Example : Subset of Integers S:

BASIS STEP: $3 \in S$.

RECURSIVE STEP: If $x \in S$ and $y \in S$, then $x + y$ is in S.

- Initially 3 is in S, then $3 + 3 = 6$, then $3 + 6 = 9$, etc.

Example: The natural numbers **N**.

BASIS STEP: $0 \in \mathbf{N}$.

RECURSIVE STEP: If n is in **N**, then $n + 1$ is in **N**.

- Initially 0 is in S, then $0 + 1 = 1$, then $1 + 1 = 2$, etc.

Strings

Definition: The set Σ* of *strings* over the alphabet Σ:

BASIS STEP: λ ∈ Σ* (λ is the empty string)

RECURSIVE STEP: If w is in Σ* and x is in Σ, then $wx \in \Sigma^*$.

Example: If Σ = {0,1}, the strings in in Σ* are the set of all bit strings, λ,0,1, 00,01,10, 11, etc.

Example: If Σ = {a,b}, show that aab is in Σ*.

- Since λ ∈ Σ* and $a \in \Sigma$, $a \in \Sigma^*$.
- Since $a \in \Sigma^*$ and $a \in \Sigma$, $aa \in \Sigma^*$.
- Since $aa \in \Sigma^*$ and $b \in \Sigma$, $aab \in \Sigma^*$.

String Concatenation

Definition: Two strings can be combined via the operation of *concatenation*. Let Σ be a set of symbols and Σ* be the set of strings formed from the symbols in Σ. We can define the concatenation of two strings, denoted by ·, recursively as follows.

BASIS STEP: If $w \in \Sigma^*$, then $w \cdot \lambda = w$.

RECURSIVE STEP: If $w_1 \in \Sigma^*$ and $w_2 \in \Sigma^*$ and $x \in \Sigma$, then $w_1 \cdot (w_2 x) = (w_1 \cdot w_2)x$.

- Often $w_1 \cdot w_2$ is written as $w_1 w_2$.
- If w_1 = *abra* and w_2 = *cadabra*, the concatenation $w_1 w_2$ = *abracadabra.*

Length of a String

Example: Give a recursive definition of $l(w)$, the length of the string w.

Solution: The length of a string can be recursively defined by:

$l(\lambda) = 0$;

$l(wx) = l(w) + 1$ if $w \in \Sigma^*$ and $x \in \Sigma$.

Balanced Parentheses

Example: Give a recursive definition of the set of balanced parentheses P.

Solution:

BASIS STEP: () ∈ P

RECURSIVE STEP: If $w \in P$, then () $w \in P$, (w) ∈ P and w () ∈ P.

- Show that (() ()) is in P.
- Why is))(() not in P?

Well-Formed Formulae in Propositional Logic

Definition: The set of *well-formed formulae* in propositional logic involving **T**, **F**, propositional variables, and operators from the set {¬,∧,∨,→,↔}.

BASIS STEP: **T**,**F**, and s, where s is a propositional variable, are well-formed formulae.

RECURSIVE STEP: If E and F are well formed formulae, then (¬ E), ($E \wedge F$), ($E \vee F$), ($E \rightarrow F$), ($E \leftrightarrow F$), are well-formed formulae.

Examples: $((p \vee q) \rightarrow (q \wedge \mathbf{F}))$ is a well-formed formula.

$pq \wedge$ is not a well formed formula.

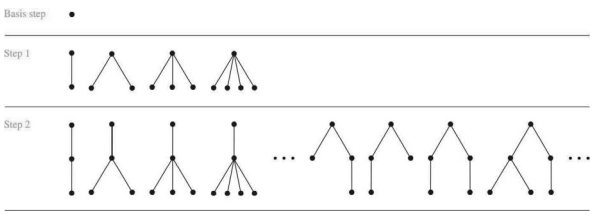
Rooted Trees

Definition: The set of *rooted trees,* where a rooted tree consists of a set of vertices containing a distinguished vertex called the *root*, and edges connecting these vertices, can be defined recursively by these steps:

BASIS STEP: A single vertex r is a rooted tree.

RECURSIVE STEP: Suppose that $T_1, T_2, \ldots, T_n$ are disjoint rooted trees with roots $r_1, r_2, \ldots, r_n$, respectively. Then the graph formed by starting with a root r, which is not in any of the rooted trees $T_1, T_2, \ldots, T_n$, and adding an edge from r to each of the vertices $r_1, r_2, \ldots, r_n$, is also a rooted tree.

Building Up Rooted Trees



- Trees are studied extensively in Chapter 11.
- Next we look at a special type of tree, the full binary tree.

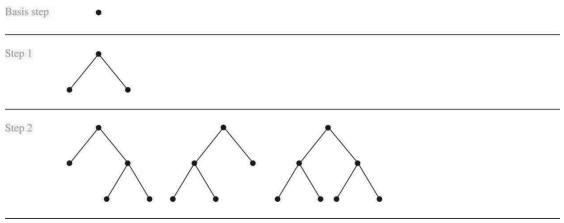
Full Binary Trees

Definition: The set of *full binary trees* can be defined recursively by these steps.

BASIS STEP: There is a full binary tree consisting of only a single vertex r.

RECURSIVE STEP: If T_1 and T_2 are disjoint full binary trees, there is a full binary tree, denoted by $T_1 \cdot T_2$, consisting of a root r together with edges connecting the root to each of the roots of the left subtree T_1 and the right subtree T_2.

Building Up Full Binary Trees



Induction and Recursively Defined Sets

Example: Show that the set S defined by specifying that $3 \in S$ and that if $x \in S$ and $y \in S$, then $x + y$ is in S, is the set of all positive integers that are multiples of 3.

Solution: Let A be the set of all positive integers divisible by 3. To prove that $A = S$, show that A is a subset of S and S is a subset of A.

- $A \subset S$: Let P(n) be the statement that $3n$ belongs to S.
  - BASIS STEP: $3 \cdot 1 = 3 \in S$, by the first part of recursive definition.
  - INDUCTIVE STEP: Assume $P(k)$ is true. By the second part of the recursive definition, if $3k \in S$, then since $3 \in S$, $3k + 3 = 3(k + 1) \in S$. Hence, $P(k + 1)$ is true.
- $S \subset A$:
  - BASIS STEP: $3 \in S$ by the first part of recursive definition, and $3 = 3 \cdot 1$.
  - INDUCTIVE STEP: The second part of the recursive definition adds $x + y$ to S, if both x and y are in S. If x and y are both in A, then both x and y are divisible by 3. By part (i) of Theorem 1 of Section 4.1, it follows that $x + y$ is divisible by 3.

- We used mathematical induction to prove a result about a recursively defined set. Next we study a more direct form induction for proving results about recursively defined sets.

Structural Induction

Definition: To prove a property of the elements of a recursively defined set, we use *structural induction*.

BASIS STEP: Show that the result holds for all elements specified in the basis step of the recursive definition.

RECURSIVE STEP: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

- The validity of structural induction can be shown to follow from the principle of mathematical induction.

Full Binary Trees

Definition: The *height h(T)* of a full binary tree T is defined recursively as follows:

- BASIS STEP: The height of a full binary tree T consisting of only a root r is $h(T) = 0$.
- RECURSIVE STEP: If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has height $h(T) = 1 + \max(h(T_1), h(T_2))$.

- The number of vertices $n(T)$ of a full binary tree T satisfies the following recursive formula:
  - **BASIS STEP**: The number of vertices of a full binary tree T consisting of only a root r is $n(T) = 1$.
  - **RECURSIVE STEP**: If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has the number of vertices $n(T) = 1 + n(T_1) + n(T_2)$.

Structural Induction and Binary Trees

Theorem: If T is a full binary tree, then $n(T) \le 2^{h(T)+1} - 1$.

Proof: Use structural induction.

- BASIS STEP: The result holds for a full binary tree consisting only of a root, $n(T) = 1$ and $h(T) = 0$. Hence, $n(T) = 1 \le 2^{0+1} - 1 = 1$.
- RECURSIVE STEP: Assume $n(T_1) \le 2^{h(T1)+1} - 1$ and also $n(T_2) \le 2^{h(T2)+1} - 1$ whenever T_1 and T_2 are full binary trees.

$$
\begin{aligned}
n(T) &= 1 + n(T_1) + n(T_2) && \text{(by recursive formula of } n(T)\text{)} \\
&\le 1 + (2^{h(T1)+1} - 1) + (2^{h(T2)+1} - 1) && \text{(by inductive hypothesis)} \\
&\le 2 \cdot \max(2^{h(T1)+1}, 2^{h(T2)+1}) - 1 \\
&= 2 \cdot 2^{\max(h(T1),h(T2))+1} - 1 && (\max(2^x, 2^y) = 2^{\max(x,y)}) \\
&= 2 \cdot 2^{h(t)} - 1 && \text{(by recursive definition of } h(T)\text{)} \\
&= 2^{h(t)+1} - 1
\end{aligned}
$$

Generalized Induction

- *Generalized induction* is used to prove results about sets other than the integers that have the well-ordering property. (*explored in more detail in Chapter* 9)
- For example, consider an ordering on $\mathbf{N} \times \mathbf{N}$, ordered pairs of nonnegative integers. Specify that (x_1, y_1) is less than or equal to (x_2, y_2) if either $x_1 < x_2$, or $x_1 = x_2$ and $y_1 < y_2$. This is called the *lexicographic ordering*.
- Strings are also commonly ordered by a *lexicographic ordering*.
- The next example uses generalized induction to prove a result about ordered pairs from $\mathbf{N} \times \mathbf{N}$.

Generalized Induction

Example: Suppose that $a_{m,n}$ is defined for $(m,n) \in \mathbf{N} \times \mathbf{N}$ by $a_{0,0} = 0$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1 & \text{if } n = 0 \text{ and } m > 0 \\ a_{m,n-1} + n & \text{if } n > 0 \end{cases}.$$

Show that $a_{m,n} = m + n(n+1)/2$ is defined for all $(m,n) \in \mathbf{N} \times \mathbf{N}$.

Solution: Use generalized induction.

BASIS STEP: $a_{0,0} = 0 = 0 + (0 \cdot 1)/2$

INDUCTIVE STEP: Assume that $a_{m',n'} = m' + n'(n'+1)/2$ whenever (m',n') is less than (m,n) in the lexicographic ordering of $\mathbf{N} \times \mathbf{N}$.

- If $n = 0$, by the inductive hypothesis we can conclude $a_{m,n} = a_{m-1,n} + 1 = m - 1 + n(n+1)/2 + 1 = m + n(n+1)/2$.
- If $n > 0$, by the inductive hypothesis we can conclude $a_{m,n} = a_{m-1,n} + 1 = m + n(n-1)/2 + n = m + n(n+1)/2$.

Recursive Algorithms

Section 5.4

Section Summary

- Recursive Algorithms
- Proving Recursive Algorithms Correct
- Recursion and Iteration (*not yet included in overheads*)
- Merge Sort

Recursive Algorithms

Definition: An algorithm is called *recursive* if it solves a problem by reducing it to an instance of the same problem with smaller input.

- For the algorithm to terminate, the instance of the problem must eventually be reduced to some initial case for which the solution is known.

Recursive Factorial Algorithm

Example: Give a recursive algorithm for computing $n!$, where n is a nonnegative integer.

- **Solution**: Use the recursive definition of the factorial function.

procedure *factorial*(n: nonnegative integer)
if $n = 0$ **then return** 1
else return $n \cdot$ *factorial* $(n-1)$
{output is $n!$}

Recursive Exponentiation Algorithm

Example: Give a recursive algorithm for computing a^n, where a is a nonzero real number and n is a nonnegative integer.

Solution: Use the recursive definition of a^n.

procedure *power*(a: nonzero real number, n: nonnegative integer)
if $n = 0$ **then return** 1
else return $a \cdot power\ (a, n-1)$
{output is a^n}

Recursive GCD Algorithm

Example: Give a recursive algorithm for computing the greatest common divisor of two nonnegative integers a and b with $a < b$.

Solution: Use the reduction

$$\gcd(a,b) = \gcd(b \bmod a, a)$$

and the condition $\gcd(0,b) = b$ when $b > 0$.

procedure *gcd*(a,b: nonnegative integers with $a < b$)
if $a = 0$ **then return** b
else return *gcd* (b **mod** a, a)
{output is *gcd*(a, b)}

Recursive Modular Exponentiation Algorithm

Example: Devise a a recursive algorithm for computing b^n **mod** m, where b, n, *and* m are integers with $m \geq 2$, $n \geq 0$, and $1 \leq b \leq m$.

- **Solution**:

(*see text for full explanation*)

procedure *mpower*(b,m,n: integers with $b > 0$ and $m \geq 2$, $n \geq 0$)
if $n = 0$ **then**
  return 1
else if n *is even* **then**
  return $mpower(b, n/2, m)^2$ **mod** m
else
  return $(mpower(b, \lfloor n/2 \rfloor, m)^2$ **mod** $m \cdot b$ **mod** $m)$ **mod** m
{output is b^n **mod** m}

Recursive Binary Search Algorithm

Example: Construct a recursive version of a binary search algorithm.

Solution: Assume we have $a_1, a_2, \ldots, a_n$, an increasing sequence of integers. Initially i is 1 and j is n. We are searching for x.

procedure *binary search*(i, j, x : integers, $1 \leq i \leq j \leq n$)
$m := \lfloor (i+j)/2 \rfloor$
if $x = a_m$ **then**
  return m
else if ($x < a_m$ and $i < m$) **then**
  return *binary search*(i,$m-1$,x)
else if ($x > a_m$ and $j > m$) **then**
  return *binary search*($m+1$,j,x)
else return 0
{output is location of x in $a_1, a_2, \ldots, a_n$ if it appears, otherwise 0}

Proving Recursive Algorithms Correct

- Both mathematical and str0ng induction are useful techniques to show that recursive algorithms always produce the correct output.

Example: Prove that the algorithm for computing the powers of real numbers is correct.

procedure *power*(a: nonzero real number, n: nonnegative integer)
if $n = 0$ **then return** 1
else return $a \cdot power\ (a, n-1)$
{output is a^n}

Solution: Use mathematical induction on the exponent n.
BASIS STEP: $a^0 = 1$ for every nonzero real number a, and *power*(a,0) = 1.
INDUCTIVE STEP: The inductive hypothesis is that *power*(a,k) = a^k, for all $a \neq 0$. Assuming the inductive hypothesis, the algorithm correctly computes a^{k+1}, since

$$power(a, k+1) = a \cdot power\ (a, k) = a \cdot a^k = a^{k+1}.$$

Merge Sort

- *Merge Sort* works by iteratively splitting a list (with an even number of elements) into two sublists of equal length until each sublist has one element.
- Each sublist is represented by a balanced binary tree.
- At each step a pair of sublists is successively merged into a list with the elements in increasing order. The process ends when all the sublists have been merged.
- The succession of merged lists is represented by a binary tree.

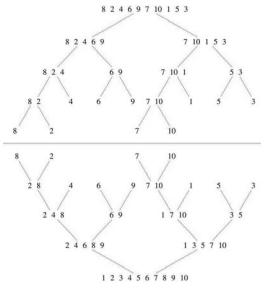
Merge Sort

Example: Use merge sort to put the list

$8,2,4,6,9,7,10, 1, 5, 3$

into increasing order.

Solution:



Recursive Merge Sort

Example: Construct a recursive merge sort algorithm.

Solution: Begin with the list of n elements L.

```
procedure mergesort(L = a_1, a_2,...,a_n )
if n > 1 then
    m := ⌊n/2⌋
    L_1 := a_1, a_2,...,a_m
    L_2 := a_{m+1}, a_{m+2},...,a_n
    L := merge(mergesort(L_1), mergesort(L_2 ))
{L is now sorted into elements in increasing order}
```

continued →

Recursive Merge Sort

- Subroutine *merge*, which merges two sorted lists.

```
procedure merge(L_1, L_2 :sorted lists)
L := empty list
while L_1 and L_2 are both nonempty
    remove smaller of first elements of L_1 and L_2 from its list;
        put at the right end of L
    if this removal makes one list empty
        then remove all elements from the other list and append them to L
return L {L is the merged list with the elements in increasing order}
```

Complexity of Merge: Two sorted lists with m elements and n elements can be merged into a sorted list using no more than $m + n - 1$ comparisons.

Merging Two Lists

Example: Merge the two lists $2,3,5,6$ and $1,4$.

Solution:

TABLE 1 Merging the Two Sorted Lists 2, 3, 5, 6 and 1, 4.

First List	*Second List*	*Merged List*	*Comparison*
2 3 5 6	1 4		$1 < 2$
2 3 5 6	4	1	$2 < 4$
3 5 6	4	1 2	$3 < 4$
5 6	4	1 2 3	$4 < 5$
5 6		1 2 3 4	
		1 2 3 4 5 6	

Complexity of Merge Sort

Complexity of Merge Sort: The number of comparisons needed to merge a list with n elements is $O(n \log n)$.

- For simplicity, assume that n is a power of 2, say 2^m.
- At the end of the splitting process, we have a binary tree with m levels, and 2^m lists with one element at level m.
- The merging process begins at level m with the pairs of 2^m lists with one element combined into 2^{m-1} lists of two elements. Each merger takes two one comparison.
- The procedure continues , at each level ($k = m, m-1, m-1,...,3,2,1$) 2^k lists with 2^{m-k} elements are merged into 2^{k-1} lists, with 2^{m-k+1} elements at level $k-1$.
  - We know (by the complexity of the merge subroutine) that each merger takes at most $2^{m-k} + 2^{m-k} - 1 = 2^{m-k+1} - 1$ comparisons.

continued →

Complexity of Merge Sort

- Summing over the number of comparisons at each level, shows that

$$\sum_{k=1}^{m} 2^{k-1}(2^{m-k+1} - 1) = \sum_{k=1}^{m} 2^m - \sum_{k=1}^{m} 2^{k-1} = m2^m - (2^m - 1) = n \log n - n + 1,$$

because $m = \log n$ and $n = 2^m$.

(The expression $\sum_{k=1}^{m} 2^{k-1}$ in the formula above is evaluated as $2^m - 1$ using the formula for the sum of the terms of a geometric progression, from Section 2.4.)

- In Chapter 11, we'll see that the fastest comparison-based sorting algorithms have $O(n \log n)$ time complexity. So, merge sort achieves the best possible big-O estimate of time complexity.

Counting

Chapter 6

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- The Basics of Counting
- The Pigeonhole Principle
- Permutations and Combinations
- Binomial Coefficients and Identities
- Generalized Permutations and Combinations
- Generating Permutations and Combinations (*not yet included in overheads*)

The Basics of Counting

Section 6.1

Section Summary

- The Product Rule
- The Sum Rule
- The Subtraction Rule
- The Division Rule
- Examples, Examples, and Examples
- Tree Diagrams

Basic Counting Principles: The Product Rule

The Product Rule: A procedure can be broken down into a sequence of two tasks. There are n_1 ways to do the first task and n_2 ways to do the second task. Then there are $n_1 \cdot n_2$ ways to do the procedure.

Example: How many bit strings of length seven are there?

Solution: Since each of the seven bits is either a 0 or a 1, the answer is $2^7 = 128$.

The Product Rule

Example: How many different license plates can be made if each plate contains a sequence of three uppercase English letters followed by three digits?

Solution: By the product rule,

there are $26 \cdot 26 \cdot 26 \cdot 10 \cdot 10 \cdot 10 = 17{,}576{,}000$ different possible license plates.

$$\underbrace{_\ _\ _}_{\text{26 choices for each letter}}\ \underbrace{_\ _\ _}_{\text{10 choices for each digit}}$$

Counting Functions

Counting Functions: How many functions are there from a set with m elements to a set with n elements?

Solution: Since a function represents a choice of one of the n elements of the codomain for each of the m elements in the domain, the product rule tells us that there are $n \cdot n \cdots n = n^m$ such functions.

Counting One-to-One Functions: How many one-to-one functions are there from a set with m elements to one with n elements?

Solution: Suppose the elements in the domain are a_1, a_2,..., a_m. There are n ways to choose the value of a_1 and $n-1$ ways to choose a_2, etc. The product rule tells us that there are $n(n-1)\,(n-2)\cdots(n-m+1)$ such functions.

Telephone Numbering Plan

Example: The *North American numbering plan* (*NANP*) specifies that a telephone number consists of 10 digits, consisting of a three-digit area code, a three-digit office code, and a four-digit station code. There are some restrictions on the digits.

- Let *X* denote a digit from 0 through 9.
- Let *N* denote a digit from 2 through 9.
- Let *Y* denote a digit that is 0 or 1.
- In the old plan (in use in the 1960s) the format was *NYX-NNX-XXX*.
- In the new plan, the format is *NXX-NXX-XXX*.

How many different telephone numbers are possible under the old plan and the new plan?

Solution: Use the Product Rule.

- There are $8 \cdot 2 \cdot 10 = 160$ area codes with the format *NYX*.
- There are $8 \cdot 10 \cdot 10 = 800$ area codes with the format *NXX*.
- There are $8 \cdot 8 \cdot 10 = 640$ office codes with the format *NNX*.
- There are $10 \cdot 10 \cdot 10 \cdot 10 = 10{,}000$ station codes with the format *XXXX*.

Number of old plan telephone numbers: $160 \cdot 640 \cdot 10{,}000 = 1{,}024{,}000{,}000$.

Number of new plan telephone numbers: $800 \cdot 800 \cdot 10{,}000 = 6{,}400{,}000{,}000$.

Basic Counting Principles: The Sum Rule

The Sum Rule: If a task can be done either in one of n_1 ways or in one of n_2, where none of the set of n_1 ways is the same as any of the n_2 ways, then there are $n_1 + n_2$ ways to do the task.

Example: The mathematics department must choose either a student or a faculty member as a representative for a university committee. How many choices are there for this representative if there are 37 members of the mathematics faculty and 83 mathematics majors and no one is both a faculty member and a student.

Solution: By the sum rule it follows that there are $37 + 83 = 120$ possible ways to pick a representative.

The Sum Rule in terms of sets.

- The sum rule can be phrased in terms of sets.
  $|A \cup B| = |A| + |B|$ as long as A and B are disjoint sets.
- Or more generally,

$$|A_1 \cup A_2 \cup \cdots \cup A_m| = |A_1| + |A_2| + \cdots + |A_m|$$
when $A_i \cap A_j = \emptyset$ for all i, j.

- The case where the sets have elements in common will be discussed when we consider the subtraction rule and taken up fully in Chapter 8.

Combining the Sum and Product Rule

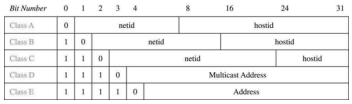
Example: Suppose statement labels in a programming language can be either a single letter or a letter followed by a digit. Find the number of possible labels.

Solution: Use the product rule.

$26 + 26 \cdot 10 = 286$

Internet Addresses

- Version 4 of the Internet Protocol (IPv4) uses 32 bits.



- **Class A Addresses**: used for the largest networks, a 0,followed by a 7-bit netid and a 24-bit hostid.
- **Class B Addresses**: used for the medium-sized networks, a 10,followed by a 14-bit netid and a 16-bit hostid.
- **Class C Addresses**: used for the smallest networks, a 110,followed by a 21-bit netid and a 8-bit hostid.
  - Neither Class D nor Class E addresses are assigned as the address of a computer on the internet. Only Classes A, B, and C are available.
  - 1111111 is not available as the netid of a Class A network.
  - Hostids consisting of all 0s and all 1s are not available in any network.

Counting Internet Addresses

Example: How many different IPv4 addresses are available for computers on the internet?

Solution: Use both the sum and the product rule. Let x be the number of available addresses, and let x_A, x_B, and x_C denote the number of addresses for the respective classes.

- To find, x_A: $2^7 - 1 = 127$ netids. $2^{24} - 2 = 16{,}777{,}214$ hostids.
  $x_A = 127 \cdot 16{,}777{,}214 = 2{,}130{,}706{,}178$.
- To find, x_B: $2^{14} = 16{,}384$ netids. $2^{16} - 2 = 16{,}534$ hostids.
  $x_B = 16{,}384 \cdot 16{,}534 = 1{,}073{,}709{,}056$.
- To find, x_C: $2^{21} = 2{,}097{,}152$ netids. $2^8 - 2 = 254$ hostids.
  $x_C = 2{,}097{,}152 \cdot 254 = 532{,}676{,}608$.
- Hence, the total number of available IPv4 addresses is
  $x = x_A + x_B + x_C$
  $= 2{,}130{,}706{,}178 + 1{,}073{,}709{,}056 + 532{,}676{,}608$
  $= 3{,}737{,}091{,}842$.

Not Enough Today !!
The newer IPv6 protocol solves the problem of too few addresses.

Basic Counting Principles: Subtraction Rule

Subtraction Rule: If a task can be done either in one of n_1 ways or in one of n_2 ways, then the total number of ways to do the task is $n_1 + n_2$ minus the number of ways to do the task that are common to the two different ways.

- Also known as, the *principle of inclusion-exclusion*:

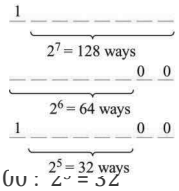
$$|A \cup B| = |A| + |B| - |A \cap B|$$

Counting Bit Strings

Example: How many bit strings of length eight either start with a 1 bit or end with the two bits 00?

Solution: Use the subtraction rule.

- Number of bit strings of length eight that start with a 1 bit: $2^7 = 128$
- Number of bit strings of length eight that end with bits 00: $2^6 = 64$
- Number of bit strings of length eight that start with a 1 bit and end with bits 00 : $2^5 = 32$



Hence, the number is $128 + 64 - 32 = 160$.

Basic Counting Principles: Division Rule

Division Rule: There are n/d ways to do a task if it can be done using a procedure that can be carried out in n ways, and for every way w, exactly d of the n ways correspond to way w.

- Restated in terms of sets: If the finite set A is the union of n pairwise disjoint subsets each with d elements, then $n = |A|/d$.
- In terms of functions: If f is a function from A to B, where both are finite sets, and for every value $y \in B$ there are exactly d values $x \in A$ such that $f(x) = y$, then $|B| = |A|/d$.

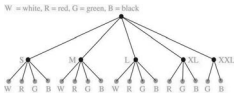
Example: How many ways are there to seat four people around a circular table, where two seatings are considered the same when each person has the same left and right neighbor?

Solution: Number the seats around the table from 1 to 4 proceeding clockwise. There are four ways to select the person for seat 1, 3 for seat 2, 2, for seat 3, and one way for seat 4. Thus there are 4! = 24 ways to order the four people. But since two seatings are the same when each person has the same left and right neighbor, for every choice for seat 1, we get the same seating.

Therefore, by the division rule, there are 24/4 = 6 different seating arrangements.

Tree Diagrams

- **Tree Diagrams**: We can solve many counting problems through the use of *tree diagrams*, where a branch represents a possible choice and the leaves represent possible outcomes.
- **Example**: Suppose that "I Love Discrete Math" T-shirts come in five different sizes: S,M,L,XL, and XXL. Each size comes in four colors (white, red, green, and black), except XL, which comes only in red, green, and black, and XXL, which comes only in green and black. What is the minimum number of shirts that the campus book store needs to stock to have one of each size and color available?
- **Solution**: Draw the tree diagram.



- The store must stock 17 T-shirts.

The Pigeonhole Principle

Section 6.2

Section Summary

- The Pigeonhole Principle
- The Generalized Pigeonhole Principle

The Pigeonhole Principle

- If a flock of 20 pigeons roosts in a set of 19 pigeonholes, one of the pigeonholes must have more than 1 pigeon.



Pigeonhole Principle: If k is a positive integer and $k + 1$ objects are placed into k boxes, then at least one box contains two or more objects.

Proof: We use a proof by contraposition. Suppose none of the k boxes has more than one object. Then the total number of objects would be at most k. This contradicts the statement that we have $k + 1$ objects. ◄

The Pigeonhole Principle

Corollary 1: A function f from a set with $k + 1$ elements to a set with k elements is not one-to-one.

Proof: Use the pigeonhole principle.

- Create a box for each element y in the codomain of f .
- Put in the box for y all of the elements x from the domain such that $f(x) = y$.
- Because there are $k + 1$ elements and only k boxes, at least one box has two or more elements.

Hence, f can't be one-to-one. ◄

Pigeonhole Principle

Example: Among any group of 367 people, there must be at least two with the same birthday, because there are only 366 possible birthdays.

The Generalized Pigeonhole Principle

The Generalized Pigeonhole Principle: If N objects are placed into k boxes, then there is at least one box containing at least $\lceil N/k \rceil$ objects.

Proof: We use a proof by contraposition. Suppose that none of the boxes contains more than $\lceil N/k \rceil - 1$ objects. Then the total number of objects is at most

$$k\left(\left\lceil \frac{N}{k} \right\rceil - 1\right) < k\left(\left(\frac{N}{k} + 1\right) - 1\right) = N,$$

where the inequality $\lceil N/k \rceil < \lceil N/k \rceil + 1$ has been used. This is a contradiction because there are a total of n objects. ◄

Example: Among 100 people there are at least $\lceil 100/12 \rceil = 9$ who were born in the same month.

The Generalized Pigeonhole Principle

Example: a) How many cards must be selected from a standard deck of 52 cards to guarantee that at least three cards of the same suit are chosen?

b) How many must be selected to guarantee that at least three hearts are selected?

Solution: a) We assume four boxes; one for each suit. Using the generalized pigeonhole principle, at least one box contains at least $\lceil N/4 \rceil$ cards. At least three cards of one suit are selected if $\lceil N/4 \rceil \geq 3$. The smallest integer N such that $\lceil N/4 \rceil \geq 3$ is $N = 2 \cdot 4 + 1 = 9$.

b) A deck contains 13 hearts and 39 cards which are not hearts. So, if we select 41 cards, we may have 39 cards which are not hearts along with 2 hearts. However, when we select 42 cards, we must have at least three hearts. (Note that the generalized pigeonhole principle is not used here.)

Permutations and Combinations

Section 6.3

Section Summary

- Permutations
- Combinations
- Combinatorial Proofs

Permutations

Definition: A *permutation* of a set of distinct objects is an ordered arrangement of these objects. An ordered arrangement of r elements of a set is called an *r-permuation*.

Example: Let $S = \{1,2,3\}$.

- The ordered arrangement $3,1,2$ is a permutation of S.
- The ordered arrangement $3,2$ is a 2-permutation of S.

- The number of *r*-permuatations of a set with *n* elements is denoted by $P(n,r)$.
  - The 2-permutations of $S = \{1,2,3\}$ are $1,2$; $1,3$; $2,1$; $2,3$; $3,1$; and $3,2$. Hence, $P(3,2) = 6$.

A Formula for the Number of Permutations

Theorem 1: If *n* is a positive integer and *r* is an integer with $1 \leq r \leq n$, then there are

$P(n, r) = n(n-1)(n-2)\cdots(n-r+1)$

r-permutations of a set with n distinct elements.

Proof: Use the product rule. The first element can be chosen in *n* ways. The second in $n-1$ ways, and so on until there are $(n-(r-1))$ ways to choose the last element.

- Note that $P(n,0) = 1$, since there is only one way to order zero elements.

Corollary 1: If *n* and *r* are integers with $1 \leq r \leq n$, then

$$P(n,r) = \frac{n!}{(n-r)!}$$

Solving Counting Problems by Counting Permutations

Example: How many ways are there to select a first-prize winner, a second prize winner, and a third-prize winner from 100 different people who have entered a contest?

Solution:

$P(100,3) = 100 \cdot 99 \cdot 98 = 970{,}200$

Solving Counting Problems by Counting Permutations (*continued*)

Example: Suppose that a saleswoman has to visit eight different cities. She must begin her trip in a specified city, but she can visit the other seven cities in any order she wishes. How many possible orders can the saleswoman use when visiting these cities?

Solution: The first city is chosen, and the rest are ordered arbitrarily. Hence the orders are:

$7! = 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5040$

If she wants to find the tour with the shortest path that visits all the cities, she must consider 5040 paths!

Solving Counting Problems by Counting Permutations (*continued*)

Example: How many permutations of the letters *ABCDEFGH* contain the string *ABC* ?

Solution: We solve this problem by counting the permutations of six objects, *ABC*, *D*, *E*, *F*, *G*, and *H*.

$6! = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$

Combinations

Definition: An *r-combination* of elements of a set is an unordered selection of *r* elements from the set. Thus, an *r*-combination is simply a subset of the set with *r* elements.

- The number of *r*-combinations of a set with n distinct elements is denoted by $C(n, r)$. The notation $\binom{n}{r}$ is also used and is called a *binomial coefficient*. (*We will see the notation again in the binomial theorem in Section* 6.4.)

Example: Let *S* be the set $\{a, b, c, d\}$. Then $\{a, c, d\}$ is a 3-combination from S. It is the same as $\{d, c, a\}$ since the order listed does not matter.

- $C(4,2) = 6$ because the 2-combinations of $\{a, b, c, d\}$ are the six subsets $\{a, b\}$, $\{a, c\}$, $\{a, d\}$, $\{b, c\}$, $\{b, d\}$, and $\{c, d\}$.

Combinations

Theorem 2: The number of *r*-combinations of a set with *n* elements, where $n \geq r \geq 0$, equals

$$C(n,r) = \frac{n!}{(n-r)!r!}.$$

Proof: By the product rule $P(n, r) = C(n,r) \cdot P(r,r)$. Therefore,

$$C(n,r) = \frac{P(n,r)}{P(r,r)} = \frac{n!/(n-r)!}{r!/(r-r)!} = \frac{n!}{(n-r)!r!}.$$

Combinations

Example: How many poker hands of five cards can be dealt from a standard deck of 52 cards? Also, how many ways are there to select 47 cards from a deck of 52 cards?

Solution: Since the order in which the cards are dealt does not matter, the number of five card hands is:

$$C(52,5) = \frac{52!}{5!47!} = \frac{52\cdot 51\cdot 50\cdot 49\cdot 48}{5\cdot 4\cdot 3\cdot 2\cdot 1} = 26\cdot 17\cdot 10\cdot 49\cdot 12 = 2,598,960$$

- The different ways to select 47 cards from 52 is

$$C(52,47) = \frac{52!}{47!5!} = C(52,5) = 2,598,960.$$

This is a special case of a general result. →

Combinations

Corollary 2: Let n and r be nonnegative integers with $r \leq n$. Then $C(n, r) = C(n, n - r)$.

Proof: From Theorem 2, it follows that

$$C(n,r) = \frac{n!}{(n-r)!r!}$$

and

$$C(n,n-r) = \frac{n!}{(n-r)![n-(n-r)]!} = \frac{n!}{(n-r)!r!}.$$

Hence, $C(n, r) = C(n, n - r)$. ◄

This result can be proved without using algebraic manipulation. →

Combinations

Example: How many ways are there to select five players from a 10-member tennis team to make a trip to a match at another school.

Solution: By Theorem 2, the number of combinations is

$$C(10,5) = \frac{10!}{5!5!} = 252.$$

Example: A group of 30 people have been trained as astronauts to go on the first mission to Mars. How many ways are there to select a crew of six people to go on this mission?

Solution: By Theorem 2, the number of possible crews is

$$C(30,6) = \frac{30!}{6!24!} = \frac{30\cdot 29\cdot 28\cdot 27\cdot 26\cdot 25}{6\cdot 5\cdot 4\cdot 3\cdot 2\cdot 1} = 593,775\ .$$

Binomial Coefficients and Identities

Section 6.4

Section Summary

- The Binomial Theorem
- Pascal's Identity and Triangle
- Other Identities Involving Binomial Coefficients (*not currently included in overheads*)

Powers of Binomial Expressions

Definition: A *binomial* expression is the sum of two terms, such as $x + y$. (More generally, these terms can be products of constants and variables.)

- We can use counting principles to find the coefficients in the expansion of $(x + y)^n$ where n is a positive integer.
- To illustrate this idea, we first look at the process of expanding $(x + y)^3$.
- $(x + y)(x + y)(x + y)$ expands into a sum of terms that are the product of a term from each of the three sums.
- Terms of the form x^3, x^2y, xy^2, y^3 arise. The question is what are the coefficients?
  - To obtain x^3, an x must be chosen from each of the sums. There is only one way to do this. So, the coefficient of x^3 is 1.
  - To obtain x^2y, an x must be chosen from two of the sums and a y from the other. There $\binom{3}{2}$ are ways to do this and so the coefficient of x^2y is 3.
  - To obtain xy^2, an x must be chosen from $\binom{3}{1}$ of the sums and a y from the other two . There are ways to do this and so the coefficient of xy^2 is 3.
  - To obtain y^3, a y must be chosen from each of the sums. There is only one way to do this. So, the coefficient of y^3 is 1.
- We have used a counting argument to show that $(x + y)^3 = x^3 + 3x^2y + 3xy^2 + y^3$.
- Next we present the binomial theorem gives the coefficients of the terms in the expansion of $(x + y)^n$.

Binomial Theorem

Binomial Theorem: Let x and y be variables, and n a nonnegative integer. Then:

$$(x+y)^n = \sum_{j=0}^{n} \binom{n}{j} x^{n-j}y^j = \binom{n}{0}x^n + \binom{n}{1}x^{n-1}y + \cdots + \binom{n}{n-1}xy^{n-1} + \binom{n}{n}y^n.$$

Proof: We use combinatorial reasoning . The terms in the expansion of $(x + y)^n$ are of the form $x^{n-j}y^j$ for $j = 0,1,2,\ldots,n$. To form the term $x^{n-j}y^j$, it is necessary to choose $n-j$ xs from the n sums. Therefore, the coefficient of $x^{n-j}y^j$ is $\binom{n}{n-j}$ which equals $\binom{n}{j}$. ◄

Using the Binomial Theorem

Example: What is the coefficient of $x^{12}y^{13}$ in the expansion of $(2x - 3y)^{25}$?

Solution: We view the expression as $(2x + (-3y))^{25}$. By the binomial theorem

$$(2x + (-3y))^{25} = \sum_{j=0}^{25} \binom{25}{j} (2x)^{25-j}(-3y)^j.$$

Consequently, the coefficient of $x^{12}y^{13}$ in the expansion is obtained when $j = 13$.

$$\binom{25}{13} 2^{12}(-3)^{13} = -\frac{25!}{13!12!}2^{12}3^{13}.$$

A Useful Identity

Corollary 1: With $n \geq 0$,

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

Proof (*using binomial theorem*): With $x = 1$ and $y = 1$, from the binomial theorem we see that:

$$2^n = (1+1)^n = \sum_{k=0}^{n} \binom{n}{k} 1^k 1^{(n-k)} = \sum_{k=0}^{n} \binom{n}{k}. \quad \blacktriangleleft$$

Proof (*combinatorial*): Consider the subsets of a set with n elements. There are $\binom{n}{0}$ subsets with zero elements, $\binom{n}{1}$ with one element, $\binom{n}{2}$ with two elements, …, and $\binom{n}{n}$ with n elements. Therefore the total is $\sum_{k=0}^{n} \binom{n}{k}$.

Since, we know that a set with n elements has 2^n subsets, we conclude:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n. \quad \blacktriangleleft$$

Pascal's Identity

Blaise Pascal
(1623-1662)



Pascal's Identity: If n and k are integers with $n \geq k \geq 0$, then

$$\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}.$$

Proof (*combinatorial*): Let T be a set where $|T| = n + 1$, $a \in T$, and $S = T - \{a\}$. There are $\binom{n+1}{k}$ subsets of T containing k elements. Each of these subsets either:

- contains a with $k - 1$ other elements, or
- contains k elements of S and not a.

There are

- $\binom{n}{k-1}$ subsets of k elements that contain a, since there are subsets of $k - 1$ elements of S,
- $\binom{n}{k}$ subsets of k elements of T that do not contain a, because there are subsets of k elements of S.

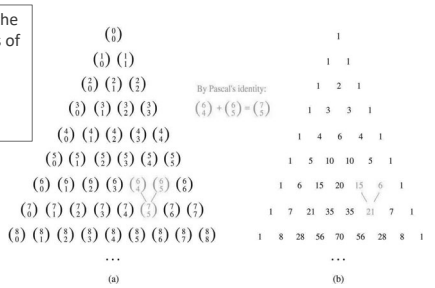
Hence,

$$\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}. \quad \blacktriangleleft$$

See Exercise 19 *for an algebraic proof.*

Pascal's Triangle

The nth row in the triangle consists of the binomial coefficients $k = 0,1,\ldots,n$. $\binom{n}{k}$



By Pascal's identity, adding two adjacent bionomial coefficients results is the binomial coefficient in the next row between these two coefficients.

Generalized Permutations and Combinations

Section 6.5

Section Summary

- Permutations with Repetition
- Combinations with Repetition
- Permutations with Indistinguishable Objects
- Distributing Objects into Boxes

Permutations with Repetition

Theorem 1: The number of r-permutations of a set of n objects with repetition allowed is n^r.

Proof: There are n ways to select an element of the set for each of the r positions in the r-permutation when repetition is allowed. Hence, by the product rule there are n^r r-permutations with repetition. ◄

Example: How many strings of length r can be formed from the uppercase letters of the English alphabet?

Solution: The number of such strings is 26^r, which is the number of r-permutations of a set with 26 elements.

Combinations with Repetition

Example: How many ways are there to select five bills from a box containing at least five of each of the following denominations: \$1, \$2, \$5, \$10, \$20, \$50, and \$100?

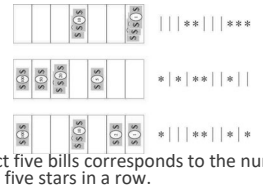
Solution: Place the selected bills in the appropriate position of a cash box illustrated below:



continued →

Combinations with Repetition

- Some possible ways of placing the five bills:



- The number of ways to select five bills corresponds to the number of ways to arrange six bars and five stars in a row.
- This is the number of unordered selections of 5 objects from a set of 11. Hence, there are

$$C(11,5) = \frac{11!}{5!6!} = 462$$

ways to choose five bills with seven types of bills.

Combinations with Repetition

Theorem 2: The number 0f r-combinations from a set with n elements when repetition of elements is allowed is

$$C(n + r - 1, r) = C(n + r - 1, n - 1).$$

Proof: Each r-combination of a set with n elements with repetition allowed can be represented by a list of $n - 1$ bars and r stars. The bars mark the n cells containing a star for each time the ith element of the set occurs in the combination.

The number of such lists is $C(n + r - 1, r)$, because each list is a choice of the r positions to place the stars, from the total of $n + r - 1$ positions to place the stars and the bars. This is also equal to $C(n + r - 1, n - 1)$, which is the number of ways to place the $n - 1$ bars. ◀

Combinations with Repetition



Example: Suppose that a cookie shop has four different kinds of cookies. How many different ways can six cookies be chosen?

Solution: The number of ways to choose six cookies is the number of 6-combinations of a set with four elements. By Theorem 2

$C(4 + 6 - 1, 6) = C(9, 3) = \frac{9 \cdot 8 \cdot 7}{1 \cdot 2 \cdot 3} = 84$

is the number of ways to choose six cookies from the four kinds.

Combinations with Repetition

Example: How many solutions does the equation

$$x_1 + x_2 + x_3 = 11$$

have, where x_1, x_2 and x_3 are nonnegative integers?

Solution: Each solution corresponds to a way to select 11 items from a set with three elements; x_1 elements of type one, x_2 of type two, and x_3 of type three.

By Theorem 2 it follows that there are

$$C(3 + 11 - 1, 11) = C(13, 11) = C(13, 2) = \frac{13 \cdot 12}{1 \cdot 2} = 78$$

solutions.

Summarizing the Formulas for Counting Permutations and Combinations with and without Repetition

TABLE 1 Combinations and Permutations With and Without Repetition.

Type	*Repetition Allowed?*	*Formula*
r-permutations	No	$\frac{n!}{(n-r)!}$
r-combinations	No	$\frac{n!}{r!\,(n-r)!}$
r-permutations	Yes	n^r
r-combinations	Yes	$\frac{(n+r-1)!}{r!\,(n-1)!}$

Permutations with Indistinguishable Objects

Example: How many different strings can be made by reordering the letters of the word *SUCCESS*.

Solution: There are seven possible positions for the three Ss, two Cs, one U, and one E.

- The three Ss can be placed in $C(7,3)$ different ways, leaving four positions free.
- The two Cs can be placed in $C(4,2)$ different ways, leaving two positions free.
- The U can be placed in $C(2,1)$ different ways, leaving one position free.
- The E can be placed in $C(1,1)$ way.

By the product rule, the number of different strings is:

$$C(7,3)C(4,2)C(2,1)C(1,1) = \frac{7!}{3!4!} \cdot \frac{4!}{2!2!} \cdot \frac{2!}{1!1!} \cdot \frac{1!}{1!0!} = \frac{7!}{3!2!1!1!} = 420.$$

The reasoning can be generalized to the following theorem. →

Permutations with Indistinguishable Objects

Theorem 3: The number of different permutations of n objects, where there are n_1 indistinguishable objects of type 1, n_2 indistinguishable objects of type 2, ..., and n_k indistinguishable objects of type k, is:

$$\frac{n!}{n_1!n_2!\cdots n_k!}.$$

Proof: By the product rule the total number of permutations is:

$C(n, n_1)\,C(n - n_1, n_2)\cdots C(n - n_1 - n_2 - \cdots - n_{k}, n_k)$ since:

- The n_1 objects of type one can be placed in the n positions in $C(n, n_1)$ ways, leaving $n - n_1$ positions.
- Then the n_2 objects of type two can be placed in the $n - n_1$ positions in $C(n - n_1, n_2)$ ways, leaving $n - n_1 - n_2$ positions.
- Continue in this fashion, until n_k objects of type k are placed in $C(n - n_1 - n_2 - \cdots - n_k, n_k)$ ways.

The product can be manipulated into the desired result as follows:

$$\frac{n!}{n_1!(n-n_1)!}\,\frac{(n-n_1)!}{n_2!(n-n_1-n_2!)}\cdots\frac{(n-n_1-\cdots-n_{k-1})!}{n_k!0!} = \frac{n!}{n_1!n_2!\cdots n_k!}.$$ ◀

Distributing Objects into Boxes

- Many counting problems can be solved by counting the ways objects can be placed in boxes.
  - The objects may be either different from each other (*distinguishable*) or identical (*indistinguishable*).
  - The boxes may be labeled (*distinguishable*) or unlabeled (*indistinguishable*).

Distributing Objects into Boxes

- *Distinguishable objects* and *distinguishable boxes*.
  - There are $n!/(n_1!n_2!\cdots n_k!)$ ways to distribute n distinguishable objects into k distinguishable boxes.
  - (*See Exercises 47 and 48 for two different proofs.*)
  - Example: There are $52!/(5!5!5!5!32!)$ ways to distribute hands of 5 cards each to four players.
- *Indistinguishable objects* and *distinguishable boxes*.
  - There are $C(n + r - 1, n - 1)$ ways to place r indistinguishable objects into n distinguishable boxes.
  - Proof based on one-to-one correspondence between n-combinations from a set with k-elements when repetition is allowed and the ways to place n indistinguishable objects into k distinguishable boxes.
  - Example: There are $C(8 + 10 - 1, 10) = C(17,10) = 19{,}448$ ways to place 10 indistinguishable objects into 8 distinguishable boxes.

Distributing Objects into Boxes

- *Distinguishable objects* and *indistinguishable boxes*.
  - Example: There are 14 ways to put four employees into three indistinguishable offices (*see Example* 10).
  - There is no simple closed formula for the number of ways to distribute *n* distinguishable objects into *j* indistinguishable boxes.
  - See the text for a formula involving *Stirling numbers of the second kind*.
- *Indistinguishable objects* and *indistinguishable boxes*.
  - Example: There are 9 ways to pack six copies of the same book into four identical boxes (*see Example* 11).
  - The number of ways of distributing *n* indistinguishable objects into *k* indistinguishable boxes equals $p_k(n)$, the number of ways to write *n* as the sum of at most *k* positive integers in increasing order.
  - No simple closed formula exists for this number.

Discrete Probability

Chapter 7

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Introduction to Discrete Probability
- Probability Theory
- Bayes' Theorem
- Expected Value and Variance

An Introduction to Discrete Probability

Section 7.1

Section Summary

- Finite Probability
- Probabilities of Complements and Unions of Events
- Probabilistic Reasoning



Probability of an Event



Pierre-Simon Laplace (1749-1827)

We first study Pierre-Simon Laplace's classical theory of probability, which he introduced in the 18th century, when he analyzed games of chance.

- We first define these key terms:
  - An *experiment* is a procedure that yields one of a given set of possible outcomes.
  - The *sample space* of the experiment is the set of possible outcomes.
  - An *event* is a subset of the sample space.
- Here is how Laplace defined the probability of an event:

  Definition: If S is a finite sample space of equally likely outcomes, and E is an event, that is, a subset of S, then the *probability* of E is $p(E) = |E|/|S|$.
- For every event E, we have $0 \leq p(E) \leq 1$. This follows directly from the definition because $0 \leq p(E) = |E|/|S| \leq |S|/|S| \leq 1$, since $0 \leq |E| \leq |S|$.

Applying Laplace's Definition

Example: An urn contains four blue balls and five red balls. What is the probability that a ball chosen from the urn is blue?

Solution: The probability that the ball is chosen is $4/9$ since there are nine possible outcomes, and four of these produce a blue ball.

Example: What is the probability that when two dice are rolled, the sum of the numbers on the two dice is 7?

Solution: By the product rule there are $6^2 = 36$ possible outcomes. Six of these sum to 7. Hence, the probability of obtaining a 7 is $6/36 = 1/6$.

Applying Laplace's Definition

Example: In a lottery, a player wins a large prize when they pick four digits that match, in correct order, four digits selected by a random mechanical process. What is the probability that a player wins the prize?

Solution: By the product rule there are $10^4 = 10{,}000$ ways to pick four digits.

- Since there is only 1 way to pick the correct digits, the probability of winning the large prize is $1/10{,}000 = 0.0001$.

A smaller prize is won if only three digits are matched. What is the probability that a player wins the small prize?

Solution: If exactly three digits are matched, one of the four digits must be incorrect and the other three digits must be correct. For the digit that is incorrect, there are 9 possible choices. Hence, by the sum rule, there a total of 36 possible ways to choose four digits that match exactly three of the winning four digits. The probability of winning the small price is $36/10{,}000 = 9/2500 = 0.0036$.

Applying Laplace's Definition

Example: There are many lotteries that award prizes to people who correctly choose a set of six numbers out of the first *n* positive integers, where *n* is usually between 30 and 60. What is the probability that a person picks the correct six numbers out of 40?

Solution: The number of ways to choose six numbers out of 40 is

$C(40,6) = 40!/(34!6!) = 3{,}838{,}380.$

Hence, the probability of picking a winning combination is $1/3{,}838{,}380 \approx 0.00000026$.

Can you work out the probability of winning the lottery with the biggest prize where you live?

Applying Laplace's Definition

Example: What is the probability that the numbers 11, 4, 17, 39, and 23 are drawn in that order from a bin with 50 balls labeled with the numbers 1,2, …, 50 if

a) The ball selected is not returned to the bin.
b) The ball selected is returned to the bin before the next ball is selected.

Solution: Use the product rule in each case.

a) Sampling without replacement: The probability is $1/254{,}251{,}200$ since there are $50 \cdot 49 \cdot 47 \cdot 46 \cdot 45 = 254{,}251{,}200$ ways to choose the five balls.

b) *Sampling with replacement*: The probability is $1/50^5 = 1/312{,}500{,}000$ since $50^5 = 312{,}500{,}000$.

The Probability of Complements and Unions of Events

Theorem 1: Let *E* be an event in sample space *S*. The probability of the event $\overline{E} = S - E$, the complementary event of *E*, is given by

$$p(\overline{E}) = 1 - p(E).$$

Proof: Using the fact that $|\overline{E}| = |S| - |E|$,

$$p(\overline{E}) = \frac{|S|-|E|}{|S|} = 1 - \frac{|E|}{|S|} = 1 - p(E). \quad \blacktriangleleft$$

The Probability of Complements and Unions of Events

Example: A sequence of 10 bits is chosen randomly. What is the probability that at least one of these bits is 0?

Solution: Let *E* be the event that at least one of the 10 bits is 0. Then $\overline{E}$ is the event that all of the bits are 1s. The size of the sample space *S* is 2^{10}. Hence,

$$p(E) = 1 - p(\overline{E}) = 1 - \frac{|E|}{|S|} = 1 - \frac{1}{2^{10}} = 1 - \frac{1}{1024} = \frac{1023}{1024}.$$

The Probability of Complements and Unions of Events

Theorem 2: Let E_1 and E_2 be events in the sample space *S*. Then

$$p(E_1 \cup E_2) = p(E_1) + p(E_2) - p(E_1 \cap E_2)$$

Proof: Given the inclusion-exclusion formula from Section 2.2, $|A \cup B| = |A| + |B| - |A \cap B|$, it follows that

$$\begin{aligned} p(E_1 \cup E_2) &= \frac{|E_1 \cup E_2|}{|S|} = \frac{|E_1|+|E_2|-|E_1 \cap E_2|}{|S|} \\ &= \frac{|E_1|}{|S|} + \frac{|E_2|}{|S|} - \frac{|E_1 \cap E_2|}{|S|} \\ &= p(E_1) + p(E_2) - p(E_1 \cap E_2). \end{aligned} \quad \blacktriangleleft$$

The Probability of Complements and Unions of Events

Example: What is the probability that a positive integer selected at random from the set of positive integers not exceeding 100 is divisible by either 2 or 5?

Solution: Let E_1 be the event that the integer is divisible by 2 and E_2 be the event that it is divisible 5? Then the event that the integer is divisible by 2 or 5 is $E_1 \cup E_2$ and $E_1 \cap E_2$ is the event that it is divisible by 2 and 5.

It follows that:

$p(E_1 \cup E_2) = p(E_1) + p(E_2) - p(E_1 \cap E_2)$
$= 50/100 + 20/100 - 10/100 = 3/5.$

Monty Hall Puzzle



Example: You are asked to select one of the three doors to open. There is a large prize behind one of the doors and if you select that door, you win the prize. After you select a door, the game show host opens one of the other doors (which he knows is not the winning door). The prize is not behind the door and he gives you the opportunity to switch your selection. Should you switch?

(*This is a notoriously confusing problem that has been the subject of much discussion . Do a web search to see why!)*

Solution: You should switch. The probability that your initial pick is correct is 1/3. This is the same whether or not you switch doors. But since the game show host always opens a door that does not have the prize, if you switch the probability of winning will be 2/3, because you win if your initial pick was not the correct door and the probability your initial pick was wrong is 2/3.

Probability Theory

Section 7.2

Section Summary

- Assigning Probabilities
- Probabilities of Complements and Unions of Events
- Conditional Probability
- Independence
- Bernoulli Trials and the Binomial Distribution
- Random Variables
- The Birthday Problem
- Monte Carlo Algorithms
- The Probabilistic Method (*not currently included in the overheads*)

Assigning Probabilities

Laplace's definition from the previous section, assumes that all outcomes are equally likely. Now we introduce a more general definition of probabilities that avoids this restriction.

- Let S be a sample space of an experiment with a finite number of outcomes. We assign a probability $p(s)$ to each outcome s, so that:
  i. $0 \le p(s) \le 1$ for each $s \in S$
  ii. $\sum_{s \in S} p(s) = 1$
- The function p from the set of all outcomes of the sample space S is called a *probability distribution*.

Assigning Probabilities

Example: What probabilities should we assign to the outcomes H(heads) and T (tails) when a fair coin is flipped? What probabilities should be assigned to these outcomes when the coin is biased so that heads comes up twice as often as tails?

Solution: For a fair coin, we have $p(H) = p(T) = ½$.

For a biased coin, we have $p(H) = 2p(T)$.

Because $p(H) + p(T) = 1$, it follows that

$$2p(T) + p(T) = 3p(T) = 1.$$

Hence, $p(T) = 1/3$ and $p(H) = 2/3$.

Uniform Distribution

Definition: Suppose that S is a set with n elements. The *uniform distribution* assigns the probability $1/n$ to each element of S. (Note that we could have used Laplace's definition here.)

Example: Consider again the coin flipping example, but with a fair coin. Now $p(H) = p(T) = 1/2$.

Probability of an Event

Definition: The probability of the event E is the sum of the probabilities of the outcomes in E.

$$p(E) = \sum_{s \in S} p(s)$$

- Note that now no assumption is being made about the distribution.

Example

Example: Suppose that a die is biased so that 3 appears twice as often as each other number, but that the other five outcomes are equally likely. What is the probability that an odd number appears when we roll this die?

Solution: We want the probability of the event $E = \{1,3,5\}$. We have $p(3) = 2/7$ and

$p(1) = p(2) = p(4) = p(5) = p(6) = 1/7$.

Hence, $p(E) = p(1) + p(3) + p(5) =$

$$1/7 + 2/7 + 1/7 = 4/7.$$

Probabilities of Complements and Unions of Events

- Complements: $p(\overline{E}) = 1 - p(E)$ still holds. Since each outcome is in either E or $\overline{E}$, but not both,

$$\sum_{s \in S} p(s) = 1 = p(E) + p(\overline{E}).$$

- Unions: $p(E_1 \cup E_2) = p(E_1) + p(E_2) - p(E_1 \cap E_2)$ also still holds under the new definition.

Combinations of Events

Theorem: If E_1, E_2, … is a sequence of pairwise disjoint events in a sample space S, then

$$p\left(\bigcup_i E_i\right) = \sum_i p(E_i)$$

see Exercises 36 and 37 for the proof

Conditional Probability

Definition: Let E and F be events with $p(F) > 0$. The conditional probability of E given F, denoted by $P(E|F)$, is defined as:

$$p(E|F) = \frac{p(E \cap F)}{p(F)}$$

Example: A bit string of length four is generated at random so that each of the 16 bit strings of length 4 is equally likely. What is the probability that it contains at least two consecutive 0s, given that its first bit is a 0?

Solution: Let E be the event that the bit string contains at least two consecutive 0s, and F be the event that the first bit is a 0.

- Since $E \cap F = \{0000, 0001, 0010, 0011, 0100\}$, $p(E \cap F) = 5/16$.
- Because 8 bit strings of length 4 start with a 0, p(F) = $8/16 = ½$.

Hence,

$$p(E|F) = \frac{p(E \cap F)}{p(F)} = \frac{5/16}{1/2} = \frac{5}{8}.$$

Conditional Probability

Example: What is the conditional probability that a family with two children has two boys, given that they have at least one boy. Assume that each of the possibilities *BB*, *BG*, *GB*, and *GG* is equally likely where *B* represents a boy and *G* represents a girl.

Solution: Let E be the event that the family has two boys and let F be the event that the family has at least one boy. Then $E = \{BB\}$, $F = \{BB, BG, GB\}$, and $E \cap F = \{BB\}$.

- It follows that p(F) = $3/4$ and $p(E \cap F) = 1/4$.

Hence,

$$p(E|F) = \frac{p(E \cap F)}{p(F)} = \frac{1/4}{3/4} = \frac{1}{3}.$$

Independence

Definition: The events E and F are independent if and only if

$$p(E \cap F) = p(E)p(F).$$

Example: Suppose E is the event that a randomly generated bit string of length four begins with a 1 and F is the event that this bit string contains an even number of 1s. Are E and F independent if the 16 bit strings of length four are equally likely?

Solution: There are eight bit strings of length four that begin with a 1, and eight bit strings of length four that contain an even number of 1s.

- Since the number of bit strings of length 4 is 16,

$$p(E) = p(F) = 8/16 = ½.$$

- Since $E \cap F = \{1111, 1100, 1010, 1001\}$, $p(E \cap F) = 4/16 = 1/4$.

We conclude that E and F are independent, because

$$p(E \cap F) = 1/4 = (½)\,(½) = p(E)\,p(F)$$

Independence

Example: Assume (as in the previous example) that each of the four ways a family can have two children (*BB*, *GG*, *BG*,*GB*) is equally likely. Are the events E, that a family with two children has two boys, and F, that a family with two children has at least one boy, independent?

Solution: Because $E = \{BB\}$, $p(E) = 1/4$. We saw previously that that $p(F) = 3/4$ and $p(E \cap F) = 1/4$. The events E and F are not independent since

$$p(E)\,p(F) = 3/16 \neq 1/4 = p(E \cap F).$$

Pairwise and Mutual Independence

Definition: The events $E_1, E_2, \ldots, E_n$ are *pairwise independent* if and only if $p(E_i \cap E_j) = p(E_i)\,p(E_j)$ for all pairs i and j with $i \leq j \leq n$.

The events are *mutually independent* if

$$p(E_{i_1} \cap E_{i_2} \cap \cdots \cap E_{i_m}) = p(E_{i_1})p(E_{i_2}) \cdots p(E_{i_m})$$

whenever i_j, $j = 1,2,\ldots, m$, are integers with

$$1 \leq i_1 < i_2 < \cdots < i_m \leq n \quad \text{and } m \geq 2.$$

Bernoulli Trials

James Bernoulli
(1854 – 1705)



Definition: Suppose an experiment can have only two possible outcomes, *e.g.*, the flipping of a coin or the random generation of a bit.

- Each performance of the experiment is called a *Bernoulli trial*.
- One outcome is called a *success* and the other a *failure*.
- If p is the probability of success and q the probability of failure, then $p + q = 1$.

- Many problems involve determining the probability of k successes when an experiment consists of n mutually independent Bernoulli trials.

Bernoulli Trials

Example: A coin is biased so that the probability of heads is $2/3$. What is the probability that exactly four heads occur when the coin is flipped seven times?

Solution: There are $2^7 = 128$ possible outcomes. The number of ways four of the seven flips can be heads is $C(7,4)$. The probability of each of the outcomes is $(2/3)^4(1/3)^3$ since the seven flips are independent. Hence, the probability that exactly four heads occur is

$$C(7,4)\,(2/3)^4(1/3)^3 = (35 \cdot 16)/\,2^7 = 560/\,2187.$$

Probability of k Successes in n Independent Bernoulli Trials.

Theorem 2: The probability of exactly k successes in n independent Bernoulli trials, with probability of success p and probability of failure $q = 1 - p$, is

$$C(n,k)p^kq^{n-k}.$$

Proof: The outcome of n Bernoulli trials is an n-tuple $(t_1,t_2,\ldots,t_n)$, where each is t_i either S (success) or F (failure). The probability of each outcome of n trials consisting of k successes and $k - 1$ failures (in any order) is p^kq^{n-k}. Because there are $C(n,k)$ n-tuples of Ss and Fs that contain exactly k Ss, the probability of k successes is $C(n,k)p^kq^{n-k}$. ◀

- We denote by $b(k:n,p)$ the probability of k successes in n independent Bernoulli trials with p the probability of success. Viewed as a function of k, $b(k:n,p)$ is the *binomial distribution*. By Theorem 2,

$$b(k:n,p) = C(n,k)p^kq^{n-k}.$$

Random Variables

Definition: A *random variable* is a function from the sample space of an experiment to the set of real numbers. That is, a random variable assigns a real number to each possible outcome.

- A random variable is a function. It is not a variable, and it is not random!
- In the late 1940s W. Feller and J.L. Doob flipped a coin to see whether both would use "random variable" or the more fitting "chance variable." Unfortunately, Feller won and the term "random variable" has been used ever since.

Random Variables

Definition: The *distribution* of a random variable X on a sample space S is the set of pairs $(r, p(X = r))$ for all $r \in X(S)$, where $p(X = r)$ is the probability that X takes the value r.

Example: Suppose that a coin is flipped three times. Let $X(t)$ be the random variable that equals the number of heads that appear when t is the outcome. Then $X(t)$ takes on the following values:

$X(HHH) = 3$, $X(TTT) = 0$,
$X(HHT) = X(HTH) = X(THH) = 2$,
$X(TTH) = X(THT) = X(HTT) = 1$.

Each of the eight possible outcomes has probability $1/8$. So, the distribution of $X(t)$ is $p(X = 3) = 1/8$, $p(X = 2) = 3/8$, $p(X = 1) = 3/8$, and $p(X = 0) = 1/8$.

The Famous Birthday Problem

- The puzzle of finding the number of people needed in a room to ensure that the probability of at least two of them having the same birthday is more than ½ has a surprising answer, which we now find.

Solution: We assume that all birthdays are equally likely and that there are 366 days in the year. First, we find the probability p_n that at least two of n people have different birthdays.

Now, imagine the people entering the room one by one. The probability that at least two have the same birthday is $1 - p_n$.

- The probability that the birthday of the second person is different from that of the first is $365/366$.
- The probability that the birthday of the third person is different from the other two, when these have two different birthdays, is $364/366$.
- In general, the probability that the jth person has a birthday different from the birthdays of those already in the room, assuming that these people all have different birthdays, is $(366 - (j - 1))/366 = (367 - j)/366$.
- Hence, $p_n = (365/366)(364/366) \cdots (367 - n)/366$.
- Therefore, $1 - p_n = 1 - (365/366)(364/366) \cdots (367 - n)/366$.

Checking various values for n with computation help tells us that for $n = 22$, $1 - p_n \approx 0.457$, and for $n = 23$, $1 - p_n \approx 0.506$. Consequently, a minimum number of 23 people are needed so that the probability that at least two of them have the same birthday is greater than $1/2$.

Monte Carlo Algorithms

- Algorithms that make random choices at one or more steps are called *probabilistic algorithms*.
- *Monte Carlo algorithms* are probabilistic algorithms used to answer decision problems, which are problems that either have "true" or "false" as their answer.
  - A Monte Carlo algorithm consists of a sequence of tests. For each test the algorithm responds "true" or 'unknown.'
  - If the response is "true," the algorithm terminates with the answer is "true."
  - After running a specified sequence of tests where every step yields "unknown", the algorithm outputs "false."
  - The idea is that the probability of the algorithm incorrectly outputting "false" should be very small as long as a sufficient number of tests are performed.

Probabilistic Primality Testing

- Probabilistic primality testing (*see Example* 16 *in text*) is an example of a Monte Carlo algorithm, which is used to find large primes to generate the encryption keys for RSA cryptography (*as discussed in Chapter* 4).
  - An integer n greater than 1 can be shown to be composite (i.e., not prime) if it fails a particular test (Miller's test), using a random integer b with $1 < b < n$ as the base. But if n passes Miller's test for a particular base b, it may either be prime or composite. The probability that a composite integer passes n Miller's test is for a random b, is less that ¼.
  - So failing the test, is the "true" response in a Monte Carlo algorithm, and passing the test is "unknown."
  - If the test is performed k times (choosing a random integer b each time) and the number n passes Miller's test at every iteration, then the probability that it is composite is less than $(1/4)^k$. So for a sufficiently, large k, the probability that n is composite even though it has passed all k iterations of Miller's test is small. For example, with 10 iterations, the probability that n is composite is less than 1 in 1,000,000.

Bayes' Theorem

Section 7.3

Section Summary

- Bayes' Theorem
- Generalized Bayes' Theorem
- Bayesian Spam Filters
- A.I. Applications (*optional*, *not currently included in the overheads*)

Motivation for Bayes' Theorem

- Bayes' theorem allows us to use probability to answer questions such as the following:
  - Given that someone tests positive for having a particular disease, what is the probability that they actually do have the disease?
  - Given that someone tests negative for the disease, what is the probability, that in fact they do have the disease?
- Bayes' theorem has applications to medicine, law, artificial intelligence, engineering, and many diverse other areas.

Bayes' Theorem

Thomas Bayes (1702-1761)



Bayes' Theorem: Suppose that *E* and *F* are events from a sample space S such that $p(E) \neq 0$ and $p(F) \neq 0$. Then:

$$p(F|E) = \frac{p(E|F)p(F)}{p(E|F)p(F) + p(E|\overline{F})p(\overline{F})}$$

Example: We have two boxes. The first box contains two green balls and seven red balls. The second contains four green balls and three red balls. Bob selects one of the boxes at random. Then he selects a ball from that box at random. If he has a red ball, what is the probability that he selected a ball from the first box.

- Let *E* be the event that Bob has chosen a red ball and *F* be the event that Bob has chosen the first box.
- By Bayes' theorem the probability that Bob has picked the first box is:

$$p(F|E) = \frac{(7/9)(1/2)}{(7/9)(1/2) + (3/7)(1/2)} = \frac{7/18}{38/63} = \frac{49}{76} \approx 0.645.$$

Derivation of Bayes' Theorem

- Recall the definition of the conditional probability $p(E|F)$:

$$p(E|F) = \frac{p(E \cap F)}{p(F)}$$

- From this definition, it follows that:

$$p(E|F) = \frac{p(E \cap F)}{p(F)} \quad , \quad p(F|E) = \frac{p(E \cap F)}{p(E)}$$

continued →

Derivation of Bayes' Theorem

On the last slide we showed that

$$p(E|F)p(F) = p(E \cap F), \; p(F|E)p(E) = p(E \cap F)$$

Equating the two formulas for p(E F) shows that

$$p(E|F)p(F) = p(F|E)p(E)$$

Solving for $p(E|F)$ and for $p(F|E)$ tells us that

$$p(E|F) = \frac{p(F|E)p(E)}{p(F)} \quad , \quad p(F|E) = \frac{p(E|F)p(F)}{p(E)}$$

continued →

Derivation of Bayes' Theorem

On the last slide we showed that:

$$p(F|E) = \frac{p(E|F)p(F)}{p(E)}$$

Note that

$$p(E) = p(E|F)p(F) + p(E|\overline{F})p(\overline{F})$$

since $p(E) = p(E \cap F) + p(E \cap \overline{F})$

because $E = E \cap S = E \cap (F \cup \overline{F}) = (E \cap F) \cup (E \cap \overline{F})$

and $(E \cap F) \cap (E \cap \overline{F}) = \emptyset$

By the definition of conditional probability,

$$p(E) = p(E \cap F) + p(E \cap \overline{F}) = p(E|F)p(F) + p(E|\overline{F})p(\overline{F})$$

Hence,

$$p(F|E) = \frac{p(E|F)p(F)}{p(E|F)p(F) + p(E|\overline{F})p(\overline{F})}$$

◀

Applying Bayes' Theorem

Example: Suppose that one person in 100,000 has a particular disease. There is a test for the disease that gives a positive result 99% of the time when given to someone with the disease. When given to someone without the disease, 99.5% of the time it gives a negative result. Find

a) the probability that a person who test positive has the disease.

b) the probability that a person who test negative does not have the disease.

- Should someone who tests positive be worried?

Applying Bayes' Theorem

Solution: Let *D* be the event that the person has the disease, and *E* be the event that this person tests positive. We need to compute $p(D|E)$ from $p(D)$, $p(E|D)$, $p(E|\overline{D})$, $p(\quad)$.

$p(D) = 1/100,000 = 0.00001 \quad p(\overline{D}) = 1 - 0.00001 = 0.99999$

$p(E|D) = .99 \quad p(\overline{E}|D) = .01 \quad p(E|\overline{D}) = .005 \quad p(\overline{E}|\overline{D}) = .995$

$$p(D|E) = \frac{p(E|D)p(D)}{p(E|D)p(D) + p(E|\overline{D})p(\overline{D})}$$

$$= \frac{(0.99)(0.00001)}{(0.99)(0.00001) + (0.005)(0.99999)}$$

$$\approx 0.002$$

Can you use this formula to explain why the resulting probability is surprisingly small?

So, don't worry too much, if your test for this disease comes back positive.

Applying Bayes' Theorem

- What if the result is negative?

$$p(\overline{D}|\overline{E}) = \frac{p(\overline{E}|\overline{D})p(\overline{D})}{p(\overline{E}|\overline{D})p(\overline{D}) + p(\overline{E}|D)p(D)}$$

$$= \frac{(0.995)(0.99999)}{(0.995)(0.99999) + (0.01)(0.00001)}$$

$$\approx 0.9999999$$

So, the probability you have the disease if you test negative is $p(D|\overline{E}) \approx 1 - 0.9999999 = 0.0000001.$

- So, it is extremely unlikely you have the disease if you test negative.

Generalized Bayes' Theorem

Generalized Bayes' Theorem: Suppose that *E* is an event from a sample space *S* and that $F_1, F_2, \ldots, F_n$ are mutually exclusive events such that $\bigcup_{i}^{n} F_i = S$.

Assume that $p(E) \neq 0$ for $i = 1, 2, \ldots, n$. Then

$$p(F_j|E) = \frac{p(E|F_j)p(F_j)}{\sum_{i=1}^{n} p(E|F_i)p(F_i)}.$$

Exercise 17 *asks for the proof.*

Bayesian Spam Filters

- How do we develop a tool for determining whether an email is likely to be spam?
- If we have an initial set B of spam messages and set G of non-spam messages. We can use this information along with Bayes' law to predict the probability that a new email message is spam.
- We look at a particular word w, and count the number of times that it occurs in B and in G; $n_B(w)$ and $n_G(w)$.
  - Estimated probability that a spam message contains w is: $p(w) = n_B(w)/|B|$
  - Estimated probability that a message that is not spam contains w is: $q(w) = n_G(w)/|G|$

continued →

Bayesian Spam Filters

Example: We find that the word "Rolex" occurs in 250 out of 2000 spam messages and occurs in 5 out of 1000 non-spam messages. Estimate the probability that an incoming message is spam. Suppose our threshold for rejecting the email is 0.9.

Solution: $p(Rolex) = 250/2000 = .0125$ and $q(Rolex) = 5/1000 = 0.005$.

$$r(Rolex) = \frac{p(Rolex)}{p(Rolex) + q(Rolex)} = \frac{0.125}{0.125 + .005} = \frac{0.125}{0.125 + .005} \approx 0.962$$

We class the message as spam and reject the email!

Bayesian Spam Filters

- Let S be the event that the message is spam, and E be the event that the message contains the word w.
- Using Bayes' Rule, $p(S|E) = \dfrac{p(E|S)p(S)}{p(E|S)p(S) + p(E|\overline{S})p(\overline{S})}$

Assuming that it is equally likely that an arbitrary message is spam and is not spam; i.e., $p(S) = ½$.

$$p(S|E) = \frac{p(E|S)}{p(E|S) + p(E|\overline{S})}$$

Note: If we have data on the frequency of spam messages, we can obtain a better estimate for p(s). (*See Exercise* 22.)

Using our empirical estimates of $p(E \mid S)$ and $p(E \mid \overline{S})$.

$$r(w) = \frac{p(w)}{p(w) + q(w)}$$

$r(w)$ estimates the probability that the message is spam. We can class the message as spam if $r(w)$ is above a threshold.

Bayesian Spam Filters using Multiple Words

- Accuracy can be improved by considering more than one word as evidence.
- Consider the case where E_1 and E_2 denote the events that the message contains the words w_1 and w_2 respectively.
- We make the simplifying assumption that the events are independent. And again we assume that $p(S) = ½$.

$$p(S|E_1 \cap E_2) = \frac{p(E_1|S)p(E_2|S)}{p(E_1|S)p(E_2|S) + p(E_1|\overline{S})p(E_2|\overline{S})}$$

$$r(w_1, w_2) = \frac{p(w_1)p(w_2)}{p(w_1)p(w_2) + q(w_1)q(w_2)}$$

Bayesian Spam Filters using Multiple Words

Example: We have 2000 spam messages and 1000 non-spam messages. The word "stock" occurs 400 times in the spam messages and 60 times in the non-spam. The word "undervalued" occurs in 200 spam messages and 25 non-spam.

Solution: $p(stock) = 400/2000 = .2$, $q(stock) = 60/1000 = .06$, $p(undervalued) = 200/2000 = .1$, $q(undervalued) = 25/1000 = .025$

$$r(stock, undervalued) = \frac{p(stock)p(undervalued)}{p(stock)p(undervalued) + q(stock)q(undervalued)}$$

$$= \frac{(0.2)(0.1)}{(0.2)(0.1) + (0.06)(0.025)} \approx 0.930$$

If our threshold is .9, we class the message as spam and reject it.

Bayesian Spam Filters using Multiple Words

- In general, the more words we consider, the more accurate the spam filter. With the independence assumption if we consider k words:

$$p(S|\bigcap_{i=1}^{k} E_i) = \frac{\Pi_{i=1}^{k} p(E_i|S)}{\Pi_{i=1}^{k} p(E_1|S) + \Pi_{i=1}^{k} p(E_i|\overline{S})}$$

$$r(w_1, w_2, ...w_n) = \frac{\Pi_i^k p(w_i)}{\Pi_{i=1}^{k} p(w_i) + \Pi_{i=1}^{k} q(w_i)}$$

We can further improve the filter by considering pairs of words as a single block or certain types of strings.

Expected Value and Variance

Section 6.4

Section Summary

- Expected Value
- Linearity of Expectations
- Average-Case Computational Complexity
- Geometric Distribution
- Independent Random Variables
- Variance
- Chebyshev's Inequality

Expected Value

Definition: The *expected value* (or *expectation* or *mean*) of the random variable $X(s)$ on the sample space S is equal to

$$E(X) = \sum_{x \in S} p(s)X(s).$$

Example-Expected Value of a Die: Let X be the number that comes up when a fair die is rolled. What is the expected value of X?

Solution: The random variable X takes the values 1, 2, 3, 4, 5, or 6. Each has probability 1/6. It follows that

$$E(X) = \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2 + \cdots \frac{1}{6} \cdot 6 = \frac{21}{6} = \frac{7}{2}.$$

Expected Value

Theorem 1: If X is a random variable and $p(X = r)$ is the probability that X = r, so that

$$p(X = r) = \sum_{s \in S, X(s) = r} p(s),$$ then

$$E(X) = \sum_{r \in X(S)} p(X = r)r.$$

Proof: Suppose that X is a random variable with range $X(S)$ and let $p(X = r)$ be the probability that X takes the value r. Consequently, $p(X = r)$ is the sum of the probabilities of the outcomes s such that $X(s) = r$. Hence,

$$E(X) = \sum_{r \in X(S)} p(X = r)r.$$ ◀

Expected Value

Theorem 2: The expected number of successes when n mutually independent Bernoulli trials are performed, where, the probability of success on each trial, $p = np$.

Proof: Let X be the random variable equal to the number of success in n trials. By Theorem 2 of section 7.2, $p(X = k) = C(n,k)p^k q^{n-k}$. Hence,

$$E(X) = \sum_{k=1}^{n} kp(X = k)$$ by Theorem 1

continued →

Expected Value

$$E(X) = \sum_{k=1}^{n} kp(X = k)$$ from previous page

$$= \sum_{k=1}^{n} kC(n,k)p^k q^{n-k}$$ by Theorem 2 in Section 7.2

$$= \sum_{k=1}^{n} nC(n-1,k-1)p^k q^{n-k}$$ by Exercise 21 in Section 6.4

$$= np \sum_{k=1}^{n} C(n-1,k-1)p^{k-1} q^{n-k}$$ factoring np from each term

$$= np \sum_{j=0}^{n-1} C(n-1,j)p^j q^{n-1-j}$$ shifting index of summation with $j = k - 1$

$$= np(p+1)^{n-1}$$ by the binomial theom

$$= np.$$ because $p + q = 1$ ◀

Linearity of Expectations

The following theorem tells us that expected values are linear. For example, the expected value of the sum of random variables is the sum of their expected values.

Theorem 3: If X_i, $i = 1, 2, \ldots, n$ with n a positive integer, are random variables on S, and if a and b are real numbers, then

(*i*) $E(X_1 + X_2 + \ldots + X_n) = E(X_1) + E(X_2) + \ldots + E(X_n)$
(*ii*) $E(aX + b) = aE(X) + b$.

see the text for the proof

Linearity of Expectations

Expected Value in the Hatcheck Problem: A new employee started a job checking hats, but forgot to put the claim check numbers on the hats. So, the n customers just receive a random hat from those remaining. What is the expected number of hat returned correctly?

Solution: Let X be the random variable that equals the number of people who receive the correct hat. Note that $X = X_1 + X_2 + \cdots + X_n$,

where $X_i = 1$ if the ith person receives the hat and $X_i = 0$ otherwise.

- Because it is equally likely that the checker returns any of the hats to the ith person, it follows that the probability that the ith person receives the correct hat is $1/n$. Consequently (by Theorem 1), for all i

  $E(X_i) = 1 \cdot p(X_i = 1) + 0 \cdot p(X_i = 0) = 1 \cdot 1/n + 0 = 1/n$.

- By the linearity of expectations (Theorem 3), it follows that:

  $E(X) = E(X_1) + E(X_2) + \cdots + E(X_n) = n \cdot 1/n - 1.$

Consequently, the average number of people who receive the correct hat is exactly 1. (Surprisingly, this answer remains the same no matter how many people have checked their hats!)

Linearity of Expectations

Expected Number of Inversions in a Permutation: The ordered pair (i,j) is an *inversion* in a permutation of the first n positive integers if $i < j$, but j precedes i in the permutation.

Example: There are six inversions in the permutation of 3,5, 1, 4, 2

(1, 3), (1, 5), (2, 3), (2, 4), (2, 5), (4, 5).

Find the average number of inversions in a random permutation of the first n integers.

Solution: Let $I_{i,j}$ be the random variable on the set of all permutations of the first n positive integers with $I_{i,j} = 1$ if (i,j) is an inversion of the permutation and $I_{i,j} = 0$ otherwise. If X is the random variable equal to the number of inversions in the permutation, then

$$X = \sum_{1 \le i < j \le n} I_{i,j}.$$

- Since it is equally likely for i to precede j in a randomly chosen permutation as it is for j to precede i, we have: $E(I_{i,j}) = 1 \cdot p(I_{i,j} = 1) + 0 \cdot p(I_{i,j} = 0) = 1 \cdot 1/2 + 0 = ½$, for all (i,j).
- Because there are $\binom{n}{2}$ pairs i and j with $1 \le i < j \le n$, by the linearity of expectations (Theorem 3), we have:

$$E(X) = \sum_{1 \le i < j \le n} E(I_{i,j}) = \binom{n}{2} \cdot \frac{1}{2} = \frac{n-1}{2} \cdot \frac{1}{2}.$$

Consequently, it follows that there is an average of $n(n-1)/4$ inversions in a random permutation of the first n positive integers.

Average-Case Computational Complexity

The average-case computational complexity of an algorithm can be found by computing the expected value of a random variable.

- Let the sample space of an experiment be the set of possible inputs a_j, $j = 1, 2, \ldots, n$, and let the random variable X be the assignment to a_j of the number of operations used by the algorithm when given a_j as input.
- Assign a probability $p(a_j)$ to each possible input value a_j.
- The expected value of X is the average-case computational complexity of the algorithm.

$$E(X) = \sum_{j=1}^{n} p(a_j)X(a_j).$$

Average-Case Complexity of Linear Search

What is the average-case complexity of linear search (described in Chapter 3) if the probability that x is in the list is p and it is equally likely that x is any of the n elements of the list?

```
procedure linear search(x: integer, a1, a2, …,an: distinct integers)
i := 1
while (i ≤ n and x ≠ ai)
    i := i + 1
    if i ≤ n then location := i
        else location := 0
return location{location is the subscript of the term that equals x, or
   is 0 if x is not found}
```

continued →

Average-Case Complexity of Linear Search

Solution: There are $n + 1$ possible types of input: one type for each of the n numbers on the list and one additional type for the numbers not on the list.

Recall that:

- $2i + 1$ comparisons are needed if x equals the ith element of the list.
- $2n + 2$ comparisons are used if x is not on the list.

The probability that x equals a_i is p/n and the probability that x is not in the list is $q = 1 - p$. The average-case case computational complexity of the linear search algorithm is:

$$\begin{aligned} E &= 3p/n + 5p/n + \ldots + (2n+1)p/n + (2n+2)q \\ &= (p/n)(3 + 5 + \ldots + (2n+1)) + (2n+2)q \\ &= (p/n)((n+1)^2 - 1) + (2n+2)q \quad (\textit{Example 2 from Section 5.1}) \\ &= p(n+2) + (2n+2)q. \end{aligned}$$

- When x is guaranteed to be in the list, p = 1, $q = 0$, so that $E = n + 2$.
- When p is ½ and q = ½, then E = $(n + 2)/2 + n + 1 = (3n + 4)/2$.
- When p is ¾ and q = ¼ then E = $(n + 2)/4 + (n + 1)/2 = (5n + 8)/4$.
- When x is guaranteed not to be in the list, p = 0 and $q = 1$, then $E = 2n + 2$.

Average-Case Complexity of Insertion Sort

- What is the average number of comparisons used by insertion sort from Chapter 3) to sort n distinct elements?
  - At step i for i = 2, ….,n, insertion sort inserts the ith element in the original list into the correct position in the sorted list of the first i -1 elements.

```
procedure insertion sort
            (a1,…,an: reals  with n ≥ 2)
    for j := 2 to n
        i := 1
        while aj > ai
            i := i + 1
        m := aj
        for k := 0 to j  − i − 1
            aj-k := aj-k-1
        ai := m
{Now a1,…,an is in increasing order}
```

continued →

Average-Case Complexity of Insertion Sort

Solution: Let X be the random variable equal to the number of comparisons used by insertion sort to sort a list of $a_1, a_2, \ldots, a_n$ distinct elements. $E(X)$ is the average number of comparisons.

- Let X_i be the random variable equal to the number of comparisons used to insert a_i into the proper position after the first $i - 1$ elements $a_1, a_2, \ldots, a_{i-1}$ have been sorted.
- Since $X = X_2 + X_3 + \cdots + X_n$,
  $E(X) = E(X_2 + X_3 + \cdots + X_n) = E(X_2) + E(X_3) + \cdots + E(X_n)$.

- To find $E(X_i)$ for $i = 2,3,\ldots,n$, let $p_j(k)$ be the probability that the largest of the first j elements in the list occurs at the kth position, that is, $\max(a_1, a_2, \ldots, a_j) = a_k$, where $1 \le k \le j$.
- Assume uniform distribution; $p_j(k) = 1/j$.
- Then $X_j(k) = k$.

continued →

Average-Case Complexity of Insertion Sort

- Since a_i could be inserted into any of the first i positions

$$E(X_i) = \sum_{k=1}^{i} p_i(k) \cdot X_i(k) = \sum_{k=1}^{i} \frac{1}{i} \cdot k \quad = \frac{1}{i} \sum_{k=1}^{i} k = \frac{1}{i} \cdot \frac{i(i+1)}{2} = \frac{i+1}{2}$$

- It follows that

$$E(X) = \sum_{i=2}^{n} E(X_i) = \sum_{i=2}^{n} \frac{i+1}{2} = \frac{1}{2} \sum_{j=3}^{n+1} j$$

$$= \frac{1}{2} \frac{(n+1)(n+2)}{2} - \frac{1}{2}(1+2) = \frac{n^2+3n-4}{4}$$

- Hence, the average-case complexity is $\theta(n^2)$.

The Geometric Distribution

Definition 2: A random variable X has *geometric distribution with parameter p* if $p(X = k) = (1 - p)^{k-1}p$ for $k = 1,2,3,\ldots$, where p is a real number with $0 \le p \le 1$.

Theorem 4: If the random variable X has the geometric distribution with parameter p, then $E(X) = 1/p$.

Example: Suppose the probability that a coin comes up tails is p. What is the expected number of flips until this coin comes up tails?

- The sample space is {T, HT, HHT, HHHT, HHHHT, …}.
- Let X be the random variable equal to the number of flips in an element of the sample space; $X(T) = 1$, $X(HT) = 2$, $X(HHT) = 3$, etc.
- By Theorem 4, $E(X) = 1/p$.

see text for full details

Independent Random Variables

Definition 3: The random variables X and Y on a sample space S are independent if

$p(X = r_1$ and $Y = r_2) = p(X = r_1) \cdot p(Y = r_2)$.

Theorem 5: If X and Y are independent variables on a sample space S, then $E(XY) = E(X)E(Y)$.

see text for the proof

Variance

Deviation: The *deviation* of X at s ∈ S is $X(s) - E(X)$, the difference between the value of X and the mean of X.

Definition 4: Let X be a random variable on the sample space S. The *variance* of X, denoted by V(X) is

$$V(X) = \sum_{s \in S} (X(s) - E(X))^2 p(s).$$

That is $V(X)$ is the weighted average of the square of the deviation of X. The standard deviation of X, denoted by $\sigma(X)$ is defined to be $\sqrt{V(X)}$.

Theorem 6: If X is a random variable on a sample space S, then $V(X) = E(X^2) - E(X)^2$.

see text for the proof

Corollary 1: If X is a random variable on a sample space S and $E(X) = \mu$, then $V(X) = E((X - \mu)^2)$.

see text for the proof.

Variance

Example: What is the variance of the random variable X, where $X(t) = 1$ if a Bernoulli trial is a success and $X(t) = 0$ if it is a failure, where p is the probability of success and q is the probability of failure?

Solution: Because X takes only the values 0 and 1, it follows that $X^2(t) = X(t)$. Hence,

$$V(X) = E(X^2) - E(X)^2 = p - p^2 = p(1 - p) = pq.$$

Variance of the Value of a Die: What is the variance of a random variable X, where X is the number that comes up when a fair die is rolled?

Solution: We have $V(X) = E(X^2) - E(X)^2$. In an earlier example, we saw that $E(X) = 7/2$. Note that

$$E(X^2) = 1/6(1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2) = 91/6.$$

$$V(X) = \frac{91}{6} - \left(\frac{7}{2}\right)^2 = \frac{35}{12}.$$

We conclude that

Chebyshev's Inequality

Pafnuty Lvovich Chebyshev (1821-1894)



Chebyschev's Inequality: Let X be a random variable on a sample space S with probability function p. If r is a positive real number, then

$$p(|X(s) - E(X)| \geq r) \leq V(X)/r^2.$$

see text for the proof

Example: Suppose that X is a random variable that counts the number of tails when a fair coin is tossed n times. Note that X is the number of successes when n independent Bernoulli trials, each with probability of success ½ are done. Hence, (by Theorem 2) $E(X) = n/2$ and (by Example 18) $V(X) = n/4$.

By Chebyschev's inequality with $r = \sqrt{n}$,

$$p(|X(s) - n/2| \geq \sqrt{n}) \leq (n/4)(\sqrt{n})^2 = ¼.$$

This means that the probability that the number of tails that come up on n tosses deviates from the mean , $n/2$, by more than $\sqrt{n}$ is no larger than ¼.

Variance

Irenée-Jules Bienaymé (1796-1878)



Bienaymé's Formula: If X and Y are two independent random variables on a sample space S, then $V(X + Y) = V(X) + V(Y)$. Furthermore, if X_i, $i = 1,2, \ldots,n$, with n a positive integer, are pairwise independent random variables on S, then

$$V(X_1 + X_2 + \cdots + X_n) = V(X_1) + V(X_2) + \cdots + V(X_n).$$

see text for the proof

Example: Find the variance of the number of successes when n independent Bernoulli trials are performed, where on each trial, p is the probability of success and q is the probability of failure.

Solution: Let X_i be the random variable with $X_i((t_1, t_2, \ldots, t_n)) = 1$ if trial t_i is a success and $X_i((t_1, t_2, \ldots, t_n)) = 0$ if it is a failure. Let $X = X_2 + X_3 + \ldots X_n$. Then X counts the number of successes in the n trials.

- By Bienaymé 's Formula, it follows that $V(X) = V(X_1) + V(X_2) + \cdots + V(X_n)$.
- By the previous example ,$V(X_i) = pq$ for $i = 1,2, \ldots,n$.

Hence, $V(X) = npq$.

Advanced Counting Techniques

Chapter 8

With Question/Answer Animations

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Applications of Recurrence Relations
- Solving Linear Recurrence Relations
  - Homogeneous Recurrence Relations
  - Nonhomogeneous Recurrence Relations
- Divide-and-Conquer Algorithms and Recurrence Relations
- Generating Functions
- Inclusion-Exclusion
- Applications of Inclusion-Exclusion

Applications of Recurrence Relations

Section 8.1

Section Summary

- Applications of Recurrence Relations
  - Fibonacci Numbers
  - The Tower of Hanoi
  - Counting Problems
- Algorithms and Recurrence Relations (*not currently included in overheads*)

Recurrence Relations
(recalling definitions from Chapter 2)

Definition: A *recurrence relation* for the sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more of the previous terms of the sequence, namely, $a_0, a_1, \ldots, a_{n-1}$, for all integers n with $n \geq n_0$, where n_0 is a nonnegative integer.

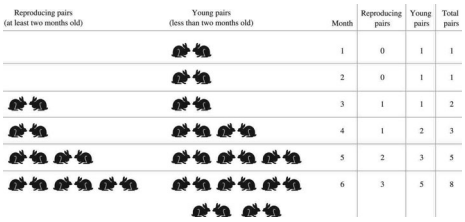
- A sequence is called a *solution* of a recurrence relation if its terms satisfy the recurrence relation.
- The *initial conditions* for a sequence specify the terms that precede the first term where the recurrence relation takes effect.

Rabbits and the Fiobonacci Numbers

Example: A young pair of rabbits (one of each gender) is placed on an island. A pair of rabbits does not breed until they are 2 months old. After they are 2 months old, each pair of rabbits produces another pair each month. Find a recurrence relation for the number of pairs of rabbits on the island after *n* months, assuming that rabbits never die.

This is the original problem considered by Leonardo Pisano (Fibonacci) in the thirteenth century.

Rabbits and the Fiobonacci Numbers (*cont.*)



Reproducing pairs (at least two months old)	Young pairs (less than two months old)	Month	Reproducing pairs	Young pairs	Total pairs
		1	0	1	1
		2	0	1	1
		3	1	1	2
		4	1	2	3
		5	2	3	5
		6	3	5	8

Modeling the Population Growth of Rabbits on an Island

Rabbits and the Fibonacci Numbers (*cont.*)

Solution: Let f_n be the the number of pairs of rabbits after *n* months.

- There are is $f_1 = 1$ pairs of rabbits on the island at the end of the first month.
- We also have $f_2 = 1$ because the pair does not breed during the first month.
- To find the number of pairs on the island after *n* months, add the number on the island after the previous month, f_{n-1}, and the number of newborn pairs, which equals f_{n-2}, because each newborn pair comes from a pair at least two months old.

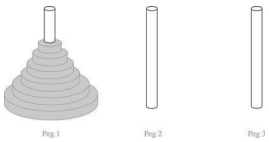
Consequently the sequence $\{f_n\}$ satisfies the recurrence relation $f_n = f_{n-1} + f_{n-2}$ for $n \geq 3$ with the initial conditions $f_1 = 1$ and $f_2 = 1$. The number of pairs of rabbits on the island after *n* months is given by the *n*th Fibonacci number.

The Tower of Hanoi

In the late nineteenth century, the French mathematician Édouard Lucas invented a puzzle consisting of three pegs on a board with disks of different sizes. Initially all of the disks are on the first peg in order of size, with the largest on the bottom.

Rules: You are allowed to move the disks one at a time from one peg to another as long as a larger disk is never placed on a smaller.
Goal: Using allowable moves, end up with all the disks on the second peg in order of size with largest on the bottom.

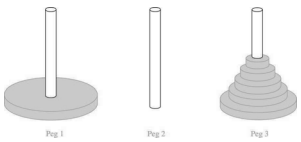
The Tower of Hanoi (*continued*)



The Initial Position in the Tower of Hanoi Puzzle

The Tower of Hanoi (*continued*)

Solution: Let $\{H_n\}$ denote the number of moves needed to solve the Tower of Hanoi Puzzle with *n* disks. Set up a recurrence relation for the sequence $\{H_n\}$. Begin with *n* disks on peg 1. We can transfer the top $n-1$ disks, following the rules of the puzzle, to peg 3 using H_{n-1} moves.



First, we use 1 move to transfer the largest disk to the second peg. Then we transfer the $n-1$ disks from peg 3 to peg 2 using H_{n-1} additional moves. This can not be done in fewer steps. Hence,

$$H_n = 2H_{n-1} + 1.$$

The initial condition is $H_1 = 1$ since a single disk can be transferred from peg 1 to peg 2 in one move.

The Tower of Hanoi (*continued*)

- We can use an iterative approach to solve this recurrence relation by repeatedly expressing H_n in terms of the previous terms of the sequence.

$$\begin{aligned} H_n &= 2H_{n-1} + 1 \\ &= 2(2H_{n-2} + 1) + 1 = 2^2 H_{n-2} + 2 + 1 \\ &= 2^2(2H_{n-3} + 1) + 2 + 1 = 2^3 H_{n-3} + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}H_1 + 2^{n-2} + 2^{n-3} + \ldots + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + 2^{n-3} + \ldots + 2 + 1 \quad \textit{because } H_1 = 1 \\ &= 2^n - 1 \quad \textit{using the formula for the sum of the terms of a geometric series} \end{aligned}$$

- There was a myth created with the puzzle. Monks in a tower in Hanoi are transferring 64 gold disks from one peg to another following the rules of the puzzle. They move one disk each day. When the puzzle is finished, the world will end.
- Using this formula for the 64 gold disks of the myth,
  $2^{64} - 1 = 18{,}446{,}744{,}073{,}709{,}551{,}615$
  days are needed to solve the puzzle, which is more than 500 billion years.
- Reve's puzzle (proposed in 1907 by Henry Dudeney) is similar but has 4 pegs. There is a well-known unsettled conjecture for the the minimum number of moves needed to solve this puzzle. (*see Exercises* 38-45)

Counting Bit Strings

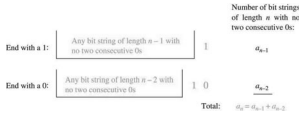
Example 3: Find a recurrence relation and give initial conditions for the number of bit strings of length *n* without two consecutive 0s. How many such bit strings are there of length five?

Solution: Let a_n denote the number of bit strings of length *n* without two consecutive 0s. To obtain a recurrence relation for $\{a_n\}$ note that the number of bit strings of length *n* that do not have two consecutive 0s is the number of bit strings ending with a 0 plus the number of such bit strings ending with a 1.

Now assume that $n \geq 3$.

- The bit strings of length *n* ending with 1 without two consecutive 0s are the bit strings of length $n-1$ with no two consecutive 0s with a 1 at the end. Hence, there are a_{n-1} such bit strings.
- The bit strings of length n ending with 0 without two consecutive 0s are the bit strings of length $n-2$ with no two consecutive 0s with 10 at the end. Hence, there are a_{n-2} such bit strings.

We conclude that $a_n = a_{n-1} + a_{n-2}$ for $n \geq 3$.



Bit Strings (*continued*)

The initial conditions are:
- $a_1 = 2$, since both the bit strings 0 and 1 do not have consecutive 0s.
- $a_2 = 3$, since the bit strings 01, 10, and 11 do not have consecutive 0s, while 00 does.

To obtain a_5, we use the recurrence relation three times to find that:

- $a_3 = a_2 + a_1 = 3 + 2 = 5$
- $a_4 = a_3 + a_2 = 5 + 3 = 8$
- $a_5 = a_4 + a_3 = 8 + 5 = 13$

Note that $\{a_n\}$ satisfies the same recurrence relation as the Fibonacci sequence. Since $a_1 = f_3$ and $a_2 = f_4$, we conclude that $a_n = f_{n+2}$.

Counting the Ways to Parenthesize a Product

Example: Find a recurrence relation for C_n, the number of ways to parenthesize the product of $n + 1$ numbers, $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$, to specify the order of multiplication.

For example, $C_3 = 5$, since all the possible ways to parenthesize 4 numbers are

$$((x_0 \cdot x_1) \cdot x_2) \cdot x_3,\quad (x_0 \cdot (x_1 \cdot x_2)) \cdot x_3,\quad (x_0 \cdot x_1) \cdot (x_2 \cdot x_3),\quad x_0 \cdot ((x_1 \cdot x_2) \cdot x_3),\quad x_0 \cdot (x_1 \cdot (x_2 \cdot x_3))$$

Solution: Note that however parentheses are inserted in $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$, one "$\cdot$" operator remains outside all parentheses. This final operator appears between two of the $n + 1$ numbers, say x_k and x_{k+1}. Since there are C_k ways to insert parentheses in the product $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_k$ and C_{n-k-1} ways to insert parentheses in the product $x_{k+1} \cdot x_{k+2} \cdot \cdots \cdot x_n$, we have

$$\begin{aligned} C_n &= C_0 C_{n-1} + C_1 C_{n-2} + \cdots + C_{n-2} C_1 + C_{n-1} C_0 \\ &= \sum_{k=0}^{n-1} C_k C_{n-k-1} \end{aligned}$$

The initial conditions are $C_0 = 1$ and $C_1 = 1$.

The sequence $\{C_n\}$ is the sequence of **Catalan Numbers**. This recurrence relation can be solved using the method of generating functions; see Exercise 41 in Section 8.4.

Solving Linear Recurrence Relations

Section 8.2

Section Summary

- Linear Homogeneous Recurrence Relations
- Solving Linear Homogeneous Recurrence Relations with Constant Coefficients.
- Solving Linear Nonhomogeneous Recurrence Relations with Constant Coefficients.

Linear Homogeneous Recurrence Relations

Definition: A *linear homogeneous recurrence relation of degree* k *with constant coefficients* is a recurrence relation of the form $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$, where $c_1, c_2, \ldots, c_k$ are real numbers, and $c_k \neq 0$

- it is *linear* because the right-hand side is a sum of the previous terms of the sequence each multiplied by a function of n.
- it is *homogeneous* because no terms occur that are not multiples of the a_js. Each coefficient is a constant.
- the *degree* is k because a_n is expressed in terms of the previous k terms of the sequence.

By strong induction, a sequence satisfying such a recurrence relation is uniquely determined by the recurrence relation and the k initial conditions $a_0 = C_1$, $a_0 = C_1, \ldots, a_{k-1} = C_{k-1}$.

Examples of Linear Homogeneous Recurrence Relations

- $P_n = (1.11)P_{n-1}$ linear homogeneous recurrence relation of degree one
- $f_n = f_{n-1} + f_{n-2}$ linear homogeneous recurrence relation of degree two
- $a_n = a_{n-1} + a_{n-2}^2$ not linear
- $H_n = 2H_{n-1} + 1$ not homogeneous
- $B_n = nB_{n-1}$ coefficients are not constants

Solving Linear Homogeneous Recurrence Relations

- The basic approach is to look for solutions of the form $a_n = r^n$, where r is a constant.
- Note that $a_n = r^n$ is a solution to the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$ if and only if $r^n = c_1 r^{n-1} + c_2 r^{n-2} + \cdots + c_k r^{n-k}$.
- Algebraic manipulation yields the *characteristic equation*:
  $r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_{k-1} r - c_k = 0$
- The sequence $\{a_n\}$ with $a_n = r^n$ is a solution if and only if r is a solution to the characteristic equation.
- The solutions to the characteristic equation are called the *characteristic roots* of the recurrence relation. The roots are used to give an explicit formula for all the solutions of the recurrence relation.

Solving Linear Homogeneous Recurrence Relations of Degree Two

Theorem 1: Let c_1 and c_2 be real numbers. Suppose that $r^2 - c_1 r - c_2 = 0$ has two distinct roots r_1 and r_2. Then the sequence $\{a_n\}$ is a solution to the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ if and only if

$$a_n = \alpha r_1^n + \alpha_2 r_2^n$$

for $n = 0, 1, 2, \ldots$, where α_1 and α_2 are constants.

Using Theorem 1

Example: What is the solution to the recurrence relation

$a_n = a_{n-1} + 2a_{n-2}$ with $a_0 = 2$ and $a_1 = 7$?

Solution: The characteristic equation is $r^2 - r - 2 = 0$.
Its roots are $r = 2$ and $r = -1$. Therefore, $\{a_n\}$ is a solution to the recurrence relation if and only if $a_n = \alpha_1 2^n + \alpha_2(-1)^n$, for some constants α_1 and α_2.

To find the constants α_1 and α_2, note that

$a_0 = 2 = \alpha_1 + \alpha_2$ and $a_1 = 7 = \alpha_1 2 + \alpha_2(-1)$.

Solving these equations, we find that $\alpha_1 = 3$ and $\alpha_2 = -1$.

Hence, the solution is the sequence $\{a_n\}$ with $a_n = 3\cdot 2^n - (-1)^n$.

An Explicit Formula for the Fibonacci Numbers

We can use Theorem 1 to find an explicit formula for the Fibonacci numbers. The sequence of Fibonacci numbers satisfies the recurrence relation $f_n = f_{n-1} + f_{n-2}$ with the initial conditions: $f_0 = 0$ and $f_1 = 1$.

Solution: The roots of the characteristic equation $r^2 - r - 1 = 0$ are

$$r_1 = \frac{1+\sqrt{5}}{2}$$

$$r_2 = \frac{1-\sqrt{5}}{2}$$

Fibonacci Numbers (*continued*)

Therefore by Theorem 1

$$f_n = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right)^n$$

for some constants α_1 and α_2.

Using the initial conditions $f_0 = 0$ and $f_1 = 1$, we have

$$f_0 = \alpha_1 + \alpha_2 = 0$$

$$f_1 = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right) + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right) = 1 .$$

Solving, we obtain $\alpha_1 = \frac{1}{\sqrt{5}}$, $\alpha_2 = -\frac{1}{\sqrt{5}}$.

Hence, $f_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2}\right)^n$

The Solution when there is a Repeated Root

Theorem 2: Let c_1 and c_2 be real numbers with $c_2 \neq 0$. Suppose that $r^2 - c_1 r - c_2 = 0$ has one repeated root r_0. Then the sequence $\{a_n\}$ is a solution to the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ if and only if

$$a_n = \alpha r_0^n + \alpha_2 n r_0^n$$

for $n = 0,1,2,\ldots$, where α_1 and α_2 are constants.

Using Theorem 2

Example: What is the solution to the recurrence relation $a_n = 6a_{n-1} - 9a_{n-2}$ with $a_0 = 1$ and $a_1 = 6$?

Solution: The characteristic equation is $r^2 - 6r + 9 = 0$.
The only root is $r = 3$. Therefore, $\{a_n\}$ is a solution to the recurrence relation if and only if

$a_n = \alpha_1 3^n + \alpha_2 n(3)^n$

where α_1 and α_2 are constants.

To find the constants α_1 and α_2, note that

$a_0 = 1 = \alpha_1$ and $a_1 = 6 = \alpha_1 \cdot 3 + \alpha_2 \cdot 3$.

Solving, we find that $\alpha_1 = 1$ and $\alpha_2 = 1$.
Hence,

$a_n = 3^n + n3^n$.

Solving Linear Homogeneous Recurrence Relations of Arbitrary Degree

This theorem can be used to solve linear homogeneous recurrence relations with constant coefficients of any degree when the characteristic equation has distinct roots.

Theorem 3: Let $c_1, c_2, \ldots, c_k$ be real numbers. Suppose that the characteristic equation

$r^k - c_1 r^{k-1} - \cdots - c_k = 0$

has k distinct roots $r_1, r_2, \ldots, r_k$. Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$

if and only if

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n$$

for $n = 0, 1, 2, \ldots$, where $\alpha_1, \alpha_2, \ldots, \alpha_k$ are constants.

The General Case with Repeated Roots Allowed

Theorem 4: Let $c_1, c_2, \ldots, c_k$ be real numbers. Suppose that the characteristic equation

$r^k - c_1 r^{k-1} - \cdots - c_k = 0$

has t distinct roots $r_1, r_2, \ldots, r_t$ with multiplicities $m_1, m_2, \ldots, m_t$, respectively so that $m_i \geq 1$ for $i = 0, 1, 2, \ldots, t$ and $m_1 + m_2 + \ldots + m_t = k$. Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$

if and only if

$$\begin{aligned} a_n = {} & (\alpha_{1,0} + \alpha_{1,1} n + \cdots + \alpha_{1,m_1-1} n^{m_1-1}) r_1^n \\ & + (\alpha_{2,0} + \alpha_{2,1} n + \cdots + \alpha_{2,m_2-1} n^{m_2-1}) r_2^n \\ & + \cdots + (\alpha_{t,0} + \alpha_{t,1} n + \cdots + \alpha_{t,m_t-1} n^{m_t-1}) r_t^n \end{aligned}$$

for $n = 0, 1, 2, \ldots$, where $\alpha_{i,j}$ are constants for $1 \leq i \leq t$ and $0 \leq j \leq m_i - 1$.

Linear Nonhomogeneous Recurrence Relations with Constant Coefficients

Definition: A *linear nonhomogeneous recurrence relation with constant coefficients* is a recurrence relation of the form:

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + F(n)$,

where $c_1, c_2, \ldots, c_k$ are real numbers, and $F(n)$ is a function not identically zero depending only on n.

The recurrence relation

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$,

is called the associated homogeneous recurrence relation.

Linear Nonhomogeneous Recurrence Relations with Constant Coefficients (*cont.*)

The following are linear nonhomogeneous recurrence relations with constant coefficients:

$a_n = a_{n-1} + 2^n$,

$a_n = a_{n-1} + a_{n-2} + n^2 + n + 1$,

$a_n = 3a_{n-1} + n3^n$,

$a_n = a_{n-1} + a_{n-2} + a_{n-3} + n!$

where the following are the associated linear homogeneous recurrence relations, respectively:

$a_n = a_{n-1}$,

$a_n = a_{n-1} + a_{n-2}$,

$a_n = 3a_{n-1}$,

$a_n = a_{n-1} + a_{n-2} + a_{n-3}$

Solving Linear Nonhomogeneous Recurrence Relations with Constant Coefficients (*continued*)

Example: Find all solutions of the recurrence relation $a_n = 3a_{n-1} + 2n$.

What is the solution with $a_1 = 3$?

Solution: The associated linear homogeneous equation is $a_n = 3a_{n-1}$.

Its solutions are $a_n^{(h)} = \alpha 3^n$, where α is a constant.

Because $F(n) = 2n$ is a polynomial in n of degree one, to find a particular solution we might try a linear function in n, say $p_n = cn + d$, where c and d are constants. Suppose that $p_n = cn + d$ is such a solution.

Then $a_n = 3a_{n-1} + 2n$ becomes $cn + d = 3(c(n-1) + d) + 2n$.

Simplifying yields $(2 + 2c)n + (2d - 3c) = 0$. It follows that $cn + d$ is a solution if and only if $2 + 2c = 0$ and $2d - 3c = 0$. Therefore, $cn + d$ is a solution if and only if $c = -1$ and $d = -3/2$.

Consequently, $a_n^{(p)} = -n - 3/2$ is a particular solution.

By Theorem 5, all solutions are of the form $a_n = a_n^{(h)} + a_n^{(p)} = -n - 3/2 + \alpha 3^n$, where α is a constant.

To find the solution with $a_1 = 3$, let $n = 1$ in the above formula for the general solution.

Then $3 = -1 - 3/2 + 3\alpha$, and $\alpha = 11/6$. Hence, the solution is $a_n = -n - 3/2 + (11/6)3^n$.

Solving Linear Nonhomogeneous Recurrence Relations with Constant Coefficients

Theorem 5: If $\{a_n^{(p)}\}$ is a particular solution of the nonhomogeneous linear recurrence relation with constant coefficients

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k} + F(n),$$

then every solution is of the form $\{a_n^{(p)} + a_n^{(h)}\}$, where $\{a_n^{(h)}\}$ is a solution of the associated homogeneous recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}.$$

Divide-and-Conquer Algorithms and Recurrence Relations

Section 8.3

Section Summary

- Divide-and-Conquer Algorithms and Recurrence Relations
- Examples
  - Binary Search
  - Merge Sort
  - Fast Multiplication of Integers
- Master Theorem
- Closest Pair of Points (*not covered yet in these slides*)

Divide-and-Conquer Algorithmic Paradigm

Definition: A *divide-and-conquer algorithm* works by first *dividing* a problem into one or more instances of the same problem of smaller size and then *conquering* the problem using the solutions of the smaller problems to find a solution of the original problem.

Examples:

- Binary search, covered in Chapters 3 and 5: It works by comparing the element to be located to the middle element. The original list is then split into two lists and the search continues recursively in the appropriate sublist.
- Merge sort, covered in Chapter 5: A list is split into two approximately equal sized sublists, each recursively sorted by merge sort. Sorting is done by successively merging pairs of lists.

Divide-and-Conquer Recurrence Relations

- Suppose that a recursive algorithm divides a problem of size n into a subproblems.
- Assume each subproblem is of size n/b.
- Suppose $g(n)$ extra operations are needed in the conquer step.
- Then $f(n)$ represents the number of operations to solve a problem of size n satisisfies the following recurrence relation:

  $$f(n) = af(n/b) + g(n)$$

- This is called a *divide-and-conquer recurrence relation.*

Example: Binary Search

- Binary search reduces the search for an element in a sequence of size n to the search in a sequence of size $n/2$. Two comparisons are needed to implement this reduction;
  - one to decide whether to search the upper or lower half of the sequence and
  - the other to determine if the sequence has elements.
- Hence, if $f(n)$ is the number of comparisons required to search for an element in a sequence of size n, then

  $$f(n) = f(n/2) + 2$$

  when n is even.

Example: Merge Sort

- The merge sort algorithm splits a list of n (assuming n is even) items to be sorted into two lists with $n/2$ items. It uses fewer than n comparisons to merge the two sorted lists.
- Hence, the number of comparisons required to sort a sequence of size n, is no more than $M(n)$ where

$$M(n) = 2M(n/2) + n.$$

Example: Fast Multiplication of Integers

- An algorithm for the fast multiplication of two $2n$-bit integers (assuming n is even) first splits each of the $2n$-bit integers into two blocks, each of n bits.
- Suppose that a and b are integers with binary expansions of length $2n$. Let

  $a = (a_{2n-1}a_{2n-2} \dots a_1a_0)_2$ and $b = (b_{2n-1}b_{2n-2} \dots b_1b_0)_2$.
- Let $a = 2^nA_1 + A_0$, $b = 2^nB_1 + B_0$, where

  $A_1 = (a_{2n-1} \dots a_{n+1}a_n)_2$, $A_0 = (a_{n-1} \dots a_1a_0)_2$,

  $B_1 = (b_{2n-1} \dots b_{n+1}b_n)_2$, $B_0 = (b_{n-1} \dots b_1b_0)_2$.
- The algorithm is based on the fact that ab can be rewritten as:

  $ab = (2^{2n} + 2^n)A_1B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0B_0$.
- This identity shows that the multiplication of two $2n$-bit integers can be carried out using three multiplications of n-bit integers, together with additions, subtractions, and shifts.
- Hence, if $f(n)$ is the total number of operations needed to multiply two n-bit integers, then

  $f(2n) = 3f(n) + Cn$

  where Cn represents the total number of bit operations; the additions, subtractions and shifts that are a constant multiple of n-bit operations.

Estimating the Size of Divide-and-Conquer Functions

Theorem 1: Let f be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + cn^d$$

whenever n is divisible by b, where $a \geq 1$, b is an integer greater than 1, and c is a positive real number. Then

$$f(n) \text{ is } \begin{cases} O(n^{\log_b a}) & \text{if } a > 1 \\ O(\log n) & \text{if } a = 1. \end{cases}$$

Furthermore, when $n = b^k$ and $a \neq 1$, where k is a positive integer,

$$f(n) = C_1 n^{\log_b a} + C_2$$

where $C_1 = f(1) + c/(a-1)$ and $C_1 = -c/(a-1)$.

Complexity of Binary Search

Binary Search Example: Give a big-O estimate for the number of comparisons used by a binary search.

Solution: Since the number of comparisons used by binary search is $f(n) = f(n/2) + 2$ where n is even, by Theorem 1, it follows that $f(n)$ is $O(\log n)$.

Estimating the Size of Divide-and-conquer Functions (*continued*)

Theorem 2. Master Theorem: Let f be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + cn^d$$

whenever $n = b^k$, where k is a positive integer greater than 1, and c and d are real numbers with c positive and d nonnegative. Then

$$f(n) \text{ is } \begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Complexity of Merge Sort

Merge Sort Example: Give a big-O estimate for the number of comparisons used by merge sort.

Solution: Since the number of comparisons used by merge sort to sort a list of n elements is less than $M(n)$ where $M(n) = 2M(n/2) + n$, by the master theorem $M(n)$ is $O(n \log n)$.

Complexity of Fast Integer Multiplication Algorithm

Integer Multiplication Example: Give a big-O estimate for the number of bit operations used needed to multiply two n-bit integers using the fast multiplication algorithm.

Solution: We have shown that $f(n) = 3f(n/2) + Cn$, when n is even, where $f(n)$ is the number of bit operations needed to multiply two n-bit integers. Hence by the master theorem with $a = 3$, $b = 2$, $c = C$, and $d = 0$ (so that we have the case where $a > b^d$), it follows that $f(n)$ is $O(n^{\log 3})$.

Note that $\log 3 \approx 1.6$. Therefore the fast multiplication algorithm is a substantial improvement over the conventional algorithm that uses $O(n^2)$ bit operations.

Generating Functions

Section 8.4

Section Summary

- Generating Functions
- Counting Problems and Generating Functions
- Useful Generating Functions
- Solving Recurrence Relations Using Generating Functions (*not yet covered in the slides*)
- Proving Identities Using Generating Functions (*not yet covered in the slides*)

Generating Functions

Definition: The *generating function for the sequence* $a_0, a_1, \ldots, a_k, \ldots$ of real numbers is the infinite series

$$G(x) = a_0 + a_1 x + \cdots + a_k x^k + \cdots = \sum_{k=0}^{\infty} a_k x^k.$$

Examples:

- The sequence $\{a_k\}$ with $a_k = 3$ has the generating function $\sum_{k=0}^{\infty} 3x^k$.
- The sequence $\{a_k\}$ with $a_k = k + 1$ has the generating function $\sum_{k=0}^{\infty} (k+1)x^k$.
- The sequence $\{a_k\}$ with $a_k = 2^k$ has the generating function $\sum_{k=0}^{\infty} 2^k x^k$.

Generating Functions for Finite Sequences

- Generating functions for finite sequences of real numbers can be defined by extending a finite sequence $a_0, a_1, \ldots, a_n$ into an infinite sequence by setting $a_{n+1} = 0$, $a_{n+2} = 0$, and so on.
- The generating function $G(x)$ of this infinite sequence $\{a_n\}$ is a polynomial of degree n because no terms of the form $a_j x^j$ with $j > n$ occur, that is,

$$G(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

Generating Functions for Finite Sequences (continued)

Example: What is the generating function for the sequence 1,1,1,1,1,1?

Solution: The generating function of 1,1,1,1,1,1 is

$$1 + x + x^2 + x^3 + x^4 + x^5.$$

By Theorem 1 of Section 2.4, we have

$$(x^6 - 1)/(x - 1) = 1 + x + x^2 + x^3 + x^4 + x^5$$

when $x \neq 1$.

Consequently $G(x) = (x^6 - 1)/(x - 1)$ is the generating function of the sequence.

Useful Generating Functions

$G(x)$	a_k
$(1+x)^n = \sum_{k=0}^{n} C(n,k)x^k = 1 + C(n,1)x + C(n,2)x^2 + \cdots + x^n$	$C(n,k)$
$(1+ax)^n = \sum_{k=0}^{n} C(n,k)a^k x^k = 1 + C(n,1)ax + C(n,2)a^2x^2 + \cdots + a^n x^n$	$C(n,k)a^k$
$(1+x^r)^n = \sum_{k=0}^{n} C(n,k)x^{rk} = 1 + C(n,1)x^r + C(n,2)x^{2r} + \cdots + x^{nr}$	$C(n,k/r)$ if $r \mid k$; 0 otherwise
$\frac{1-x^{n+1}}{1-x} = \sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n$	1 if $k \leq n$; 0 otherwise
$\frac{1}{1-x} = \sum_{k=0}^{\infty} x^k = 1 + x + x^2 + \cdots$	1
$\frac{1}{1-ax} = \sum_{k=0}^{\infty} a^k x^k = 1 + ax + a^2x^2 + \cdots$	a^k
$\frac{1}{1-x^r} = \sum_{k=0}^{\infty} x^{rk} = 1 + x^r + x^{2r} + \cdots$	1 if $r \mid k$; 0 otherwise
$\frac{1}{(1-x)^2} = \sum_{k=0}^{\infty} (k+1)x^k = 1 + 2x + 3x^2 + \cdots$	$k+1$
$\frac{1}{(1-x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)x^k = C(n,0) + C(n+1,1)x + C(n+2,2)x^2 + \cdots$	$C(n+k-1,k) = C(n+k-1,n-1)$
$\frac{1}{(1+x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)(-1)^k x^k = 1 - C(n,1)x + C(n+1,2)x^2 - \cdots$	$(-1)^k C(n+k-1,k) = (-1)^k C(n+k-1,n-1)$
$\frac{1}{(1-ax)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)a^k x^k = 1 + C(n,1)ax + C(n+1,2)a^2x^2 + \cdots$	$C(n+k-1,k)a^k = C(n+k-1,n-1)a^k$
$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \cdots$	$1/k!$
$\ln(1+x) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} x^k = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$	$(-1)^{k+1}/k$

Note: The series for the last two generating functions can be found in most calculus books when power series are discussed.

Counting Problems and Generating Functions (*continued*)

Example: Use generating functions to find the number of k-combinations of a set with n elements, i.e., $C(n,k)$.

Solution: Each of the n elements in the set contributes the term $(1 + x)$ to the generating function

$$f(x) = \sum_{k=0}^{n} a_k x^k$$

Hence $f(x) = (1 + x)^n$ where $f(x)$ is the generating function for $\{a^k\}$, where a^k represents the number of k-combinations of a set with n elements.

By the binomial theorem, we have

$$f(x) = \sum_{k=0}^{n} \binom{n}{k} x^k,$$

where

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Hence,

$$C(n,k) = \frac{n!}{k!(n-k)!}.$$

Counting Problems and Generating Functions

Example: Find the number of solutions of

$$e_1 + e_2 + e_3 = 17,$$

where e_1, e_2, and e_3 are nonnegative integers with $2 \leq e_1 \leq 5$, $3 \leq e_2 \leq 6$, and $4 \leq e_3 \leq 7$.

Solution: The number of solutions is the coefficient of x^{17} in the expansion of

$$(x^2 + x^3 + x^4 + x^5)(x^3 + x^4 + x^5 + x^6)(x^4 + x^5 + x^6 + x^7).$$

This follows because a term equal to is obtained in the product by picking a term in the first sum x^{e_1}, a term in the second sum x^{e_2}, and a term in the third sum x^{e_3}, where $e_1 + e_2 + e_3 = 17$.

There are three solutions since the coefficient of x^{17} in the product is 3.

Inclusion-Exclusion

Section 8.5

Section Summary

- The Principle of Inclusion-Exclusion
- Examples

Principle of Inclusion-Exclusion

- In Section 2.2, we developed the following formula for the number of elements in the union of two finite sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

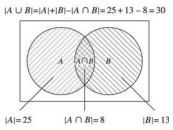
- We will generalize this formula to finite sets of any size.

Two Finite Sets

Example: In a discrete mathematics class every student is a major in computer science or mathematics or both. The number of students having computer science as a major (possibly along with mathematics) is 25; the number of students having mathematics as a major (possibly along with computer science) is 13; and the number of students majoring in both computer science and mathematics is 8. How many students are in the class?

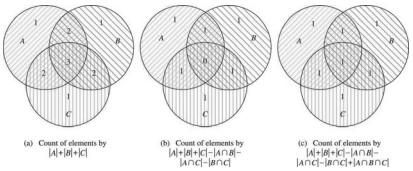
Solution: $|A \cup B| = |A| + |B| - |A \cap B|$

$= 25 + 13 - 8 = 30$



Three Finite Sets

$$|A \cup B \cup C| =$$
$$|A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$



Three Finite Sets Continued

Example: A total of 1232 students have taken a course in Spanish, 879 have taken a course in French, and 114 have taken a course in Russian. Further, 103 have taken courses in both Spanish and French, 23 have taken courses in both Spanish and Russian, and 14 have taken courses in both French and Russian. If 2092 students have taken a course in at least one of Spanish French and Russian, how many students have taken a course in all 3 languages.

Solution: Let S be the set of students who have taken a course in Spanish, F the set of students who have taken a course in French, and R the set of students who have taken a course in Russian. Then, we have $|S| = 1232$, $|F| = 879$, $|R| = 114$, $|S \cap F| = 103$, $|S \cap R| = 23$, $|F \cap R| = 14$, and $|S \cup F \cup R| = 23$.

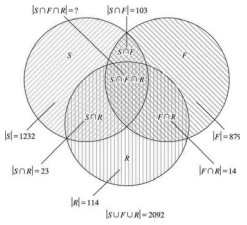
Using the equation

$$|S \cup F \cup R| = |S| + |F| + |R| - |S \cap F| - |S \cap R| - |F \cap R| + |S \cap F \cap R|,$$

we obtain $2092 = 1232 + 879 + 114 - 103 - 23 - 14 + |S \cap F \cap R|$.

Solving for $|S \cap F \cap R|$ yields 7.

Illustration of Three Finite Set Example



The Principle of Inclusion-Exclusion

Theorem 1. The Principle of Inclusion-Exclusion: Let $A_1, A_2, \ldots, A_n$ be finite sets. Then:

$$|A_1 \cup A_2 \cup \cdots \cup A_n| =$$

$$\sum_{1 \le i \le n} |A_i| - \sum_{1 \le i \le j \le n} |A_i \cap A_j| +$$

$$\sum_{1 \le i \le j \le k \le n} |A_i \cap A_j \cap A_k| - \ldots + (-1)^{n+1} |A_1 \cap A_2 \cap \ldots \cap A_n|$$

The Principle of Inclusion-Exclusion (*continued*)

Proof: An element in the union is counted exactly once in the right-hand side of the equation. Consider an element a that is a member of r of the sets $A_1, \ldots, A_n$ where $1 \le r \le n$.

- It is counted $C(r,1)$ times by $\sum |A_i|$
- It is counted $C(r,2)$ times by $\sum |A_i \cap A_j|$
- In general, it is counted $C(r,m)$ times by the summation of m of the sets A_i.

The Principle of Inclusion-Exclusion (cont)

- Thus the element is counted exactly
  $C(r,1) - C(r,2) + C(r,3) - \cdots + (-1)^{r+1} C(r,r)$
  times by the right hand side of the equation.
- By Corollary 2 of Section 6.4, we have
  $C(r,0) - C(r,1) + C(r,2) - \cdots + (-1)^{r} C(r,r) = 0.$
- Hence,
  $1 = C(r,0) = C(r,1) - C(r,2) + \cdots + (-1)^{r+1} C(r,r).$

Applications of Inclusion-Exclusion

Section 8.6

Section Summary

- Counting Onto-Functions
- Derangements

The Number of Onto Functions

Example: How many onto functions are there from a set with six elements to a set with three elements?

Solution: Suppose that the elements in the codomain are b_1, b_2, and b_3. Let P_1, P_2, and P_3 be the properties that b_1, b_2, and b_3 are not in the range of the function, respectively. The function is onto if none of the properties P_1, P_2, and P_3 hold.

By the inclusion-exclusion principle the number of onto functions from a set with six elements to a set with three elements is

$$N - [N(P_1) + N(P_2) + N(P_3)] + [N(P_1P_2) + N(P_1P_3) + N(P_2P_3)] - N(P_1P_2P_3)$$

- Here the total number of functions from a set with six elements to one with three elements is $N = 3^6$.
- The number of functions that do not have in the range is $N(P_1) = 2^6$. Similarly, $N(P_2) = N(3_1) = 2^6$.
- Note that $N(P_1P_2) = N(P_1P_3) = N(P_2P_3) = 1$ and $N(P_1P_2P_3) = 0$.

Hence, the number of onto functions from a set with six elements to a set with three elements is:

$$3^6 - 3 \cdot 2^6 + 3 = 729 - 192 + 3 = 540$$

The Number of Onto Functions (continued)

Theorem 1: Let m and n be positive integers with $m \geq n$. Then there are

$$n^m - C(n,1)(n-1)^m + C(n,2)(n-2)^m - \cdots + (-1)^{n-1} C(n,n-1) \cdot 1^m$$

onto functions from a set with m elements to a set with n elements.

Proof follows from the principle of inclusion-exclusion (*see Exercise* 27).

Derangements

Definition: A *derangement* is a permutation of objects that leaves no object in the original position.

Example: The permutation of 21453 is a derangement of 12345 because no number is left in its original position. But 21543 is not a derangement of 12345, because 4 is in its original position.

Derangements (continued)

Theorem 2: The number of derangements of a set with n elements is

$$D_n = n! \left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}\right].$$

Proof follows from the principle of inclusion-exclusion (*see text*).

Derangements (continued)

The Hatcheck Problem: A new employee checks the hats of n people at restaurant, forgetting to put claim check numbers on the hats. When customers return for their hats, the checker gives them back hats chosen at random from the remaining hats. What is the probability that no one receives the correct hat.

Solution: The answer is the number of ways the hats can be arranged so that there is no hat in its original position divided by $n!$, the number of permutations of n hats.

Remark: It can be shown that the probability of a derangement approaches $1/e$ as n grows without bound.

$$\frac{D_n}{n!} = \left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}\right]$$

TABLE 1 **The Probability of a Derangement.**

n	2	3	4	5	6	7
$D_n/n!$	0.50000	0.33333	0.37500	0.36667	0.36806	0.36786

Relations

Chapter 9

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Relations and Their Properties
- *n*-ary Relations and Their Applications (*not currently included in overheads*)
- Representing Relations
- Closures of Relations (*not currently included in overheads*)
- Equivalence Relations
- Partial Orderings

Relations and Their Properties

Section 9.1

Section Summary

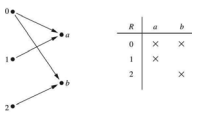
- Relations and Functions
- Properties of Relations
  - Reflexive Relations
  - Symmetric and Antisymmetric Relations
  - Transitive Relations
- Combining Relations

Binary Relations

Definition: A *binary relation R* from a set *A* to a set *B* is a subset $R \subseteq A \times B$.

Example:

- Let $A = \{0,1,2\}$ and $B = \{a,b\}$
- $\{(0, a), (0, b), (1,a), (2, b)\}$ is a relation from *A* to *B*.
- We can represent relations from a set *A* to a set *B* graphically or using a table:



R	*a*	*b*
0	×	×
1	×	
2		×

Relations are more general than functions. A function is a relation where exactly one element of *B* is related to each element of *A*.

Binary Relation on a Set

Definition: A binary relation *R on a set A* is a subset of $A \times A$ or a relation from *A* to *A*.

Example:

- Suppose that $A = \{a,b,c\}$. Then $R = \{(a,a),(a,b),(a,c)\}$ is a relation on *A*.
- Let $A = \{1, 2, 3, 4\}$. The ordered pairs in the relation $R = \{(a,b) \mid a \text{ divides } b\}$ are $(1,1), (1, 2), (1,3), (1, 4), (2, 2), (2, 4), (3, 3), \text{and } (4, 4)$.

Binary Relations on a Set (*cont.*)

Example: Consider these relations on the set of integers:

$R_1 = \{(a,b) \mid a \leq b\}$, $R_4 = \{(a,b) \mid a = b\}$,
$R_2 = \{(a,b) \mid a > b\}$, $R_5 = \{(a,b) \mid a = b + 1\}$,
$R_3 = \{(a,b) \mid a = b \text{ or } a = -b\}$, $R_6 = \{(a,b) \mid a + b \leq 3\}$.

Note that these relations are on an infinite set and each of these relations is an infinite set.

Which of these relations contain each of the pairs

$(1,1), (1, 2), (2, 1), (1, -1), \text{and } (2, 2)$?

Solution: Checking the conditions that define each relation, we see that the pair $(1,1)$ is in R_1, R_3, R_4, and R_6: $(1,2)$ is in R_1 and R_6: $(2,1)$ is in R_2, R_5, and R_6: $(1, -1)$ is in R_2, R_3, and R_6 : $(2,2)$ is in R_1, R_3, and R_4.

Reflexive Relations

Definition: *R* is *reflexive* iff $(a,a) \in R$ for every element $a \in$ A. Written symbolically, R is reflexive if and only if

$$\forall x[x \in U \longrightarrow (x,x) \in R]$$

Example: The following relations on the integers are reflexive:

If $A = \emptyset$ then the empty relation is reflexive vacuously. That is the empty relation on an empty set is reflexive!

$R_1 = \{(a,b) \mid a \leq b\}$,
$R_3 = \{(a,b) \mid a = b \text{ or } a = -b\}$,
$R_4 = \{(a,b) \mid a = b\}$.

The following relations are not reflexive:

$R_2 = \{(a,b) \mid a > b\}$ (note that $3 \not> 3$),
$R_5 = \{(a,b) \mid a = b + 1\}$ (note that $3 \neq 3 + 1$),
$R_6 = \{(a,b) \mid a + b \leq 3\}$ (note that $4 + 4 \not\leq 3$).

Symmetric Relations

Definition: R is *symmetric* iff $(b,a) \in R$ whenever $(a,b) \in R$ for all $a,b \in A$. Written symbolically, R is symmetric if and only if

$\forall x \forall y\, [(x,y) \in R \rightarrow (y,x) \in R]$

Example: The following relations on the integers are symmetric:

$R_3 = \{(a,b) \mid a = b \text{ or } a = -b\}$,
$R_4 = \{(a,b) \mid a = b\}$,
$R_6 = \{(a,b) \mid a + b \leq 3\}$.

The following are not symmetric:

$R_1 = \{(a,b) \mid a \leq b\}$ (note that $3 \leq 4$, but $4 \not\leq 3$),
$R_2 = \{(a,b) \mid a > b\}$ (note that $4 > 3$, but $3 \not> 4$),
$R_5 = \{(a,b) \mid a = b + 1\}$ (note that $4 = 3 + 1$, but $3 \neq 4 + 1$).

Antisymmetric Relations

Definition:A relation R on a set A such that for all $a,b \in A$ if $(a,b) \in R$ and $(b,a) \in R$, then $a = b$ is called *antisymmetric*. Written symbolically, R is antisymmetric if and only if

$\forall x \forall y\, [(x,y) \in R \wedge (y,x) \in R \rightarrow x = y]$

- **Example**: The following relations on the integers are antisymmetric:

$R_1 = \{(a,b) \mid a \leq b\}$,
$R_2 = \{(a,b) \mid a > b\}$,
$R_4 = \{(a,b) \mid a = b\}$,
$R_5 = \{(a,b) \mid a = b + 1\}$.

For any integer, if $a \leq b$ and $a \leq b$, then $a = b$.

The following relations are not antisymmetric:

$R_3 = \{(a,b) \mid a = b \text{ or } a = -b\}$
(note that both $(1,-1)$ and $(-1,1)$ belong to R_3),
$R_6 = \{(a,b) \mid a + b \leq 3\}$ (note that both $(1,2)$ and $(2,1)$ belong to R_6).

Transitive Relations

Definition: A relation R on a set A is called transitive if whenever $(a,b) \in R$ and $(b,c) \in R$, then $(a,c) \in R$, for all $a,b,c \in A$. Written symbolically, R is transitive if and only if

$\forall x \forall y\, \forall z[(x,y) \in R \wedge (y,z) \in \mathrm{R} \rightarrow (x,z) \in R\,]$

- **Example**: The following relations on the integers are transitive:

$R_1 = \{(a,b) \mid a \leq b\}$,
$R_2 = \{(a,b) \mid a > b\}$,
$R_3 = \{(a,b) \mid a = b \text{ or } a = -b\}$,
$R_4 = \{(a,b) \mid a = b\}$.

For every integer, $a \leq b$ and $b \leq c$, then $b \leq c$.

The following are not transitive:

$R_5 = \{(a,b) \mid a = b + 1\}$ (note that both (3,2) and (4,3) belong to R_5, but not (3,3)),
$R_6 = \{(a,b) \mid a + b \leq 3\}$ (note that both (2,1) and (1,2) belong to R_6, but not (2,2)).

Combining Relations

- Given two relations R_1 and R_2, we can combine them using basic set operations to form new relations such as $R_1 \cup R_2$, $R_1 \cap R_2$, $R_1 - R_2$, and $R_2 - R_1$.
- **Example**: Let $A = \{1,2,3\}$ and $B = \{1,2,3,4\}$. The relations $R_1 = \{(1,1),(2,2),(3,3)\}$ and $R_2 = \{(1,1),(1,2),(1,3),(1,4)\}$ can be combined using basic set operations to form new relations:

$R_1 \cup R_2 = \{(1,1),(1,2),(1,3),(1,4),(2,2),(3,3)\}$

$R_1 \cap R_2 = \{(1,1)\}$ $\qquad$ $R_1 - R_2 = \{(2,2),(3,3)\}$

$R_2 - R_1 = \{(1,2),(1,3),(1,4)\}$

Composition

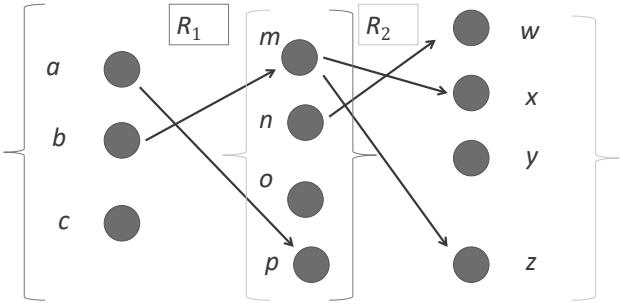
Definition: Suppose

- R_1 is a relation from a set A to a set B.
- R_2 is a relation from B to a set C.

Then the *composition* (or *composite*) of R_2 with R_1, is a relation from A to C where

- if (x,y) is a member of R_1 and (y,z) is a member of R_2, then (x,z) is a member of $R_2 \circ R_1$.

Representing the Composition of a Relation



$R_1 \circ R_2 = \{(b,D),(b,B)\}$

Powers of a Relation

Definition: Let R be a binary relation on A. Then the powers R^n of the relation R can be defined inductively by:

- Basis Step: $R^1 = R$
- Inductive Step: $R^{n+1} = R^n \circ R$

(*see the slides for Section* 9.3 *for further insights*)

The powers of a transitive relation are subsets of the relation. This is established by the following theorem:

Theorem 1: The relation R on a set A is transitive iff $R^n \subseteq R$ for $n = 1,2,3 \ldots$.

(*see the text for a proof via mathematical induction*)

Representing Relations

Section 9.3

Section Summary

- Representing Relations using Matrices
- Representing Relations using Digraphs

Representing Relations Using Matrices

- A relation between finite sets can be represented using a zero-one matrix.
- Suppose R is a relation from $A = \{a_1, a_2, \ldots, a_m\}$ to $B = \{b_1, b_2, \ldots, b_n\}$.
  - The elements of the two sets can be listed in any particular arbitrary order. When $A = B$, we use the same ordering.
- The relation R is represented by the matrix $M_R = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 \text{ if } (a_i, b_j) \in R, \\ 0 \text{ if } (a_i, b_j) \notin R. \end{cases}$$

- The matrix representing R has a 1 as its (i,j) entry when a_i is related to b_j and a 0 if a_i is not related to b_j.

Examples of Representing Relations Using Matrices

Example 1: Suppose that $A = \{1,2,3\}$ and $B = \{1,2\}$. Let R be the relation from A to B containing (a,b) if $a \in A$, $b \in B$, and $a > b$. What is the matrix representing R (assuming the ordering of elements is the same as the increasing numerical order)?

Solution: Because $R = \{(2,1), (3,1),(3,2)\}$, the matrix is

$$M_R = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

Examples of Representing Relations Using Matrices (*cont.*)

Example 2: Let $A = \{a_1,a_2, a_3\}$ and $B = \{b_1,b_2, b_3,b_4, b_5\}$. Which ordered pairs are in the relation R represented by the matrix

$$M_R = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}?$$

Solution: Because R consists of those ordered pairs (a_i,b_j) with $m_{ij} = 1$, it follows that:

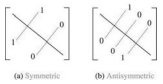
$R = \{(a_1, b_2), (a_2, b_1),(a_2, b_3), (a_2, b_4),(a_3, b_1), \{(a_3, b_3), (a_3, b_5)\}$.

Matrices of Relations on Sets

- If R is a reflexive relation, all the elements on the main diagonal of M_R are equal to 1.



- R is a symmetric relation, if and only if $m_{ij} = 1$ whenever $m_{ji} = 1$. R is an antisymmetric relation, if and only if $m_{ij} = 0$ or $m_{ji} = 0$ when $i \neq j$.



Example of a Relation on a Set

Example 3: Suppose that the relation R on a set is represented by the matrix

$$M_R = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Is R reflexive, symmetric, and/or antisymmetric?

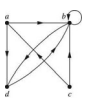
Solution: Because all the diagonal elements are equal to 1, R is reflexive. Because M_R is symmetric, R is symmetric and not antisymmetric because both $m_{1,2}$ and $m_{2,1}$ are 1.

Representing Relations Using Digraphs

Definition: A *directed graph*, or *digraph*, consists of a set V of *vertices* (or *nodes*) together with a set E of ordered pairs of elements of V called *edges* (or *arcs*). The vertex a is called the *initial vertex* of the edge (a,b), and the vertex b is called the *terminal vertex* of this edge.

- An edge of the form (a,a) is called a *loop*.

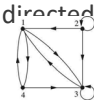
Example 7: A drawing of the directed graph with vertices a, b, c, and d, and edges (a, b), (a, d), (b, b), (b, d), (c, a), (c, b), and (d, b) is shown here.



Examples of Digraphs Representing Relations

Example 8: What are the ordered pairs in the relation

represented by this directed graph?



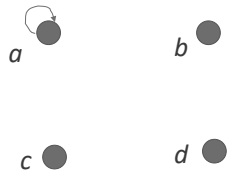
Solution: The ordered pairs in the relation are

(1, 3), (1, 4), (2, 1), (2, 2), (2, 3), (3, 1), (3, 3), (4, 1), and (4, 3)

Determining which Properties a Relation has from its Digraph

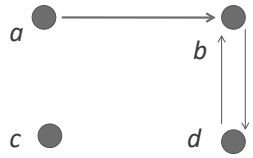
- *Reflexivity*: A loop must be present at all vertices in the graph.
- *Symmetry*: If (x,y) is an edge, then so is (y,x).
- *Antisymmetry*: If (x,y) with $x \neq y$ is an edge, then (y,x) is not an edge.
- *Transitivity*: If (x,y) and (y,z) are edges, then so is (x,z).

Determining which Properties a Relation has from its Digraph – Example 1



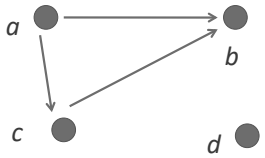
- *Reflexive?* No, not every vertex has a loop
- *Symmetric?* Yes (trivially), there is no edge from one vertex to another
- *Antisymmetric?* Yes (trivially), there is no edge from one vertex to another
- *Transitive?* Yes, (trivially) since there is no edge from one vertex to another

Determining which Properties a Relation has from its Digraph – Example 2



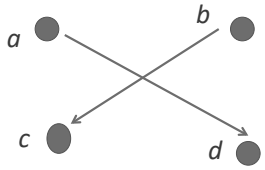
- *Reflexive?* No, there are no loops
- *Symmetric?* No, there is an edge from *a* to *b*, but not from *b* to *a*
- *Antisymmetric?* No, there is an edge from *d* to *b* and *b* to *d*
- *Transitive?* No, there are edges from *a* to *c* and from *c* to *b*, but there is no edge from *a* to *d*

Determining which Properties a Relation has from its Digraph – Example 3



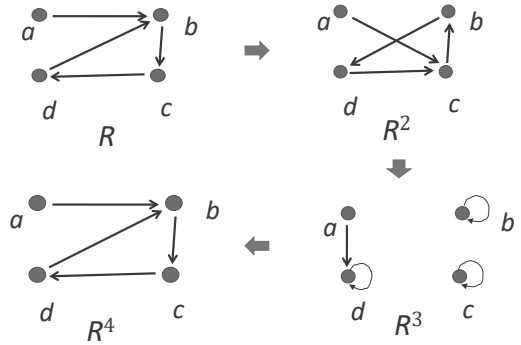
Reflexive? No, there are no loops
Symmetric? No, for example, there is no edge from *c* to *a*
Antisymmetric? Yes, whenever there is an edge from one vertex to another, there is not one going back
Transitive? No, there is no edge from *a* to *b*

Determining which Properties a Relation has from its Digraph – Example 4



- *Reflexive?* No, there are no loops
- *Symmetric?* No, for example, there is no edge from *d* to *a*
- *Antisymmetric?* Yes, whenever there is an edge from one vertex to another, there is not one going back
- *Transitive?* Yes (trivially), there are no two edges where the first edge ends at the vertex where the second edge begins

Example of the Powers of a Relation



The pair (x,y) is in R^n if there is a path of length *n* from *x* to *y* in *R* (following the direction of the arrows).

Equivalence Relations

Section 9.5

Section Summary

- Equivalence Relations
- Equivalence Classes
- Equivalence Classes and Partitions

Equivalence Relations

Definition 1: A relation on a set *A* is called an *equivalence relation* if it is reflexive, symmetric, and transitive.

Definition 2: Two elements *a*, and *b* that are related by an equivalence relation are called *equivalent.* The notation $a \sim b$ is often used to denote that *a* and *b* are equivalent elements with respect to a particular equivalence relation.

Strings

Example: Suppose that *R* is the relation on the set of strings of English letters such that *aRb* if and only if $l(a) = l(b)$, where $l(x)$ is the length of the string *x*. Is *R* an equivalence relation?

Solution: Show that all of the properties of an equivalence relation hold.

- *Reflexivity*: Because $l(a) = l(a)$, it follows that *aRa* for all strings *a*.
- *Symmetry*: Suppose that *aRb.* Since $l(a) = l(b)$, $l(b) = l(a)$ also holds and *bRa*.
- *Transitivity*: Suppose that a*R*b and *bRc*. Since $l(a) = l(b)$,and $l(b) = l(c)$, $l(a) = l(a)$ also holds and *aRc*.

Congruence Modulo *m*

Example: Let *m* be an integer with $m > 1$. Show that the relation

$R = \{(a,b) \mid a \equiv b \pmod m\}$

is an equivalence relation on the set of integers.

Solution: Recall that $a \equiv b \pmod m$ if and only if *m* divides $a - b$.

- *Reflexivity*: $a \equiv a \pmod m$ since $a - a = 0$ is divisible by *m* since $0 = 0 \cdot m$.
- *Symmetry*: Suppose that $a \equiv b \pmod m$. Then $a - b$ is divisible by *m*, and so $a - b = km$, where *k* is an integer. It follows that $b - a = (-k)\, m$, so $b \equiv a \pmod m$.
- *Transitivity*: Suppose that $a \equiv b \pmod m$ and $b \equiv c \pmod m$. Then *m* divides both $a - b$ and $b - c$. Hence, there are integers *k* and *l* with $a - b = km$ and $b - c = lm$. We obtain by adding the equations:

  $a - c = (a - b) + (b - c) = km + lm = (k + l)\, m.$

  Therefore, $a \equiv c \pmod m$.

Divides

Example: Show that the "divides" relation on the set of positive integers is not an equivalence relation.

Solution: The properties of reflexivity, and transitivity do hold, but there relation is not transitive. Hence, "divides" is not an equivalence relation.

- *Reflexivity*: $a \mid a$ for all a.
- *Not Symmetric*: For example, $2 \mid 4$, but $4 \nmid 2$. Hence, the relation is not symmetric.
- *Transitivity*: Suppose that *a* divides *b* and *b* divides *c*. Then there are positive integers *k* and *l* such that $b = ak$ and $c = bl$. Hence, $c = a(kl)$, so *a* divides *c*. Therefore, the relation is transitive.

Equivalence Classes

Definition 3: Let *R* be an equivalence relation on a set *A*. The set of all elements that are related to an element *a* of *A* is called the *equivalence class* of *a*. The equivalence class of *a* with respect to *R* is denoted by $[a]_R$.

When only one relation is under consideration, we can write $[a]$, without the subscript *R*, for this equivalence class.

Note that $[a]_R = \{s \mid (a,s) \in R\}$.

- If $b \in [a]_R$, then *b* is called a representative of this equivalence class. Any element of a class can be used as a representative of the class.
- The equivalence classes of the relation congruence modulo *m* are called the *congruence classes modulo m*. The congruence class of an integer a modulo m is denoted by $[a]_m$, so $[a]_m = \{\ldots, a-2m, a-m, a+2m, a+2m, \ldots\}$. For example,

$[0]_4 = \{\ldots, -8, -4, 0, 4, 8, \ldots\}$ $\quad$ $[1]_4 = \{\ldots, -7, -3, 1, 5, 9, \ldots\}$

$[2]_4 = \{\ldots, -6, -2, 2, 6, 10, \ldots\}$ $\quad$ $[3]_4 = \{\ldots, -5, -1, 3, 7, 11, \ldots\}$

Equivalence Classes and Partitions

Theorem 1: let *R* be an equivalence relation on a set *A*. These statements for elements *a* and *b* of *A* are equivalent:

(*i*) *aRb*

(*ii*) $[a] = [b]$

(*iii*) $[a] \cap [b] = \emptyset$

Proof: We show that (*i*) implies (*ii*). Assume that *aRb*. Now suppose that $c \in [a]$. Then *aRc*. Because *aRb* and *R* is symmetric, *bRa*. Because *R* is transitive and *bRa* and *aRc*, it follows that *bRc*. Hence, $c \in [b]$. Therefore, $[a] \subseteq [b]$. A similar argument (omitted here) shows that $[b] \subseteq [a]$. Since $[a] \subseteq [b]$ and $[b] \subseteq [a]$, we have shown that $[a] = [b]$.

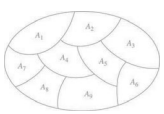
(*see text for proof that* (*ii*) *implies* (*iii*) *and* (*iii*) *implies* (*i*))

Partition of a Set

Definition: A *partition* of a set *S* is a collection of disjoint nonempty subsets of *S* that have *S* as their union. In other words, the collection of subsets A_i, where $i \in I$ (where *I* is an index set), forms a partition of *S* if and only if

- $A_i \neq \emptyset$ for $i \in I$,
- $A_i \cap A_j = \emptyset$ when $i \neq j$,
- and

$$\bigcup_{i \in I} A_i = S.$$



A Partition of a Set

An Equivalence Relation Partitions a Set

- Let *R* be an equivalence relation on a set *A*. The union of all the equivalence classes of *R* is all of *A*, since an element *a* of *A* is in its own equivalence class $[a]_R$. In other words,

$$\bigcup_{a \in A} [a]_R = A.$$

- From Theorem 1, it follows that these equivalence classes are either equal or disjoint, so $[a]_R \cap [b]_R = \emptyset$ when $[a]_R \neq [b]_R$.
- Therefore, the equivalence classes form a partition of *A*, because they split *A* into disjoint subsets.

An Equivalence Relation Partitions a Set (*continued*)

Theorem 2: Let R be an equivalence relation on a set S. Then the equivalence classes of R form a partition of S. Conversely, given a partition $\{A_i \mid i \in I\}$ of the set S, there is an equivalence relation R that has the sets A_i, $i \in I$, as its equivalence classes.

Proof: We have already shown the first part of the theorem.

For the second part, assume that $\{A_i \mid i \in I\}$ is a partition of S. Let R be the relation on S consisting of the pairs (x, y) where x and y belong to the same subset A_i in the partition. We must show that R satisfies the properties of an equivalence relation.

- *Reflexivity:* For every $a \in S$, $(a,a) \in R$, because a is in the same subset as itself.
- *Symmetry*: If $(a,b) \in R$, then b and a are in the same subset of the partition, so $(b,a) \in R$.
- *Transitivity*: If $(a,b) \in R$ and $(b,c) \in R$, then a and b are in the same subset of the partition, as are b and c. Since the subsets are disjoint and b belongs to both, the two subsets of the partition must be identical. Therefore, $(a,c) \in R$ since a and c belong to the same subset of the partition.

Partial Orderings

Section 9.6

Section Summary

- Partial Orderings and Partially-ordered Sets
- Lexicographic Orderings
- Hasse Diagrams
- Lattices (*not currently in overheads*)
- Topological Sorting (*not currently in overheads*)

Partial Orderings

Definition 1: A relation R on a set S is called a *partial ordering,* or *partial order,* if it is reflexive, antisymmetric, and transitive. A set together with a partial ordering R is called a *partially ordered set*, or *poset*, and is denoted by (S, R). Members of S are called *elements* of the poset.

Partial Orderings (*continued*)

Example 1: Show that the "greater than or equal" relation ($\geq$) is a partial ordering on the set of integers.

- *Reflexivity*: $a \geq a$ for every integer a.
- *Antisymmetry*: If $a \geq b$ and $b \geq a$, then $a = b$.
- *Transitivity*: If $a \geq b$ and $b \geq c$, then $a \geq c$.

These properties all follow from the order axioms for the integers. (*See Appendix* 1).

Partial Orderings (*continued*)

Example 2: Show that the divisibility relation ($|$) is a partial ordering on the set of integers.

- *Reflexivity*: $a \mid a$ for all integers a. (*see Example* 9 *in Section* 9.1)
- *Antisymmetry*: If a and b are positive integers with $a \mid b$ and $b \mid a$, then $a = b$. (*see Example* 12 *in Section* 9.1)
- *Transitivity*: Suppose that a divides b and b divides c. Then there are positive integers k and l such that $b = ak$ and $c = bl$. Hence, $c = a(kl)$, so a divides c. Therefore, the relation is transitive.

- $(\mathbf{Z}^+, |)$ is a poset.

Partial Orderings (*continued*)

Example 3: Show that the inclusion relation ($\subseteq$) is a partial ordering on the power set of a set S.

- *Reflexivity*: $A \subseteq A$ whenever A is a subset of S.
- *Antisymmetry*: If A and B are positive integers with $A \subseteq B$ and $B \subseteq A$, then $A = B$.
- *Transitivity*: If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.

The properties all follow from the definition of set inclusion.

Comparability

Definition 2: The elements a and b of a poset $(S,\preccurlyeq)$ are *comparable* if either $a \preccurlyeq b$ or $b \preccurlyeq a$. When a and b are elements of S so that neither $a \preccurlyeq b$ nor $b \preccurlyeq a$, then a and b are called i*ncomparable*.

The symbol $\preccurlyeq$ is used to denote the relation in any poset.

Definition 3: If $(S,\preccurlyeq)$ is a poset and every two elements of S are comparable, S is called a *totally ordered* or *linearly ordered set*, and $\preccurlyeq$ is called a *total order* or a *linear order.* A totally ordered set is also called a *chain*.

Definition 4: $(S,\preccurlyeq)$ is a well-ordered set if it is a poset such that $\preccurlyeq$ is a total ordering and every nonempty subset of S has a least element.

Lexicographic Order

Definition: Given two posets $(A_1,\preccurlyeq_1)$ and $(A_2,\preccurlyeq_2)$, the *lexicographic ordering* on $A_1 \times A_2$ is defined by specifying that (a_1, a_2) is less than (b_1,b_2), that is,

$$(a_1, a_2) \prec (b_1,b_2),$$

either if $a_1 \prec_1 b_1$ or if $a_1 = b_1$ and $a_2 \prec_2 b_2$.

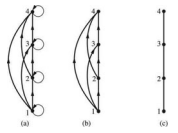
- This definition can be easily extended to a lexicographic ordering on strings (*see text*).

Example: Consider strings of lowercase English letters. A lexicographic ordering can be defined using the ordering of the letters in the alphabet. This is the same ordering as that used in dictionaries.

- *discreet* $\prec$ *discrete*, because these strings differ in the seventh position and $e \prec t$.
- *discreet* $\prec$ *discreetness*, because the first eight letters agree, but the second string is longer.

Hasse Diagrams

Definition: A *Hasse diagram* is a visual representation of a partial ordering that leaves out edges that must be present because of the reflexive and transitive properties.



A partial ordering is shown in (a) of the figure above. The loops due to the reflexive property are deleted in (b). The edges that must be present due to the transitive property are deleted in (c). The Hasse diagram for the partial ordering (a), is depicted in (c).

Procedure for Constructing a Hasse Diagram

- To represent a finite poset $(S,\preccurlyeq)$ using a Hasse diagram, start with the directed graph of the relation:
  - Remove the loops (a, a) present at every vertex due to the reflexive property.
  - Remove all edges (x, y) for which there is an element $z \in S$ such that $x \prec z$ and $z \prec y$. These are the edges that must be present due to the transitive property.
  - Arrange each edge so that its initial vertex is below the terminal vertex. Remove all the arrows, because all edges point upwards toward their terminal vertex.

Graphs

Chapter 10

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Graphs and Graph Models
- Graph Terminology and Special Types of Graphs
- Representing Graphs and Graph Isomorphism
- Connectivity
- Euler and Hamiltonian Graphs
- Shortest-Path Problems (*not currently included in overheads*)
- Planar Graphs (*not currently included in overheads*)
- Graph Coloring (*not currently included in overheads*)

Graphs and Graph Models

Section 10.1

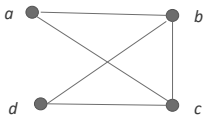
Section Summary

- Introduction to Graphs
- Graph Taxonomy
- Graph Models

Graphs

Definition: A *graph G* = (*V, E*) consists of a nonempty set *V* of *vertices* (or *nodes*) and a set *E* of *edges*. Each edge has either one or two vertices associated with it, called its *endpoints*. An edge is said to *connect* its endpoints.

Example:
This is a graph with four vertices and five edges.



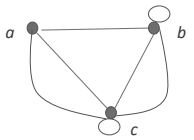
Remarks:

- The graphs we study here are unrelated to graphs of functions studied in Chapter 2.
- We have a lot of freedom when we draw a picture of a graph. All that matters is the connections made by the edges, not the particular geometry depicted. For example, the lengths of edges, whether edges cross, how vertices are depicted, and so on, do not matter
- A graph with an infinite vertex set is called an *infinite graph.* A graph with a finite vertex set is called a *finite graph*. We (following the text) restrict our attention to finite graphs.

Some Terminology

- In a *simple graph* each edge connects two different vertices and no two edges connect the same pair of vertices.
- *Multigraphs* may have multiple edges connecting the same two vertices. When *m* different edges connect the vertices *u* and *v*, we say that {*u,v*} is an edge of *multiplicity m*.
- An edge that connects a vertex to itself is called a *loop*.
- A *pseudograph* may include loops, as well as multiple edges connecting the same pair of vertices.

Example: This pseudograph has both multiple edges and a loop.

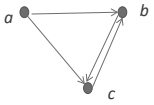


Remark: There is no standard terminology for graph theory. So, it is crucial that you understand the terminology being used whenever you read material about graphs.

Some Terminology (*continued*)

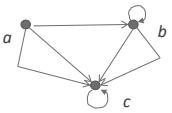
- A *simple directed graph* has no loops and no multiple edges.

  Example:
  This is a directed graph with three vertices and four edges.



- A *directed multigraph* may have multiple directed edges. When there are *m* directed edges from the vertex *u* to the vertex *v*, we say that (*u,v*) is an edge of *multiplicity m*.

  Example:
  In this directed multigraph the multiplicity of (*a,b*) is 1 and the multiplicity of (*b,c*) is 2.

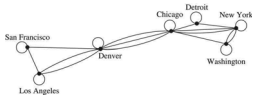


Graph Models: Computer Networks (*continued*)

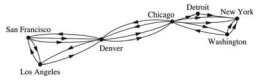
- To model a computer network where we care about the number of links between data centers, we use a multigraph.



- To model a computer network with diagnostic links at data centers, we use a pseudograph, as loops are needed.



- To model a network with multiple one-way links, we use a directed multigraph. Note that we could use a directed graph without multiple edges if we only care whether there is at least one link from a data center to another data center.



Other Applications of Graphs

- We will illustrate how graph theory can be used in models of:
  - Social networks
  - Communications networks
  - Information networks
  - Software design
  - Transportation networks
  - Biological networks
- It's a challenge to find a subject to which graph theory has not yet been applied. Can you find an area without applications of graph theory?

Directed Graphs

Definition: An *directed graph* (or *digraph*) *G* = (*V, E*) consists of a nonempty set *V* of *vertices* (or *nodes*) and a set *E* of *directed edges* (or *arcs*). Each edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (*u*,*v*) is said to *start at u* and *end at v*.

Remark:

- Graphs where the end points of an edge are not ordered are said to be *undirected graphs*.

Graph Models: Computer Networks

- When we build a graph model, we use the appropriate type of graph to capture the important features of the application.
- We illustrate this process using graph models of different types of computer networks. In all these graph models, the vertices represent data centers and the edges represent communication links.
- To model a computer network where we are only concerned whether two data centers are connected by a communications link, we use a simple graph. This is the appropriate type of graph when we only care whether two data centers are directly linked (and not how many links there may be) and all communications links work in both directions.



Graph Terminology: Summary

- To understand the structure of a graph and to build a graph model, we ask these questions:
  - Are the edges of the graph undirected or directed (or both)?
  - If the edges are undirected, are multiple edges present that connect the same pair of vertices? If the edges are directed, are multiple directed edges present?
  - Are loops present?

TABLE 1 **Graph Terminology.**

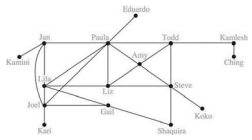
Type	*Edges*	*Multiple Edges Allowed?*	*Loops Allowed?*
Simple graph	Undirected	No	No
Multigraph	Undirected	Yes	No
Pseudograph	Undirected	Yes	Yes
Simple directed graph	Directed	No	No
Directed multigraph	Directed	Yes	Yes
Mixed graph	Directed and undirected	Yes	Yes

Graph Models: Social Networks

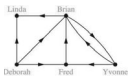
- Graphs can be used to model social structures based on different kinds of relationships between people or groups.
- In a *social network*, vertices represent individuals or organizations and edges represent relationships between them.
- Useful graph models of social networks include:
  - *friendship graphs* - undirected graphs where two people are connected if they are friends (in the real world, on Facebook, or in a particular virtual world, and so on.)
  - *collaboration graphs* - undirected graphs where two people are connected if they collaborate in a specific way
  - *influence graphs* - directed graphs where there is an edge from one person to another if the first person can influence the second person

Graph Models: Social Networks (*continued*)

Example: A friendship graph where two people are connected if they are Facebook friends.



Example: An influence graph



Next Slide: Collaboration Graphs

Examples of Collaboration Graphs

- The *Hollywood graph* models the collaboration of actors in films.
  - We represent actors by vertices and we connect two vertices if the actors they represent have appeared in the same movie.
  - We will study the Hollywood Graph in Section 10.4 when we discuss Kevin Bacon numbers.
- An *academic collaboration graph* models the collaboration of researchers who have jointly written a paper in a particular subject.
  - We represent researchers in a particular academic discipline using vertices.
  - We connect the vertices representing two researchers in this discipline if they are coauthors of a paper.
  - We will study the academic collaboration graph for mathematicians when we discuss *Erdős numbers* in Section 10.4.

Applications to Information Networks

- Graphs can be used to model different types of networks that link different types of information.
- In a *web graph*, web pages are represented by vertices and links are represented by directed edges.
  - A web graph models the web at a particular time.
  - We will explain how the web graph is used by search engines in Section 11.4.
- In a *citation network*:
  - Research papers in a particular discipline are represented by vertices.
  - When a paper cites a second paper as a reference, there is an edge from the vertex representing this paper to the vertex representing the second paper.

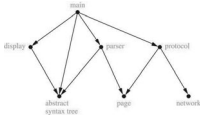
Transportation Graphs

- Graph models are extensively used in the study of transportation networks.
- Airline networks can be modeled using directed multigraphs where
  - airports are represented by vertices
  - each flight is represented by a directed edge from the vertex representing the departure airport to the vertex representing the destination airport
- Road networks can be modeled using graphs where
  - vertices represent intersections and edges represent roads.
  - undirected edges represent two-way roads and directed edges represent one-way roads.

Software Design Applications

- Graph models are extensively used in software design. We will introduce two such models here; one representing the dependency between the modules of a software application and the other representing restrictions in the execution of statements in computer programs.
- When a top-down approach is used to design software, the system is divided into modules, each performing a specific task.
- We use a *module dependency graph* to represent the dependency between these modules. These dependencies need to be understood before coding can be done.
  - In a module dependency graph vertices represent software modules and there is an edge from one module to another if the second module depends on the first.

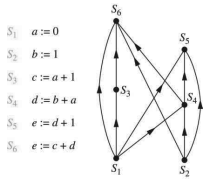
Example: The dependencies between the seven modules in the design of a web browser are represented by this module dependency graph.



Software Design Applications (*continued*)

- We can use a directed graph called a *precedence graph* to represent which statements must have already been executed before we execute each statement.
  - Vertices represent statements in a computer program
  - There is a directed edge from a vertex to a second vertex if the second vertex cannot be executed before the first

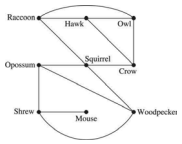
Example: This precedence graph shows which statements must already have been executed before we can execute each of the six statements in the program.



Biological Applications

- Graph models are used extensively in many areas of the biological science. We will describe two such models, one to ecology and the other to molecular biology.
- *Niche overlap graphs* model competition between species in an ecosystem
  - Vertices represent species and an edge connects two vertices when they represent species who compete for food resources.

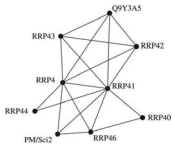
Example: This is the niche overlap graph for a forest ecosystem with nine species.



Biological Applications (*continued*)

- We can model the interaction of proteins in a cell using a *protein interaction network.*
- In a *protein interaction graph*, vertices represent proteins and vertices are connected by an edge if the proteins they represent interact.
- Protein interaction graphs can be huge and can contain more than 100,000 vertices, each representing a different protein, and more than 1,000,000 edges, each representing an interaction between proteins
- Protein interaction graphs are often split into smaller graphs, called *modules*, which represent the interactions between proteins involved in a particular function.

Example: This is a module of the protein interaction graph of proteins that degrade RNA in a human cell.



Graph Terminology and Special Types of Graphs

Section 10.2

Section Summary

- Basic Terminology
- Some Special Types of Graphs
- Bipartite Graphs
- Bipartite Graphs and Matchings (*not currently included in overheads*)
- Some Applications of Special Types of Graphs (*not currently included in overheads*)
- New Graphs from Old

Basic Terminology

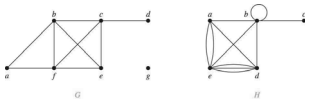
Definition 1. Two vertices *u*, *v* in an undirected graph *G* are called *adjacent* (or *neighbors*) in *G* if there is an edge *e* between *u* and *v*. Such an edge *e* is called *incident with* the vertices *u* and *v* and *e* is said to *connect u* and *v*.

Definition 2. The set of all neighbors of a vertex *v* of *G* = (*V*, *E*), denoted by *N*(*v*), is called the *neighborhood* of *v*. If *A* is a subset of *V*, we denote by *N*(*A*) the set of all vertices in *G* that are adjacent to at least one vertex in *A*. So, $N(A) = \bigcup_{v \in A} N(v)$.

Definition 3. The *degree of a vertex in a undirected graph* is the number of edges incident with it, except that a loop at a vertex contributes two to the degree of that vertex. The degree of the vertex *v* is denoted by deg(*v*).

Degrees and Neighborhoods of Vertices

Example: What are the degrees and neighborhoods of the vertices in the graphs *G* and *H*?



Solution:

G: deg(*a*) = 2, deg(*b*) = deg(*c*) = deg(*f*) = 4, deg(*d*) = 1, deg(*e*) = 3, deg(*g*) = 0.
N(*a*) = {*b, f* }, *N*(*b*) = {*a, c, e, f* }, *N*(*c*) = {*b, d, e, f* }, *N*(*d*) = {*c*}, *N*(*e*) = {*b, c , f* }, *N*(*f*) = {*a, b, c, e*}, *N*(*g*) = ∅ .

H: deg(*a*) = 4, deg(*b*) = deg(*e*) = 6, deg(*c*) = 1, deg(*d*) = 5.
N(*a*) = {*b, d, e*}, *N*(*b*) = {*a, b, c, d, e*}, *N*(*c*) = {*b*}, *N*(*d*) = {*a, b, e*}, *N*(*e*) = {*a, b ,d*}.

Degrees of Vertices

Theorem 1 (*Handshaking Theorem*): If *G* = (*V*,*E*) is an undirected graph with *m* edges, then

$$2m = \sum_{v \in V} \deg(v)$$

Proof:

Each edge contributes twice to the degree count of all vertices. Hence, both the left-hand and right-hand sides of this equation equal twice the number of edges. ◀

Think about the graph where vertices represent the people at a party and an edge connects two people who have shaken hands.

Handshaking Theorem

We now give two examples illustrating the usefulness of the handshaking theorem.

Example: How many edges are there in a graph with 10 vertices of degree six?

Solution: Because the sum of the degrees of the vertices is $6 \cdot 10 = 60$, the handshaking theorem tells us that $2m = 60$. So the number of edges $m = 30$.

Example: If a graph has 5 vertices, can each vertex have degree 3?

Solution: This is not possible by the handshaking thorem, because the sum of the degrees of the vertices $3 \cdot 5 = 15$ is odd.

Degree of Vertices (*continued*)

Theorem 2: An undirected graph has an even number of vertices of odd degree.

***Proof*:** Let V_1 be the vertices of even degree and V_2 be the vertices of odd degree in an undirected graph *G* = (*V*, *E*) with *m* edges. Then

even ⟶ $2m = \sum_{v \in V} \deg(v) = \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v).$

must be even since deg(*v*) is even for each $v \in V_1$

This sum must be even because 2*m* is even and the sum of the degrees of the vertices of even degrees is also even. Because this is the sum of the degrees of all vertices of odd degree in the graph, there must be an even number of such vertices.

Directed Graphs

Recall the definition of a directed graph.

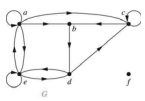
Definition: An *directed graph G =* (*V, E)* consists of *V,* a nonempty set of *vertices* (or *nodes*), and *E,* a set of *directed edges* or *arcs.* Each edge is an ordered pair of vertices. The directed edge (*u*,*v*) is said to start at *u* and end at *v*.

Definition: Let (*u,v*) be an edge in *G*. Then *u* is the *initial vertex* of this edge and is *adjacent to v* and *v* is the *terminal* (or *end*) *vertex* of this edge and is *adjacent from u*. The initial and terminal vertices of a loop are the same.

Directed Graphs (*continued*)

Definition: The *in-degree of a vertex v*, denoted $deg^-(v)$, is the number of edges which terminate at *v*. The *out-degree of v*, denoted $deg^+(v)$, is the number of edges with *v* as their initial vertex. Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of the vertex.

Example: In the graph *G* we have



$\deg^-(a) = 2$, $\deg^-(b) = 2$, $\deg^-(c) = 3$, $\deg^-(d) = 2$, $\deg^-(e) = 3$, $\deg^-(f) = 0$.

$\deg^+(a) = 4$, $\deg^+(b) = 1$, $\deg^+(c) = 2$, $\deg^+(d) = 2$, $\deg^+(e) = 3$, $\deg^+(f) = 0$.

Directed Graphs (*continued*)

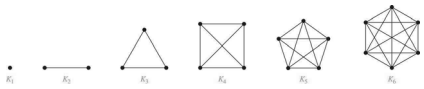
Theorem 3: Let $G = (V, E)$ be a graph with directed edges. Then:

$$|E| = \sum_{v \in V} deg^-(v) = \sum_{v \in V} deg^+(v).$$

Proof: The first sum counts the number of outgoing edges over all vertices and the second sum counts the number of incoming edges over all vertices. It follows that both sums equal the number of edges in the graph. ◀

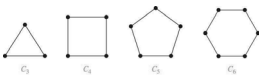
Special Types of Simple Graphs: Complete Graphs

A *complete graph on n vertices*, denoted by K_n, is the simple graph that contains exactly one edge between each pair of distinct vertices.

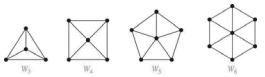


Special Types of Simple Graphs: Cycles and Wheels

A *cycle* C_n for $n \geq 3$ consists of *n* vertices $v_1, v_2, \cdots, v_n$, and edges $\{v_1, v_2\}, \{v_2, v_3\}, \cdots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$.

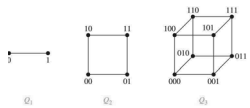


A *wheel* W_n is obtained by adding an additional vertex to a cycle C_n for $n \geq 3$ and connecting this new vertex to each of the *n* vertices in C_n by new edges.



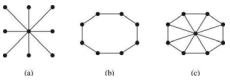
Special Types of Simple Graphs: *n*-Cubes

An *n-dimensional hypercube*, or *n-cube,* $\boldsymbol{Q_n}$, is a graph with 2^n vertices representing all bit strings of length *n*, where there is an edge between two vertices that differ in exactly one bit position.

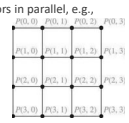


Special Types of Graphs and Computer Network Architecture

Various special graphs play an important role in the design of computer networks.



- Some local area networks use a *star topology*, which is a complete bipartite graph $K_{1,n}$, as shown in (a). All devices are connected to a central control device.
- Other local networks are based on a *ring topology*, where each device is connected to exactly two others using C_n, as illustrated in (b). Messages may be sent around the ring.
- Others, as illustrated in (c), use a W_n – based topology, combining the features of a star topology and a ring topology.
- Various special graphs also play a role in parallel processing where processors need to be interconnected as one processor may need the output generated by another.
  - The *n-dimensional hypercube*, or *n-cube,* $\boldsymbol{Q_n}$, is a common way to connect processors in parallel, e.g., Intel Hypercube.
  - Another common method is the *mesh* network, illustrated here processors.

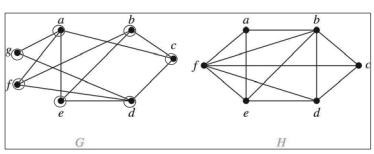


Bipartite Graphs

Definition: A simple graph *G* is bipartite if *V* can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 and a vertex in V_2. In other words, there are no edges which connect two vertices in V_1 or in V_2.

It is not hard to show that an equivalent definition of a bipartite graph is a graph where it is possible to color the vertices red or blue so that no two adjacent vertices are the same color.

G is bipartite

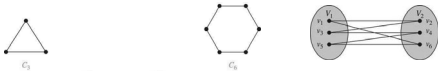


H is not bipartite since if we color *a* red, then the adjacent vertices *f* and *b* must both be blue.

Bipartite Graphs (*continued*)

Example: Show that C_6 is bipartite.

Solution: We can partition the vertex set into $V_1 = \{v_1, v_3, v_5\}$ and $V_2 = \{v_2, v_4, v_6\}$ so that every edge of C_6 connects a vertex in V_1 and V_2 .



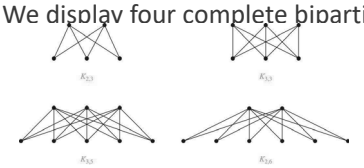
Example: Show that C_3 is not bipartite.

Solution: If we divide the vertex set of C_3 into two nonempty sets, one of the two must contain two vertices. But in C_3 every vertex is connected to every other vertex. Therefore, the two vertices in the same partition are connected. Hence, C_3 is not bipartite.

Complete Bipartite Graphs

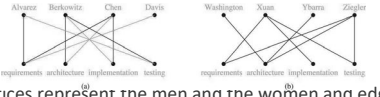
Definition: A *complete bipartite graph* $K_{m,n}$ is a graph that has its vertex set partitioned into two subsets V_1 of size m and V_2 of size n such that there is an edge from every vertex in V_1 to every vertex in V_2.

Example: We display four complete bipartite graphs here.



Bipartite Graphs and Matchings

- Bipartite graphs are used to model applications that involve matching the elements of one set to elements in another, for example:
- *Job assignments* - vertices represent the jobs and the employees, edges link employees with those jobs they have been trained to do. A common goal is to match jobs to employees so that the most jobs are done.



- *Marriage* - vertices represent the men and the women and edges link a a man and a woman if they are an acceptable spouse. We may wish to find the largest number of possible marriages.

See the text for more about matchings in bipartite graphs.

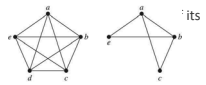
Representing Graphs and Graph Isomorphism

Section 10.3

New Graphs from Old

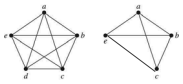
Definition: A *subgraph of a graph* $G = (V,E)$ is a graph (W,F), where $W \subset V$ and $F \subset E$. A subgraph H of G is a proper subgraph of G if $H \neq G$.

Example: Here we show K_5 and its subgraphs.



Definition: Let $G = (V, E)$ be a simple graph. The *subgraph induced* by a subset W of the vertex set V is the graph (W,F), where the edge set F contains an edge in E if and only if both endpoints are in W.

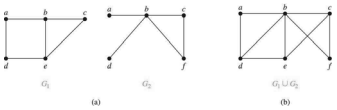
Example: Here we show K_5 and the subgraph induced by $W = \{a,b,c,e\}$.



New Graphs from Old (*continued*)

Definition: The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.

Example:



Section Summary

- Adjacency Lists
- Adjacency Matrices
- Incidence Matrices
- Isomorphism of Graphs

Representing Graphs: Adjacency Lists

Definition: An *adjacency list* can be used to represent a graph with no multiple edges by specifying the vertices that are adjacent to each vertex of the graph.

Example:



TABLE 1 An Adjacency List for a Simple Graph.

Vertex	Adjacent Vertices
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

Example:

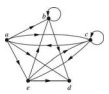


TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	Terminal Vertices
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

Representation of Graphs: Adjacency Matrices

Definition: Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Arbitrarily list the vertices of G as $v_1, v_2, \ldots, v_n$. The *adjacency matrix* $\mathbf{A}_G$ of G, with respect to the listing of vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j)th entry when v_i and v_j are adjacent, and 0 as its (i, j)th entry when they are not adjacent.

- In other words, if the graphs adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

Adjacency Matrices (*continued*)

Example:



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.

When a graph is sparse, that is, it has few edges relatively to the total number of possible edges, it is much more efficient to represent the graph using an adjacency list than an adjacency matrix. But for a dense graph, which includes a high percentage of possible edges, an adjacency matrix is preferable.

Note: The adjacency matrix of a simple graph is symmetric, i.e., $a_{ij} = a_{ji}$
Also, since there are no loops, each diagonal entry a_{ij} for $i = 1, 2, 3, \ldots, n$, is 0.

Adjacency Matrices (*continued*)

- Adjacency matrices can also be used to represent directed graphs. The matrix for a directed graph $G = (V, E)$ has a 1 in its (i, j)th position if there is an edge from v_i to v_j, where $v_1, v_2, \ldots v_n$ is a list of the vertices.
  - In other words, if the graphs adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then
$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$
  - The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_i to v_j, when there is an edge from v_j to v_i.
  - To represent directed multigraphs, the value of a_{ij} is the number of edges connecting v_i to v_j.

Adjacency Matrices (*continued*)

- Adjacency matrices can also be used to represent graphs with loops and multiple edges.
- A loop at the vertex v_i is represented by a 1 at the (i, j)th position of the matrix.
- When multiple edges connect the same pair of vertices v_i and v_j, (or if multiple loops are present at the same vertex), the (i, j)th entry equals the number of edges connecting the pair of vertices.

Example: We give the adjacency matrix of the pseudograph shown here using the ordering of vertices *a*, *b*, *c*, *d*.



$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

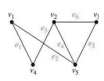
Representation of Graphs: Incidence Matrices

Definition: Let $G = (V, E)$ be an undirected graph with vertices where $v_1, v_2, \ldots v_n$ and edges $e_1, e_2, \ldots e_m$. The incidence matrix with respect to the ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

Incidence Matrices (*continued*)

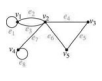
Example: Simple Graph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_6.

Example: Pseudograph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_8.

Isomorphism of Graphs

Definition: The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there is a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2, for all a and b in V_1. Such a function f is called an *isomorphism.* Two simple graphs that are not isomorphic are called *nonisomorphic*.

Isomorphism of Graphs (*cont.*)

Example: Show that the graphs $G = (V, E)$ and $H = (W, F)$ are isomorphic.





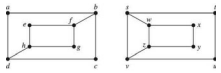
Solution: The function f with $f(u_1) = v_1$, $f(u_2) = v_4$, $f(u_3) = v_3$, and $f(u_4) = v_2$ is a one-to-one correspondence between V and W. Note that adjacent vertices in G are u_1 and u_2, u_1 and u_3, u_2 and u_4, and u_3 and u_4. Each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4$, $f(u_1) = v_1$ and $f(u_3) = v_3$, $f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H.

Isomorphism of Graphs (*cont.*)

- It is difficult to determine whether two simple graphs are isomorphic using brute force because there are $n!$ possible one-to-one correspondences between the vertex sets of two simple graphs with n vertices.
- The best algorithms for determining weather two graphs are isomorphic have exponential worst case complexity in terms of the number of vertices of the graphs.
- Sometimes it is not hard to show that two graphs are not isomorphic. We can do so by finding a property, preserved by isomorphism, that only one of the two graphs has. Such a property is called *graph invariant*.
- There are many different useful graph invariants that can be used to distinguish nonisomorphic graphs, such as the number of vertices, number of edges, and degree sequence (list of the degrees of the vertices in nonincreasing order). We will encounter others in later sections of this chapter.

Isomorphism of Graphs (*cont.*)

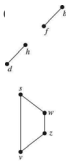
Example: Determine whether these two graphs are isomorphic.



Solution: Both graphs have eight vertices and ten edges.
They also both have four vertices of degree two and four of degree three.

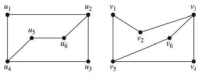
However, *G* and *H* are not isomorphic. Note that since $deg(a) = 2$ in *G*, *a* must correspond to *t*, *u*, *x*, or *y* in H, because these are the vertices of degree 2. But each of these vertices is adjacent to another vertex of degree two in *H*, which is not true for *a* in *G*.

Alternatively, note that the subgraphs of *G* and *H* made up of vertices of degree three and the edges connecting them must be isomorphic. But the subgraphs, as shown at the right, are not isomorphic.



Isomorphism of Graphs (*cont.*)

Example: Determine whether these two graphs are isomorphic.



Solution: Both graphs have six vertices and seven edges.
They also both have four vertices of degree two and two of degree three.
The subgraphs of *G* and *H* consisting of all the vertices of degree two and the edges connecting them are isomorphic. So, it is reasonable to try to find an isomorphism *f*.

We define an injection *f* from the vertices of *G* to the vertices of *H* that preserves the degree of vertices. We will determine whether it is an isomorphism.

The function *f* with $f(u_1) = v_6$, $f(u_2) = v_3$, $f(u_3) = v_4$, and $f(u_4) = v_5$, $f(u_5) = v_1$, and $f(u_6) = v_2$ is a one-to-one correspondence between *G* and *H*. Showing that this correspondence preserves edges is straightforward, so we will omit the details here. Because *f* is an isomorphism, it follows that *G* and *H* are isomorphic graphs.

See the text for an illustration of how adjacency matrices can be used for this verification.

Algorithms for Graph Isomorphism

- The best algorithms known for determining whether two graphs are isomorphic have exponential worst-case time complexity (in the number of vertices of the graphs).
- However, there are algorithms with linear average-case time complexity.
- You can use a public domain program called NAUTY to determine in less than a second whether two graphs with as many as 100 vertices are isomoprhic.
- Graph isomorphism is a problem of special interest because it is one of a few NP problems not known to be either tractable or NP-complete (see Section 3.3).

Applications of Graph Isomorphism

- The question whether graphs are isomorphic plays an important role in applications of graph theory. For example,
  - chemists use molecular graphs to model chemical compounds. Vertices represent atoms and edges represent chemical bonds. When a new compound is synthesized, a database of molecular graphs is checked to determine whether the graph representing the new compound is isomorphic to the graph of a compound that this already known.
  - Electronic circuits are modeled as graphs in which the vertices represent components and the edges represent connections between them. Graph isomorphism is the basis for
    - the verification that a particular layout of a circuit corresponds to the design's original schematics.
    - determining whether a chip from one vendor includes the intellectual property of another vendor.

Connectivity

Section 10.4

Section Summary

- Paths
- Connectedness in Undirected Graphs
- Vertex Connectivity and Edge Connectivity (*not currently included in overheads*)
- Connectedness in Directed Graphs
- Paths and Isomorphism (*not currently included in overheads*)
- Counting Paths between Vertices

Paths

Informal Definition: A *path* is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these.

Applications: Numerous problems can be modeled with paths formed by traveling along edges of graphs such as:

- determining whether a message can be sent between two computers.
- efficiently planning routes for mail delivery.

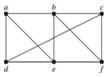
Paths

Definition: Let *n* be a nonnegative integer and *G* an undirected graph. A *path* of *length n* from *u* to *v* in *G* is a sequence of *n* edges $e_1, \ldots, e_n$ of *G* for which there exists a sequence $x_0 = u, x_1, \ldots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \ldots, n$, the endpoints x_{i-1} and x_i.

- When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \ldots, x_n$ (since listing the vertices uniquely determines the path).
- The path is a *circuit* if it begins and ends at the same vertex ($u = v$) and has length greater than zero.
- The path or circuit is said to *pass through* the vertices $x_1, x_2, \ldots, x_{n-1}$ and *traverse* the edges $e_1, \ldots, e_n$.
- A path or circuit is *simple* if it does not contain the same edge more than once.

This terminology is readily extended to directed graphs. (*see text*)

Paths (*continued*)

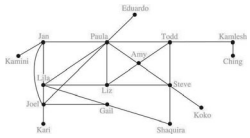


Example: In the simple graph here:

- *a*, *d*, *c*, *f*, *e* is a simple path of length 4.
- *d*, *e*, *c*, *a* is not a path because *e* is not connected to *c*.
- *b*, *c*, *f*, *e*, *b* is a circuit of length 4.
- *a*, *b*, *e*, *d*, *a*, *b* is a path of length 5, but it is not a simple path.

Degrees of Separation

Example: *Paths in Acquaintanceship Graphs*. In an acquaintanceship graph there is a path between two people if there is a chain of people linking these people, where two people adjacent in the chain know one another. In this graph there is a chain of six people linking Kamini and Ching.



Some have speculated that almost every pair of people in the world are linked by a small chain of no more than six, or maybe even, five people. The play *Six Degrees of Separation* by John Guare is based on this notion.

Erdős numbers



Paul Erdős

Example: *Erdős numbers*.
In a collaboration graph, two people *a* and *b* are connected by a path when there is a sequence of people starting with *a* and ending with *b* such that the endpoints of each edge in the path are people who have collaborated.

- In the academic collaboration graph of people who have written papers in mathematics, the *Erdős number* of a person *m* is the length of the shortest path between *m* and the prolific mathematician Paul Erdős.
- To learn more about Erdős numbers, visit http://www.ams.org/mathscinet/collaborationDistance.html

TABLE 1 **The Number of Mathematicians with a Given Erdős Number (as of early 2006).**

Erdős Number	*Number of People*
0	1
1	504
2	6,593
3	33,605
4	83,642
5	87,760
6	40,014
7	11,591
8	3,146
9	819
10	244
11	68
12	23
13	5

Bacon Numbrers

- In the Hollywood graph, two actors *a* and *b* are linked when there is a chain of actors linking *a* and *b*, where every two actors adjacent in the chain ha acted in the same movie.
- The *Bacon number* of an actor *c* is defined to be the length of the shortest path connecting *c* and the well-known actor Kevin Bacon. (Note that we can define a similar number by replacing Kevin Bacon by a different actor.)
- The *oracle of Bacon* web site http://oracleofbacon.org/how.php provides a tool for finding Bacon numbers.

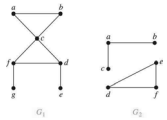
TABLE 2 **The Number of Actors with a Given Bacon Number (as of early 2011).**

Bacon Number	*Number of People*
0	1
1	2,367
2	242,407
3	785,389
4	200,602
5	14,048
6	1,277
7	114
8	16

Connectedness in Undirected Graphs

Definition: An undirected graph is called *connected* if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*. We say that we *disconnect* a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.

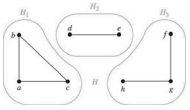
Example: G_1 is connected because there is a path between any pair of its vertices, as can be easily seen. However G_2 is not connected because there is no path between vertices *a* and *f*, for example.



Connected Components

Definition: A *connected component* of a graph *G* is a connected subgraph of *G* that is not a proper subgraph of another connected subgraph of *G*. A graph *G* that is not connected has two or more connected components that are disjoint and have *G* as their union.

Example: The graph *H* is the union of three disjoint subgraphs H_1, H_2, and H_3, none of which are proper subgraphs of a larger connected subgraph of *G*.These three subgraphs are the connected components of *H*.



Connectedness in Directed Graphs

Definition: A directed graph is *strongly connected* if there is a path from *a* to *b* and a path from *b* to *a* whenever *a* and *b* are vertices in the graph.

Definition: A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph, which is the undirected graph obtained by ignoring the directions of the edges of the directed graph.

Connectedness in Directed Graphs (*continued*)

Example: *G* is strongly connected because there is a path between any two vertices in the directed graph. Hence, *G* is also weakly connected. The graph *H* is not strongly connected, since there is no directed path from *a* to *b*, but it is weakly connected.



Definition: The subgraphs of a directed graph *G* that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* or *strong components* of *G*.

Example (*continued*): The graph *H* has three strongly connected components, consisting of the vertex *a*; the vertex *e;* and the subgraph consisting of the vertices *b*, *c*, *d* and edges (*b*,*c*), (*c*,*d*), and (*d*,*b*).

The Connected Components of the Web Graph

- Recall that at any particular instant the web graph provides a snapshot of the web, where vertices represent web pages and edges represent links. According to a 1999 study, the Web graph at that time had over 200 million vertices and over 1.5 billion edges. (The numbers today are several orders of magnitude larger.)
- The underlying undirected graph of this Web graph has a connected component that includes approximately 90% of the vertices.
- There is a *giant strongly connected component (GSCC)* consisting of more than 53 million vertices. A Web page in this component can be reached by following links starting in any other page of the component. There are three other categories of pages with each having about 44 million vertices:
  - pages that can be reached from a page in the GSCC, but do not link back.
  - pages that link back to the GSCC, but can not be reached by following links from pages in the GSCC.
  - pages that cannot reach pages in the GSCC and can not be reached from pages in the GSCC.

Counting Paths between Vertices

- We can use the adjacency matrix of a graph to find the number of paths between two vertices in the graph.

Theorem: Let G be a graph with adjacency matrix **A** with respect to the ordering $v_1, \ldots, v_n$ of vertices (with directed or undirected edges, multiple edges and loops allowed). The number of different paths of length r from v_i to v_j, where $r > 0$ is a positive integer, equals the (i,j)th entry of $\mathbf{A}^r$.

Proof by mathematical induction:

Basis Step: By definition of the adjacency matrix, the number of paths from v_i to v_j of length 1 is the (i,j)th entry of **A**.

Inductive Step: For the inductive hypothesis, we assume that that the (i,j)th entry of $\mathbf{A}^r$ is the number of different paths of length r from v_i to v_j.

- Because $\mathbf{A}^{r+1} = \mathbf{A}^r \mathbf{A}$, the (i,j)th entry of $\mathbf{A}^{r+1}$ equals $b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj}$, where b_{ik} is the (i,k)th entry of $\mathbf{A}^r$. By the inductive hypothesis, b_{ik} is the number of paths of length r from v_i to v_k.
- A path of length $r + 1$ from v_i to v_j is made up of a path of length r from v_i to some v_k, and an edge from v_k to v_j. By the product rule for counting, the number of such paths is the product of the number of paths of length r from v_i to v_k (i.e., b_{ik}) and the number of edges from from v_k to v_j (i.e, a_{kj}). The sum over all possible intermediate vertices v_k *is* $b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj}$.

◀

Counting Paths between Vertices (*continued*)

Example: How many paths of length four are there from *a* to *d* in the graph G.

G



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

adjacency matrix of G

Solution: The adjacency matrix of *G* (ordering the vertices as *a*, *b*, *c*, *d*) is given above. Hence the number of paths of length four from *a* to *d* is the $(1, 4)$th entry of $\mathbf{A}^4$. The eight paths are as:

$$\begin{bmatrix} 8 & 0 & 0 & 8 \\ 0 & 8 & 8 & 0 \\ 0 & 8 & 8 & 0 \\ 8 & 0 & 0 & 8 \end{bmatrix}$$

a, *b*, *a*, *b*, *d*
a, *b*, *d*, *b*, *d*
a, *c*, *a*, *b*, *d*
a, *c*, *d*, *b*, *d*
a, *b*, *a*, *c*, *d*
a, *b*, *d*, *c*, *d*
a, *c*, *a*, *c*, *d*
a, *c*, *d*, *c*, *d*

Euler and Hamiltonian Graphs

Section 10.5

Section Summary

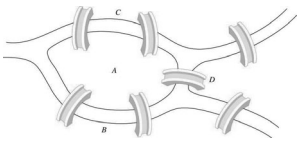
- Euler Paths and Circuits
- Hamilton Paths and Circuits
- Applications of Hamilton Circuits

Euler Paths and Circuits

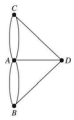


Leonard Euler
(1707-1783)

- The town of Königsberg, Prussia (now Kalingrad, Russia) was divided into four sections by the branches of the Pregel river. In the 18th century seven bridges connected these regions.
- People wondered whether whether it was possible to follow a path that crosses each bridge exactly once and returns to the starting point.
- The Swiss mathematician Leonard Euler proved that no such path exists. This result is often considered to be the first theorem ever proved in graph theory.



The 7 Bridges of Königsberg

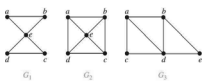


Multigraph Model of the Bridges of Königsberg

Euler Paths and Circuits (*continued*)

Definition: An *Euler circuit* in a graph *G* is a simple circuit containing every edge of *G*. An *Euler path* in *G* is a simple path containing every edge of *G*.

Example: Which of the undirected graphs G_1, G_2, and G_3 has a Euler circuit? Of those that do not, which has an Euler path?



Solution: The graph G_1 has an Euler circuit (e.g., *a*, *e*, *c*, *d*, *e*, *b*, *a*). But, as can easily be verified by inspection, neither G_2 nor G_3 has an Euler circuit. Note that G_3 has an Euler path (e.g., *a*, *c*, *d*, *e*, *b*, *d*, *a*, *b*), but there is no Euler path in G_2, which can be verified by inspection.

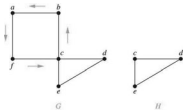
Necessary Conditions for Euler Circuits and Paths

- An Euler circuit begins with a vertex *a* and continues with an edge incident with *a*, say {*a*, *b*}. The edge {*a*, *b*} contributes one to deg(*a*).
- Each time the circuit passes through a vertex it contributes two to the vertex's degree.
- Finally, the circuit terminates where it started, contributing one to deg(*a*). Therefore deg(*a*) must be even.
- We conclude that the degree of every other vertex must also be even.
- By the same reasoning, we see that the initial vertex and the final vertex of an Euler path have odd degree, while every other vertex has even degree. So, a graph with an Euler path has exactly two vertices of odd degree.
- In the next slide we will show that these necessary conditions are also sufficient conditions.

Sufficient Conditions for Euler Circuits and Paths

Suppose that G is a connected multigraph with ≥ 2 vertices, all of even degree. Let $x_0 = a$ be a vertex of even degree. Choose an edge $\{x_0, x_1\}$ incident with a and proceed to build a simple path $\{x_0, x_1\}, \{x_1, x_2\}, ..., \{x_{n-1}, x_n\}$ by adding edges one by one until another edge can not be added.

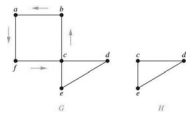
We illustrate this idea in the graph G here. We begin at a and choose the edges $\{a, f\}$, $\{f, c\}$, $\{c, b\}$, and $\{b, a\}$ in succession.



- The path begins at a with an edge of the form $\{a, x\}$; we show that it must terminate at a with an edge of the form $\{y, a\}$. Since each vertex has an even degree, there must be an even number of edges incident with this vertex. Hence, every time we enter a vertex other than a, we can leave it. Therefore, the path can only end at a.
- If all of the edges have been used, an Euler circuit has been constructed. Otherwise, consider the subgraph H obtained from G by deleting the edges already used.

In the example, H consists of the vertices a, c, d, e.

Sufficient Conditions for Euler Circuits and Paths (*continued*)



- Because G is connected, H must have at least one vertex in common with the circuit that has been deleted.

In the example, the vertex is c.

- Every vertex in H must have even degree because all the vertices in G have even degree and for each vertex, pairs of edges incident with this vertex have been deleted. Beginning with the shared vertex construct a path ending in the same vertex (as was done before). Then splice this new circuit into the original circuit.

In the example, we end up with the circuit a, f, c, d, e, c, b, a

- Continue this process until all edges have been used. This produces an Euler circuit. Since every edge is included and no edge is included more than once.
- Similar reasoning can be used to show that a graph with exactly two vertices of odd degree must have an Euler path connecting these two vertices of odd degree

Algorithm for Constructing an Euler Circuits

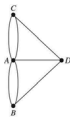
In our proof we developed this algorithms for constructing a Euler circuit in a graph with no vertices of odd degree.

procedure *Euler*(*G*: connected multigraph with all vertices of even degree)
circuit := a circuit in *G* beginning at an arbitrarily chosen vertex with edges successively added to form a path that returns to this vertex.
H := *G* with the edges of this circuit removed
while *H* has edges
subciruit := a circuit in *H* beginning at a vertex in *H* that also is an endpoint of an edge in circuit.
H := *H* with edges of *subciruit* and all isolated vertices removed
circuit := *circuit* with *subcircuit* inserted at the appropriate vertex.
return *circuit*{*circuit* is an Euler circuit}

Necessary and Sufficient Conditions for Euler Circuits and Paths (*continued*)

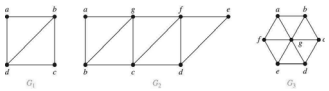
Theorem: A connected multigraph with at least two vertices has an Euler circuit if and only if each of its vertices has an even degree and it has an Euler path if and only if it has exactly two vertices of odd degree.

Example: Two of the vertices in the multigraph model of the Königsberg bridge problem have odd degree. Hence, there is no Euler circuit in this multigraph and it is impossible to start at a given point, cross each bridge exactly once, and return to the starting point.



Euler Circuits and Paths

Example:



G_1 contains exactly two vertices of odd degree (b and d). Hence it has an Euler path, e.g., d, a, b, c, d, b.

G_2 has exactly two vertices of odd degree (b and d). Hence it has an Euler path, e.g., $b, a, g, f, e, d, c, g, b, c, f, d$.

G_3 has six vertices of odd degree. Hence, it does not have an Euler path.

Applications of Euler Paths and Circuits

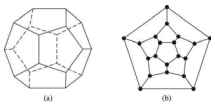
- Euler paths and circuits can be used to solve many practical problems such as finding a path or circuit that traverses each
  - street in a neighborhood,
  - road in a transportation network,
  - connection in a utility grid,
  - link in a communications network.
- Other applications are found in the
  - layout of circuits,
  - network multicasting,
  - molecular biology, where Euler paths are used in the sequencing of DNA.

Hamilton Paths and Circuits



William Rowan Hamilton (1805-1865)

- Euler paths and circuits contained every edge only once. Now we look at paths and circuits that contain every vertex exactly once.
- William Hamilton invented the *Icosian puzzle* in 1857. It consisted of a wooden dodecahedron (with 12 regular pentagons as faces), illustrated in (a), with a peg at each vertex, labeled with the names of different cities. String was used to used to plot a circuit visiting 20 cities exactly once
- The graph form of the puzzle is given in (b).



- The solution (a Hamilton circuit) is given here.



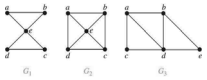
Hamilton Paths and Circuits

Definition: A simple path in a graph G that passes through every vertex exactly once is called a *Hamilton path*, and a simple circuit in a graph G that passes through every vertex exactly once is called a *Hamilton circuit.*

That is, a simple path $x_0, x_1, ..., x_{n-1}, x_n$ in the graph $G = (V, E)$ is called a Hamilton path if $V = \{x_0, x_1, ..., x_{n-1}, x_n\}$ and $x_i \neq x_j$ for $0 \leq i < j \leq n$, and the simple circuit $x_0, x_1, ..., x_{n-1}, x_n, x_0$ (with $n > 0$) is a Hamilton circuit if $x_0, x_1, ..., x_{n-1}, x_n$ is a Hamilton path.

Hamilton Paths and Circuits (*continued*)

Example: Which of these simple graphs has a Hamilton circuit or, if not, a Hamilton path?



Solution: G_1 has a Hamilton circuit: *a*, *b*, *c*, *d*, *e*, *a*.

G_2 does not have a Hamilton circuit (Why?), but does have a Hamilton path : *a*, *b*, *c*, *d*.

G_3 does not have a Hamilton circuit, or a Hamilton path. Why?

Necessary Conditions for Hamilton Circuits



Gabriel Andrew Dirac
(1925-1984)

- Unlike for an Euler circuit, no simple necessary and sufficient conditions are known for the existence of a Hamiton circuit.
- However, there are some useful necessary conditions. We describe two of these now.

Dirac's Theorem: If *G* is a simple graph with $n \geq 3$ vertices such that the degree of every vertex in *G* is $\geq n/2$, then *G* has a Hamilton circuit.

Ore's Theorem: If *G* is a simple graph with $n \geq 3$ vertices such that $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices, then G has a Hamilton circuit.

Øysten Ore
(1899-1968)



Applications of Hamilton Paths and Circuits

- Applications that ask for a path or a circuit that visits each intersection of a city, each place pipelines intersect in a utility grid, or each node in a communications network exactly once, can be solved by finding a Hamilton path in the appropriate graph.
- The famous *traveling salesperson problem* (*TSP*) asks for the shortest route a traveling salesperson should take to visit a set of cities. This problem reduces to finding a Hamilton circuit such that the total sum of the weights of its edges is as small as possible.
- A family of binary codes, known as *Gray codes*, which minimize the effect of transmission errors, correspond to Hamilton circuits in the *n*-cube Q_n. (*See the text for details*.)

Trees

Chapter 11

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Introduction to Trees
- Applications of Trees (*not currently included in overheads*)
- Tree Traversal
- Spanning Trees
- Minimum Spanning Trees (*not currently included in overheads*)

Introduction to Trees

Section 11.1

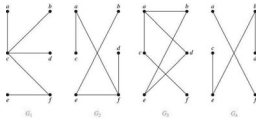
Section Summary

- Introduction to Trees
- Rooted Trees
- Trees as Models
- Properties of Trees

Trees

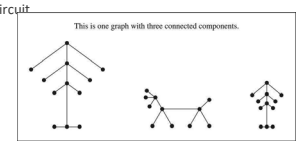
Definition: A *tree* is a connected undirected graph with no simple circuits.

Example: Which of these graphs are trees?



Solution: G_1 and G_2 are trees - both are connected and have no simple circuits. Because *e*, *b*, *a*, *d*, *e* is a simple circuit, G_3 is not a tree. G_4 is not a tree because it is not connected.

Definition: A *forest* is a graph that has no simple circuit but is not connected. Each of the connected components in a forest is a tree.



Trees (*continued*)

Theorem: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Proof: Assume that *T* is a tree. Then *T* is connected with no simple circuits. Hence, if *x* and *y* are distinct vertices of *T*, there is a simple path between them (by Theorem 1 of Section 10.4). This path must be unique - for if there were a second path, there would be a simple circuit in *T* (by Exercise 59 of Section 10.4). Hence, there is a unique simple path between any two vertices of a tree.

Now assume that there is a unique simple path between any two vertices of a graph *T*. Then *T* is connected because there is a path between any two of its vertices. Furthermore, *T* can have no simple circuits since if there were a simple circuit, there would be two paths between some two vertices.

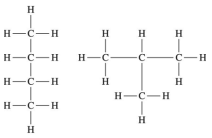
Hence, a graph with a unique simple path between any two vertices is a tree. ◄

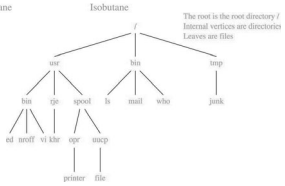
Trees as Models

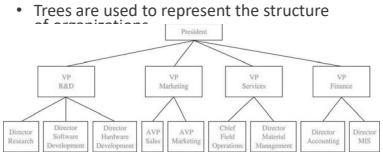
Arthur Cayley
(1821-1895)



- Trees are used as models in computer science, chemistry, geology, botany, psychology, and many other areas.
- Trees were introduced by the mathematician Cayley in 1857 in his work counting the number of isomers of saturated hydrocarbons. The two isomers of butane are shown at the right.
- The organization of a computer file system into directories, subdirectories, and files is naturally represented as a tree.
- Trees are used to represent the structure of organizations.



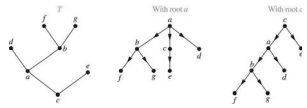




Rooted Trees

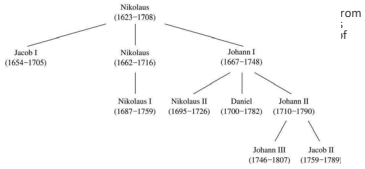
Definition: A *rooted tree* is a tree in which one vertex has been designated as the *root* and every edge is directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.



Rooted Tree Terminology

- Terminology for rooted trees is a botany and family tree mathematicians).



- If *v* is a vertex of a rooted tree other than the root, the *parent* of *v* is the unique vertex *u* such that there is a directed edge from *u* to *v*. When *u* is a parent of *v*, *v* is called a *child* of *u*. Vertices with the same parent are called *siblings*.
- The *ancestors* of a vertex are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root. The *descendants* of a vertex *v* are those vertices that have *v* as an ancestor.
- A vertex of a rooted tree with no children is called a *leaf*. Vertices that have children are called *internal vertices*.
- If *a* is a vertex in a tree, the *subtree* with *a* as its root is the subgraph of the tree consisting of *a* and its descendants and all edges incident to these descendants.

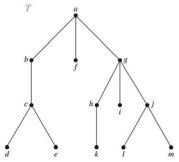
Terminology for Rooted Trees

Example: In the rooted tree *T* (with root *a*):

(i) Find the parent of *c*, the children of *g*, the siblings of *h*, the ancestors of *e*, and the descendants of *b*.

(ii) Find all internal vertices and all leaves.

(iii) What is the subtree rooted at *G*?

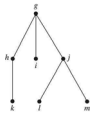


Solution:

(i) The parent of *c* is *b*. The children of *g* are *h*, *i*, and *j*. The siblings of *h* are *i* and *j*. The ancestors of *e* are c, *b*, and *a*. The descendants of *b* are *c*, *d*, and *e*.

(ii) The internal vertices are *a*, *b*, *c*, *g*, *h*, and *j*. The leaves are *d*, *e*, *f*, *i*, *k*, *l*, and *m*.

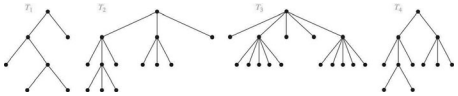
(iii) We display the subtree rooted at *g*.



m-ary Rooted Trees

Definition: A rooted tree is called an *m-ary tree* if every internal vertex has no more than *m* children. The tree is called a *full m-ary tree* if every internal vertex has exactly *m* children. An *m*-ary tree with *m* = 2 is called a *binary* tree.

Example: Are the following rooted trees full *m*-ary trees for some positive integer *m*?



Solution: T_1 is a full binary tree because each of its internal vertices has two children. T_2 is a full 3-ary tree because each of its internal vertices has three children. In T_3 each internal vertex has five children, so T_3 is a full 5-ary tree. T_4 is not a full *m*-ary tree for any m because some of its internal vertices have two children and others have three children.

Ordered Rooted Trees

Definition: An *ordered rooted tree* is a rooted tree where the children of each internal vertex are ordered.

- We draw ordered rooted trees so that the children of each internal vertex are shown in order from left to right.

Definition: A *binary tree* is an ordered rooted where where each internal vertex has at most two children. If an internal vertex of a binary tree has two children, the first is called the *left child* and the second the *right child*. The tree rooted at the left child of a vertex is called the *left subtree* of this vertex, and the tree rooted at the right child of a vertex is called the *right subtree* of this vertex.

Example: Consider the binary tree *T*.

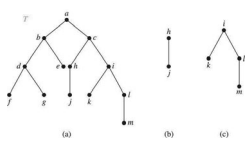
(*i*) What are the left and right children of *d*?

(*ii*) What are the left and right subtrees of *c*?

Solution:

(*i*) The left child of *d* is *f* and the right child is *g*.

(*ii*) The left and right subtrees of *c* are displayed in (b) and (c).



Properties of Trees

Theorem 2: A tree with *n* vertices has $n - 1$ edges.

***Proof* (*by mathematical induction*):**

BASIS STEP: When $n = 1$, a tree with one vertex has no edges. Hence, the theorem holds when $n = 1$.

INDUCTIVE STEP: Assume that every tree with *k* vertices has $k - 1$ edges.

Suppose that a tree *T* has $k + 1$ vertices and that *v* is a leaf of *T*. Let *w* be the parent of *v*. Removing the vertex *v* and the edge connecting *w* to *v* produces a tree *T'* with *k* vertices. By the inductive hypothesis, *T'* has $k - 1$ edges. Because *T* has one more edge than *T'*, we see that *T* has *k* edges. This completes the inductive step. ◄

Counting Vertices in Full *m*-Ary Trees

Theorem 3: A full *m*-ary tree with *i* internal vertices has $n = mi + 1$ vertices.

Proof : Every vertex, except the root, is the child of an internal vertex. Because each of the *i* internal vertices has *m* children, there are *mi* vertices in the tree other than the root. Hence, the tree contains $n = mi + 1$ vertices. ◄

Counting Vertices in Full *m*-Ary Trees (*continued*)

Theorem 4: A full *m*-ary tree with

(*i*) *n* vertices has $i = (n-1)/m$ internal vertices and $l = [(m-1)n+1]/m$ leaves,

(*ii*) *i* internal vertices has $n = mi + 1$ vertices and $l = (m-1)i + 1$ leaves,

(*iii*) *l* leaves has $n = (ml-1)/(m-1)$ vertices and $i = (l-1)/(m-1)$ internal vertices.

proofs of parts (ii) and (iii) are left as exercises

***Proof* (*of part i*):** Solving for *i* in $n = mi + 1$ (from Theorem 3) gives $i = (n-1)/m$. Since each vertex is either a leaf or an internal vertex, $n = l + i$. By solving for *l* and using the formula for *i*, we see that $l = n - i = n - (n-1)/m = [(m-1)n+1]/m$. ◄

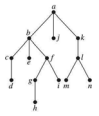
Level of vertices and height of trees

- When working with trees, we often want to have rooted trees where the subtrees at each vertex contain paths of approximately the same length.
- To make this idea precise we need some definitions:
  - The *level* of a vertex *v* in a rooted tree is the length of the unique path from the root to this vertex.
  - The *height* of a rooted tree is the maximum of the levels of the vertices.

Example:

(*i*) Find the level of each vertex in the tree to the right.

(*ii*) What is the height of the tree?



Solution:

(*i*) The root *a* is at level 0. Vertices *b*, *j*, and *k* are at level 1.

Vertices *c*, *e*, *f*, and *l* are at level 2. Vertices *d*, *g*, *i*, *m*, and *n* are at level 3.

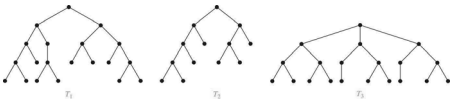
Vertex *h* is at level 4.

(*ii*) The height is 4, since 4 is the largest level of any vertex.

Balanced *m*-Ary Trees

Definition: A rooted *m*-ary tree of height *h* is *balanced* if all leaves are at levels *h* or $h-1$.

Example: Which of the rooted trees shown below is balanced?



Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

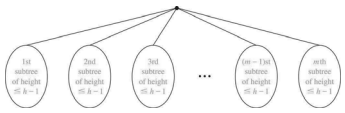
The Bound for the Number of Leaves in an *m*-Ary Tree

Theorem 5: There are at most m^h leaves in an *m*-ary tree of height *h*.

***Proof* (*by mathematical induction on height*):**

BASIS STEP: Consider an *m*-ary trees of height 1. The tree consists of a root and no more than *m* children, all leaves. Hence, there are no more than $m^1 = m$ leaves in an *m*-ary tree of height 1.

INDUCTIVE STEP: Assume the result is true for all *m*-ary trees of height < *h*. Let *T* be an *m*-ary tree of height *h*. The leaves of *T* are the leaves of the subtrees of *T* we get when we delete the edges from the root to each of the vertices of level 1.



Each of these subtrees has height ≤ $h-1$. By the inductive hypothesis, each of these subtrees has at most m^{h-1} leaves. Since there are at most *m* such subtrees, there are at most $m \cdot m^{h-1} = m^h$ leaves in the tree. ◄

Corollary 1: If an *m*-ary tree of height *h* has *l* leaves, then $h \geq \lceil \log_m l \rceil$. If the *m*-ary tree is full and balanced, then $h = \lceil \log_m l \rceil$. (*see text for the proof*)

Tree Traversal

Section 11.3

Section Summary

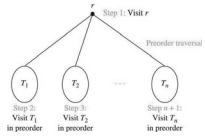
- Universal Address Systems (*not currently included in overheads*)
- Traversal Algorithms
- Infix, Prefix, and Postfix Notation

Tree Traversal

- Procedures for systematically visiting every vertex of an ordered tree are called *traversals*.
- The three most commonly used *traversals* are *preorder traversal*, *inorder traversal*, and *postorder traversal*.

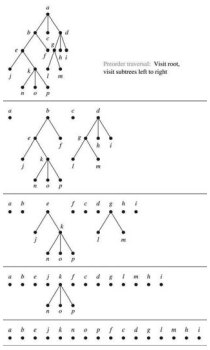
Preorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *preorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The preorder traversal begins by visiting r, and continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.



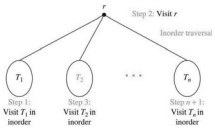
Preorder Traversal (*continued*)

```
procedure preorder (T: ordered rooted tree)
r := root of T
list r
for each child c of r from left to right
   T(c) := subtree with c as root
   preorder(T(c))
```



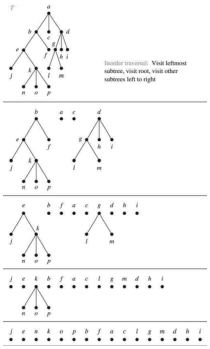
Inorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *inorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The inorder traversal begins by traversing T_1 in inorder, then visiting r, and continues by traversing T_2 in inorder, and so on, until T_n is traversed in inorder.



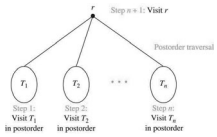
Inorder Traversal (*continued*)

```
procedure inorder (T: ordered rooted tree)
r := root of T
if r is a leaf then list r
else
   l := first child of r from left to right
   T(l) := subtree with l as its root
   inorder(T(l))
   list(r)
   for each child c of r from left to right
      T(c) := subtree with c as root
      inorder(T(c))
```



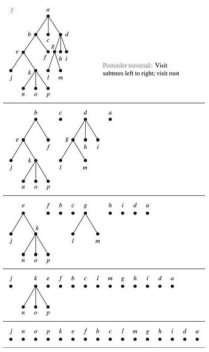
Postorder Traversal

Definition: Let T be an ordered rooted tree with root r. If T consists only of r, then r is the *postorder traversal* of T. Otherwise, suppose that $T_1, T_2, \ldots, T_n$ are the subtrees of r from left to right in T. The postorder traversal begins by traversing T_1 in postorder, then T_2 in postorder, and so on, after T_n is traversed in postorder, r is visited.



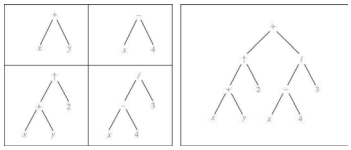
Postorder Traversal (*continued*)

```
procedure postordered (T: ordered rooted tree)
r := root of T
for each child c of r from left to right
   T(c) := subtree with c as root
   postorder(T(c))
list r
```



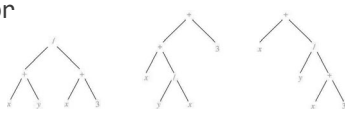
Expression Trees

- Complex expressions can be represented using ordered rooted trees.
- Consider the expression $((x + y) \uparrow 2) + ((x - 4)/3)$.
- A binary tree for the expression can be built from the bottom up, as is illustrated here.



Infix Notation

- An inorder traversal of the tree representing an expression produces the original expression when parentheses are included except for unary operations, which now immediately follow their operands.
- We illustrate why parentheses are needed with an example that displays three trees all yield the same infix representation



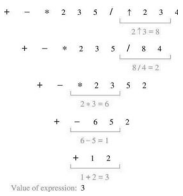
Prefix Notation



Jan Łukasiewicz (1878-1956)

- When we traverse the rooted tree representation of an expression in preorder, we obtain the *prefix* form of the expression. Expressions in prefix form are said to be in *Polish notation*, named after the Polish logician Jan Łukasiewicz.
- Operators precede their operands in the prefix form of an expression. Parentheses are not needed as the representation is unambiguous.
- The prefix form of $((x + y) \uparrow 2) + ((x - 4)/3)$ is $+ \uparrow + x\ y\ 2\ /\ -\ x\ 4\ 3$.
- Prefix expressions are evaluated by working from right to left. When we encounter an operator, we perform the corresponding operation with the two operations to the right.

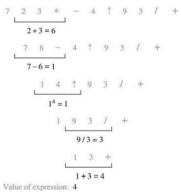
Example: We show the steps used to evaluate a particular prefix expression:



Postfix Notation

- We obtain the *postfix form* of an expression by traversing its binary trees in postorder. Expressions written in postfix form are said to be in *reverse Polish notation.*
- Parentheses are not needed as the postfix form is unambiguous.
- $x\ y + 2 \uparrow x\ 4 - 3\ / +$ is the postfix form of $((x + y) \uparrow 2) + ((x - 4)/3)$.
- A binary operator follows its two operands. So, to evaluate an expression one works from left to right, carrying out an operation represented by an operator on its preceding operands.

Example: We show the steps used to evaluate a particular postfix expression.



Spanning Trees

Section 11.4

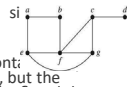
Section Summary

- Spanning Trees
- Depth-First Search
- Breadth-First Search
- Backtracking Applications (*not currently included in overheads*)
- Depth-First Search in Directed Graphs

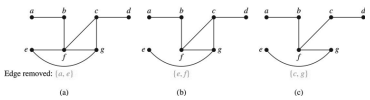
Spanning Trees

Definition: Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.

Example: Find the spanning tree of this si



Solution: The graph is connected, but is not a tree because it cont circuits. Remove the edge $\{a, e\}$. Now one simple circuit is gone, but the remaining subgraph still has a simple circuit. Remove the edge $\{e, f\}$ and then the edge $\{c, g\}$ to produce a simple graph with no simple circuits. It is a spanning tree, because it contains every vertex of the original graph.



Spanning Trees (*continued*)

Theorem: A simple graph is connected if and only if it has a spanning tree.

Proof: Suppose that a simple graph G has a spanning tree T. T contains every vertex of G and there is a path in T between any two of its vertices. Because T is a subgraph of G, there is a path in G between any two of its vertices. Hence, G is connected.

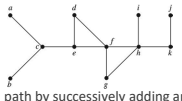
Now suppose that G is connected. If G is not a tree, it contains a simple circuit. Remove an edge from one of the simple circuits. The resulting subgraph is still connected because any vertices connected via a path containing the removed edge are still connected via a path with the remaining part of the simple circuit. Continue in this fashion until there are no more simple circuits. A tree is produced because the graph remains connected as edges are removed. The resulting tree is a spanning tree because it contains every vertex of G.

Depth-First Search

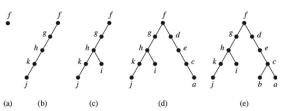
- To use *depth-first search* to build a spanning tree for a connected simple graph first arbitrarily choose a vertex of the graph as the root.
  - Form a path starting at this vertex by successively adding vertices and edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path. Continue adding vertices and edges to this path as long as possible.
  - If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
  - Otherwise, move back to the next to the last vertex in the path, and if possible, form a new path starting at this vertex and passing through vertices not already visited. If this cannot be done, move back another vertex in the path.
  - Repeat this procedure until all vertices are included in the spanning tree.

Depth-First Search (*continued*)

Example: Use depth-first search to find a spanning tree of this graph.

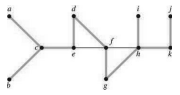


Solution: We start arbitrarily with vertex f. We build a path by successively adding an edge that connects the last vertex added to the path and a vertex not already in the path, as long as this is possible. The result is a path that connects f, g, h, k, and j. Next, we return to k, but find no new vertices to add. So, we return to h and add the path with one edge that connects h and i. We next return to f, and add the path connecting f, d, e, c, and a. Finally, we return to c and add the path connecting c and b. We now stop because all vertices have been added.



Depth-First Search (*continued*)

- The edges selected by depth-first search of a graph are called *tree edges*. All other edges of the graph must connect a vertex to an ancestor or descendant of the vertex in the graph. These are called *back edges*.
- In this figure, the tree edges are shown with heavy blue lines. The two thin black edges are back edges.



Depth-First Search Algorithm

- We now use pseudocode to specify depth-first search. In this recursive algorithm, after adding an edge connecting a vertex *v* to the vertex *w*, we finish exploring *w* before we return to *v* to continue exploring from *v*.

procedure *DFS*(*G*: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
visit(v_1)

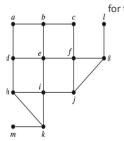
procedure *visit*(*v*: vertex of *G*)
for each vertex w adjacent to v and not yet in T
  add vertex w and edge $\{v,w\}$ to T
  visit(*w*)

Breadth-First Search

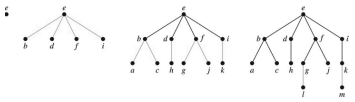
- We can construct a spanning tree using *breadth-first search*. We first arbitrarily choose a root from the vertices of the graph.
  - Then we add all of the edges incident to this vertex and the other endpoint of each of these edges. We say that these are the vertices at level 1.
  - For each vertex added at the previous level, we add each edge incident to this vertex, as long as it does not produce a simple circuit. The new vertices we find are the vertices at the next level.
  - We continue in this manner until all the vertices have been added and we have a spanning tree.

Breadth-First Search (*continued*)

Example: Use breadth-first search to find a spanning tree for this graph.



Solution: We arbitrarily choose vertex *e* as the root. We then add the edges from *e* to *b*, *d*, *f*, and *i*. These four vertices make up level 1 in the tree. Next, we add the edges from *b* to *a* and *c*, the edges from *d* to *h*, the edges from *f* to *j* and *g*, and the edge from *i* to *k*. The endpoints of these edges not at level 1 are at level 2. Next, add edges from these vertices to adjacent vertices not already in the graph. So, we add edges from *g* to *l* and from *k* to *m*. We see that level 3 is made up of the vertices *l* and *m*. This is the last level because there are no new vertices to find.



Breadth-First Search Algorithm

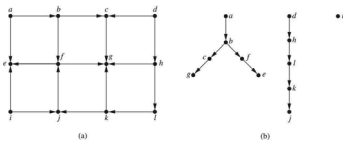
- We now use pseudocode to describe breadth-first search.

procedure *BFS*(*G*: connected graph with vertices $v_1, v_2, \ldots, v_n$)
T := tree consisting only of the vertex v_1
L := empty list *visit*(v_1)
put v_1 in the list L of unprocessed vertices
while L is not empty
  remove the first vertex, *v*, from L
  for each neighbor *w* of *v*
    if *w* is not in L and not in T **then**
      add *w* to the end of the list L
      add *w* and edge {*v*,*w*} to T

Depth-First Search in Directed Graphs

- Both depth-first search and breadth-first search can be easily modified to run on a directed graph. But the result is not necessarily a spanning tree, but rather a spanning forest.

Example: For the graph in (a), if we begin at vertex *a*, depth-first search adds the path connecting *a*, *b*, *c*, and *g*. At *g*, we are blocked, so we return to *c*. Next, we add the path connecting *f* to *e*. Next, we return to *a* and find that we cannot add a new path. So, we begin another tree with *d* as its root. We find that this new tree consists of the path connecting the vertices *d*, *h*, *l*, *k*, and *j*. Finally, we add a new tree, which only contains *i*, its root.



- To index websites, search engines such as Google systematically explore the web starting at known sites. The programs that do this exploration are known as *Web spiders*. They may use both breath-first search or depth-first search to explore the Web graph.

Boolean Algebra

Chapter 12

Chapter Summary



Claude Shannon
(1916 - 2001)

- Boolean Functions
- Representing Boolean Functions
- Logic Gates
- Minimization of Circuits (*not currently included in overheads*)

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Boolean Functions

Section 12.1

Section Summary

- Introduction to Boolean Algebra
- Boolean Expressions and Boolean Functions
- Identities of Boolean Algebra
- Duality
- The Abstract Definition of a Boolean Algebra

Introduction to Boolean Algebra

- Boolean algebra has rules for working with elements from the set $\{0, 1\}$ together with the operators + (Boolean sum), $\cdot$ (Boolean product), and $\bar{}$ (complement).
- These operators are defined by:
  - *Boolean sum*: $1 + 1 = 1, 1 + 0 = 1, 0 + 1 = 1, 0 + 0 = 0$
  - *Boolean product*: $1 \cdot 1 = 1, 1 \cdot 0 = 0, 0 \cdot 1 = 0, 0 \cdot 0 = 0$
  - *complement*: $\bar{0} = 1, \bar{1} = 0$

Example: Find the value of $1 \cdot 0 + \overline{(0 + 1)}$

Solution : $1 \cdot 0 + \overline{(0 + 1)} = 0 + \bar{1}$
$= 0 + 0$
$= 0$

Boolean Expressions and Boolean Functions

Definition: Let $B = \{0, 1\}$. Then $B^n = \{(x_1, x_2, \ldots, x_n) \mid x_i \in B \text{ for } 1 \le i \le n\}$ is the set of all possible n-tuples of 0s and 1s. The variable x is called a *Boolean variable* if it assumes values only from B, that is, if its only possible values are 0 and 1. A function from B^n to B is called a *Boolean function of degree n*.

Example: The function $F(x, y) = x$ from the set of ordered pairs of Boolean variables to the set $\{0, 1\}$ is a Boolean function of degree 2.

TABLE 1

x	y	$F(x, y)$
1	1	1
1	0	0
0	1	0
0	0	1

Boolean Expressions and Boolean Functions (*continued*)

Example: Find the values of the Boolean function represented by $F(x, y, z) = xy + \bar{z}$.

Solution: We use a table with a row for each combination of values of x, y, and z to compute the values of $F(x,y,z)$.

TABLE 2

x	y	z	xy	$\bar{z}$	$F(x, y, z) = xy + \bar{z}$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	0	0	0
0	0	0	0	1	1

Boolean Expressions and Boolean Functions (*continued*)

Definition: Boolean functions F and G of n variables are equal if and only if $F(b_1, b_2, \ldots, b_n) = G(b_1, b_2, \ldots, b_n)$ whenever $b_1, b_2, \ldots, b_n$ belong to B. Two different Boolean expressions that represent the same function are *equivalent*.

Definition: The complement of the Boolean function F is the function $\bar{F}$, where $\bar{F}(x_1, x_2, \ldots, x_n) = \overline{F(x_1, x_2, \ldots, x_n)}$.

Definition: Let F and G be Boolean functions of degree n. The Boolean sum $F + G$ and the Boolean product FG are defined by

$(F + G)(x_1, x_2, \ldots, x_n) = F(x_1, x_2, \ldots, x_n) + G(x_1, x_2, \ldots, x_n)$

$(FG)(x_1, x_2, \ldots, x_n) = F(x_1, x_2, \ldots, x_n)G(x_1, x_2, \ldots, x_n)$

Boolean Functions

Example: How many different Boolean functions of degree n are there?

Solution: By the product rule for counting, there are 2^n different n-tuples of 0s and 1s. Because a Boolean function is an assignment of 0 or 1 to each of these different n-tuples, by the product rule there are 2^{2^n} different Boolean functions of degree n.

TABLE 4 **The Number of Boolean Functions of Degree *n*.**

Degree	*Number*
1	4
2	16
3	256
4	65,536
5	4,294,967,296
6	18,446,744,073,709,551,616

The example tells us that there are 16 different Boolean functions of degree two. We display these in Table 3.

TABLE 3 **The 16 Boolean Functions of Degree Two.**

x	y	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
1	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
0	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

Identities of Boolean Algebra

TABLE 5 **Boolean Identities.**

Identity	*Name*
$\bar{\bar{x}} = x$	Law of the double complement
$x + x = x$ $x \cdot x = x$	Idempotent laws
$x + 0 = x$ $x \cdot 1 = x$	Identity laws
$x + 1 = 1$ $x \cdot 0 = 0$	Domination laws
$x + y = y + x$ $xy = yx$	Commutative laws
$x + (y + z) = (x + y) + z$ $x(yz) = (xy)z$	Associative laws
$x + yz = (x + y)(x + z)$ $x(y + z) = xy + xz$	Distributive laws
$\overline{(xy)} = \bar{x} + \bar{y}$ $\overline{(x + y)} = \bar{x}\,\bar{y}$	De Morgan's laws
$x + xy = x$ $x(x + y) = x$	Absorption laws
$x + \bar{x} = 1$	Unit property
$x\bar{x} = 0$	Zero property

Each identity can be proved using a table.

All identities in Table 5, except for the first and the last two come in pairs. Each element of the pair is the *dual* of the other (obtained by switching Boolean sums and Boolean products and 0's and 1's.

The Boolean identities correspond to the identities of propositional logic (Section 1.3) and the set identities (Section 2.2).

Identities of Boolean Algebra

Example: Show that the distributive law $x(y + x) = xy + xz$ is valid.

Solution: We show that both sides of this identity always take the same value by constructing this table.

TABLE 6 Verifying One of the Distributive Laws.

x	y	z	$y+z$	xy	xz	$x(y+z)$	$xy+xz$
1	1	1	1	1	1	1	1
1	1	0	1	1	0	1	1
1	0	1	1	0	1	1	1
1	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0

Formal Definition of a Boolean Algebra

Definition: A *Boolean algebra* is a set B with two binary operations $\vee$ and $\wedge$, elements 0 and 1, and a unary operation $^-$ such that for all x, y, and z in B:

$x \vee 0 = x$
$x \wedge 1 = x$ — *identity laws*

$x \vee \bar{x} = 1$
$x \wedge \bar{x} = 0$ — *complement laws*

$(x \vee y) \vee z = x \vee (y \vee z)$
$(x \wedge y) \wedge z = x \wedge (y \wedge z)$ — *associative laws*

$x \vee y = y \vee x$
$x \wedge y = y \wedge x$ — *commutative laws*

$x \vee (y \wedge z) = (x \vee y) \wedge (y \vee z)$
$x \wedge (y \vee z) = (x \wedge y) \vee (y \wedge z)$ — *distributive laws*

The set of propositional variables with the operators $\wedge$ and $\vee$, elements **T** and **F**, and the negation operator $\neg$ is a Boolean algebra.

The set of subsets of a universal set with the operators $\cup$ and $\cap$, the empty set ($\varnothing$), universal set (U), and the set complementation operator ($\bar{}$) is a Boolean algebra.

Representing Boolean Functions

Section 12.2

Section Summary

- Sum-of-Products Expansions
- Functional Completeness

Sum-of-Products Expansion

Example: Find Boolean expressions that represent the functions (i) $F(x, y, z)$ and (ii) $G(x, y, z)$ in Table 1.

TABLE 1

x	y	z	F	G
1	1	1	0	0
1	1	0	0	1
1	0	1	1	0
1	0	0	0	0
0	1	1	0	0
0	1	0	0	1
0	0	1	0	0
0	0	0	0	0

Solution:

(i) To represent F we need the one term $x\bar{y}z$ because t expression has the value 1 when $x = z = 1$ and $y = 0$.

(ii) To represent the function G, we use the sum $xyz + x\bar{y}z$ because this expression has the value 1 when $x = y = 1$ and $z = 0$, or $x = z = 0$ and $y = 1$.

The general principle is that each combination of values of the variables for which the function has the value 1 requires a term in the Boolean sum that is the Boolean product of the variables or their complements.

Sum-of-Products Expansion (*cont*)

Definition: A *literal* is a Boolean variable or its complement. A *minterm* of the Boolean variables $x_1, x_2, \ldots, x_n$ is a Boolean product $y_1 y_2 \cdots y_n$, where $y_i = x_i$ or $y_i = \bar{x}_i$. Hence, a minterm is a product of n literals, with one literal for each variable.

The minterm $y_1, y_2, \ldots, y_n$ has value has value 1 if and only if each x_i is 1.This occurs if and only if $x_i = 1$ when $y_i = x_i$ and $x_i = 0$ when $y_i = \bar{x}_i$.

Definition: The sum of minterms that represents the function is called the *sum-of-products expansion* or the *disjunctive normal form* of the Boolean function.

Sum-of-Products Expansion (*cont*)

Example: Find the sum-of-products expansion for the function $F(x,y,z) = (x + y)\bar{z}$.

Solution: We use two methods, first using a table and second using Boolean identities.

(*i*) Form the sum of the minterms corresponding to each row of the table that has the value 1.

TABLE 2

x	y	z	$x+y$	$\bar{z}$	$(x+y)\bar{z}$
1	1	1	1	0	0
1	1	0	1	1	1
1	0	1	1	0	0
1	0	0	1	1	1
0	1	1	1	0	0
0	1	0	1	1	1
0	0	1	0	0	0
0	0	0	0	1	0

Including a tem for each row of the table for which $F(x,y,z) = 1$ gives us $F(x, y, z) = xy\bar{z} + x\bar{y}\bar{z} + \bar{x}y\bar{z}$.

Sum-of-Products Expansion (*cont*)

(*ii*) We now use Boolean identities to find the disjunctive normal form of $F(x,y,z)$:

$F(x,y,z) = (x + y)\bar{z}$
$= x\bar{z} + y\bar{z}$ *distributive law*
$= x1\bar{z} + 1y\bar{z}$ *identity law*
$= x(y + \bar{y})\bar{z} + (x + \bar{x})y\bar{z}$ *unit property*
$= xy\bar{z} + x\bar{y}\bar{z} + xy\bar{z} + \bar{x}y\bar{z}$ *distributive law*
$= xy\bar{z} + x\bar{y}\bar{z} + \bar{x}y\bar{z}$ *idempotent law*

Functional Completeness

Definition: Because every Boolean function can be represented using the Boolean operators ·, +, and $^-$, we say that the set {·, +, $^-$} is *functionally complete*.

- The set {·, $^-$} is functionally complete since $x + y = \overline{\bar{x}\bar{y}}$.
- The set {+, $^-$} is functionally complete since $xy = \overline{\bar{x} + \bar{y}}$.
- The *nand* operator, denoted by |, is defined by $1|1 = 0$, and $1|0 = 0|1 = 0|0 = 1$. The set consisting of just the one operator nand {|} is functionally complete. Note that $\bar{x} = x \mid x$ and $xy = (x|y)|(x|y)$.
- The *nor* operator, denoted by ↓, is defined by $0 \downarrow 0 = 1$, and $1 \downarrow 0 = 0 \downarrow 1 = 1 \downarrow 1 = 0$. The set consisting of just the one operator nor {↓} is functionally complete. (*see Exercises* 15 *and* 16)

Logic Gates

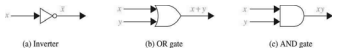
Section 12.3

Section Summary

- Logic Gates
- Combinations of Gates
- Examples of Circuits

Logic Gates

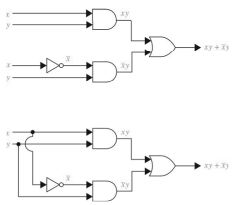
- We construct circuits using *gates*, which take as input the values of two or more Boolean variables and produce one or more bits as output, and *inverters*, which take the value of a Boolean variable as input and produce the complement of this value as output.





Combinations of Gates

- Combinatorial circuits can be constructed using a combination of inverters, OR gates, and AND gates. Gates may share input and the output of one or more gates may be input to another.
- We show two ways of constructing a circuit that produces the output $xy + \bar{x}y$.



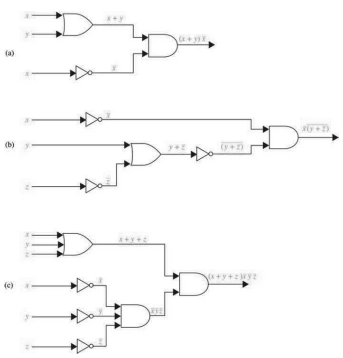
Combinations of Gates

Example: Construct circuits that produce these outputs

(a) $(x + y)\bar{x}$

(b) $\bar{x}\,\overline{(y + \bar{z})}$

(c) $(x + y + z)(\bar{x}\,\bar{y}\,\bar{z})$

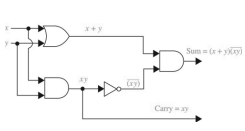


Adders

- Logic circuits can be used to add two positive integers from their binary expansions.
- The first step is to build a *half adder* that adds two bits, but which does not accept a carry from a previous addition.
- Since the circuit has more than one output, it is a *multiple output circuit*.

TABLE 3
Input and Output for the Half Adder.

Input		*Output*	
x	y	s	c
1	1	0	1
1	0	1	0
0	1	1	0
0	0	0	0

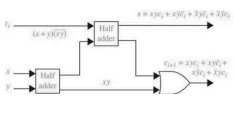


Adders (continued)

- A *full adder* is used to compute the sum bit and the carry bit when two bits and a carry are added.

TABLE 4
Input and Output for the Full Adder.

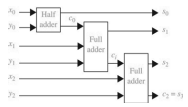
Input			*Output*	
x	y	c_i	s	c_{i+1}
1	1	1	1	1
1	1	0	0	1
1	0	1	0	1
1	0	0	1	0
0	1	1	0	1
0	1	0	1	0
0	0	1	1	0
0	0	0	0	0



Adders (continued)

- A half adder and multiple full adders can be used to produce the sum of *n* bit integers.

  Example: Here is a circuit to compute the sum of two three-bit integers.



Modeling Computation

Chapter 13

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Languages and Grammars
- Finite-State Machines with Output
- Finite-State Machines with No Output
- Language Recognition
- Turing Machines

Languages and Grammars

Section 13.1

Section Summary

- Phrase-Structure Grammars
- Types of Phrase-Structure Grammars
- Derivation Trees
- Backus-Naur Form

Introduction

- *Syntax* (form of a sentence) vs. *semantics* (meaning of a sentence)
- The sentence *the frog writes neatly* is a valid sentence according to the rules of English grammar. That is, it is syntactically correct, even though it's nonsensical (unless we are talking about a fantasy world).
- The sequence of words *swims quickly mathematics* is not a valid sentence according to the rules of English grammar.

Grammars

- The rules that specify the syntactically correct sentences of a natural language such as English are complex.
- Instead of studying natural languages, we can define *formal languages* that have well-defined rules of syntax.
- These rules of syntax are important both in linguistics (the study of natural languages) and in the study of programming languages.

An Example Grammar

1. a **sentence** is made up of a **noun phrase** followed by a **verb phrase**;
2. a **noun phrase** is made up of an **article** followed by an **adjective** followed by a **noun**, or
3. a **noun phrase** is made up of an **article** followed by a **noun**;
4. a **verb phrase** is made up of a **verb** followed by an **adverb**, or
5. a **verb phrase** is made up of a **verb**;
6. an **article** is *a*, or
7. an **article** is *the*;
8. an **adjective** is *large*, or
9. an **adjective** is *hungry*;
10. a **noun** is *rabbit*, or
11. a **noun** is *mathematician*;
12. a **verb** is *eats*, or
13. a **verb** is *hops*;
14. an **adverb** is *quickly*, or
15. an **adverb** is *wildly*.

We use these rules to form valid sentences by making a series of replacements until no more rules can be used.

An example sequence of replacements:

noun phrase verb phrase
article adjective noun verb phrase
article adjective noun verb adverb
the **adjective noun verb adverb**
the large **noun verb adverb**
the large rabbit **verb adverb**
the large rabbit hops **adverb**
the large rabbit hops quickly

Some additional valid sentences are:

a hungry mathematician eats wildly,
a large mathematician hops,
the rabbit eats quickly, etc.

But note that the following is not valid:

the quickly eats mathematician

Phrase-Structure Grammars

- A *vocabulary* (or *alphabet*) *V* is a finite, nonempty set of elements called *symbols*.
- A *word* (or *sentence*) over *V* is a string of finite length of elements of *V*.
- The *empty string* or *null string*, denoted by λ, is the string containing no symbols.
- The set of all words over *V* is denoted by *V**. A *language* over *V* is a subset of *V**.
- The elements of *V* that can not be replaced by other symbols are called *terminals*, e.g., *a*, *the*, and *rabbit* in the example grammar.
- Those that can be replaced by other symbols are called *nonterminals*, e.g., **sentence**, **noun phrase**, etc.
- The rules that specify when we can replace a string *V** with another string are called *productions* of the grammar. We denote by $z_0 \rightarrow z_1$ the production that specifies that z_0 can be replaced by z_1 within a string.

Phrase-Structure Grammars (*cont.*)

- A *phrase-structure grammar G =*(*V, T, S, P*) consists of a vocabulary *V,* a subset *T* of *V* consisting of terminal symbols, a *start symbol S* from *V*, and a finite set of *productions P*.
- The set $N = V - T$ is the set of nonterminal symbols.
- Every production in *P* must contain at least one nonterminal on its left side.

Example (Grammar 1): Let *G =*(*V, T, S, P*), where *V* = {*a*, *b*, *A*, *B*, *S*}, *T* = {*a*,*b*}, *S* is the start symbol, and *P* = {$S \rightarrow Aba$, $A \rightarrow BB$, $B \rightarrow ab$, $AB \rightarrow b$}.

Derivations

- Let *G =*(*V, T, S, P*) be a phrase-structure grammar. Let $w_0 = lz_0r$ (that is the concatenation of l, z_0, and r) and $w_1 = lz_1r$ be strings over *V*. If $z_0 \rightarrow z_1$ is a production of *G*, we say that w_1 is *directly derivable* from w_0 and write $w_0 \Rightarrow w_1$.
- If $w_0, w_1, \ldots, w_n$ are strings over *V* such that $w_0 \Rightarrow w_1$, $w_1 \Rightarrow w_2$, ..., $w_{n-1} \Rightarrow w_n$, then we say that w_n is *derivable from* w_0 and write $w_0 \stackrel{*}{\Rightarrow} w_n$.
- The sequence of steps used to obtain w_n from w_0 is called a *derivation*.

Example (Grammar 1): *Aaba* is directly derivable from *ABa* because $B \rightarrow ab$ is a production and *abababa* is derivable from *ABa* because $ABa \Rightarrow Aaba \Rightarrow BBaba \Rightarrow Bababa \Rightarrow abababa$ using the productions $B \rightarrow ab$, $A \rightarrow BB$, and $B \rightarrow ab$ in both of the last two steps of the derivation.

Language Generation

- Let *G =*(*V, T, S, P*) be a phrase-structure grammar. The *language generated by G,* denoted by *L*(*G*), is the set of all strings or terminals that are derivable from the starting state *S*.
- In other words, $L(G) = \{w \in T^* \mid S \stackrel{*}{\Rightarrow} w\}$.
- Let *G* be the grammar with the vocabulary *V* = {*S*, *A*, *a*, *b*}, a set of terminals *T* = {*a*, *b*}, starting symbol *S*, and productions *P* = {$S \rightarrow aA$, $S \rightarrow b$, $A \rightarrow aa$}.
- *L*(*G*) = {*b*, *aaa*}, because we can begin a derivation with $S \rightarrow aA$ or with $S \rightarrow b$, and from *aA* we can derive *aaa* using $A \rightarrow aa$. There are no other possible derivations.

Types of Phrase Structure Grammars



Avram Noam Chomsky (Born 1928)

- Phrase-structure grammars are classified by the types of allowable productions.

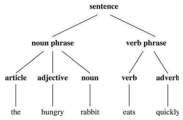
TABLE 1 **Types of Grammars.**

Type	*Restrictions on Productions* $w_1 \rightarrow w_2$
0	No restrictions
1	$w_1 = lAr$ and $w_2 = lwr$, where $A \in N$, $l, r, w \in (N \cup T)^*$ and $w \neq \lambda$; or $w_1 = S$ and $w_2 = \lambda$ as long as S is not on the right-hand side of another production
2	$w_1 = A$, where A is a nonterminal symbol
3	$w_1 = A$ and $w_2 = aB$ or $w_2 = a$, where $A \in N$, $B \in N$, and $a \in T$; or $w_1 = S$ and $w_2 = \lambda$

- Type 2 grammars are called *context-free grammars.* A language generated by a context-free grammar is called a *context-free language*.
- Type 3 grammars are called *context-sensitive grammars* (or a *regular grammar*). A language generated by a context-sensitive grammar is called a *context-sensitive language* (or a *regular language*).

Derivation Trees

- We can represent a derivation in the language generated by a context-free grammar by an ordered rooted tree, called a *derivation*, or *parse tree*.
  - The root of the tree represents the start symbol.
  - The internal vertices represent the nonterminal symbols that arise in the derivation.
  - The leaves represent the terminal symbols that arise.
  - If the production $A \rightarrow w$, where w is a word, arises in the derivation, the vertex that represents A has as children vertices that represent each symbol in w, in order from left to right.
- A derivation tree for the derivation of *the hungry rabbit eats quickly*, given the grammar described earlier.



Backus-Naur Form



John Backus (1924-2007)



Peter Naur (Born 1928)

- *Backus-Naur form* (*BNF*) is sometimes used to specify a type 2 grammar. It is often used to specify the syntactic rules of computer languages.
- The productions of a type 2 grammar have a single nonterminal symbol on their left-hand side.
- All the productions with the same nonterminal symbol on the left-hand side are combined into one statement using the symbol ::= instead of $\rightarrow$. Additionally,, all nonterminal symbols are enclosed in brackets (⟨⟩), and the right-hand side of productions are spearated by bars.
- For example, the productions $A \rightarrow Aa$, $A \rightarrow a$, and $A \rightarrow AB$ are written as ⟨*A*⟩ ::= ⟨*A*⟩*a* | *a* | ⟨*A*⟩ ⟨*B*⟩.

BNF and ALGOL 60

- In the programming language ALGOL 60 an identifier consists of a string of alphanumeric characters and must begin with a letter.
- The BNF description of allowable identifiers is:

  ⟨*identifier*⟩ ::=⟨*letter*⟩ | ⟨*identifier*⟩⟨*letter*⟩ | ⟨*identifier*⟩⟨*digit*⟩

  ⟨*letter* ⟩ ::= *a* | *b* | ⋯ | *y* | *z*

  ⟨*digit*⟩ ::= 0 | 1 | ⋯ | 8 | 9
- *x*99*a* is a valid identifier since the first rule can be used to replace ⟨*identifier*⟩ by ⟨*identifier*⟩⟨*letter*⟩ , the second rule to obtain ⟨*identifier*⟩ *a*, the first rule twice to obtain ⟨*identifier*⟩⟨*digit*⟩⟨*digit*⟩ *a*, the third rule twice to obtain ⟨*identifier*⟩99*a,* the first rule to obtain ⟨*letter*⟩99*a*, and finally the second rule to obtain *x*99*a.*

Finite-State Machines with Output

Section 13.2

Section Summary

- Finite-State Machines (FSMs) with Outputs
- Types of Finite-State Machines with Outputs (*not currently included in overheads*)

Introduction

- Many kinds of machines, including computers, can be modeled using a structure called a *finite-state machine (*or *finite automaton*).
- A finite-state machine consists of a finite set of states, a designated start state, an input alphabet, and a transition function that assigns a next state to every (state, input) pair
- As we will see in Sections $13.2 - 13.4$, some types of finite-state machines produce output, while for other types of finite-state machines that do not produce output some states are designated as accepting states.
- Finite-state machines are used in many diverse applications, including spell-checking programs, grammar checking, indexing, searching large bodies of text, speech recognition, XML, HTML, and network protocols

An Example of a Finite-State Machine with Output

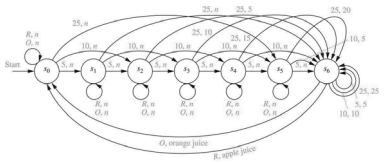
- A vending machine accepts nickels (5 cents) , dimes (10 cents) , and quarters (25 cents). When 30 cents or more has been deposited, the machine returns the amount over 30 cents. The customer can then press an orange button to receive a container of orange juice or a red button to receive a container of apple juice.
- The machine can be in any of the states s_i, $i = 0, \ldots, 6$, where s_i is the state where the machine has received $5i$ cents. The machine starts in state s_0, with 0 cents received. The possible inputs are 5 cents, 10 cents, 25 cents, the orange button (*O*), and the red button (*R*). The possible outputs are nothing (*n*), 5 cents, 15 cents, 20 cents, 25 cents, an orange juice, and an apple juice.

TABLE 1 **State Table for a Vending Machine.**

	Next State					Output				
	Input					*Input*				
State	**5**	**10**	**25**	***O***	***R***	**5**	**10**	**25**	***O***	***R***
s_0	s_1	s_2	s_5	s_0	s_0	*n*	*n*	*n*	*n*	*n*
s_1	s_2	s_3	s_6	s_1	s_1	*n*	*n*	*n*	*n*	*n*
s_2	s_3	s_4	s_6	s_2	s_2	*n*	*n*	5	*n*	*n*
s_3	s_4	s_5	s_6	s_3	s_3	*n*	*n*	10	*n*	*n*
s_4	s_5	s_6	s_6	s_4	s_4	*n*	*n*	15	*n*	*n*
s_5	s_6	s_6	s_6	s_5	s_5	*n*	5	20	*n*	*n*
s_6	s_6	s_6	s_6	s_0	s_0	5	10	25	OJ	AJ

An Example (*cont.*)

- We represent this vending machine using a directed graph with labeled edges, where each state is represented by a circle, edges represent transitions, and edges are labeled with the input and output for that transition.



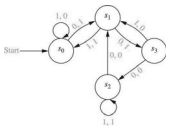
- We will trace the transitions and outputs of the vending machine when a student puts in a dime followed by a quarter, receives 5 cents back, and then pushes the orange button and receives an orange juice.
- The machine starts in state s_0.
- The first input is 10 cents, which changes the state to s_2 and gives no output.
- After the second input of 25 cents, the state changes to s_6 and gives 5 cents as output.
- The last input is the orange button, which changes the state back to s_0 and gives an orange juice as output.

FSMs with Outputs

- A *finite-state machine* $M = (S, I, O, f, g, s_0)$ consists of a finite set S of *states*, a finite *input alphabet* I, a finite *output alphabet* O, a *transition function* f that assigns to each state and input pair a new state, an *output function* g that assigns to each state and input pair an output, and an *initial state* s_0 .
- A state table is used to represent the values of the transition function f and the output function g for all (state, input).
- Alternatively, a finite-state machine can be represented by a state diagram, which is a directed graph with labeled edges. Each state is represented by a circle, and arrows labeled with the input and output pair represent the transitions.
- The state table and state diagram both represent the finite state machine with $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, and $O = \{0, 1\}$.

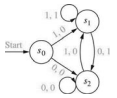
TABLE 2

	f		g	
	Input		*Input*	
State	**0**	**1**	**0**	**1**
s_0	s_1	s_0	1	0
s_1	s_3	s_0	1	1
s_2	s_1	s_2	0	1
s_3	s_2	s_1	0	0



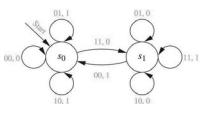
Unit-delay Machine

- An important element in many electronic devices is a *unit-delay machine*, which produces as output the input string delayed by a specified amount of time, i.e., padded with an initial string of 0s.
- How can a finite-state machine be constructed that delays an input string by one unit of time, that is, produces as output the bit string $0x_1x_2\ldots x_{k-1}$ given the input bit string $x_1x_2\ldots x_{k-1}$?
- A delay machine can be constructed that has 0 or 1 as possible inputs. The machine has the start state s_0. The transition from s_0 produces an output of 0. The machine is in state s_1 if the previous input was a 1 and it produces 1 as output for its next transition, and in state s_2 if the previous input was 0 and it produces an output of 0 for its next transition.



Addition Machine

- We will construct a finite-state machine that adds two positive integers using their binary expansions.
- Recall the conventional procedure to add $(x_n\ldots x_1x_0)_2$ and $(y_n\ldots y_1y_0)_2$.
  - First, the bits x_0 and y_0 are added, producing a sum bit z_0 and a carry bit c_0. Next the bits x_1 and y_1 are added together with the carry bit c_0. This gives a sum bit z_1 and a carry bit c_1.
  - The procedure continues until the nth stage, where x_n, y_n and the previous carry c_{n-1} are added to produce the sum bit z_n and the carry bit c_n, which is equal to the sum bit z_{n+1}.
- We can construct a finite state machine that uses just two states.
  - The start state s_0 is used to remember that the previous carry is 0.
  - The other state s_1 is used to remember that the previous carry is 1. (For simplicity, we assume that both x_n and y_n are 0.)
  - The inputs are pairs of bits. The transitions and the outputs are constructed from the sum of the two bits in the input and the carry represented by the state.
  - For example, when the machine is in state s_1 and receives 01 as input, the next state is s_1 and the output is 0, because the sum $0 + 1 + 1 = (10)_2$.



Finite-State Machines with No Output

Section 13.3

Section Summary

- Set of Strings
- Finite-State Automata
- Language Recognition by Finite-State Machines
- Designing Finite-State Automata
- Equivalent Finite-State Automata (*not currently included in overheads*)
- Nondeterministic Finite-State Automata

Set of Strings



Stephen Cole Kleene
(1909-1994)

- FSMs with no output, but with some states designated as *accepting states*, are specifically designed for recognizing languages.
- The *concatenation* of A and B, where A and B are subsets of V^*, denoted by AB, is the set of all strings of the form xy, where x is a string in A and y is a string in B.
- Let $A = \{0, 11\}$ and $B = \{1, 10, 110\}$. Then
  $AB = \{01, 010, 0110, 111, 110, 11110\}$ and
  $BA = \{10, 111, 100, 1011, 1100, 11011\}$
- If A is a subset of V^*, the *Kleene closure* of A, denoted by A^*, is the set consisting of arbitrarily long strings of elements of A. That is, $A^* = \bigcup_{k=0}^{\infty} A^k$.
- The Kleene closures of the sets $A = \{0\}$, $B = \{0,1\}$ and $C = \{11\}$ are $A^* = \{0^n \mid n = 0, 1, 2, \ldots.\}$, $B^* = V^*$, and $C^* = \{1^{2n} \mid n = 0, 1, 2, \ldots.\}$

Finite-State Automata (FSA)

- A *finite-state automaton* $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I, a *transition function* f that assigns a next state to every pair of state and input (so that f: $S \times I \rightarrow S$), an *initial* or *start state* s_0, and a subset F of S consisting of *final* (or *accepting*) *states*.
- FSAs can be represented using either state tables or state diagrams, in which final states are indicated with a double circle.
- The state diagram for the FSA $M = (S, I, f, s_0, F)$, where $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, $F = \{s_0, s_3\}$, and the transition diagram is in Table 1, is shown here.

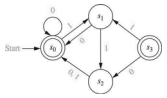


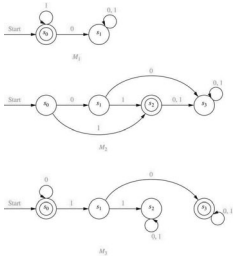
TABLE 1

	f	
	Input	
State	**0**	**1**
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_0
s_3	s_2	s_1

Language Recognition by FSAs

- A string x is said to be *recognized* (or *accepted*) by the machine $M = (S, I, f, s_0, F)$ if it takes the initial state s_0 to a final state, that is, $f(s_0, x)$. The *language recognized* (or *accepted*) by M, denoted by $L(M)$, is the set of all strings that are recognized by M. Two finite-state automata are called *equivalent* if they recognize the same language.

- The only final state of M_1 is s_0. The strings that take s_0 to itself consist of zero or more consecutive 1s. Hence, $L(M_1) = \{1^n \mid n = 0, 1, 2, \ldots.\}$.
- The only final state of M_2 is s_2. The strings that take s_0 to s_2 are 1 and 01 . Hence , $L(M_2) = \{1, 01\}$.
- The final state of M_3 are s_0 and s_3. The strings that take s_0 to itself are λ, 0, 00, 000,… . The strings that take s_0 to s_3 are a string of zero or more consecutive 0s, followed by 10, followed by any string. Hence, $L(M_3) = \{0^n,0^n10x \mid n = 0, 1, 2, \ldots., \text{ and } x \text{ is any string}\}$

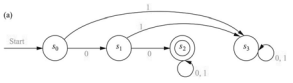


Language Recognition by FSAs (*cont.*)

Example: Construct a FSA that recognizes the set of bit strings that begin with two 0s.

Solution:

- Besides the start state s_0, we include a nonfinal state s_1; we move to s_1 from s_0 if the first bit is a 0.
- Next, we add a final state s_2, which we move to from s_1, if the second bit is a 0. We stay in this state no matter what the succeeding bits (if any) are.
- We need a nonfinal state s_3, so that we can move to it from s_0 if the first bit is a 1 and from s_1 if the second bit is a 1.

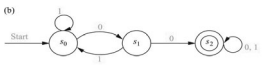


Language Recognition by FSA (*cont.*)

Example: Construct a FSA that recognizes the set of bit strings that contain two consecutive 0s.

Solution:

- Besides the start state s_0, we include a nonfinal state s_1, which tells us that the last input bit seen is a 0, but either the bit before was a 1, or this is the initial bit.
- Next, we add a final state s_2, which we move to from s_1, if the next bit after a 0 is also 0. We stay in this state no matter what the succeeding bits (if any) are.
- We return from s_0 or s_1, if a 1 follows a 0 in the string, before we come to two consecutive 0s.



NDFSA

- A nondeterministic finite-state automaton $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I, a *transition function* f that assigns a set of states to every pair of state and input (so that f: $S \times I \rightarrow P(S)$), an *initial* or *start state* s_0, and a subset F of S consisting of *final* (or *accepting*) *states*.
  - We can represent a nondeterministic finite-state automaton using a state table where we give a list of possible next states for each pair of a state and an input value.
  - We construct a state diagram for a nondeterministic automaton by including an edge from each state to all possible next states, labeling edges with the input or inputs that lead to this transition.
  - We use the abbreviation NDFSA for a nondeterministic finite-state automaton and DFSA for a deterministic finite-state automata when we needed to distinguish between NDFSA and DFSA.

Example: Find the state diagram for the NDFSA with the state table shown in Table 2.
The final states are s_2 and s_3.

Solution:

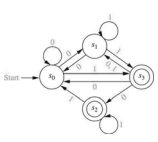


TABLE 2

	f	
	Input	
State	**0**	**1**
s_0	s_0, s_1	s_3
s_1	s_0	s_1, s_3
s_2		s_0, s_2
s_3	s_0, s_1, s_2	s_1

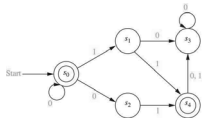
Finding a DFSA Equivalent to a NFSA

- For every NFSA there is an equivalent DFSA. That is, if the language L is recognized by a NFSA M_0, then L is also recognized by a DFSA M_1.

  We construct the DFSA M_1 so that
  - Each state in M_1 is made up of a set of states in M_0.
  - The start symbol of M_1 is $\{s_0\}$.
  - The input set of M_1 is the same as the input set of M_0.
  - Given a state $\{s_{i_1}, s_{i_2}, \ldots, s_{i_k}\}$ of M_1, the input symbol x takes this state to the union of the sets of next states for the elements of this set, that is, the union of the sets $f(s_{i_1}, x), f(s_{i_2}, x), \ldots, f(s_{i_k}, x)$.
  - We continue in this way to construct the states of M_1 from those of M_0.
  - The final states of M_1 are those sets that contain a final state of M_0.
- To see that M_0 and M_1 are equivalent, first suppose that an input string is recognized by M_0. This means that one of the states that can be reached from s_0 is a final state. So, in M_1, this input string leads from $\{s_0\}$ to a set of states of M_0 that contains the final state. Since this is a final state of M_1, this string is also recognized by M_1.
- Conversely, a string that is not recognized by M_0 does not lead to any final states in M_0. Consequently, this input string does not lead from $\{s_0\}$ to a final state of M_1.

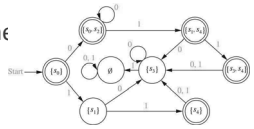
Finding an Equivalent DFSA (*cont.*)

Example: Find a DFSA that recognizes the same language as the NFSA:



State	f: Input 0	f: Input 1
s_0	s_0, s_2	s_1
s_1	s_3	s_4
s_2		s_4
s_3	s_3	
s_4	s_3	s_3

Solution: Following the steps of the procedure described on the previous slide, we obtain the DFSA shown here.



Language Recognition

Section 13.4

Section Summary

- Regular Expressions
- Kleene's Theorem
- Regular Sets and Regular Grammars
- More Powerful Types of Machines

Regular Expressions

- The *regular expressions* over a set I are defined recursively by:

  the symbol $\emptyset$ is a regular expression;
  the symbol λ is a regular expression;
  the symbol x is a regular expression whenever $x \in I$;
  the symbols (**AB**), (**A** ∪ **B**), and **A*** are regular expressions whenever **A** and **B** are regular expressions.
- Each regular expression represents a set specified by these rules:

  $\emptyset$ represents the empty set, that is, the set with no strings;
  λ represents the set $\{\lambda\}$, which is the set containing the empty string;
  x represents the set $\{x\}$ containing the string with one symbol x;
  (**AB**) represents the concatenation of the sets represented by **A** and by **B**;
  (**A** ∪ **B**) represents the union of the sets represented by **A** and by **B**;
  A* represents the Kleene closure of the set represented by **A**.
- Sets represented by regular expressions are called *regular sets*.

Regular Expressions (*cont.*)

Example: What are the strings in the regular sets specified by the regular expressions **10***, **(10)***, **0** ∪ **01**, **0(0** ∪ **1)***, and **(0*1)***?

Solution:

Expression	Strings
10*	a 1 followed by any number of 0s (including no zeros)
(10)*	any number of copies of 10 (including the null string)
0 ∪ **01**	the string 0 or the string 01
0(0 ∪ **1)***	any string beginning with 0
(0*1)*	any string not ending with 0

Finite-State Automata, Regular Sets, and Regular Grammars

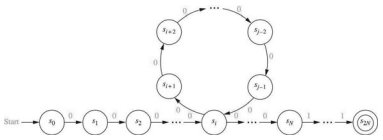
- In 1956 Kleene established the connection between regular sets and sets recognized by a FSA.
  - He showed that a set is regular if and only if it is recognized by a FSA. This result is known as *Kleene's theorem*.
  - See the text for the lengthy proof of this theorem.
- There is a close connection between regular grammars and regular sets.
  - Specifically, a set is generated by a regular grammar if and only if it is a regular set.
  - See the text for a proof.
  - We will give an example of a set that is not regular later in this section by finding a set that is not recognized by an FSA.

A Set Not Recognized by a FSA

- The set $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$ of all strings consisting of a block of 0s followed by a block of an equal number of 1s is not regular.
- To show that this set is not regular, suppose that this set was regular. Then there would be a NDFSA $M = (S, I, f, s_0, F)$ recognizing it.
- Let N be the number of states in this machine, i.e., $N = |S|$.
- M must recognize 0^N1^N since it is made up of a block of 0s followed by a block of an equal number of 1s.
- Let $s_0, s_1, s_2, \ldots, s_{2N}$ be the sequence of states obtained starting at s_0 and using the symbols of 0^N1^N as input. So, $s_1 = f(s_0, 0)$, $s_2 = f(s_1, 0)$, …, $s_N = f(s_{N-1}, 0)$, $s_{N+1} = f(s_N, 1)$, …, $s_{2N} = f(s_{2N-1}, 1)$, and s_{2N} is a final state.
- Because there are only N states, by the pigeonhole principle at least two of the first $N + 1$ states $s_0, s_2, \ldots, s_N$ must be the same.
- Suppose that s_i and s_j are identical states with $0 \le i < j \le N$. This means that $f(s_i, 0^t) = s_i$, where $t = j - i$. Hence, there is a loop leading from s_i back to itself, using 0 a total of t times.

continued

A Set Not Recognized by a FSA (*cont.*)



- Now consider the input string $0^N0^t1^N = 0^{N+t}1^N$. The string is not of the correct form and so, it is not recognized by *M*.
- Consequently, $f(s_0, 0^{N+t}1^N)$ can not be a final state.
- However, when we use the string $0^{N+t}1^N$ as input, we end up in the same state as before, namely, s_{2N}. The reason is that we go through the loop one more time.
- This contradiction shows that $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$ is not regular.

More Powerful Types of Machines

- The main limitation of finite-state automata is their finite amount of memory. This has led to the development of more powerful types of machines.
  - *Pushdown Automaton* (*PDA*): includes a stack, which provides unlimited memory. We can use a PDA to recognize $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$, but no PDA recognizes the set $\{0^n1^n2^n \mid n = 0, 1, 2, \ldots\}$.
  - *Linear Bounded Automaton* (*LBA*): More powerful than pushdown automata. We can use a LBA to recognize $\{0^n1^n2^n \mid n = 0, 1, 2, \ldots\}$, but there are languages generated by phrase-structure grammars that cannot be recognized by a LBA.
  - *Turing Machine* (*TM*): Yet more powerful machines (to be studied in the next section) which can recognize all languages generated by phrase-structure grammars.

Turing Machines

Section 13.5

Section Summary

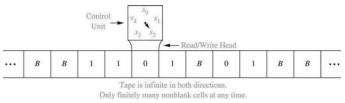
- Definition of Turing Machines
- Using Turing Machines to Recognize Sets
- Computing Functions with Turing Machines
- Different Types of Turing Machines (*not currently included in overheads*)
- The Church-Turing Thesis
- Computational Complexity, Computability, and Decidability

Introduction

Alan Mathison Turing
(1912-1954)



- Informally, a Turing machine consists of a *control unit*, which at any step is in one of finitely many different states, together with a *tape,* infinite in both directions, which is divided into *cells*.
- Turing machines have read and write capabilities on the tape as the control unit moves back and forth along this tape, changing states depending on the tape symbol read.
- Turing machines are more powerful than finite-state machines because they include additional memory capability.
- Turing machines are the most general models of computation; essentially they can do whatever a computer can do.



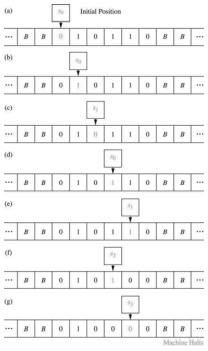
Definition of Turing Machines (TM)

- A *Turing machine* $T = (S, I, f, s_0)$ consists of
  - a finite set *S* of states,
  - an alphabet *I* containing the blank symbol *B*,
  - a partial function *f* from $S \times I$ to $S \times I \times \{R, L\}$, and
  - a starting state s_0.
- For some (state, symbol) pairs the partial function *f* may be undefined, but for a pair for which it is defined, there is a unique (state, symbol, direction) triple associated to this pair.
- The five-tuples corresponding to the partial function in the definition of a TM are called the *transition rules* of the machine.
- At each step, the control unit reads the current tape symbol *x*. If the control unit is in state *s* and if the partial function *f* is defined for the pair (s, x) with $f(s, x) = (s', x', d)$, the control unit:
  - enters the state s',
  - writes the symbol x' in the current cell, erasing *x*, and
  - moves right one cell if $d = R$ or moves left one cell if $d = L$.
- This step is written as the five-tuple (s, x, s', x', d). Turing machines are defined by specifying a set of such five-tuples. If the partial function *f* is undefined for the pair (s, x) then *T* will *halt*.
- At the beginning of its operation a TM is assumed to be in the initial state s_0 and to be positioned over the leftmost nonblank symbol on the tape. This is the *initial position* of the machine.

A TM in Operation

Example: What is the final tape when the TM *T* defined by the seven five-tuples $(s_0, 0, s_0, 0, R)$, $(s_0, 1, s_1, 1, R)$, (s_0, B, s_3, B, R), $(s_1, 0, s_0, 0, R)$, $(s_1, 1, s_2, 0, L)$, (s_1, B, s_3, B, R), and $(s_2, 1, s_3, 0, R)$ is run on the tape shown here in (a)?

Solution: The transitions of this TM are shown to the right. The final tape is shown in (g).



Using TM to Recognize Sets

- Let *V* be a subset of an alphabet *I*. A TM $T = (S, I, f, s_0)$ *recognizes* a string *x* in V^* if and only if *T*, starting in the initial position when *x* is written on the tape, halts in a final state.
- *T* is said to *recognize* a subset *A* of V^* if *x* is recognized by *T* if and only if *x* belongs to *A*.
- Note that to recognize a subset *A* of V^* we can use symbols not in *V*. This means that the input alphabet *I* may include symbols not in *V*. We will see that these extra symbols are used as markers.
- A TM operating on a tape containing the symbols of a string *x* in consecutive cells, does not recognize *x* if it does not halt or halts in a state that is not final.

Using TMs to Recognize Sets (*cont.*)

- In Section 13.4 we showed that there is no DFA that recognizes the set the set $\{0^n1^n \mid n \geq 1\}$. We will now construct a TM that recongizes this set.
  - We use an auxiliary tape symbol *M* as a marker, and specify that $V = \{0, 1\}$ and $I = \{0, 1, M\}$.
  - Our TM has one final state, s_6. The TM successively replaces a 0 at the leftmost position of the string with an *M* and a 1 at the rightmost position of the string with an *M*, sweeping back and forth, terminating in a final state if and only if the string consists of a block of 0s followed by a block of the same number of 1s.
  - The five-tuples are $(s_0, 0, s_1, M, R)$, $(s_1, 0, s_1, 0, R)$, $(s_1, 1, s_1, 1, R)$, (s_1, M, s_2, M, L), (s_1, B, s_2, B, L), $(s_2, 1, s_3, M, L)$, $(s_3, 1, s_3, 1, L)$, $(s_3, 0, s_4, 0, L)$, (s_3, M, s_5, M, R), $(s_4, 0, s_4, 0, L)$, (s_4, M, s_0, M, R), and (s_5, M, s_6, M, R).
- For example, the string 000111 would successively become *M*00111, *M*0011*M*, *MM*011*M*, *MM*01*MM*, *MMM*1*MM*, *MMMMMM* as the machine operates until it halts.

Computing Functions with TMs

- A Turing machine can be used to compute the values of a partial function.
- Suppose that the TM *T*, when given the string *x* as input, halts with the string *y* on its tape. We can then define $T(x) = y$.
- To consider a TM as a computer of functions from the set of *k*-tuples of nonnegative integers to the set of nonnegative integers, we use the *unary representation* of integers.
- A nonnegative integer *n* is represented by a string of $n + 1$ 1s. So, 0 is represented by 1, 5 by 111111, etc.
- To represent an input that is a *k*-tuple of integers, we represent each integer in the *k*-tuple separately and separate these representations using asterisks. For example, (2,0,1,3) is represented by 111*1*11*1111.
- Constructing a Turing machine that computes a particular function can be extremely complicated. Fortunately, the ability of Turing machines to compute functions is of theoretical, rather than practical, interest.

Computing Functions with TMs (*cont.*)

- To construct a TM *T* that computes the function $f(n_1,n_2) = n_1 + n_2$, we first represent the pair (n_1,n_2) by a string of $n_1 + 1$ 1s followed by an asterisk, followed by $n_2 + 1$ 1s.
- The machine starts at the leftmost 1 of the input string, and proceeds to erase this 1.
- If the next character is an asterisk, $n_1 = 0$. In this case, it replaces the asterisk with a blank and halts.
- Otherwise, it erases the next 1, and then passes over the remaining 1s, until it comes to the asterisk.
- The asterisk is then replaced by a 1.
- The five-tuples defining this Turing machine are: $(s_0, 1, s_1, B, R)$, $(s_1, *, s_3, B, R)$, $(s_1, 1, s_2, B, R)$, $(s_2, 1, s_2, 1, R)$, $(s_2, *, s_3, 1, R)$.

The Church-Turing Thesis

- The *Church-Turing Thesis* says that given any problem that can be solved with an effective algorithm, there is a TM that can solve this problem.
- It is called a *thesis* rather than a theorem because the concept of solvability by an effective algorithm is informal and imprecise, as opposed to the concept of solvability by a TM, which is formal and precise.

Decidability and Complexity

- A *decision problem* asks whether statements from a particular class of statements are true. Decision problems are also known as *yes-or-no problems.*
  - Consider the question for a particular integer *n*, "Is *n* prime?" The answer is "yes" or "no."
  - The *halting problem* is the decision problem that asks whether a Turing machine *T* eventually halts when given an input string *x*.
- When there is an effective algorithm that decides whether instances of a decision problem are true, we say that this problem is *solvable* or *decidable*.
  - In Section 3.5, an algorithm was given for determining whether a positive integer *n* is prime by checking whether it is divisible by primes not exceeding its square root.
- However, if no effective algorithm exists for solving a problem, then we say the problem is *unsolvable* or *undecidable*.
  - The halting problem is an unsolvable decision problem (proved in Section 3.1). That is, no TM exists that, when given an encoding of a TM *T* and its input string *x* as input, can determine whether *T* eventually halts when started with *x* written on its tape.
- A function that can be computed by a TM is called *computable* and a function that cannot be computed by a TM is called *uncomputable*.
  - The *busy beaver function*, which when given a positive integer *n* gives the maximum number of 1s that a TM with *n* states and alphabet {1,*B*} may print on an initially blank tape is uncomputable (see Exercise 35 in Section 13.5).

The Classes *P* and *NP*

- In a *nondeterministic Turing machine* (*NDTM*), the restriction that no two transition rules begin with the same pair (s, x) is eliminated.
- Hence, there may be more than one transition rule beginning with each (state, tape symbol) pair, so that there may be a choice as to which rule to use at each step.
- A NDTM *T* recognizes a string *x* if and only if there exists some sequence of transitions of *T* that ends in a final state when the machine starts in the initial position with *x* written on the tape.

The Classes *P* and *NP* (*cont.*)

- A decision problem is in *P*, the *class of polynomial-time problems*, if it can be solved by a deterministic Turing machine in polynomial time in terms of the size of its input.
- That is, a decision problem is in *P* if there is a deterministic Turing machine *T* that solves the decision problem and a polynomial $p(n)$ such that for all integers *n*, *T* halts in a final state after no more than $p(n)$ transitions whenever the input to *T* is a string of length *n*.
- A decision problem is in *NP*, the *class of nondeterministic polynomial-time problems*, if it can be solved by a nondeterministic Turing machine in polynomial time in terms of the size of its input.
- That is, a decision problem is in *NP* if there is a nondeterministic Turing machine *T* that solves the problem and a polynomial $p(n)$ such that for all integers *n*, *T* halts for every choice of transitions after no more than $p(n)$ transitions whenever the input to *T* is a string of length *n*.
- Problems in *P* are called *tractable*, whereas problems not in *P* are called *intractable*.

The Classes *P* and *NP* (*cont.*)

- For a problem to be in *NP*, it is necessary only that there be a NDTM that when given a true statement from the set of statements addressed by the problem, can verify its truth in polynomial time by making the correct guess at each step.
  - The problem of determining whether a given graph has a Hamilton circuit is an *NP* problem, because a NDTM can easily verify that a simple circuit in a graph passes through each vertex exactly once.
  - It can do this by making a series of correct guesses corresponding to successively adding edges to form the circuit.

Wrapping Everything up with a Millenium Problem

- Because every DTM can also be a considered to be a NDTM, where each (state, tape symbol) pair occurs in exactly one transition rule, P ⊆ NP.
- The most famous open question in theoretical CS, and one of the millennium problems with a $1,000,000 prize, is whether every problem in *NP* is also in *P*, that is, whether *P* = *NP*.
- There is an important class of problems, known as *NP-complete* problems, where a problem is in this class if it is the class *NP* and if this problem was also in the class *P*, then every problem in *NP* must also be in *P*.
- That is, a problem is *NP*-complete if the existence of a polynomial-time algorithm for solving it implies the existence of a polynomial-time algorithm for every problem in *NP*.
- We have studied several problems that can be shown to be *NP*-complete in this text, including determining whether a simple graph has a Hamilton circuit and determining whether a proposition in *n* variables is a tautology.
- This concludes our introduction to discrete mathematics, but it should come as no surprise that there is a lot more to learn!